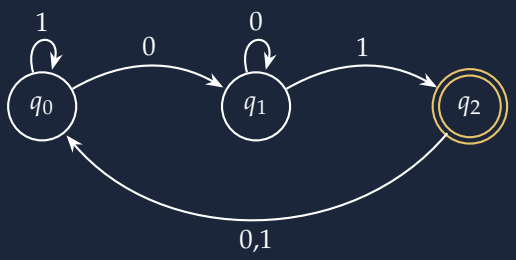
BUET • CSE 211

Theory of Computation

Topic-wise questions with
complete worked solutions



16
topics

363
question entries

17
exam papers

Sessions 2006–2007 to 2023–2024, including the January 2020 Term.

Every question is followed by its solution. Questions whose answers are identical are **stacked** and solved once. All automata, PDAs, parse trees and Turing-machine diagrams are drawn in Ti*k*Z.

MAHDI HASAN

Credit: Claude Opus 5.5

Contents

01 Automata concepts and state modelling . . . 1
02 DFA design and numerical languages . . . 11
03 NFA design, ε-transitions and conversion . . . 25
04 Regular expressions and language operations . . . 45
05 Regular-language closure and equivalence . . . 56
06 Regular pumping lemma and nonregular languages . . . 65
07 CFG construction, derivations and parse trees . . . 74
08 Ambiguous grammars and inherent ambiguity . . . 86
09 Grammar simplification, CNF and CYK . . . 94
10 Pushdown automata and CFG–PDA equivalence . . . 100
11 CFL pumping lemma and non-context-free languages . . . 119
12 Turing-machine design and computation . . . 123
13 Turing-machine models, simulation and encoding . . . 144
14 Decidability, RE languages and countability . . . 154
15 Complexity, reductions and cryptography . . . 164
16 Out of syllabus . . . 173

How to read this book

2099–2100 • Q0(a) Session • question number **MARKS 5**

A question card. The coral badge on the right shows the printed marks. Figures from the original paper are redrawn inside the card.

SOLUTION

The solution panel always follows its question. Key results are highlighted:

Answer. like this.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

Paper A Q1 **MARKS 8**

A question that was asked again…

Paper B Q4 **MARKS 10**

…and again (possibly with slightly different wording)…

Paper C Q2 **MARKS 6**

…is stacked, and solved *once* below.

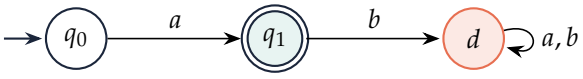
SOLUTION

One shared solution covers every stacked card.

NOTE **Gold boxes flag an issue in the source paper (a typo, an obscured line, a contradictory example) and say which reading the solution uses.**

SCOPE *says which part of a shared solution answers which paper.*

Notation used in diagrams



- An arrow from nowhere marks the start state; a double circle (tinted) is an accepting state; a rose state is a dead (trap) state.
- In a transition table, $\rightarrow$ marks the start state and $*$ an accepting state.
- PDA edges are written $a, X/\gamma$: read a (or ε), replace the top X by γ (leftmost symbol on top; $\gamma = \varepsilon$ pops). Z_0 is the start stack symbol.
- TM edges are written $X/Y\,D$: read X, write Y, move $D \in \{L, R\}$. B is the blank.

01

Automata concepts and state modelling

2023–2024 • Q2(c) Source N01 **MARKS 5**

Does the number of accepting DFA states affect the numbers of rows or columns in its transition table? Explain.

SOLUTION

No. A transition table of a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is simply a tabulation of the total function $\delta : Q \times \Sigma \to Q$.

- There is exactly **one row per state** $q \in Q$, so the number of rows is $|Q|$, regardless of how many of those states are in F.
- There is exactly **one column per input symbol** $a \in \Sigma$, so the number of columns is $|\Sigma|$.

Accepting states are only *marked* (with $*$) in the row label; changing F changes the markings, never the shape. For example, the parity DFA below has a 2×2 table whether $F = \{E\}$ (even number of 1's) or $F = \{O\}$ (odd) or $F = \{E, O\}$ (everything):

δ	0	1
$\to *E$	E	O
O	O	E

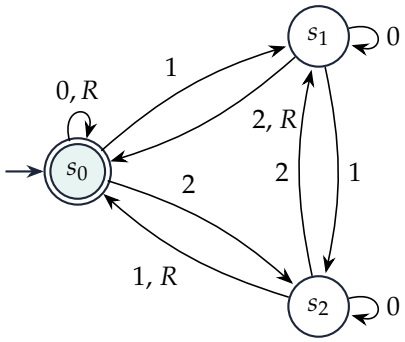
Answer. Rows $= |Q|$ and columns $= |\Sigma|$; the size of F only affects which rows carry a $*$.

2022–2023 • Q1(b) Source M49 **MARKS 12**

An automaton over $\{R, 0, 1, 2\}$ keeps the numerical-symbol sum modulo 3, resets it to 0 on R, and accepts remainder 0. A three-state design tracks all remainders. Null Pointer proposes using two states, one for remainder 0 and one for a nonzero remainder. Analyze this claim in detail.

SOLUTION

The correct design. States s_0, s_1, s_2 remember the current sum mod 3; on digit d go from s_i to $s_{(i+d) \bmod 3}$, on R go to s_0; accept in s_0.



Why two states cannot work. Suppose a DFA with states Z (remainder 0) and N ("nonzero") existed. After reading `1` it is in N, and after reading `2` it is also in N. Now append `1`:

$$1 \cdot 1 = 11 :\ 2 \not\equiv 0 \text{ (reject)}, \qquad 2 \cdot 1 = 21 :\ 3 \equiv 0 \text{ (accept)}.$$

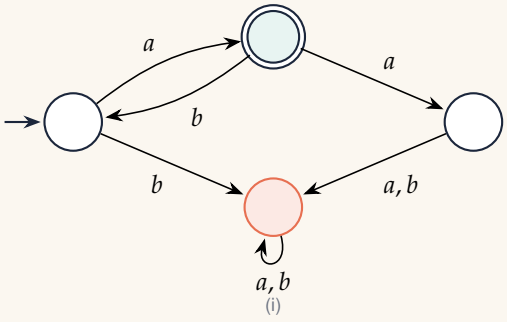
A DFA is deterministic, so from the *same* state N the same suffix `1` must lead to the same state and the same verdict — contradiction. The state N has lost the information "is the remainder 1 or 2?", which is exactly what is needed to decide the future.

Lower bound (distinguishable prefixes). The prefixes ε, `1`, `2` are pairwise distinguishable: suffix `2` separates `1` (sum 3, accept) from ε and `2` (sums 2 and 4, reject); suffix `1` separates `2` (sum 3, accept) from ε (sum 1, reject). If two of them led to the same state, appending the separating suffix would force the same verdict for both, which is impossible. Hence *every* DFA for the language needs at least 3 states.

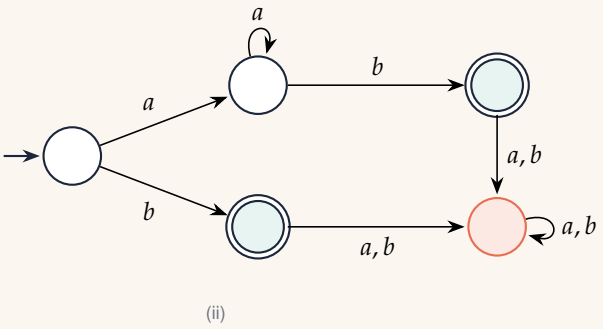
Answer. The claim is wrong: "nonzero" merges remainders 1 and 2, which react differently to future input (e.g. suffix `1`). Three states are necessary and sufficient.

2022–2023 • Q2(a)(i–v) Source M49–M50 **MARKS 20 (shared)**

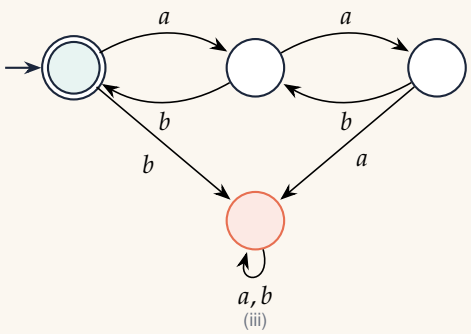
Describe informally the language accepted by each of the five shown automata.



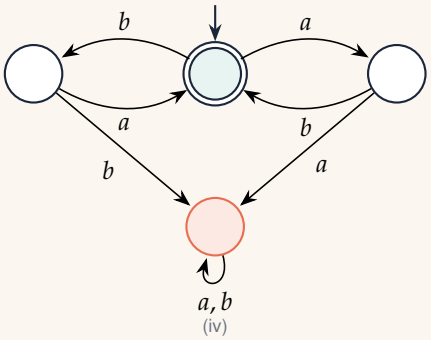
(i)



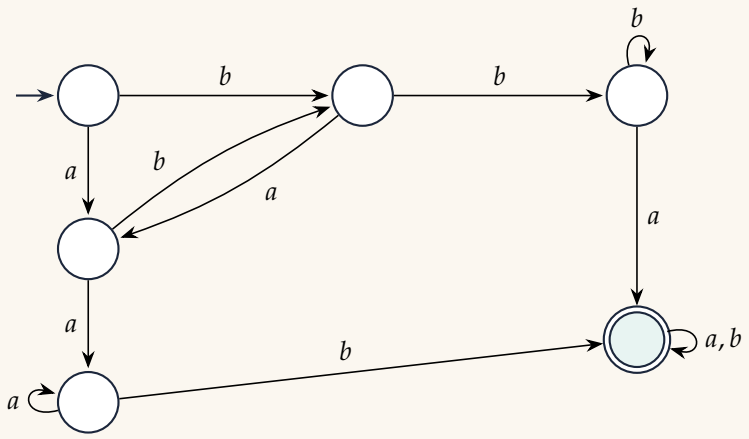
(ii)



(iii)



(iv)



(v)

SOLUTION

Trace what each state "remembers".

(i) From the start, a leads to the only accepting state; from there b returns to the start and a falls (via an extra state) into the trap. Accepted strings are therefore a, aba, $ababa$, ...

Answer. $L_1 = (ab)^*a$: strings that alternate a, b, starting and ending with a.

(ii) The upper branch reads a^+ then a single b to accept; the lower branch accepts the single symbol b. Any further symbol goes to the trap.

Answer. L_2 = any number of a's (possibly none) followed by exactly one b, which is the last symbol; i.e. strings with exactly one b, at the end. In symbols: $a^*b = \{a^n b \mid n \geq 0\}$.

(iii) Think of a as "(" and b as ")". The three non-trap states record the current depth $0, 1, 2$; a third unmatched a (depth 3) or a b at depth 0 goes to the trap, and we accept at depth 0.

Answer. $L_3 = (a(ab)^*b)^*$: strings with equally many a's and b's in which every prefix satisfies $0 \leq \#_a - \#_b \leq 2$ (balanced "parentheses" of nesting depth at most 2).

(iv) From the centre (accepting) state the machine must read either ab or ba to return; aa or bb inside a block leads to the trap.

Answer. $L_4 = (ab \cup ba)^*$: even-length strings in which positions $2i-1, 2i$ always hold different symbols.

(v) The left column remembers "last symbol a"; the lower-left state remembers "ends in aa" (it loops on a); a b there completes aab. Symmetrically the top-right state remembers "ends in bb" and an a completes bba. The final state is a sink.

Answer. $L_5 = \{w \mid w$ contains aab or bba as a substring$\}$.

2021–2022 • Q1(a) Source M44 **MARKS 4**

Which of the three traditionally central areas of Theory of Computation is applied in a password-protected computer login system? Explain clearly.

SOLUTION

The three areas are **automata theory**, **computability theory** and **complexity theory**. A login system

uses two of them, for two different jobs.

- **Automata theory — checking the password rules.** When a password is chosen, the system checks rules such as "at least 8 characters, contains a digit, contains an uppercase letter, contains no spaces" (the "Password Rules" example of Lecture 1). This needs only *finite memory*: a DFA that remembers $\min(\text{length}, 8)$, whether a digit has been seen and whether an uppercase letter has been seen ($9 \cdot 2 \cdot 2 = 36$ states), plus a dead state entered on a space, reads the password once and accepts exactly the valid ones. So the set of valid passwords is a regular language.
- **Complexity theory — keeping stored passwords secure.** The system stores only a hash $h(\text{password})$, where h is a *one-way function*: computing h is easy (polynomial time), so a login attempt is checked quickly, but inverting h, or guessing the password by brute force over an exponentially large space, is believed computationally infeasible. Security is an argument about the *difficulty* of computation (the same idea as in cryptography), not about what is computable at all.

Computability theory plays no direct role: every task here is decidable; the question is only how much memory or time it needs.

2021–2022 • Q2(a) Source M45 **MARKS 4**

DFA and NFA recognize the same class of languages. Why is this equivalence both surprising and useful?

SOLUTION

Surprising. An NFA may branch into many simultaneous computations, use ε-moves and "guess" (e.g. where a suffix begins). Intuitively this looks strictly more powerful than a single deterministic thread — for Turing-machine time complexity the analogous question (P vs NP) is still open, and for PDAs nondeterminism *does* add power. Yet for finite automata the whole set of currently-active NFA states is itself a finite object (a subset of Q), so a DFA can track it: the subset construction removes nondeterminism at the cost of at most $2^{|Q|}$ states.

Useful.

- *Design*: NFAs are much easier to build and smaller (e.g. "the 3rd symbol from the end is 1" needs 4 NFA states but 8 DFA states).
- *Proofs*: closure under union, concatenation and star, and the conversion regex $\to$ automaton, are trivial with NFAs; the equivalence transfers them to DFAs.
- *Implementation*: DFAs run in one table lookup per symbol, can be complemented by swapping F, and tested for equivalence. We design with NFAs and execute with DFAs.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q2(a) Source M40 **MARKS 5**

Briefly explain the role of a state in a finite automaton.

2007–2008 • Q2(c) Source M04 **MARKS $5\frac{1}{3}$**

Describe the most important role played by states in an automaton. Can this role have infinite capacity? Why?

SOLUTION

SCOPE *2019–2020 Q2(a) asks only for the role of a state (first paragraph).*

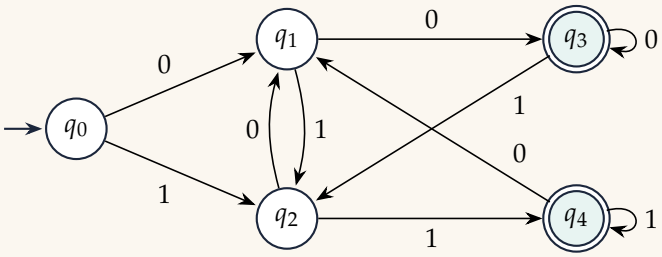
Role: states are the machine's only memory. A finite automaton has no tape or stack; after reading a prefix x it is in one state, and that state must summarise *everything about x that matters for the future*. In short, **a state is a compressed summary of the past**: it captures exactly the information needed to determine future behaviour, ignoring irrelevant history. Two prefixes may therefore lead to the same state only if no continuation can ever tell them apart. Example: for "even number of 1's" it suffices to remember the parity, so two states E, O suffice; for "ends in 01" we remember the relevant suffix (ε, 0, 01).

Infinite capacity? No. A finite automaton has finitely many states $|Q| = k$, so it can remember at most $\lceil \log_2 k \rceil$ bits about an arbitrarily long input. It cannot, for instance, count unboundedly: to recognise $\{0^n 1^n\}$ it would have to distinguish $0^1, 0^2, 0^3, \ldots$, i.e. infinitely many states. If we allowed infinitely many states the device would no longer be a finite automaton — in fact it could accept *any* language (one state per prefix, a "trie"), and it could not be built. Unbounded memory is exactly what PDAs (a stack) and Turing machines (a tape) add.

Answer. The state is a finite memory summarising the relevant past; its capacity is necessarily finite, which is why FAs recognise only regular languages.

January 2020 Term • Q2(a) Source M36 **MARKS 7**

State informally the language accepted by the shown DFA.



SOLUTION

q_1 means "the last symbol is 0 but the last two are not 00"; q_3 means "the last two symbols are 00". Symmetrically q_2/q_4 for 1. Every transition on 0 goes to q_1 or q_3, every transition on 1 to q_2 or q_4, and we accept exactly in q_3, q_4.

Answer. $L = \{w \in \{0,1\}^* \mid w \text{ ends in } 00 \text{ or } 11\} = (0 \cup 1)^*(00 \cup 11)$, i.e. the last two symbols are equal.

2016–2017 • Q1(a–d) Source M28 **MARKS (a) 5; (b) 10; (c) 6; (d) 14**

On planet Xustaeter there are red, green, and blue stones. Contact between different colors destroys those two stones and creates two stones of the third color. The planet fails when all stones have one color. Represent each state by (red count, green count, blue count). An R-event

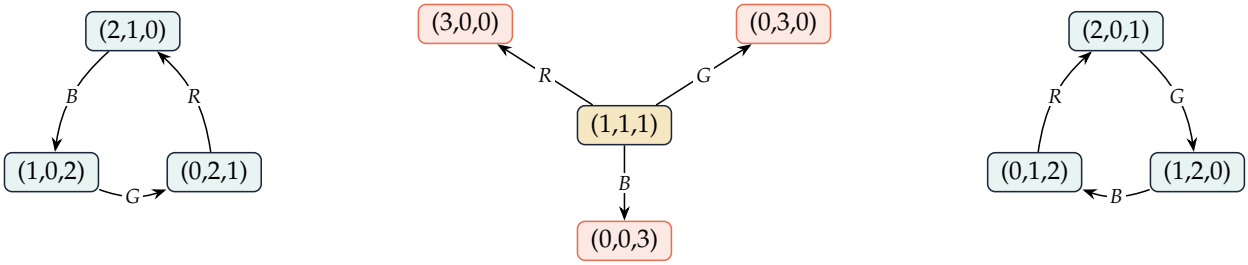
replaces a green–blue pair by two red stones; define G- and B-events analogously. (a) For 3 stones, give a state diagram and classify must-fail, can't-fail, and might-fail states. (b) Do the same for 4 stones. (c) Use symmetries in (b) to propose a reduced state-diagram scheme. (d) Using the scheme from (c), do the same for 6 stones.

SOLUTION

Events: $R : (r,g,b) \mapsto (r+2, g-1, b-1)$ (needs $g, b \geq 1$), and similarly G, B. Events happen in any (nondeterministic) order. Classification: *must-fail* = every sequence of events reaches a monochrome state; *can't-fail* = no sequence ever does; *might-fail* = some sequences do and some avoid failure forever.

Key invariant. An R-event changes $r-g$ by $+3$, $g-b$ by 0 and $r-b$ by $+3$; the other events behave alike. Hence the pairwise differences of the counts *mod 3 never change*. A failed state $(n,0,0)$ has two equal counts, so **a state whose three counts are pairwise incongruent mod 3 can never fail.**

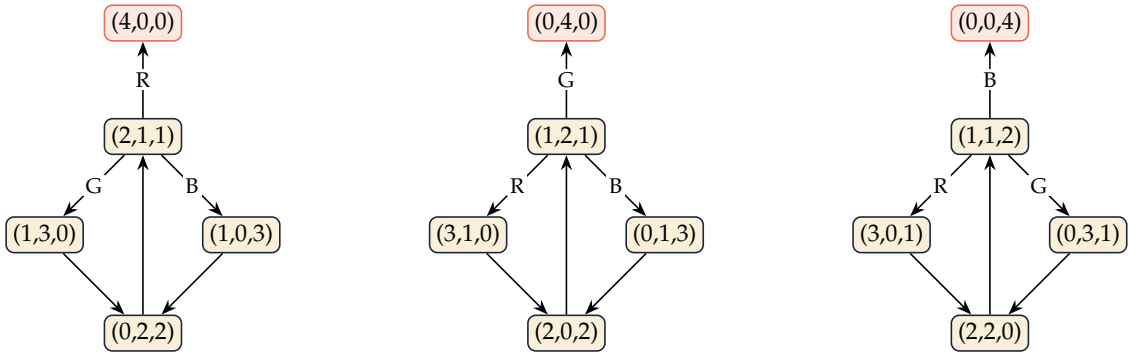
(a) Three stones (10 states)



Failed: $(3,0,0), (0,3,0), (0,0,3)$. **Must-fail**: $(1,1,1)$ — each of its three events produces a monochrome state. **Can't-fail**: the six permutations of $(2,1,0)$; each has exactly one possible event and they form two 3-cycles (residues $2,1,0$ are distinct). **Might-fail**: none.

(b) Four stones (15 states)

The diagram splits into three disjoint, identical clusters (one per possible failure colour):

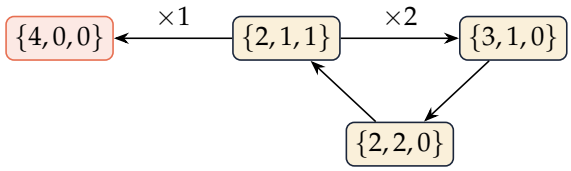


(The unlabeled edges are the single possible event of that state, e.g. $(1,3,0) \xrightarrow{B} (0,2,2) \xrightarrow{R} (2,1,1)$.) Failed: $(4,0,0), (0,4,0), (0,0,4)$. Every other state lies on a cycle that avoids failure (e.g. $(2,1,1) \to (1,3,0) \to (0,2,2) \to (2,1,1)$) and can also reach the hub's failing edge. Hence **all 12 non-failed states are might-fail**; there are no must-fail or can't-fail states. (Consistent with the invariant: $4 \equiv 1 \pmod 3$, so every state has two congruent counts.)

(c) Reduced scheme

The rules are symmetric under any permutation of the colours, so a permuted state has the same fate. Represent a state by the **multiset** of counts $\{x \geq y \geq z\}$ (sorted triple) and draw one node

per multiset; an edge $\{x,y,z\} \to \{x',y',z'\}$ exists if some event turns one representative into some representative of the other (optionally labelled by the number of ways). For 4 stones the 15 states collapse to 4 classes:

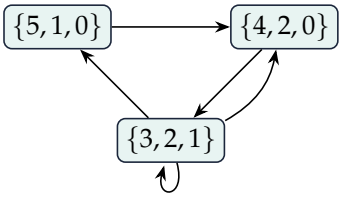


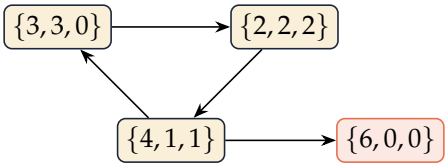
(d) Six stones with the reduced scheme

The 28 states form 7 classes. Applying the events to a representative of each class:

$$\{5,1,0\} \to \{4,2,0\} \to \{3,2,1\}, \qquad \{3,2,1\} \to \{5,1,0\},\ \{4,2,0\},\ \{3,2,1\};$$

$$\{3,3,0\} \to \{2,2,2\} \to \{4,1,1\}, \qquad \{4,1,1\} \to \{6,0,0\}\ \text{(fail)},\ \{3,3,0\}.$$

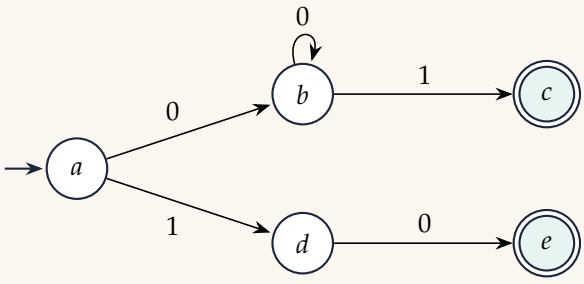




- **Failed**: $\{6,0,0\}$ (3 states).
- **Can't-fail**: $\{5,1,0\}, \{4,2,0\}, \{3,2,1\}$ — 18 states. Their counts are $\equiv 0,1,2$ (all different) mod 3, so by the invariant they never fail; indeed the left component never reaches $\{6,0,0\}$.
- **Might-fail**: $\{3,3,0\}, \{2,2,2\}, \{4,1,1\}$ — 7 states. The cycle $\{3,3,0\} \to \{2,2,2\} \to \{4,1,1\} \to \{3,3,0\}$ avoids failure, but from $\{4,1,1\}$ the event that merges the two singletons fails.
- **Must-fail**: none (other than already-failed states).

2015–2016 • Q1(a–d) Source M24 **MARKS (a) $8\frac{1}{3}$; (b) 4; (c) 4; (d) 4**

For the FSM F_1 in the figure: (a) Formally define an FSM and formally describe F_1. (b) What language does F_1 recognize? (c) Add only $\delta(d,1) = d$ to obtain F_2. What language does F_2 recognize? (d) A friend changes F_1 without changing its numbers of states and transitions, obtaining F_3, and claims $|L(F_3)| = |L(F_1)| + 1$. Is that possible? Justify.



SOLUTION

(a) A finite-state machine (DFA) is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$: Q a finite set of states, Σ a finite input alphabet, $\delta : Q \times \Sigma \to Q$ the transition function, $q_0 \in Q$ the start state, $F \subseteq Q$ the accepting states. M accepts $w = w_1 \cdots w_n$ iff the run $r_0 = q_0$, $r_i = \delta(r_{i-1}, w_i)$ ends in $r_n \in F$.
The figure omits the moves that lead nowhere; by convention every missing move goes to a non-accepting trap state t, which makes δ total. Formally $F_1 = (\{a, b, c, d, e, t\}, \{0, 1\}, \delta, a, \{c, e\})$ with

δ	0	1
$\to a$	b	d
b	b	c
$*c$	t	t
d	e	t
$*e$	t	t
t	t	t

(b) There are only two ways to reach an accepting state. Through b the machine reads one or more 0's (the loop on b) and then a single 1, after which any further symbol leads to the trap. Through d it reads 1 and then 0, and again nothing more may follow. So F_1 accepts the string 10, together with every string made of one or more 0's followed by exactly one 1, where that 1 is the last symbol (e.g. 01, 001, 0001). It rejects strings such as 0101: they start with 0 and end with 01, but contain a second 1.

Answer. $L(F_1)$ = the string 10, or one or more 0's followed by a single final 1. In symbols: $0^+1 \cup \{10\} = \{0^n1 \mid n \geq 1\} \cup \{10\}$ (an infinite language).

(c) The new loop on d lets the machine read any number of extra 1's before the final 0, so the lower path now accepts one or more 1's followed by a single 0. The upper path is unchanged. The two parts mirror each other: a string is accepted iff it has length at least 2, all of its symbols except the last are the same, and the last symbol is the other one.

Answer. $L(F_2)$ = one or more 0's followed by a single 1, or one or more 1's followed by a single 0. In symbols: $0^+1 \cup 1^+0$.

(d) *Yes, it is possible.* Simply make the start state a accepting. No state or transition is added or removed, and $L(F_3) = L(F_1) \cup \{\varepsilon\}$: exactly one extra string. (Even read purely as cardinalities the claim is consistent: $|L(F_1)| = \aleph_0$ and $\aleph_0 + 1 = \aleph_0$.)

2015–2016 • Q1(e) Source M24 MARKS 3

A regular expression R_1 represents exactly five strings. You design an equivalent DFA D_1 and correctly implement it in C. The program accepts 1000 distinct random test strings. Your younger brother claims your DFA is wrong. Do you agree? Justify.

SOLUTION

Yes, the brother is right. If $L(D_1) = L(R_1)$ then $|L(D_1)| = 5$, so *at most five* distinct strings can ever be accepted. Since the program (a correct implementation of D_1) accepted 1000 distinct strings, at least 995 accepted strings are not in $L(R_1)$; hence $L(D_1) \neq L(R_1)$ — the DFA design is wrong. (A correct DFA for a finite language of 5 strings must have every path to an accepting state of bounded length, i.e. no cycle can lie on a path from the start to an accepting state.)

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2012–2013 • Q8(a) Source M18 **MARKS 5**

Give three differences between DFA and NFA, with examples.

2011–2012 • Q5(a) Source M15 **MARKS 6**

Give three differences between DFA and NFA, with examples.

2008–2009 • Q5(c) Source M07 **MARKS $5\frac{1}{3}$**

Define deterministic and nondeterministic finite automata.

SOLUTION

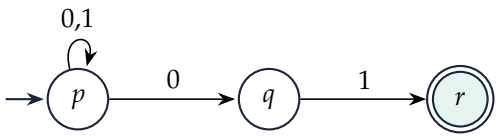
SCOPE *2008–2009 Q5(c) asks only for the definitions (first paragraph).*

Definitions. A **DFA** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$: Q a finite set of states, Σ the input alphabet, $\delta : Q \times \Sigma \to Q$ the transition function, $q_0 \in Q$ the start state, $F \subseteq Q$ the accepting states. M accepts $w = w_1 w_2 \cdots w_n$ ($w_i \in \Sigma$) if there is a sequence of states $r_0, r_1, \ldots, r_n$ with $r_0 = q_0$, $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \le i \le n-1$, and $r_n \in F$. An **NFA** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$ with the same components except $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$, where $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$. N accepts w if w can be written as $w = y_1 y_2 \cdots y_m$ with each $y_i \in \Sigma_\varepsilon$ and there is a sequence of states $r_0, r_1, \ldots, r_m$ with $r_0 = q_0$, $r_{i+1} \in \delta(r_i, y_{i+1})$ for $0 \le i \le m-1$, and $r_m \in F$ — i.e. *at least one* computation branch accepts.

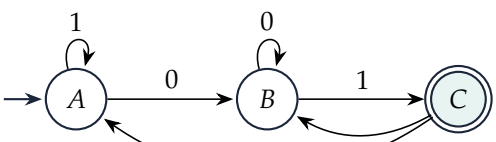
Differences.

	DFA	NFA
1. Next move	exactly one next state for each (state, symbol)	zero, one or several next states
2. ε-moves	not allowed	allowed (state change without input)
3. Acceptance	the unique run ends in F	at least one of the possible runs ends in F
4. Missing moves	δ is total	a missing move just kills that branch
5. Size/ease	may need up to 2^n states	often exponentially smaller, easier to design
6. Execution	one table lookup per symbol	must track a set of states (or backtrack)

Example: $L = \{w \in \{0,1\}^* \mid w \text{ ends in } 01\}$.



NFA: guesses where 01 starts



DFA: remembers the relevant suffix

In the NFA, $\delta(p, 0) = \{p, q\}$ (two choices) and $\delta(q, 0) = \emptyset$; the DFA has exactly one arrow per symbol from each state. Both accept the same language.

2007–2008 • Q1(c) Source M04 **MARKS $5\frac{1}{3}$**

Explain how set-formers can be used to define a language.

SOLUTION

A language is a set of strings, $L \subseteq \Sigma^*$, so it can be written with set-builder ("set-former") notation

$$L = \{\, w \mid \text{property } P(w) \text{ holds} \,\} \qquad \text{or} \qquad L = \{\, \text{pattern} \mid \text{conditions on its parameters} \,\}.$$

Examples:

- $\{w \in \{0,1\}^* \mid w \text{ has an equal number of 0's and 1's}\}$ — a property of the string;
- $\{0^n 1^n \mid n \geq 1\}$ — a pattern with a parameter n and a condition $n \geq 1$;
- $\{0^i 1^j \mid 0 \leq i \leq j\}$, $\{w \mid w \text{ is a binary integer that is prime}\}$, $\{ww^R \mid w \in \{a,b\}^*\}$.

The notation is precise and compact, can describe infinite languages, and makes membership a logical question ("does w satisfy P?"); it is the starting point for designing automata or grammars.

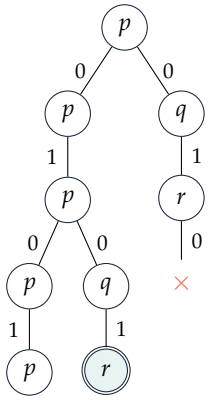
2006–2007 • Q2(c) Source M01 **MARKS** $5\frac{1}{3}$

Elaborate, with an example: 'One way to think of a nondeterministic computation is as a tree of possibilities.'

SOLUTION

At each step an NFA may have several choices. Draw the start configuration as the root; each child is a configuration reachable in one move. A path from root to a leaf is one possible computation; the input is **accepted iff at least one leaf** (after the whole input is read) is an accepting state. Branches with no move simply die. Equivalently the machine "explores all branches in parallel"; the set of nodes at depth i is the set of states after i symbols — exactly the DFA state of the subset construction.

Example. NFA for "ends in 01" ($p \xrightarrow{0,1} p$, $p \xrightarrow{0} q$, $q \xrightarrow{1} r$, r accepting) on input `0101`:



One leaf at depth 4 is the accepting state r, so `0101` is accepted; the branch through r at depth 2 dies because r has no move on 0.

02

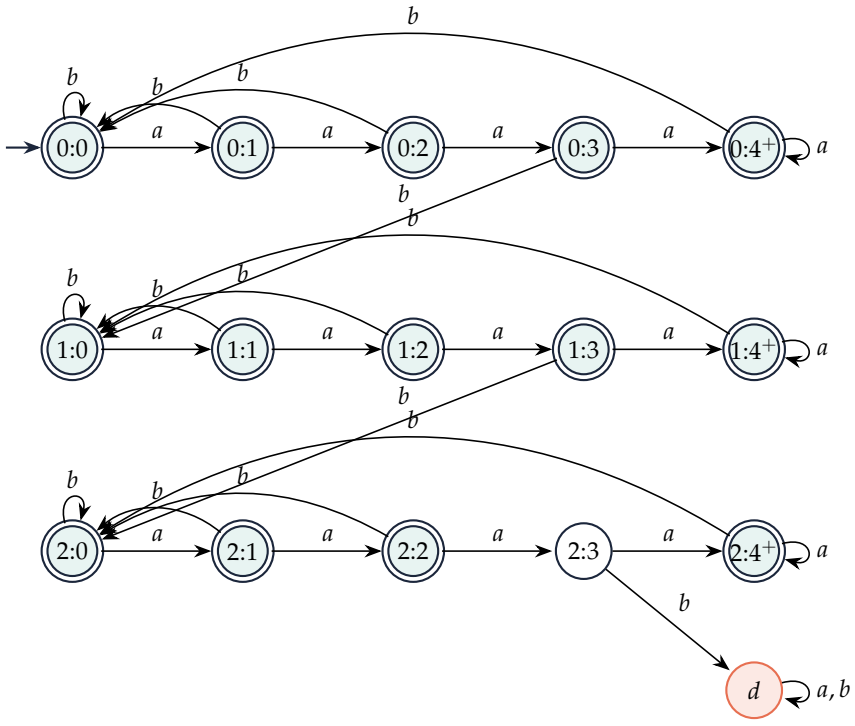
DFA design and numerical languages

2023–2024 • Q1(b) Source N01 **MARKS 17**

A run is a maximal substring of length at least two consisting of one repeated symbol. For example, *abbbaab* has a *b*-run of length 3 and an *a*-run of length 2. Design a DFA over $\{a, b\}$ for strings with at most two runs of *a*'s of length exactly three. Give only the state diagram.

SOLUTION

Idea. Remember two things: c = how many completed *a*-runs of length exactly 3 have been seen $(0, 1, 2)$, and ℓ = length of the current block of *a*'s $(0, 1, 2, 3$ or "4^+"$)$. On a: $\ell \mapsto \min(\ell + 1, 4)$. On b: the block ends; if $\ell = 3$ then c increases (a third one sends us to the dead state d), and $\ell \mapsto 0$. At the end of input a block of length exactly 3 that is still open also counts, so state $(2{:}3)$ is *rejecting* while every other non-dead state accepts. State $c{:}\ell$ is drawn in row c, column ℓ.



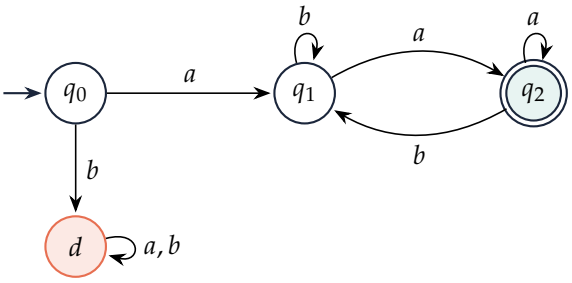
All 15 states $c{:}\ell$ except 2:3 are accepting. (Brute-force check: this DFA agrees with the definition on all $2^{13} - 1$ strings of length ≤ 12.)

2023–2024 • Q2(b) Source N01 **MARKS 15**

Show that $L = \{awa \mid w \in \{a, b\}^*\}$ is regular by constructing a DFA.

SOLUTION

L = strings of length ≥ 2 that begin and end with a. After the first a, state q_2 means "last symbol is a" (accept) and q_1 means "last symbol is b or only the first a was read".



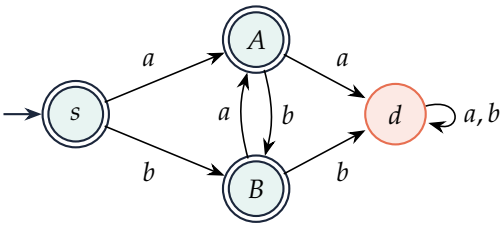
Note that the single string a ends in q_1 and is rejected, as required ($|awa| \geq 2$). Since a DFA recognises L, L is regular.

2022–2023 • Q1(c) Source M49 **MARKS 13**

Construct a DFA for $\{w \in \{a,b\}^* \mid \text{neither } aa \text{ nor } bb \text{ is a substring}\}$. A transition-table representation is required.

SOLUTION

The strings are exactly the alternating strings. Remember the last symbol.



δ	a	b
$\to *s$	A	B
$*A$	d	B
$*B$	A	d
d	d	d

s: nothing read; A/B: last symbol a/b; d: dead.

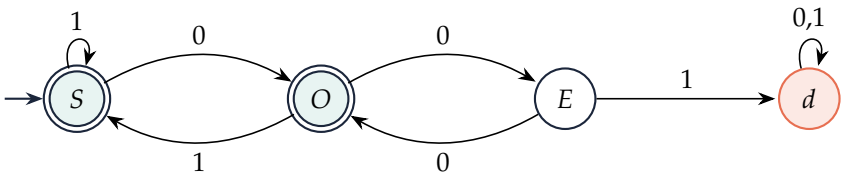
2022–2023 • Q3(a) Source M51 **MARKS 15**

Construct a DFA accepting binary strings in which no run of 0's has even length ≥ 2. A run is a maximal block of consecutive 0's: 00 has one run of length 2; 000010 has runs of lengths 4 and 1. Reject 0001100, accept 01110001 and 11111. Give a state diagram and explain the design idea.

SOLUTION

Idea. Every maximal 0-run must have *odd* length. While inside a run we only need its parity; the verdict on a run is given when it ends — either by a 1 or by the end of the input.

- S: not inside a 0-run (start, or last symbol 1) — accept.
- O: inside a 0-run of odd length so far — accept (if the input ends here the run is odd).
- E: inside a 0-run of even length so far — reject at end; a 1 here closes an even run $\Rightarrow$ dead d.



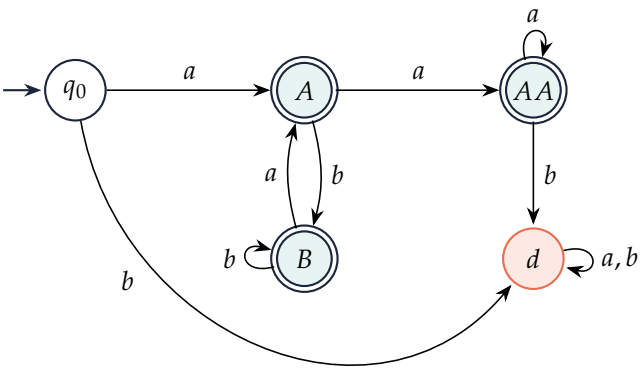
Checks: 0001100: runs 000 (odd), 00 (even, ends in E) $\Rightarrow$ reject. 01110001: $S \xrightarrow{0} O \xrightarrow{111} S \xrightarrow{000} O \xrightarrow{1} S$ accept. 11111: stays in S, accept.

2021–2022 • Q1(b) Source M44 **MARKS 10**

Design a DFA with no more than five states over $\{a, b\}$, accepting strings that begin with a and do not contain aab. Give only the state diagram.

SOLUTION

States: q_0 start; A = last symbol a (single); AA = ends in aa (a following b would complete aab); B = last symbol b; d = dead. Five states.

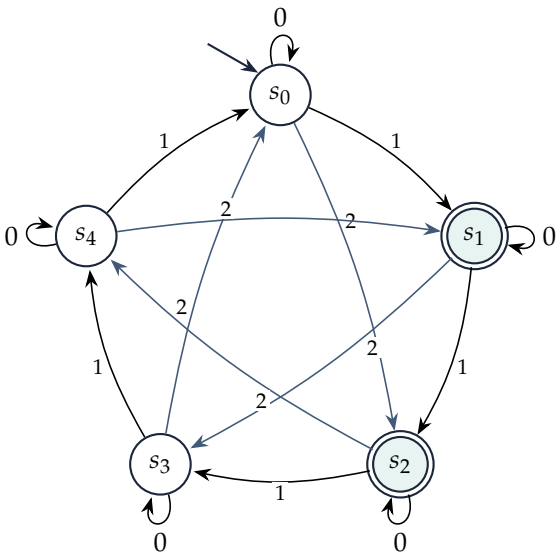


2019–2020 • Q1(c) Source M40 **MARKS 15**

Draw a DFA over $\{0, 1, 2, R\}$ keeping the sum of numerical input symbols, resetting it to 0 on R. Accept if the sum is 1 or 2 modulo 5. A state diagram suffices.

SOLUTION

Only the sum mod 5 matters, so use states $s_0, \ldots, s_4$; on digit d: $s_i \to s_{(i+d) \bmod 5}$; on R: $\to s_0$. Accept s_1, s_2.



In addition, **every** state has an R-edge to s_0 (omitted from the drawing for clarity; s_0 also loops on R). Transition table:

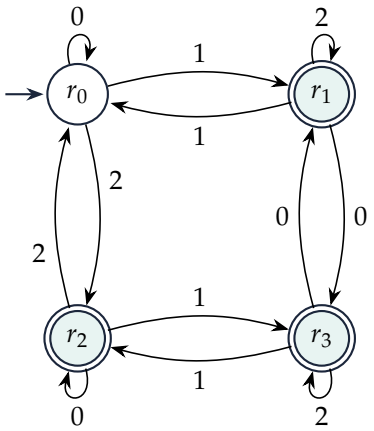
	0	1	2	R
$\to s_0$	s_0	s_1	s_2	s_0
$*s_1$	s_1	s_2	s_3	s_0
$*s_2$	s_2	s_3	s_4	s_0
s_3	s_3	s_4	s_0	s_0
s_4	s_4	s_0	s_1	s_0

2019–2020 • Q2(c) Source M40 **MARKS 15**

Draw a DFA accepting base-3 numbers (digits 0, 1, 2) that are not divisible by 4.

SOLUTION

Reading digit d after a prefix with value v gives value $3v + d$, so with $r = v \bmod 4$: $r \mapsto (3r + d) \bmod 4$. The start state is $r = 0$ (empty prefix, value 0 — non-accepting, so ε is rejected). Accept $r \in \{1, 2, 3\}$.



r	0	1	2
$\to r_0$	r_0	r_1	r_2
$*r_1$	r_3	r_0	r_1
$*r_2$	r_2	r_3	r_0
$*r_3$	r_1	r_2	r_3

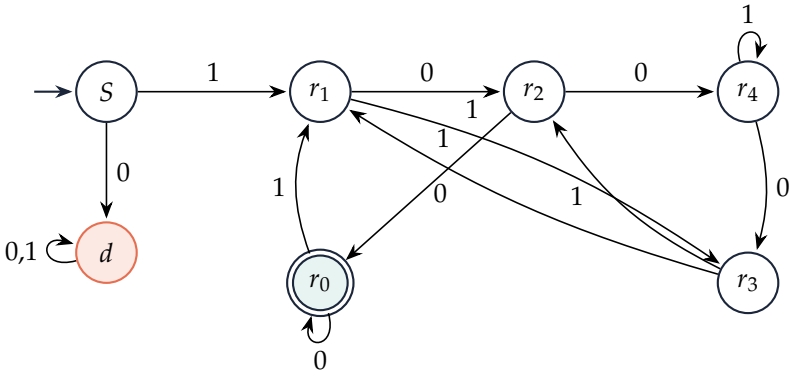
e.g. $12_3 = 5$: $r_0 \xrightarrow{1} r_1 \xrightarrow{2} r_1$ accept ($5 \bmod 4 = 1$).

2017–2018 • Q2(b) Source M32 **MARKS 10 + 7 = 17**

Draw DFA state diagrams over $\{0,1\}$ for (i) strings beginning with 1 whose binary value is divisible by 5 (accept 101, 1010, 1111; reject 0, 100, 0101, 111); (ii) strings beginning with 1 whose binary value is a power of 2.

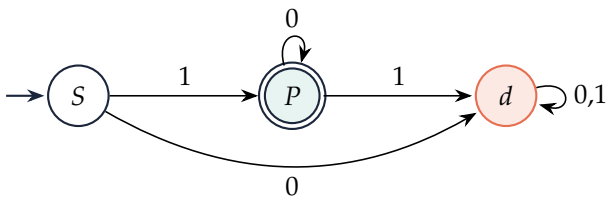
SOLUTION

(i) Appending bit b maps value v to $2v+b$, so the remainder $r = v \bmod 5$ updates as $r \mapsto (2r+b) \bmod 5$. A start state S forces the first bit to be 1.



	0	1
$\to S$	d	r_1
$*r_0$	r_0	r_1
r_1	r_2	r_3
r_2	r_4	r_0
r_3	r_1	r_2
r_4	r_3	r_4

Checks: 1111: $r_1 \to r_3 \to r_2 \to r_0$ (15) accept; 111: $r_1 \to r_3 \to r_2$ (7) reject; 0101: $S \xrightarrow{0} d$ reject.
(ii) Powers of 2 starting with 1 are exactly 10^*.



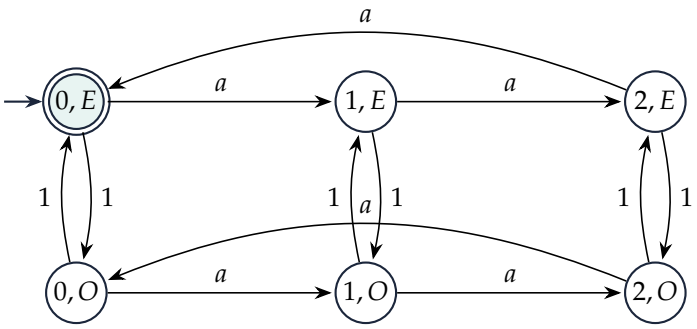
2016–2017 • Q2(a) Source M28 **MARKS 17**

Draw a DFA D_1 over $\{a,b,1,2\}$ accepting strings whose number of a's is divisible by 3 and whose sum of digits is even. Accepted examples: ε, $aab12a1$, $aab1bbaaba2a1$, $aaba$, 121. Rejected: $aaab12a1$, $aab11bbaaba2a1$, $aaaba$, 1121.

SOLUTION

Track $(\#_a \bmod 3,\ \text{digit-sum parity})$: six states (i,p). a increments i; digit 1 flips p; digit 2 and b

change nothing. Accept only $(0, E)$.



In addition every state has a self-loop on b and on 2 (not drawn). Check $aab12a1$: a's $= 3$, digits $1 + 2 + 1 = 4$ even $\Rightarrow (0, E)$ accept; 1121: sum 5 odd $\Rightarrow (0, O)$ reject.

2015–2016 • Q2(a) Source M25 **MARKS $8\frac{1}{3}$**

Design a DFA accepting binary numbers NOT divisible by 3. It rejects 1111 (15) and accepts 111 (7). Discuss the intuition and explain how it works; a diagram alone is insufficient.

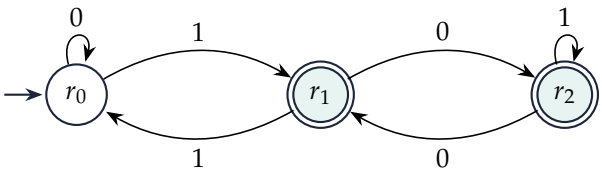
SOLUTION

Intuition. We cannot store the whole number, but we do not need to: reading the bits left to right, appending a bit b to a prefix of value v produces $2v + b$. Divisibility by 3 depends only on $v \bmod 3$, and

$$(2v + b) \bmod 3 = (2\,(v \bmod 3) + b) \bmod 3.$$

So three states r_0, r_1, r_2 (r_i: the prefix read so far has remainder i modulo 3) are enough, with $\delta(r_i, b) = r_{(2i+b) \bmod 3}$:

r	0	1
$\to r_0$	r_0	r_1
$*r_1$	r_2	r_0
$*r_2$	r_1	r_2



The number is *not* divisible by 3 exactly when the remainder is 1 or 2, so $F = \{r_1, r_2\}$. The start state r_0 (empty prefix, value 0) is non-accepting, so ε is rejected as well.

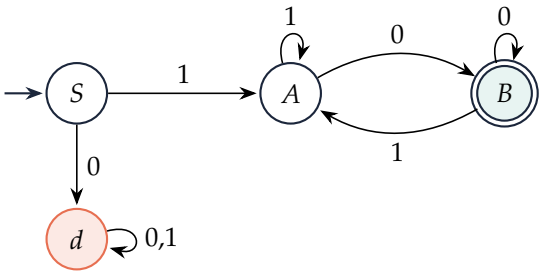
How it works on the examples: 1111: $r_0 \xrightarrow{1} r_1 \xrightarrow{1} r_0 \xrightarrow{1} r_1 \xrightarrow{1} r_0$ — remainder 0 ($15 = 3 \cdot 5$), $r_0 \notin F$, rejected. 111: $r_0 \xrightarrow{1} r_1 \xrightarrow{1} r_0 \xrightarrow{1} r_1$ — remainder 1 ($7 = 3 \cdot 2 + 1$), $r_1 \in F$, accepted.

2015–2016 • Q3(a) Source M25 • DFA part **MARKS part of 18**

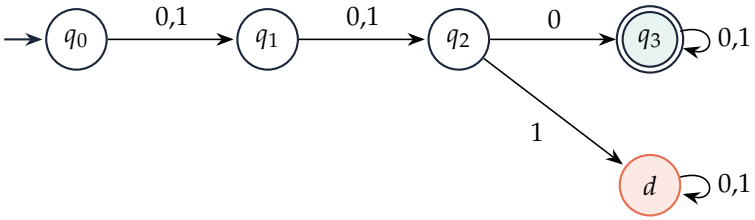
Compute a DFA for each binary language: (i) begins with 1 and ends with 0; (ii) length at least 3 and third symbol 0; (iii) an even number of 0's or exactly two 1's; (iv) every string except 11 and 111; (v) at least two 0's and at most one 1.

SOLUTION

(i) begins with 1 and ends with 0

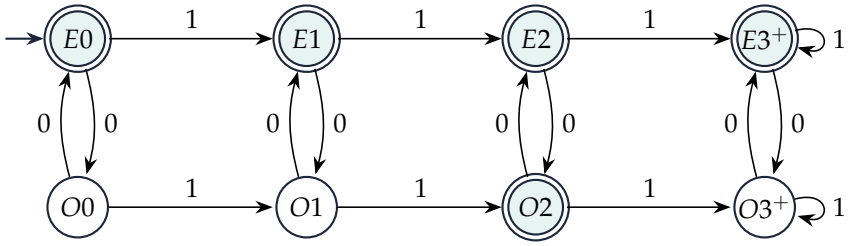


(ii) length at least 3, third symbol 0



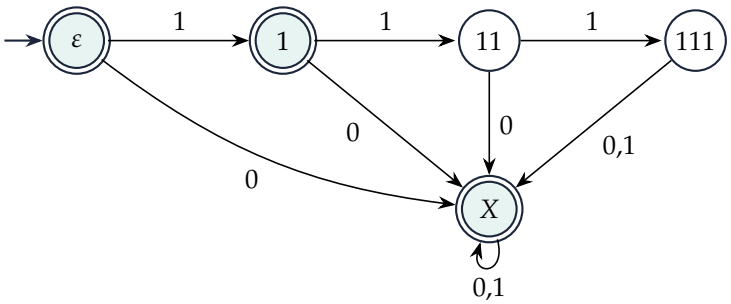
(iii) even number of 0's or exactly two 1's

Product of "parity of 0's" (E/O) and "number of 1's" ($0, 1, 2, 3^+$); accept when parity is E or count is 2. A 0 moves vertically, a 1 moves right.



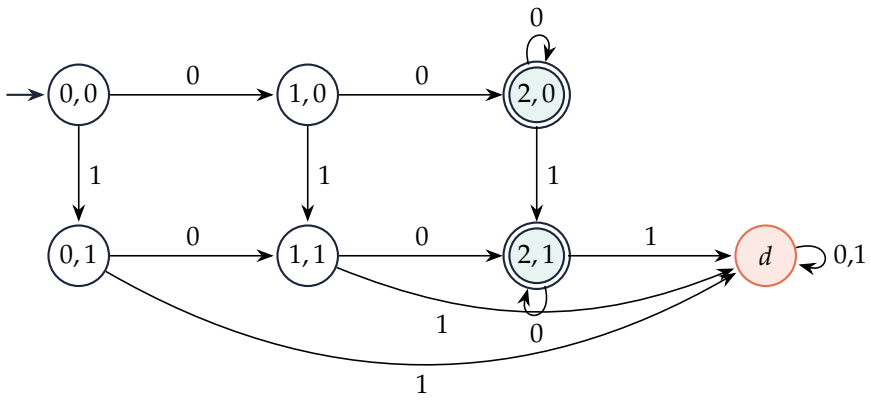
(iv) every string except 11 and 111

Track the prefix while it is still a candidate for 11 or 111; any 0 (or a 4th symbol) escapes to an accepting sink X.



(v) at least two 0's and at most one 1

States (z, o) with $z = \min(\#_0, 2)$, $o = \#_1 \in \{0, 1\}$; a second 1 kills.



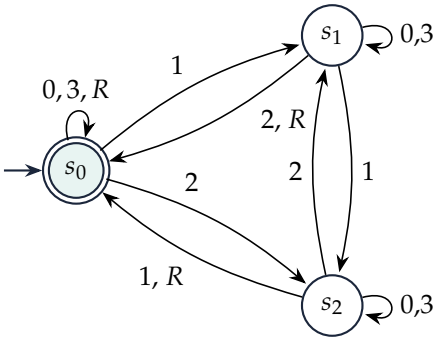
(All five DFAs were cross-checked by enumeration on all strings up to length 10. The regular expressions asked for in the same question are in the next block.)

2014–2015 • Q1(a) Source M22 MARKS 10

Design a DFA over $\{0, 1, 2, 3, R\}$. Accept exactly strings whose sum of numerical symbols is a multiple of 3, resetting the sum to 0 whenever R occurs. A diagrammatic description suffices.

SOLUTION

Keep the sum mod 3. Digits 0 and 3 do not change the remainder; 1 adds 1; 2 adds 2; R returns to s_0.

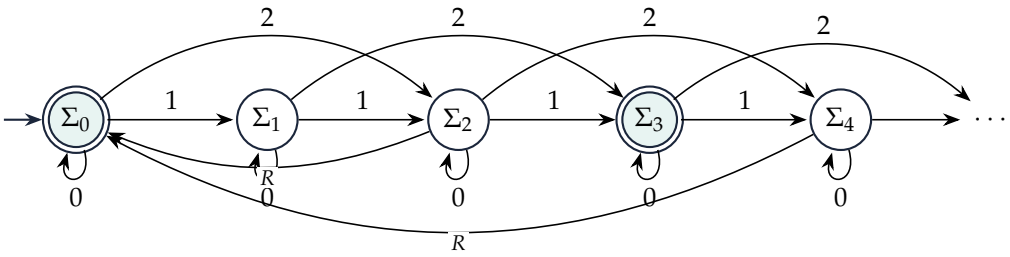


2014–2015 • Q1(b) Source M22 MARKS $13\frac{1}{3}$

Change the alphabet in 1(a) to $\{0, 1, 2, R\}$ and use states to remember actual sums. Show diagrammatically how to construct this DFA. What is its problem? Explain clearly.

SOLUTION

Use one state Σ_k for each possible actual sum k: digit d goes from Σ_k to Σ_{k+d}, R goes back to Σ_0, and Σ_k is accepting iff $3 \mid k$.



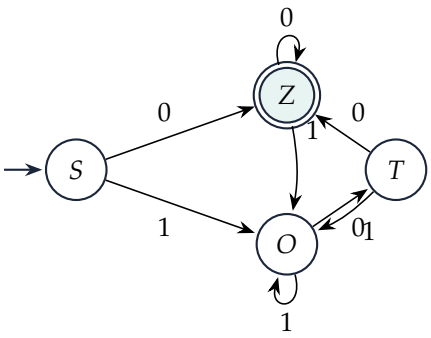
(Every Σ_k has an R-edge to Σ_0; only two are drawn.)
Problem: the sum can grow without bound ($111\cdots1$), so this construction needs *infinitely many* states. It is therefore not a finite automaton at all — it cannot be built or written as a finite table. The fix is to observe that acceptance depends only on k mod 3, so Σ_k and Σ_{k+3} behave identically on every future input; merging them gives the 3-state DFA of part (a).

2014–2015 • Q2(b) Source M22 **MARKS 13 $\frac{1}{3}$**

Design a DFA accepting exactly binary-number strings divisible by 4. Explain how it works.

SOLUTION

Every bit except the last two has place value 2^k with $k \geq 2$, a multiple of 4. Hence the value modulo 4 is just the value of the last two bits, and a binary number is divisible by $4 = 2^2$ iff it ends in 00 or is the single digit 0. So the DFA only has to remember the relevant suffix of what it has read. DFA (states = relevant suffix):



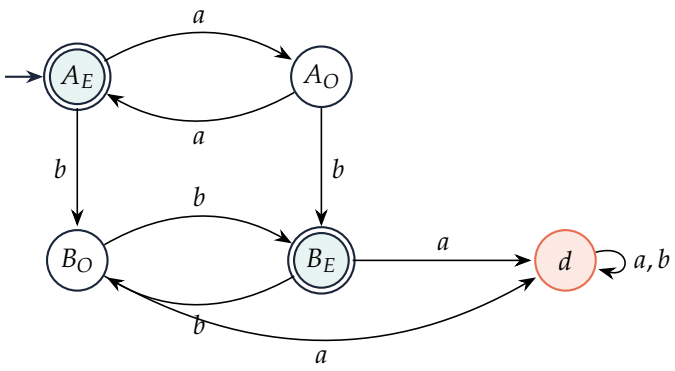
Z: value $\equiv 0 \pmod 4$ (ends in 00, or the string is 0/00); O: last bit 1 (value odd); T: ends in 10 (value $\equiv 2$). Reading 0 from O gives $\equiv 2$; reading 0 from T gives $\equiv 0$; reading 1 always gives an odd number. S is a non-accepting start state so that ε is not accepted. Example: 1100 (12): $S \to O \to O \to T \to Z$ accept; 110 (6): ends in T, reject.

2013–2014 • Q5(c) Source M20 **MARKS 10 $\frac{1}{3}$**

Construct a DFA recognizing $\{w \in \{a,b\}^* \mid w = a^p b^q \text{ and } p+q \text{ is even}\}$.

SOLUTION

L = even-length strings of the form a^*b^*. Track phase (a-part or b-part) and length parity; an a after a b is fatal.



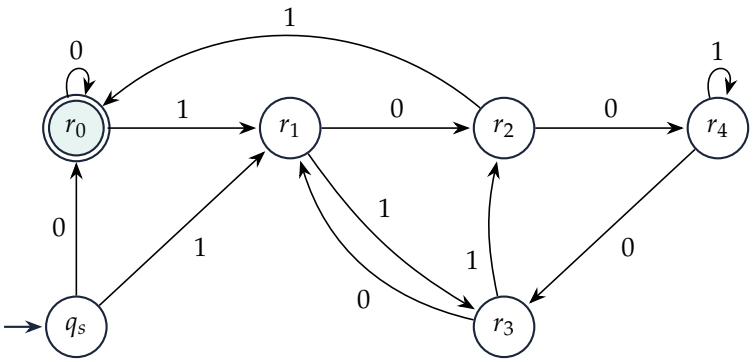
A_E / A_O: only a's so far, even/odd length; B_E / B_O: in the b-part, even/odd total length.

2012–2013 • Q7(c) Source M18 **MARKS 8**

Design a DFA for binary strings whose decimal values are multiples of five.

SOLUTION

Same idea as for divisibility by 3 (2015–2016 Q2(a), 2008–2009 Q6(a)): remainder $r = v \bmod 5$, update $r \mapsto (2r + b) \bmod 5$, accept r_0. The empty string has no decimal value, so a separate non-accepting start state q_s (no bit read yet) has the moves of r_0: $q_s \xrightarrow{0} r_0$, $q_s \xrightarrow{1} r_1$.



r	0	1	2	3	4
on 0	r_0	r_2	r_4	r_1	r_3
on 1	r_1	r_3	r_0	r_2	r_4

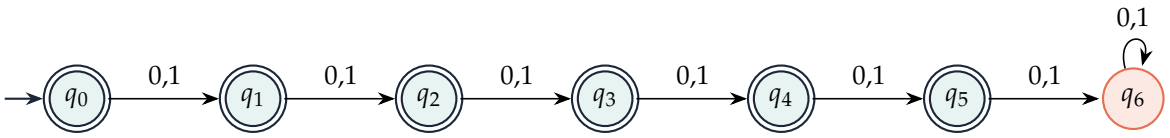
Example 1010 (10): $q_s \to r_1 \to r_2 \to r_0 \to r_0$ accept; ε stays in q_s, rejected.

2011–2012 • Q5(c) Source M15 **MARKS $5\frac{1}{3}$**

Give the state diagram of a DFA recognizing $\{w \mid |w| \leq 5\}$ over $\{0,1\}$.

SOLUTION

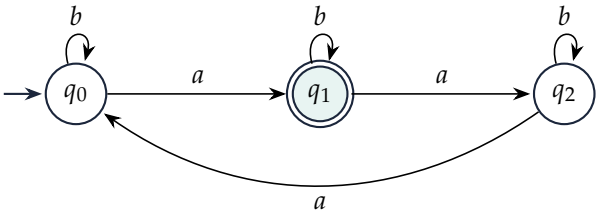
Count the length up to 5; the sixth symbol leads to a dead state.



2010–2011 • Q2 Source M11 **MARKS 10**

Design a DFA for strings over $\{a, b\}$ satisfying $n \bmod 3 = 1$, where n is the number of a's. Give a transition diagram and mathematical representation.

SOLUTION



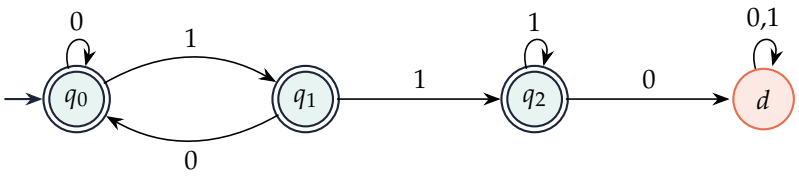
$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_1\})$ with $\delta(q_i, a) = q_{(i+1) \bmod 3}$ and $\delta(q_i, b) = q_i$. State q_i means $\#_a \equiv i \pmod 3$.

2010–2011 • Q4 Source M11 • DFA part **MARKS 7**

Design a DFA for binary strings not containing substring 110.

SOLUTION

States record the longest suffix that is a prefix of 110: q_0 (none), q_1 ("1"), q_2 ("11"), d (110 seen).



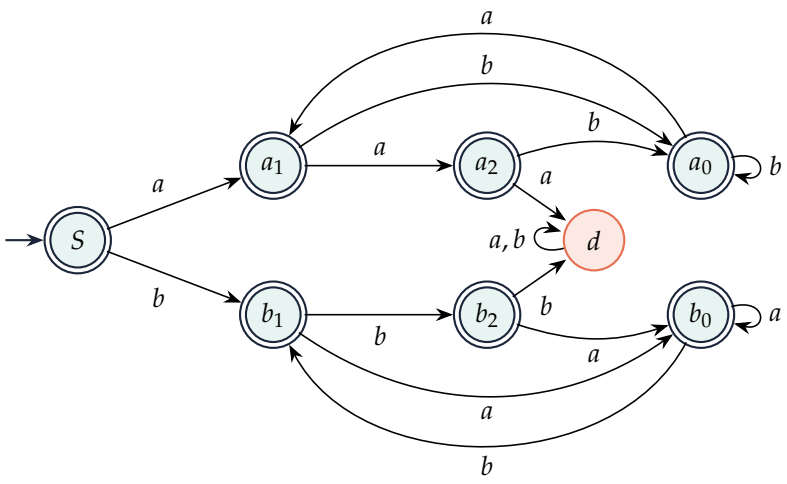
All three non-dead states accept. (The regular expression asked for in the same question is in the next block.)

2009–2010 • Q2 Source M08 **MARKS 10**

Design a DFA for strings over $\{a, b\}$ that never contain three consecutive occurrences of the character that started the string. Thus *abaab* and *baaabba* are accepted; *abaaab* and *bbbaa* are rejected. Give a transition diagram.

SOLUTION

The first symbol selects one of two symmetric sub-machines; each counts the current run of the starting character (1 or 2). ε is accepted vacuously.



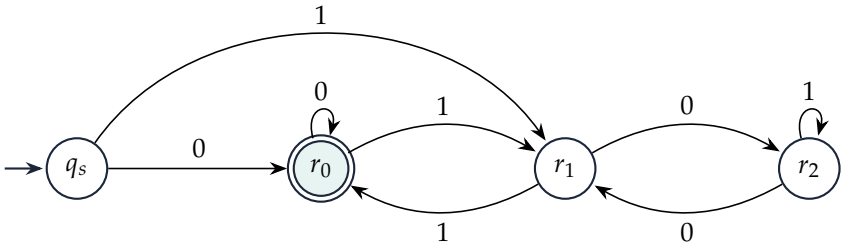
a_k ($k = 0, 1, 2$): the string started with a and currently ends in exactly k consecutive a's; symmetric for b_k. Checks: *abaaab*: $S \to a_1 \to a_0 \to a_1 \to a_2 \to d$ reject; *baaabba*: $S \to b_1 \to b_0 \to b_0 \to b_0 \to b_1 \to b_2 \to b_0$ accept.

2008–2009 • Q6(a) Source M07 **MARKS 9**

Design a DFA accepting binary strings whose decimal values are multiples of three.

SOLUTION

Same remainder states r_0, r_1, r_2 and moves $\delta(r_i, b) = r_{(2i+b) \bmod 3}$ as in 2015–2016 Q2(a) above, but now the accepting set is $F = \{r_0\}$ (remainder 0). The empty string has no decimal value, so — as in the slides — we add a separate non-accepting start state q_s (no bit read yet) whose moves are those of r_0; we accept when at least one bit has been read and the remainder is 0.



$M = (\{q_s, r_0, r_1, r_2\}, \{0, 1\}, \delta, q_s, \{r_0\})$. E.g. 1001 (9): $q_s \xrightarrow{1} r_1 \xrightarrow{0} r_2 \xrightarrow{0} r_1 \xrightarrow{1} r_0$, accepted; 100 (4): ends in r_1, rejected.

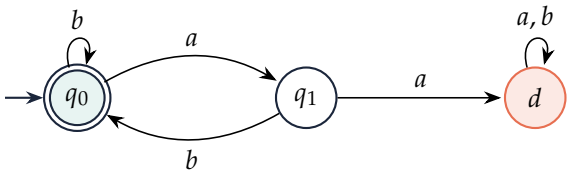
2007–2008 • Q1(b) Source M04 **MARKS 8**

Design a DFA for strings over $\{a, b\}$ in which every a is followed by at least one b. Mention the design steps and give both a transition diagram and a transition table.

SOLUTION

Steps. (1) Fix the reading: every a must be *immediately* followed by a b, i.e. $L = (b \cup ab)^*$. (2) Decide what to remember: whether an a is still "waiting" for its b. (3) States: q_0 = nothing pending (accepting), q_1 = an a is pending, d = violation (an a was followed by a). (4) Add transitions, mark

$F = \{q_0\}$.



	a	b
$\to *q_0$	q_1	q_0
q_1	d	q_0
d	d	d

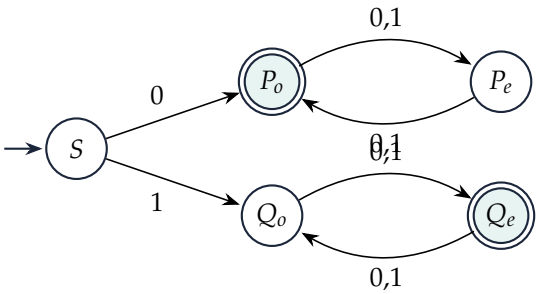
(If "followed" is read as "followed somewhere later", the language is just "ε or ends in b" and the same diagram works with the $q_1 \xrightarrow{a} d$ edge replaced by a self-loop on q_1.)

2006–2007 • Q1(a) Source M01 **MARKS 10**

Design a DFA accepting all strings which start with 0 and have odd length, or start with 1 and have even length. Depict the automaton using a transition diagram and a transition table.

SOLUTION

The first symbol chooses a branch; each branch tracks length parity.



	0	1
$\to S$	P_o	Q_o
$*P_o$	P_e	P_e
P_e	P_o	P_o
Q_o	Q_e	Q_e
$*Q_e$	Q_o	Q_o

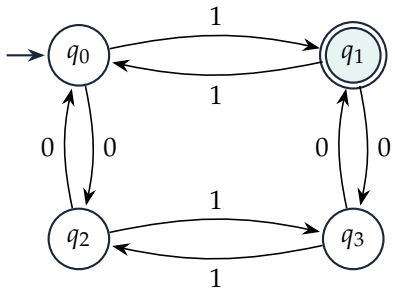
P = started with 0, Q = started with 1; subscript = parity of the length so far.

2006–2007 • Q2(b) Source M01 **MARKS 8**

Design a DFA for $L = \{w \mid w$ has an even number of 0's and an odd number of 1's$\}$. Verify it by finding $\hat{\delta}(q_0, 11010)$, where q_0 is the start state.

SOLUTION

Four states, one for each pair of parities: $q_0 = (E,E)$, $q_1 = (E,O)$, $q_2 = (O,E)$, $q_3 = (O,O)$ (parity of 0's, parity of 1's). 0 flips the first parity, 1 flips the second. Accept q_1.



$$\hat{\delta}(q_0, 11010) : \; q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0 \xrightarrow{0} q_2 \xrightarrow{1} q_3 \xrightarrow{0} q_1.$$

$\hat{\delta}(q_0, 11010) = q_1 \in F$, so it is accepted — correct, since 11010 has two 0's (even) and three 1's (odd).

03

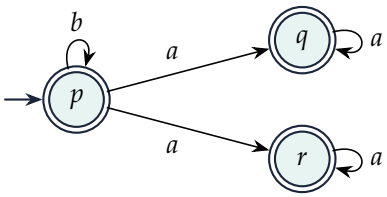
NFA design, ε-transitions and conversion

2023–2024 • Q3(a) Source N02 **MARKS 15**

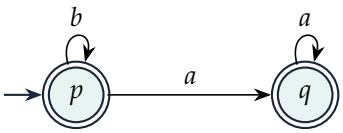
Find a three-state NFA for $L = \{a^n \mid n \geq 1\} \cup \{b^m a^k \mid m, k \geq 0\}$. Give only the diagram. Can an NFA with fewer than three states accept it? Elaborate.

SOLUTION

Three-state NFA (one branch per part of the union, sharing the start):



$p \xrightarrow{a} r \xrightarrow{a} \cdots$ gives a^n $(n \geq 1)$; $p \xrightarrow{b^m} p \xrightarrow{a} q \xrightarrow{a^{k-1}} q$ gives $b^m a^k$.
Fewer states? Yes — *two* suffice. Observe $\{a^n \mid n \geq 1\} \subseteq \{b^m a^k\}$ (take $m = 0$), so $L = b^* a^*$, and state r above is redundant:

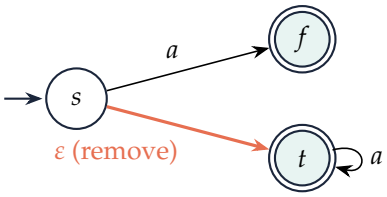


One state is impossible. With one state s: since $\varepsilon \in L$, s must be accepting, so $L(N) = S^*$ where S is the set of symbols on its self-loops. $S = \{a, b\}$ accepts $ab \notin L$; $S = \{a\}$ misses b; $S = \{b\}$ misses a; $S = \emptyset$ gives $\{\varepsilon\}$. Hence the minimum is **2 states**.

2023–2024 • Q4(a) Source N02 **MARKS 10**

Find an NFA accepting $\{a\}^*$ such that removing just one edge, with no other changes, makes it accept $\{a\}$. Give only the state diagram.

SOLUTION



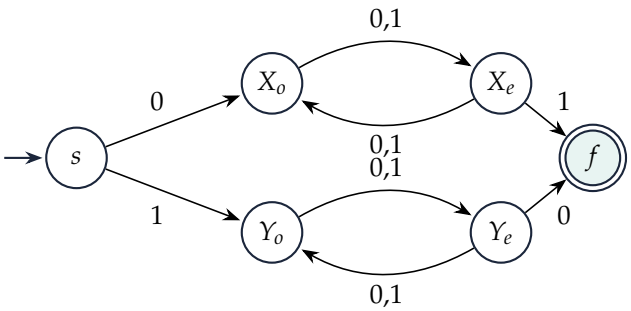
With the coral ε-edge, $L = \{a\} \cup a^* = a^*$. Removing it leaves only $s \xrightarrow{a} f$, so $L = \{a\}$. (The removable edge must be an ε-edge: $\varepsilon \in a^*$ but $\varepsilon \notin \{a\}$, and an accepting path for ε uses only ε-edges.)

2022–2023 • Q4(b) Source M51 **MARKS 15**

Draw an NFA for binary strings beginning and ending with different symbols and having odd length. A state diagram suffices.

SOLUTION

Branch on the first symbol, track length parity, and guess that the next symbol is the last one — allowed only when it differs from the first symbol and makes the length odd (i.e. it is read from an even-length state).



X: began with 0, Y: began with 1; subscript = parity of the length read so far. Example: 001: $s \to X_o \to X_e \to f$ ✓; 01 (even length) cannot reach f.

2021–2022 • Q2(b) Source M45 **MARKS 10**

Using basic principles, convert $(((00)^*(11)) \cup 01)^*$ into an NFA.

SOLUTION

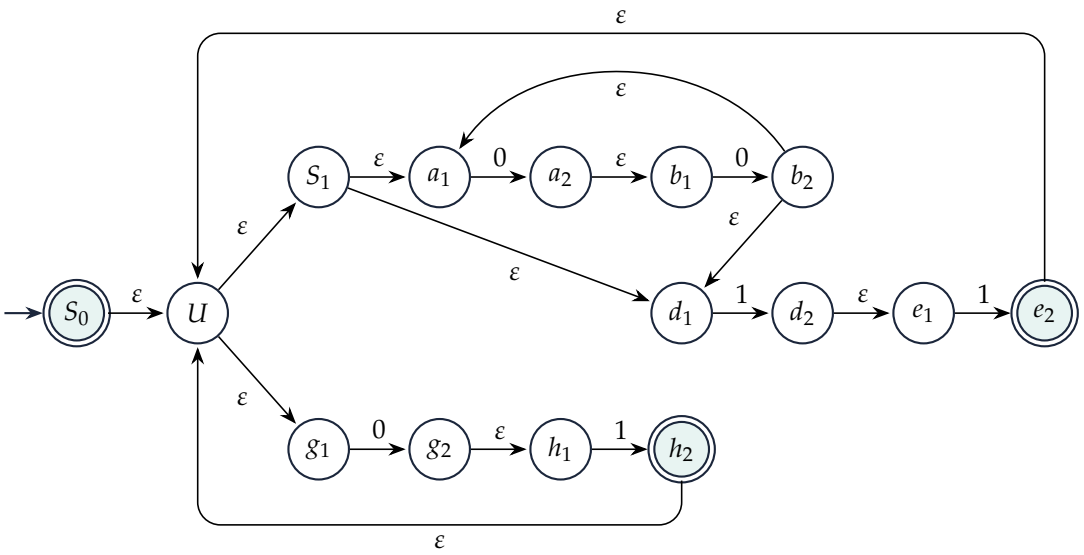
Build bottom-up with the slide (Sipser) constructions: a symbol x is the two-state NFA $q \xrightarrow{x} q'$ (q' accepting), and every concatenation joins the pieces by an ε-edge.

(i) 00: $a_1 \xrightarrow{0} a_2 \xrightarrow{\varepsilon} b_1 \xrightarrow{0} b_2$ (accepting b_2). $(00)^*$: new accepting start S_1, $S_1 \xrightarrow{\varepsilon} a_1$, $b_2 \xrightarrow{\varepsilon} a_1$; accepting S_1, b_2.

(ii) 11: $d_1 \xrightarrow{1} d_2 \xrightarrow{\varepsilon} e_1 \xrightarrow{1} e_2$. $(00)^*11$: ε-edges from the accepting states S_1, b_2 of $(00)^*$ to d_1; only e_2 accepts.

(iii) 01: $g_1 \xrightarrow{0} g_2 \xrightarrow{\varepsilon} h_1 \xrightarrow{1} h_2$. Union: new start U with $U \xrightarrow{\varepsilon} S_1$, $U \xrightarrow{\varepsilon} g_1$; accepting e_2, h_2.

(iv) Star: new accepting start S_0, $S_0 \xrightarrow{\varepsilon} U$, and $e_2 \xrightarrow{\varepsilon} U$, $h_2 \xrightarrow{\varepsilon} U$. Accepting: S_0, e_2, h_2.

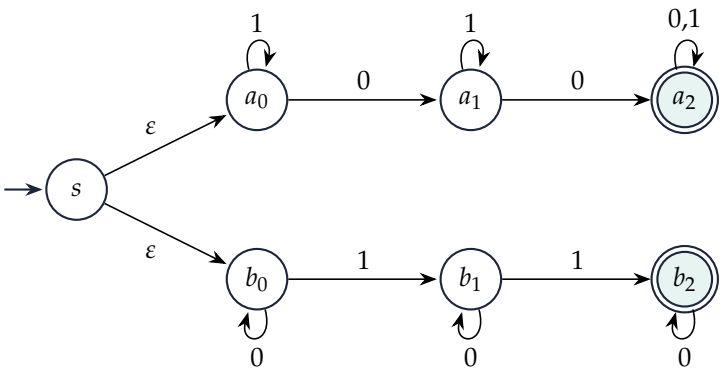


2021–2022 • Q3(b) Source M45 **MARKS 10**

Draw an NFA accepting binary strings containing at least two 0's or exactly two 1's. A state diagram suffices.

SOLUTION

Union of two simple machines via a new start state with ε-edges.



Upper branch counts 0's up to 2; lower branch counts 1's and has no move on a third 1 (that branch dies).

2021–2022 • Q3(c) Source M45–M46 **MARKS 9**

Convert the given NFA into a DFA. Only the DFA state diagram is required.

State	a	b	ε
q_1	$\{q_3\}$	$\emptyset$	$\{q_2\}$
q_2	$\{q_1\}$	$\emptyset$	$\emptyset$
q_3	$\{q_2\}$	$\{q_2, q_3\}$	$\emptyset$

SOLUTION

> **NOTE** The table marks neither start nor accepting states. This is the NFA of Sipser Exercise 1.16(b), whose start state is q_1 and accepting state is q_2; we use that.

Step 1: remove the ε-transitions

$\delta'(q,x) = E\big(\bigcup_{r\in E(q)} \delta(r,x)\big)$ and $F' = \{q \mid E(q) \cap F \neq \emptyset\}$ with $F = \{q_2\}$:

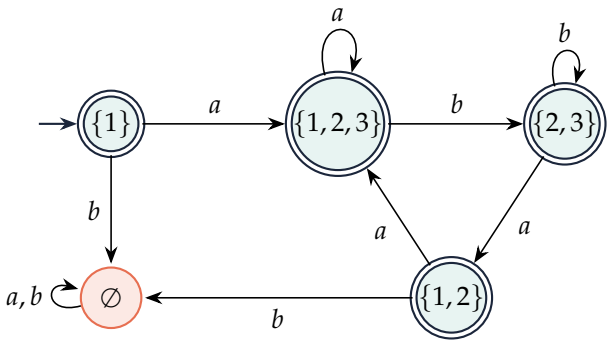
q	$E(q)$	$\delta'(q,a)$	$\delta'(q,b)$
$*q_1$	$\{q_1,q_2\}$	$E(\{q_3,q_1\}) = \{q_1,q_2,q_3\}$	$\emptyset$
$*q_2$	$\{q_2\}$	$E(\{q_1\}) = \{q_1,q_2\}$	$\emptyset$
q_3	$\{q_3\}$	$E(\{q_2\}) = \{q_2\}$	$E(\{q_2,q_3\}) = \{q_2,q_3\}$

$F' = \{q_1,q_2\}$ (q_1 reaches q_2 by an ε-move).

Step 2: subset construction from $\{q_1\}$

$\delta_D(R,x) = \bigcup_{r\in R} \delta'(r,x)$; a subset is accepting iff it meets F'. Only reachable subsets are built:

DFA state	a	b
$\to *\{q_1\}$	$\{q_1,q_2,q_3\}$	$\emptyset$
$*\{q_1,q_2,q_3\}$	$\{q_1,q_2,q_3\}$	$\emptyset\cup\emptyset\cup\{q_2,q_3\} = \{q_2,q_3\}$
$*\{q_2,q_3\}$	$\{q_1,q_2\}\cup\{q_2\} = \{q_1,q_2\}$	$\{q_2,q_3\}$
$*\{q_1,q_2\}$	$\{q_1,q_2,q_3\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$



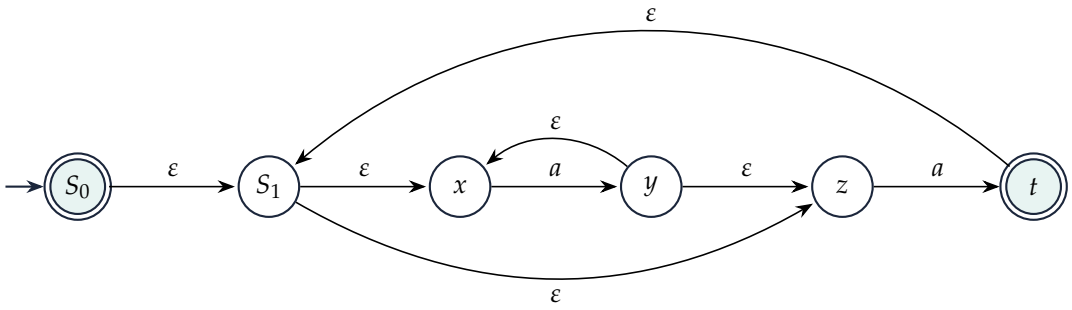
Five reachable states (numbers abbreviate q_i); all except $\emptyset$ are accepting.

2019–2020 • Q3(a) Source M40 **MARKS 5**

Construct an NFA for $((a^*)a)^*$ using basic principles.

SOLUTION

a: $x \xrightarrow{a} y$. a^*: new accepting start S_1, $S_1 \xrightarrow{\varepsilon} x$, $y \xrightarrow{\varepsilon} x$. $(a^*)a$: ε from S_1, y to $z \xrightarrow{a} t$. $((a^*)a)^*$: new accepting start $S_0 \xrightarrow{\varepsilon} S_1$, $t \xrightarrow{\varepsilon} S_1$.



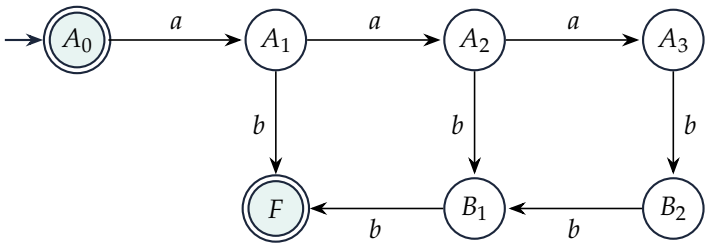
(The language is simply a^*, but the question asks for the systematic construction.)

2019–2020 • Q3(b) Source M40 **MARKS 15**

Design an NFA for $L = \{a^n b^n \mid n \leq 3\}$ over $\{a, b\}$. Give only a state diagram. Using more than the required minimum states may be penalized.

SOLUTION

$L = \{\varepsilon, ab, aabb, aaabbb\}$. After k a's jump to the point of the b-chain that still needs k b's:



Fewer than 7 states is impossible. Let N be any NFA with $L(N) = L$. Since $a^3b^3 = s_1 s_2 \cdots s_6 \in L$, N has an accepting computation on it; let r_i be the state it is in after reading the first i symbols ($i = 0, 1, \ldots, 6$). Suppose $r_i = r_j$ for some $i < j$. Then the part of the computation between these two moments reads the $j - i \geq 1$ symbols $s_{i+1} \cdots s_j$ and returns to the same state, so it can be traversed twice: N also accepts $s_1 \cdots s_i \, (s_{i+1} \cdots s_j)^2 \, s_{j+1} \cdots s_6$, a string of length $6 + (j - i) > 6$. But every string of L has length at most 6 — contradiction. Hence $r_0, \ldots, r_6$ are 7 different states, and every NFA for L has at least 7 states; the NFA above is optimal.

2019–2020 • Q4(a) Source M41 **MARKS 5**

When a five-state NFA is converted to a DFA, what are the minimum and maximum possible numbers of DFA states? Justify.

SOLUTION

Each DFA state of the subset construction is a subset of the 5 NFA states, so there are at most $2^5 = \mathbf{32}$ states (counting $\emptyset$). The bound is attainable. Example: states $0, \ldots, 4$, start 0, alphabet $\{a, b, c\}$ with a: $i \mapsto \{i + 1 \bmod 5\}$; b: $0 \mapsto \{0, 1\}$, $i \mapsto \{i\}$ for $i \neq 0$; c: $0 \mapsto \emptyset$, $i \mapsto \{i\}$ for $i \neq 0$. In the subset construction a rotates a subset, b adds 1 to a subset containing 0, and c deletes 0. From $\{0\}$ the word $(ba^4)^3 b$ reaches $\{0, 1, 2, 3, 4\}$, and any element j can then be deleted by $a^{5-j} c a^j$ (rotate j to 0, delete, rotate back). So all 32 subsets are reachable DFA states. At least **1** state is always

needed, and 1 can happen: e.g. an NFA whose start state is accepting with self-loops on every symbol (accepts Σ^*) yields the single DFA state $E(\{q_0\})$, which maps to itself.

Answer. Minimum 1, maximum $2^5 = 32$.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

January 2020 Term • Q1(c) Source M36 **MARKS 12**

For the given NFA, give general forms of all accepted strings of length at most 3, with explanations.

State	a	b	c	ε
$\to p$	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
q	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$
$*r$	$\{r\}$	$\emptyset$	$\{p\}$	$\{q\}$

2017–2018 • Q4(a) Source M33 **MARKS 15**

For the given NFA (same table as above): (i) give, with explanations, general forms of every accepted string of length at most 3; (ii) convert it to a DFA (detailed computations optional).

2008–2009 • Q6(c) Source M07 **MARKS $5\frac{1}{3}$**

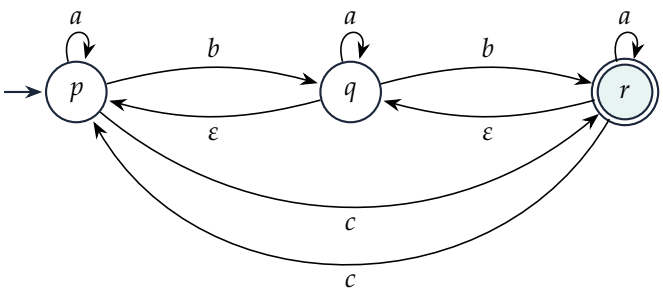
For the ε-NFA in the table: (i) compute the ε-closure of each state; (ii) convert it to a DFA.

State	ε	a	b	c
$\to p$	$\emptyset$	$\{p\}$	$\{q\}$	$\{r\}$
q	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
$*r$	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$

SOLUTION

SCOPE *January 2020 Term Q1(c) asks only for the accepted strings of length ≤ 3 (last part); 2008–2009 Q6(c) asks only for the ε-closures and the DFA (Steps 1–2); 2017–2018 Q4(a) asks for the strings of length ≤ 3 and the DFA.*

All three tables describe the *same* ε-NFA (only the column order differs).



Step 1: ε-closures and removal of the ε-transitions

$E(p) = \{p\}$, $E(q) = \{q, p\}$, $E(r) = \{r, q, p\}$. With $\delta'(s, x) = E\big(\bigcup_{t \in E(s)} \delta(t, x)\big)$ and $F' = \{s \mid E(s) \cap F \neq \emptyset\}$:

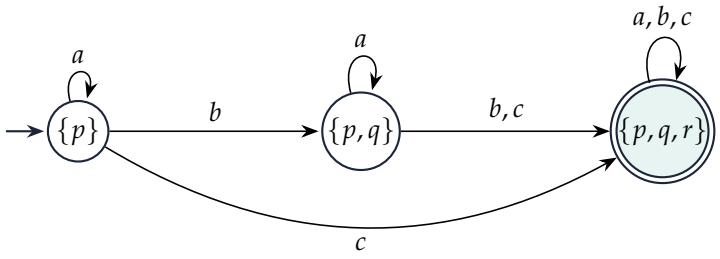
	a	b	c
$\to p$	$E(\{p\}) = \{p\}$	$E(\{q\}) = \{p,q\}$	$E(\{r\}) = \{p,q,r\}$
q	$E(\{p,q\}) = \{p,q\}$	$E(\{q,r\}) = \{p,q,r\}$	$E(\{r\}) = \{p,q,r\}$
$*r$	$E(\{p,q,r\}) = \{p,q,r\}$	$E(\{q,r\}) = \{p,q,r\}$	$E(\{r,p\}) = \{p,q,r\}$

$F' = \{r\}$ (neither $E(p)$ nor $E(q)$ contains r).

Step 2: subset construction from $\{p\}$

$\delta_D(R,x) = \bigcup_{s\in R} \delta'(s,x)$, reachable subsets only:

	a	b	c
$\to A = \{p\}$	$\{p\} = A$	$\{p,q\} = B$	$\{p,q,r\} = C$
$B = \{p,q\}$	$\{p,q\} = B$	$\{p,q\} \cup \{p,q,r\} = C$	C
$*C = \{p,q,r\}$	C	C	C



So $L = \{w \in \{a,b,c\}^* \mid w$ contains a c or at least two b's$\}$.

Accepted strings of length ≤ 3 (general forms)

Reading a never changes the DFA state; one b moves $\{p\} \to \{p,q\}$; a second b or any c reaches the accepting sink.

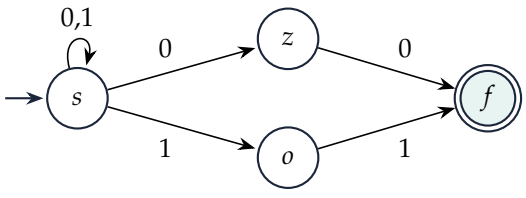
- Length 0, 1: ε is rejected; of length 1 only c.
- Length 2: xc and cx for any $x \in \{a,b,c\}$, and bb — i.e. $\{ac, bc, cc, ca, cb, bb\}$.
- Length 3: *every* string containing at least one c (19 strings: Σ^3 minus the 8 strings over $\{a,b\}$), plus abb, bab, bba, bbb (the strings over $\{a,b\}$ with ≥ 2 b's) — 23 strings in total.

January 2020 Term • Q2(b) Source M36 MARKS 13

Draw an equivalent NFA for the DFA in 2(a) (the five-state DFA accepting strings that end in 00 or 11; see Topic 01).

SOLUTION

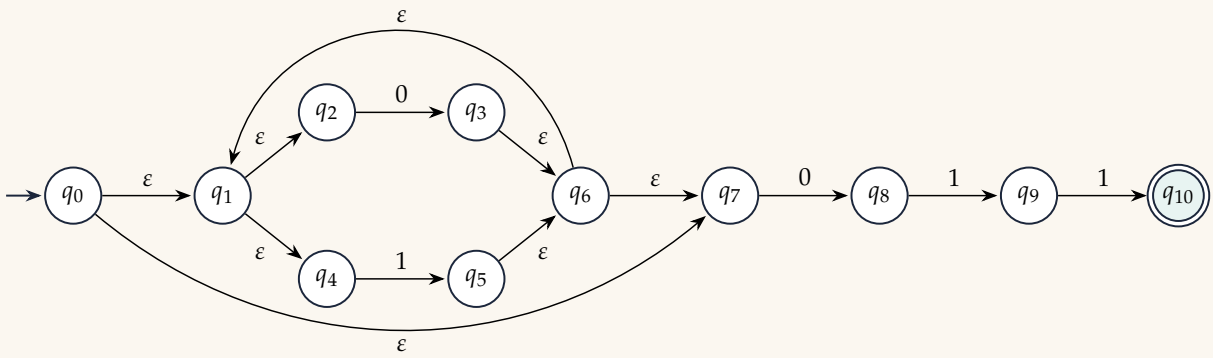
The DFA accepts $(0 \cup 1)^*(00 \cup 11)$. An NFA simply guesses where the final pair starts:



4 states instead of 5, with one nondeterministic choice at s for each symbol.

January 2020 Term • Q4(b) Source M37 MARKS 20

Convert the shown NFA to an equivalent DFA, showing computations.



SOLUTION

(This NFA recognizes $(0 \cup 1)^*011$. Numbers abbreviate q_i.)

Step 1: remove the ε-transitions

$\delta'(q,x) = E\big(\bigcup_{r\in E(q)} \delta(r,x)\big)$, $F' = \{q \mid E(q) \ni q_{10}\} = \{q_{10}\}$ (no other state reaches q_{10} by ε-moves alone):

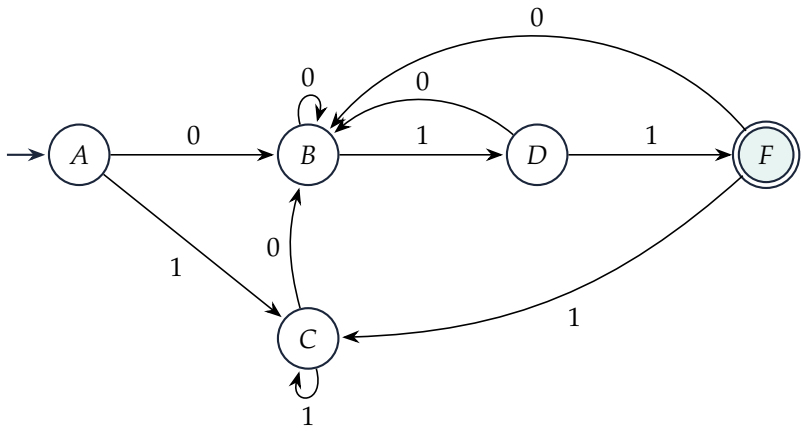
q	$E(q)$	$\delta'(q,0)$	$\delta'(q,1)$
$\to 0$	$\{0,1,2,4,7\}$	$E(\{3,8\}) = \{1,2,3,4,6,7,8\}$	$E(\{5\}) = \{1,2,4,5,6,7\}$
1	$\{1,2,4\}$	$E(\{3\}) = \{1,2,3,4,6,7\}$	$\{1,2,4,5,6,7\}$
2	$\{2\}$	$\{1,2,3,4,6,7\}$	$\emptyset$
3	$\{1,2,3,4,6,7\}$	$\{1,2,3,4,6,7,8\}$	$\{1,2,4,5,6,7\}$
4	$\{4\}$	$\emptyset$	$\{1,2,4,5,6,7\}$
5	$\{1,2,4,5,6,7\}$	$\{1,2,3,4,6,7,8\}$	$\{1,2,4,5,6,7\}$
6	$\{1,2,4,6,7\}$	$\{1,2,3,4,6,7,8\}$	$\{1,2,4,5,6,7\}$
7	$\{7\}$	$\{8\}$	$\emptyset$
8	$\{8\}$	$\emptyset$	$\{9\}$
9	$\{9\}$	$\emptyset$	$\{10\}$
$*10$	$\{10\}$	$\emptyset$	$\emptyset$

Step 2: subset construction from $\{q_0\}$

$\delta_D(R,x) = \bigcup_{r\in R} \delta'(r,x)$, reachable subsets only:

DFA state	on 0	on 1
$\to A = \{0\}$	$\{1,2,3,4,6,7,8\} = B$	$\{1,2,4,5,6,7\} = C$
$B = \{1,2,3,4,6,7,8\}$	B	$\{1,2,4,5,6,7\} \cup \{9\} = \{1,2,4,5,6,7,9\} = D$
$C = \{1,2,4,5,6,7\}$	B	C
$D = \{1,2,4,5,6,7,9\}$	B	$\{1,2,4,5,6,7\} \cup \{10\} = \{1,2,4,5,6,7,10\} = F$
$*F = \{1,2,4,5,6,7,10\}$	B	C

Only F contains $q_{10} \in F'$, so F is the sole accepting state.

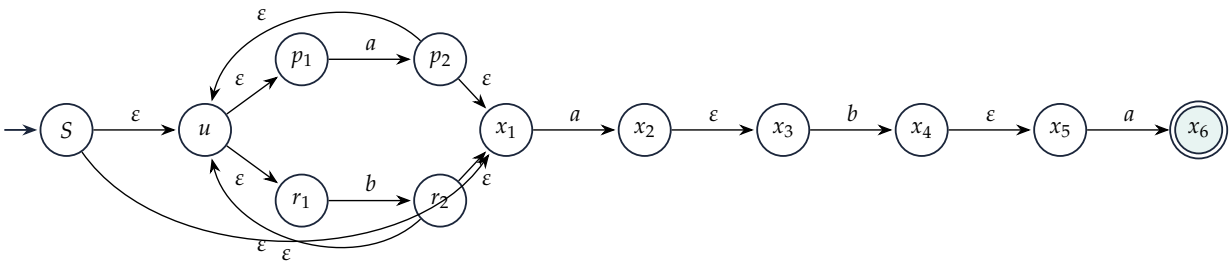


2017–2018 • Q2(c) Source M32 **MARKS 8**

Using regular-language closure constructions, draw an NFA for $(a \cup b)^* aba$.

SOLUTION

$a \cup b$: start u with $u \xrightarrow{\varepsilon} p_1 \xrightarrow{a} p_2$ and $u \xrightarrow{\varepsilon} r_1 \xrightarrow{b} r_2$. Star: new accepting start S, $S \xrightarrow{\varepsilon} u$, $p_2, r_2 \xrightarrow{\varepsilon} u$. Concatenate with a, b, a (each a two-state NFA) using ε-edges from the previous accepting states.



(After removing ε-moves this collapses to the familiar 4-state NFA $s \xrightarrow{a,b} s$, $s \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{a} 3$.)

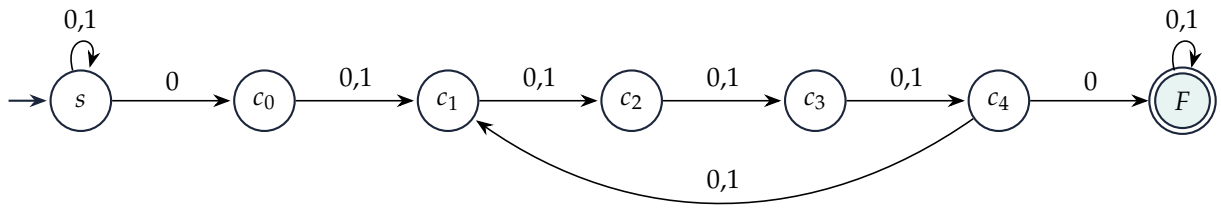
2017–2018 • Q3(a) Source M32 **MARKS 18**

Design an NFA for binary strings containing two 0's with a nonzero multiple of 4 characters between them; other 0's may occur. Give a state diagram and transition table. Source accepted examples: 00000, 101111010, 01000010111011. Source rejected examples: 00, 10101010.

SOLUTION

> **NOTE** **With the stated condition, 00000 should be *rejected*: any two of its 0's have $0, 1, 2$ or 3 characters between them, never a nonzero multiple of 4. The NFA below follows the stated condition; the other four examples are handled correctly.**

Guess the first of the two 0's, then count the characters after it modulo 4 (at least 4 of them), and guess that a 0 read when the count is $\equiv 0$ (and ≥ 4) is the second one.

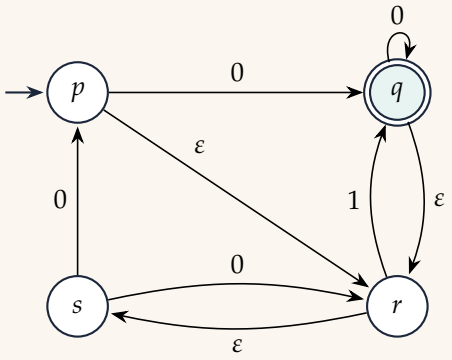


State	0	1
$\to s$	$\{s, c_0\}$	$\{s\}$
c_0	$\{c_1\}$	$\{c_1\}$
c_1	$\{c_2\}$	$\{c_2\}$
c_2	$\{c_3\}$	$\{c_3\}$
c_3	$\{c_4\}$	$\{c_4\}$
c_4	$\{F, c_1\}$	$\{c_1\}$
$*F$	$\{F\}$	$\{F\}$

c_k: k characters read since the guessed first 0 (c_4 stands for any count $\equiv 0 \pmod 4$, ≥ 4). Example 101111010: 0 at position 2 $\to c_0$; 1111 $\to c_4$; next 0 $\to F$ ✓. 10101010: gaps are 1, 3, 5 characters, never $\equiv 0$, so F is unreachable.

2016–2017 • Q2(b) Source M29 **MARKS 8**

Convert the given NFA to an equivalent DFA. Detailed computations are optional.



SOLUTION

Step 1: remove the ε-transitions

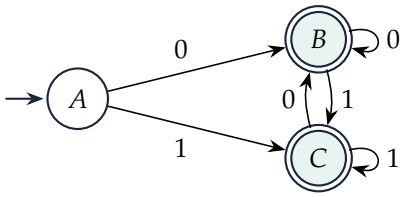
$\delta'(x,c) = E\big(\bigcup_{y\in E(x)} \delta(y,c)\big)$, $F' = \{x \mid E(x) \ni q\} = \{q\}$:

	$E(\cdot)$	$\delta'(\cdot,0)$	$\delta'(\cdot,1)$
$\to p$	$\{p,r,s\}$	$E(\{q,p,r\}) = \{p,q,r,s\}$	$E(\{q\}) = \{q,r,s\}$
$*q$	$\{q,r,s\}$	$E(\{q,p,r\}) = \{p,q,r,s\}$	$E(\{q\}) = \{q,r,s\}$
r	$\{r,s\}$	$E(\{p,r\}) = \{p,r,s\}$	$E(\{q\}) = \{q,r,s\}$
s	$\{s\}$	$E(\{p,r\}) = \{p,r,s\}$	$\emptyset$

Step 2: subset construction from $\{p\}$

$\delta_D(R,x) = \bigcup_{y\in R} \delta'(y,x)$, reachable subsets only:

	0	1
$\to A = \{p\}$	$\{p,q,r,s\} = B$	$\{q,r,s\} = C$
$*B = \{p,q,r,s\}$	$\{p,q,r,s\} \cup \{p,r,s\} = B$	$\{q,r,s\} \cup \emptyset = C$
$*C = \{q,r,s\}$	$\{p,q,r,s\} \cup \{p,r,s\} = B$	$\{q,r,s\} = C$



A is non-accepting, so ε is rejected; every non-empty string leads to B or C, both accepting (no edge returns to A), so the language is $(0 \cup 1)^+$: every non-empty string.

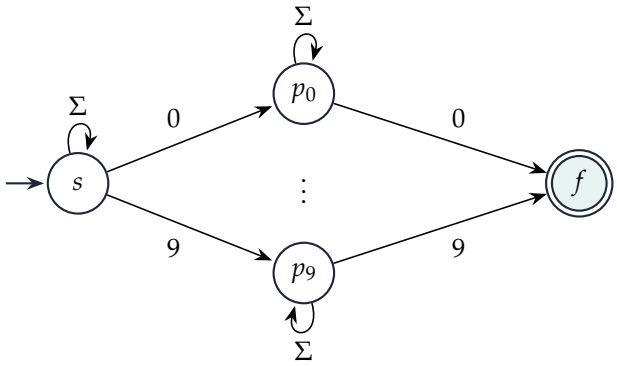
2016–2017 • Q3(a) Source M29 **MARKS 5 + 8 + 7 = 20**

Draw NFAs for: (i) strings over $\{0,1,\ldots,9\}$ whose final digit appeared earlier; (ii) strings over $\{0,1,\ldots,9\}$ whose final digit did not appear earlier; (iii) binary strings with at least two 0's separated by a number of positions divisible by 4. Other 0's may occur; zero is allowed as a multiple of 4. Accepted examples: 00, 10111010, 01010111011.

SOLUTION

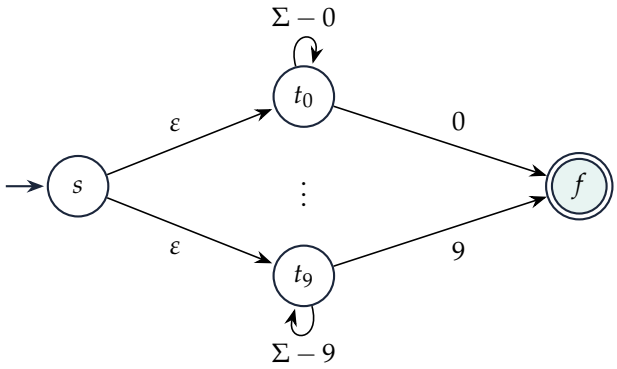
Let $\Sigma = \{0,\ldots,9\}$; $\Sigma - d$ means every digit except d. Only the branches for digits 0 and 9 are drawn; the other eight are identical.

(i) Final digit appeared earlier



Guess an earlier occurrence of digit d (go to p_d), skip anything, then read d as the final symbol (f has no outgoing edges). 12 states.

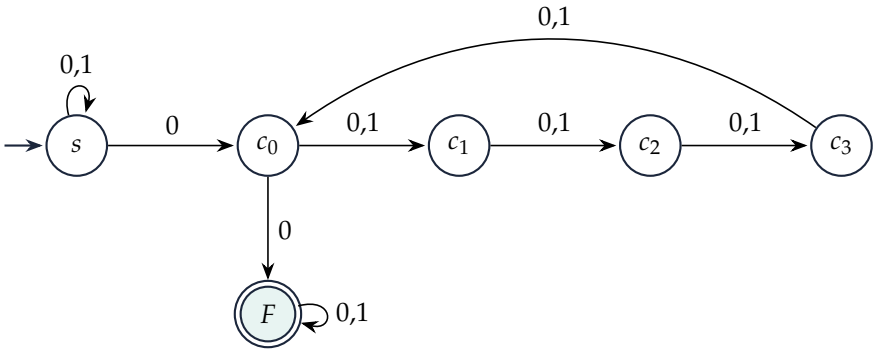
(ii) Final digit did not appear earlier



Guess at the very beginning which digit d will be last; read only digits $\neq d$, then d once. Note the different design: in (ii) the guess is made *first*, because "did not appear" must be checked on the whole prefix.

(iii) Two 0's separated by $\equiv 0 \pmod 4$ symbols

> **NOTE** The text says the number of symbols ***between*** the 0's is a multiple of 4 (so 00, with 0 symbols between, is accepted). The other two examples instead have 0's whose ***positions*** differ by a multiple of 4 (3, 7, ... symbols between). We follow the stated condition and show the one-edge change for the other reading.



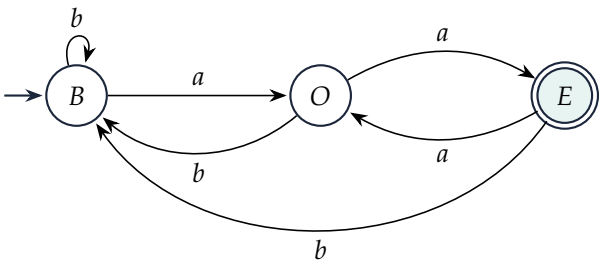
c_k: number of symbols read since the first guessed 0 is $\equiv k \pmod 4$. A 0 read in c_0 has $0, 4, 8, \ldots$ symbols before it since the first $0 \Rightarrow F$. (For the "position difference" reading, replace the edge $c_0 \xrightarrow{0} F$ by $c_3 \xrightarrow{0} F$.)

2014–2015 • Q3(a) Source M22 **MARKS 13 $\frac{1}{3}$**

Design an NFA over $\{a, b\}$ accepting nonempty strings with a nonzero even number of a's at the end. Accept aa, $aaaa$, $abbaa$, $abaababaaaa$; reject aaa, ab, b, $abbaaa$, $abaabbbb$.

SOLUTION

The trailing block of a's must have even length ≥ 2. Track the parity of the current trailing block; a b resets it. (A DFA is a special NFA.)



B: no trailing a; O: odd trailing block; E: even (≥ 2). Checks: *abbaaa* ends in O (3 a's) reject; *abaababaaaa* ends in E (4 a's) accept. A genuinely nondeterministic alternative: $s \xrightarrow{a,b} s$, $s \xrightarrow{b} t$, a new start state with ε-edges to s and t, $t \xrightarrow{a} t_1 \xrightarrow{a} t_2$ (accepting), $t_2 \xrightarrow{a} t_1$ — it guesses the b just before the final block.

2014–2015 • Q4(a) Source M22 **MARKS $7\frac{1}{3} + 4 = 11\frac{1}{3}$**

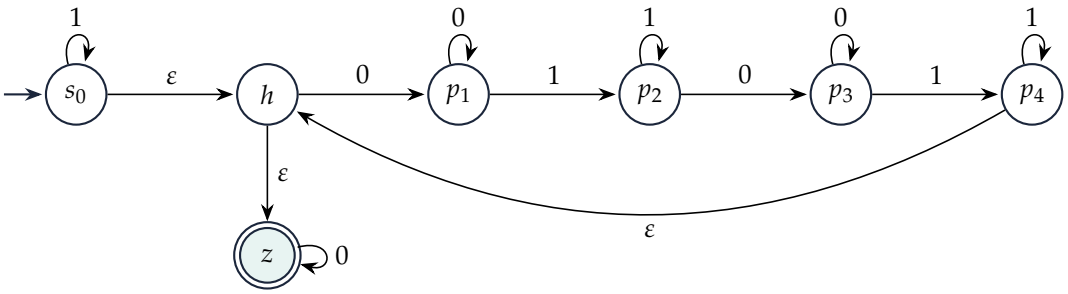
Construct an NFA over $\{0,1\}$ accepting strings containing an even number of occurrences of substring 01. Convert it to a DFA, giving only states reachable from the start.

SOLUTION

Idea. Every binary string is uniquely $1^*(0^+1^+)^k0^*$ and contains exactly k occurrences of 01. Hence

$$L = 1^*\,(0^+1^+0^+1^+)^*\,0^*.$$

NFA (from the expression; h is a hub, z handles the trailing 0^*):



Step 1: remove the ε-transitions

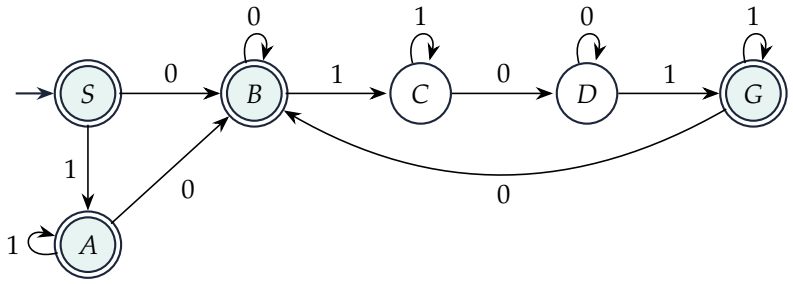
$\delta'(q,x) = E\big(\bigcup_{r\in E(q)} \delta(r,x)\big)$; $F' = \{q \mid z \in E(q)\} = \{s_0, h, p_4, z\}$:

q	$E(q)$	$\delta'(q,0)$	$\delta'(q,1)$
$\to *s_0$	$\{s_0,h,z\}$	$E(\{p_1,z\}) = \{p_1,z\}$	$E(\{s_0\}) = \{s_0,h,z\}$
$*h$	$\{h,z\}$	$\{p_1,z\}$	$\emptyset$
p_1	$\{p_1\}$	$\{p_1\}$	$\{p_2\}$
p_2	$\{p_2\}$	$\{p_3\}$	$\{p_2\}$
p_3	$\{p_3\}$	$\{p_3\}$	$E(\{p_4\}) = \{p_4,h,z\}$
$*p_4$	$\{p_4,h,z\}$	$E(\{p_1,z\}) = \{p_1,z\}$	$E(\{p_4\}) = \{p_4,h,z\}$
$*z$	$\{z\}$	$\{z\}$	$\emptyset$

Step 2: subset construction from $\{s_0\}$

$\delta_D(R,x) = \bigcup_{r\in R} \delta'(r,x)$, reachable subsets only; a subset is accepting iff it meets F'.

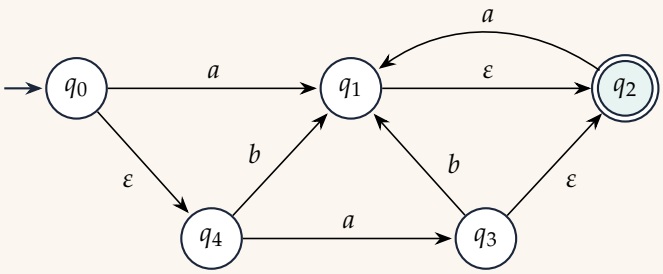
	0	1
$\to *S = \{s_0\}$	$\{p_1, z\} = B$	$\{s_0, h, z\} = A$
$*A = \{s_0, h, z\}$	$\{p_1, z\} \cup \{p_1, z\} \cup \{z\} = B$	$\{s_0, h, z\} \cup \emptyset \cup \emptyset = A$
$*B = \{p_1, z\}$	$\{p_1\} \cup \{z\} = B$	$\{p_2\} = C$
$C = \{p_2\}$	$\{p_3\} = D$	C
$D = \{p_3\}$	D	$\{p_4, h, z\} = G$
$*G = \{p_4, h, z\}$	$\{p_1, z\} \cup \{p_1, z\} \cup \{z\} = B$	$\{p_4, h, z\} = G$



Six reachable states; S, A, B, G are accepting.

2013–2014 • Q7(a) Source M20 MARKS 9

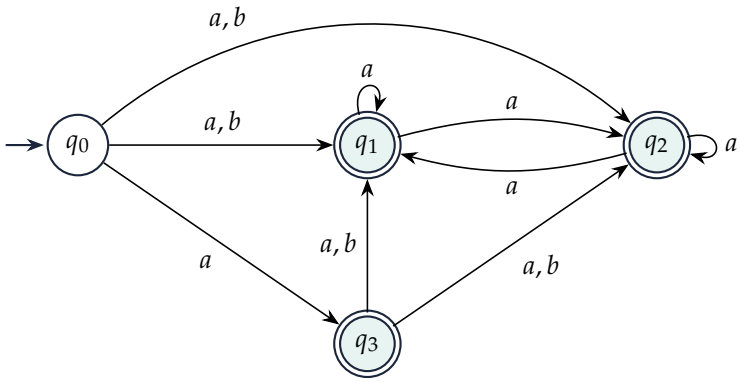
Eliminate all ε-transitions from the given NFA.



SOLUTION

ε-closures: $E(q_0) = \{q_0, q_4\}$, $E(q_1) = \{q_1, q_2\}$, $E(q_2) = \{q_2\}$, $E(q_3) = \{q_3, q_2\}$, $E(q_4) = \{q_4\}$. New transitions $\delta'(q, x) = E\big(\delta(E(q), x)\big)$; a state is accepting iff its closure contains q_2:

	a	b
$\to q_0$	$E(\{q_1, q_3\}) = \{q_1, q_2, q_3\}$	$E(q_1) = \{q_1, q_2\}$
$*q_1$	$E(q_1) = \{q_1, q_2\}$	$\emptyset$
$*q_2$	$\{q_1, q_2\}$	$\emptyset$
$*q_3$	$\{q_1, q_2\}$	$\{q_1, q_2\}$
q_4	$\{q_2, q_3\}$	$\{q_1, q_2\}$



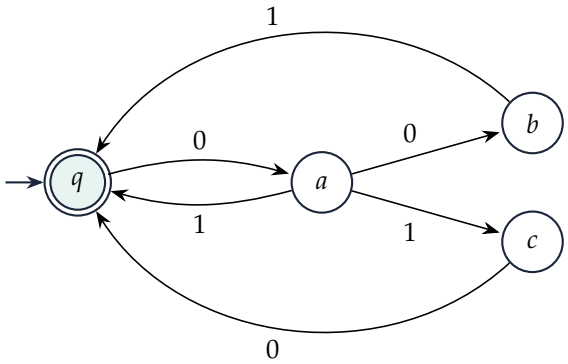
q_4 is no longer reachable from q_0 (it was entered only by ε), so it is dropped. The start state q_0 stays non-accepting since $E(q_0) \not\ni q_2$.

2011–2012 • Q7(a–b) Source M16 **MARKS (a) 8; (b) 8**

(a) Construct an NFA recognizing $(01 \cup 001 \cup 010)^*$. (b) Convert that NFA to a DFA, drawing only states reachable from the start state.

SOLUTION

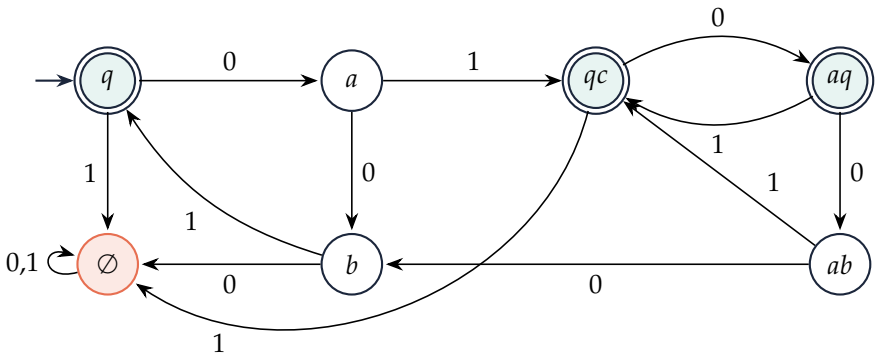
(a) Let q be both start and accepting; each word of the union is a loop through q, sharing the first 0:



$01: q \xrightarrow{0} a \xrightarrow{1} q$; $001: q \xrightarrow{0} a \xrightarrow{0} b \xrightarrow{1} q$; $010: q \xrightarrow{0} a \xrightarrow{1} c \xrightarrow{0} q$.

(b) Subset construction:

	0	1
$\to *\{q\}$	$\{a\}$	$\emptyset$
$\{a\}$	$\{b\}$	$\{q,c\}$
$\{b\}$	$\emptyset$	$\{q\}$
$*\{q,c\}$	$\{a,q\}$	$\emptyset$
$*\{a,q\}$	$\{a,b\}$	$\{q,c\}$
$\{a,b\}$	$\{b\}$	$\{q,c\}$
$\emptyset$	$\emptyset$	$\emptyset$



Seven reachable states; accepting are those containing q. Example $01001 = 010 \cdot 01$: $\{q\} \to \{a\} \to \{q,c\} \to \{a,q\} \to \{a,b\} \to \{q,c\}$ accept.

2010–2011 • Q3 Source M11 **MARKS 10**

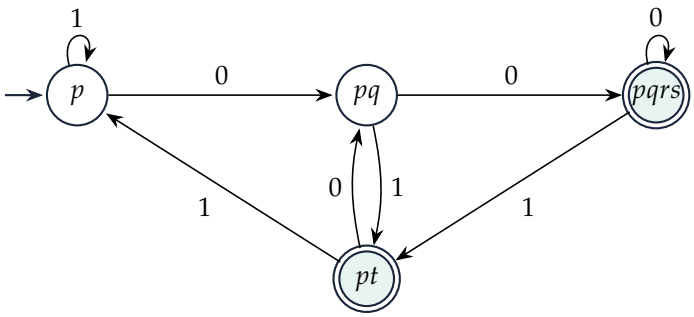
Convert the given NFA to a DFA using lazy evaluation.

State	0	1
$\to p$	$\{p,q\}$	$\{p\}$
q	$\{r,s\}$	$\{t\}$
r	$\{p,r\}$	$\{t\}$
$*s$	$\emptyset$	$\emptyset$
$*t$	$\emptyset$	$\emptyset$

SOLUTION

Lazy evaluation: start from $\{p\}$ and only compute subsets that actually appear.

DFA state	0	1
$\to \{p\}$	$\{p,q\}$	$\{p\}$
$\{p,q\}$	$\{p,q\} \cup \{r,s\} = \{p,q,r,s\}$	$\{p\} \cup \{t\} = \{p,t\}$
$*\{p,q,r,s\}$	$\{p,q\} \cup \{r,s\} \cup \{p,r\} \cup \emptyset = \{p,q,r,s\}$	$\{p,t\}$
$*\{p,t\}$	$\{p,q\}$	$\{p\}$



Only 4 of the $2^5 = 32$ subsets are reachable; accepting are the ones containing s or t.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2009–2010 • Q1 Source M08 **MARKS 10**

Explain: 'Nondeterminism is an inessential feature of finite automata.'

2007–2008 • Q3(c) Source M04 **MARKS $5\frac{1}{3}$**

Explain: 'Nondeterminism is an inessential feature of finite automata.'

SOLUTION

The statement means: *adding nondeterminism does not increase the set of languages a finite automaton can recognise.* Every DFA is already an NFA, and conversely every NFA $N = (Q, \Sigma, \delta, q_0, F)$ (possibly with ε-moves) has an equivalent DFA, built in two steps.

Step 1: remove the ε-moves. Let $E(q)$ be the ε-closure of q (the states reachable from q by zero or more ε-moves, q included). Build $N' = (Q, \Sigma, \delta', q_0, F')$ with

$$\delta'(q,a) = E\Big(\bigcup_{r \in E(q)} \delta(r,a)\Big), \qquad F' = \{q \in Q \mid E(q) \cap F \neq \emptyset\}.$$

(δ' = take free moves, read a, take free moves again; q becomes accepting if it can reach F by free moves.) N' has no ε-moves and $L(N') = L(N)$.

Step 2: subset construction. From N' build the DFA $M = (\mathcal{P}(Q), \Sigma, \delta_M, \{q_0\}, F_M)$ with

$$\delta_M(R,a) = \bigcup_{r \in R} \delta'(r,a), \qquad F_M = \{R \subseteq Q \mid R \cap F' \neq \emptyset\},$$

constructing only the subsets reachable from the start state $\{q_0\}$. The DFA state after reading w is *exactly* the set of N'-states reachable on w (induction on $|w|$), so M accepts w iff some branch of N' — equivalently of N — accepts it. Hence $L(M) = L(N)$, and DFA, NFA and ε-NFA all recognise precisely the regular languages.

Example. The NFA for "ends in 01": $\delta(p,0) = \{p,q\}$, $\delta(p,1) = \{p\}$, $\delta(q,1) = \{r\}$, all other moves $\emptyset$, start p, $F = \{r\}$. It has no ε-moves, so Step 1 leaves it unchanged ($E(q) = \{q\}$ for every q). Step 2 from $\{p\}$:

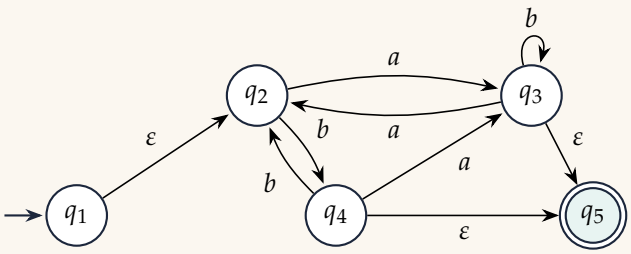
δ_M	0	1
$\to \{p\}$	$\{p,q\}$	$\{p\}$
$\{p,q\}$	$\{p,q\}$	$\{p,r\}$
$*\{p,r\}$	$\{p,q\}$	$\{p\}$

No new subsets appear, so only 3 of the $2^3 = 8$ subsets are reachable: a 3-state DFA accepting exactly the strings that end in 01.

Nondeterminism is "inessential" only for *power*; it is still valuable for *succinctness* and convenience: some n-state NFAs need 2^n-state DFAs. (Contrast: for PDAs nondeterminism is essential — DPDAs cannot recognise $\{ww^R\}$.)

2009–2010 • Q3 Source M08 **MARKS 10**

Convert the given NFA into an equivalent DFA using lazy evaluation.



SOLUTION

Step 1: remove the ε-transitions

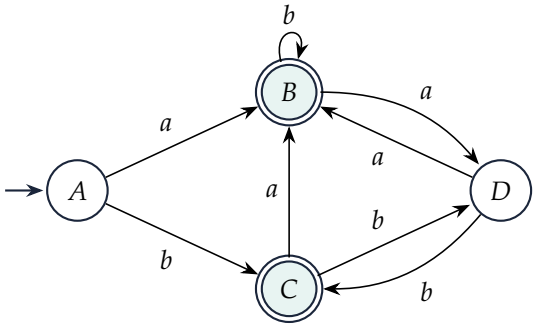
$\delta'(q,x) = E\big(\bigcup_{r\in E(q)} \delta(r,x)\big)$; $F' = \{q \mid q_5 \in E(q)\} = \{q_3, q_4, q_5\}$:

q	$E(q)$	$\delta'(q,a)$	$\delta'(q,b)$
$\to q_1$	$\{q_1, q_2\}$	$E(\{q_3\}) = \{q_3, q_5\}$	$E(\{q_4\}) = \{q_4, q_5\}$
q_2	$\{q_2\}$	$\{q_3, q_5\}$	$\{q_4, q_5\}$
$*q_3$	$\{q_3, q_5\}$	$E(\{q_2\}) = \{q_2\}$	$E(\{q_3\}) = \{q_3, q_5\}$
$*q_4$	$\{q_4, q_5\}$	$E(\{q_3\}) = \{q_3, q_5\}$	$E(\{q_2\}) = \{q_2\}$
$*q_5$	$\{q_5\}$	$\emptyset$	$\emptyset$

Step 2: lazy subset construction from $\{q_1\}$

$\delta_D(R,x) = \bigcup_{r\in R} \delta'(r,x)$; only subsets that actually appear are expanded:

DFA state	a	b
$\to A = \{q_1\}$	$\{q_3, q_5\} = B$	$\{q_4, q_5\} = C$
$*B = \{q_3, q_5\}$	$\{q_2\} \cup \emptyset = \{q_2\} = D$	$\{q_3, q_5\} \cup \emptyset = B$
$*C = \{q_4, q_5\}$	$\{q_3, q_5\} = B$	$\{q_2\} = D$
$D = \{q_2\}$	B	C

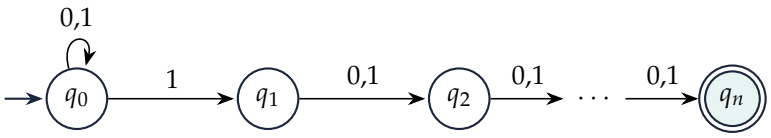


2008–2009 • Q6(b) Source M07 **MARKS 9**

Give an illustrative example of an NFA with $n+1$ states that has no equivalent DFA with fewer than 2^n states.

SOLUTION

$L_n = \{w \in \{0,1\}^* \mid$ the n-th symbol from the end is $1\}$. NFA with $n+1$ states: guess the position of that 1.



Every DFA needs $\geq 2^n$ states. Suppose a DFA D has fewer than 2^n states. There are 2^n strings of length n, so two different ones, $x \neq y$, lead D to the same state. They differ at some position i (say $x_i = 1, y_i = 0$). Append $z = 0^{i-1}$: in xz and yz the symbol at position n from the end is $x_i = 1$ resp. $y_i = 0$, so $xz \in L_n$, $yz \notin L_n$. But D is in the same state after x and y, hence also after xz and yz — contradiction. (Example $n = 3$: 4 NFA states, 8 DFA states, one per possible last-three-symbols window.)

2008–2009 • Q7(c) Source M07 **MARKS $4\frac{1}{3}$**

Convert $(0+1)^*1(0+1)$ into an ε-NFA.

SOLUTION

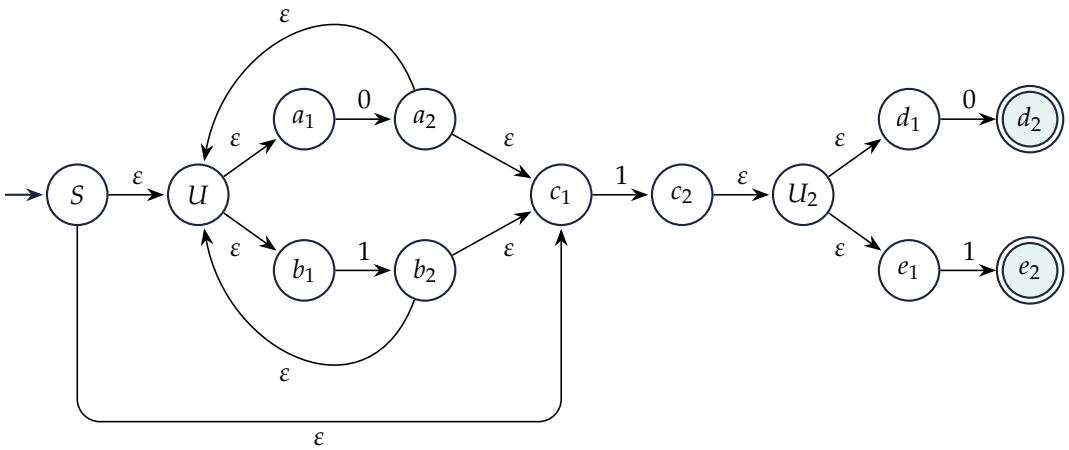
Build bottom-up with the slide (Sipser) constructions; concatenated pieces are joined by ε-edges from the accepting states of the left piece to the start of the right piece (which then stop accepting).

(i) $0 \cup 1$: new start U, $U \xrightarrow{\varepsilon} a_1 \xrightarrow{0} a_2$ and $U \xrightarrow{\varepsilon} b_1 \xrightarrow{1} b_2$; accepting a_2, b_2.

(ii) $(0 \cup 1)^*$: new accepting start S, $S \xrightarrow{\varepsilon} U$, and $a_2 \xrightarrow{\varepsilon} U$, $b_2 \xrightarrow{\varepsilon} U$; accepting S, a_2, b_2.

(iii) Concatenate with 1 ($c_1 \xrightarrow{1} c_2$): ε-edges from S, a_2, b_2 to c_1; only c_2 accepts.

(iv) Concatenate with a second copy of $0 \cup 1$ (start U_2, $U_2 \xrightarrow{\varepsilon} d_1 \xrightarrow{0} d_2$, $U_2 \xrightarrow{\varepsilon} e_1 \xrightarrow{1} e_2$): $c_2 \xrightarrow{\varepsilon} U_2$. Accepting: d_2, e_2.



(Equivalent compact NFA: $p \xrightarrow{0,1} p$, $p \xrightarrow{1} q$, $q \xrightarrow{0,1} r$ with r accepting — "second-last symbol is 1".)

2006–2007 • Q3(b) Source M01 **MARKS 8**

Describe in detail the steps for finding the equivalent DFA of an ε-NFA using lazy evaluation.

SOLUTION

Given an ε-NFA $N = (Q, \Sigma, \delta, q_0, F)$, build a DFA $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$:

(i) **ε-closures.** For each q, $E(q)$ = all states reachable from q using zero or more ε-edges (so $q \in E(q)$); for a set S, $E(S) = \bigcup_{q \in S} E(q)$.

(ii) **Remove the ε-transitions.** Build $N' = (Q, \Sigma, \delta', q_0, F')$ with $\delta'(q, a) = E\big(\bigcup_{r \in E(q)} \delta(r, a)\big)$ (free moves, read a, free moves again) and $F' = \{q \mid E(q) \cap F \neq \emptyset\}$ (a state that reaches an accepting state by free moves becomes accepting). N' has no ε-edges and $L(N') = L(N)$.

(iii) **Start state.** $q_D = \{q_0\}$. Put it in a work-list; $Q_D = \{q_D\}$.

(iv) **Lazy expansion.** Remove a set R from the work-list. For each $a \in \Sigma$ compute $\delta_D(R, a) = \bigcup_{r \in R} \delta'(r, a)$. If this set is new, add it to Q_D and to the work-list. (Only subsets that are actually reached are ever created — this is the "lazy" part; the full $2^{|Q|}$ power set is never listed.)

(v) **Repeat** step 4 until the work-list is empty. $\emptyset$, if it appears, is a dead state with self-loops.

(vi) **Accepting states.** $F_D = \{R \in Q_D \mid R \cap F' \neq \emptyset\}$.

By induction on $|w|$, $\delta_D(\{q_0\}, w)$ is exactly the set of states N' can be in after reading w, so $L(D) = L(N') = L(N)$. Worked examples appear above (e.g. January 2020 Q4(b), 2021–2022 Q3(c)).

04

Regular expressions and language operations

NOTE Exam papers often write $+$ for union; the slides (Sipser) write $\cup$. Both mean the same; the solutions use $\cup$.

2023–2024 • Q4(b) Source N02 **MARKS 12**

For strings over $\{a, b\}$ containing no more than one occurrence of aa, determine which options are correct. Give reasons for every option: A. $(b + ab)^*(\varepsilon + a + aa)(b + ab)^*$; B. $(b + ab)^*(\varepsilon + a + aa)(b + ba)^*$; C. $(b + ab)^*(\varepsilon + a)(b + ba)^*$; D. $(b + ba)^*(\varepsilon + a + aa)(b + ba)^*$.

SOLUTION

Useful facts: $(b + ab)^*$ = strings with no aa that are empty or *end with* b; $(b + ba)^*$ = strings with no aa that are empty or *start with* b. Occurrences are counted with overlap (aaa contains aa twice).

× **A.** The right factor may begin with a: $aa \cdot ab = aaab$ contains aa twice (positions 1–2 and 2–3). Over-generates.

✓ **B.** *Sound*: the left part ends in b (or is empty) and the right part starts with b (or is empty), so the middle ε, a or aa is isolated; at most one aa arises. *Complete*: if w has exactly one aa, write $w = u\,aa\,v$; u cannot end in a (else aaa) so $u \in (b + ab)^*$, and v cannot start with a so $v \in (b + ba)^*$. If w has no aa: if it ends in a, $w = u\,a$ with $u \in (b + ab)^*$; otherwise $w \in (b + ab)^*$.

× **C.** Sound but incomplete: it generates only strings with *zero* occurrences, missing e.g. aa, $baab$.

× **D.** The left factor may end in a: $ba \cdot aa = baaa$ contains aa twice. Over-generates.

Answer. Only B is correct. (Verified by enumerating all strings of length ≤ 9.)

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q3(c) Source M51 **MARKS 8**

Explain in detail why the Kleene star of a null set is an empty string.

2021–2022 • Q3(a) Source M45 **MARKS 4**

Why is the Kleene star of the null set an empty string?

2016–2017 • Q2(c) Source M29 **MARKS 10**

Why is the Kleene star of the null set an empty string? Explain clearly.

SOLUTION

Precisely: $\emptyset^* = \{\varepsilon\}$ — the language containing only the empty string (not $\emptyset$, and not the string ε itself).

From the definition. For any language L,

$$L^* = \bigcup_{k \geq 0} L^k = L^0 \cup L^1 \cup L^2 \cup \cdots, \qquad L^0 = \{\varepsilon\}, \quad L^{k+1} = L^k \circ L.$$

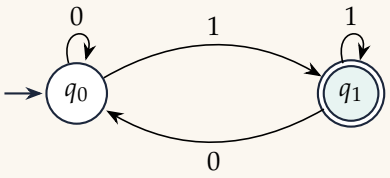
$L^0 = \{\varepsilon\}$ for *every* L: concatenating zero strings gives the empty string, and $\{\varepsilon\}$ is the identity of concatenation ($\{\varepsilon\} \circ L = L$), just as $x^0 = 1$ in arithmetic. For $L = \emptyset$ and $k \geq 1$, $\emptyset^k = \emptyset$ because a concatenation needs a string from each factor and $\emptyset$ has none ($\emptyset \circ M = \emptyset$, like multiplying by 0). Hence

$$\emptyset^* = \{\varepsilon\} \cup \emptyset \cup \emptyset \cup \cdots = \{\varepsilon\}.$$

Intuition. L^* = "choose any number of strings from L and glue them". From $\emptyset$ you can choose zero strings only, and gluing nothing yields ε.
Automaton view. The star construction adds a new start state that is accepting. Applied to an NFA for $\emptyset$ (no accepting states), the only accepted string is ε.

2022–2023 • Q4(c) Source M51 **MARKS 10**

Which option(s) describe the shown DFA? Explain each choice as correct or incorrect: A. $0^*1(1 + 00^*1)^*$; B. $0^*1^*1 + 11^*0^*1$; C. $(0 + 1)^*1$.



SOLUTION

Every 1 leads to q_1 and every 0 to q_0, so the DFA accepts exactly the strings **ending in 1**.

✓ **A.** 0^*1 reaches q_1; afterwards each block is either 1 (loop) or 0^+1 (go to q_0, loop, come back). Any string ending in 1 splits uniquely this way. Correct.

× **B.** 0^*1^*1 has no 0 after a 1; 11^*0^*1 has at most one 0-block. It misses e.g. 0101 and 10101. Incorrect (it is a proper subset).

✓ **C.** Literally "anything, then 1". Correct.

Answer. A and C.

2021–2022 • Q2(c) Source M45 **MARKS 6**

Over $\{a, b\}$, let $L_1 = \{a, aa, aaa\}$, $L_2 = \{a, ab, aab, aabb\}$, and $L_3 = \{w \mid$ the number of b's is divisible by $3\}$. State whether each is finite or infinite, with reasons: (i) $L_1^* \circ L_2^*$; (ii) $L_1 \circ (L_1 \cup L_2)$; (iii) $L_2 \circ L_3$.

SOLUTION

(i) Infinite. $L_1^* \supseteq \{a^n \mid n \geq 0\}$ and $\varepsilon \in L_2^*$, so $a^n \in L_1^* L_2^*$ for every n. (Star of a language containing a non-empty string is always infinite.)

(ii) Finite. Concatenation of finite languages is finite: $|L_1 \circ (L_1 \cup L_2)| \leq 3 \cdot 6 = 18$. (In fact it has

12 distinct strings: $a^2, \ldots, a^6$, $a^k b$ for $2 \le k \le 5$, and $a^k bb$ for $3 \le k \le 5$.)

(iii) Infinite. L_3 is infinite ($a^n \in L_3$ for all n, also $b^3, b^6, \ldots$) and $L_2 \neq \emptyset$; e.g. $a \cdot a^n \in L_2 L_3$ for all n.

2021–2022 • Q3(d) Source M46 **MARKS 12**

Which regular expression(s) represent binary strings having both consecutive 00 and consecutive 11? Explain why each option is correct or incorrect: A. $(0+1)^*0011(0+1)^* + (0+1)^*1100(0+1)^*$; B. $(0+1)^*(00(0+1)^*11 + 11(0+1)^*00)(0+1)^*$; C. $(0+1)^*00(0+1)^* + (0+1)^*11(0+1)^*$; D. $00(0+1)^*11 + 11(0+1)^*00$.

SOLUTION

× **A.** Requires 00 and 11 to be *adjacent*. Misses 001011 (has 00 and 11 but neither 0011 nor 1100).

✓ **B.** Any string with both has an occurrence of 00 and one of 11; they cannot overlap, so one comes first: $x\,00\,y\,11\,z$ or $x\,11\,y\,00\,z$. Exactly what B says.

× **C.** "00 *or* 11": generates 00, which has no 11.

× **D.** Forces the string to start with one pair and end with the other; misses 00110, 10011.

Answer. Only B.

2019–2020 • Q2(b) Source M40 **MARKS 15**

Simplify each regular expression to an equivalent expression, with explanations: (i) $\emptyset^* \cup a^* \cup b^* \cup (a \cup b)^*$; (ii) $((a^*b^*)^*(b^*a^*)^*)^*$; (iii) $(a^*b)^* \cup (b^*a)^*$.

SOLUTION

(i) $\emptyset^* = \{\varepsilon\}$, and $\{\varepsilon\}$, a^*, b^* are all subsets of $(a \cup b)^*$. A union with a superset is the superset:

Answer. $(a \cup b)^*$.

(ii) a^*b^* contains a and b, so $(a^*b^*)^* \supseteq (a \cup b)^*$, hence $= (a \cup b)^*$. Likewise $(b^*a^*)^* = (a \cup b)^*$. Then $(a \cup b)^*(a \cup b)^* = (a \cup b)^*$ and its star is itself:

Answer. $(a \cup b)^*$.

(iii) $(a^*b)^* = \varepsilon$ plus every string ending in b (split after each b). $(b^*a)^* = \varepsilon$ plus every string ending in a. Every string is empty or ends in a or b:

Answer. $(a \cup b)^*$.

2019–2020 • Q4(b) Source M41 **MARKS 15**

Over $\{a, b\}$, write regular expressions for (i) the number of a's divisible by 3; (ii) no more than three a's; (iii) exactly one occurrence of substring aaa.

SOLUTION

(i) Group the a's in threes, with b's anywhere:

Answer. $b^*(ab^*ab^*ab^*)^*$

(ii) At most three optional a's separated by b's:

Answer. $b^*(a \cup \varepsilon)b^*(a \cup \varepsilon)b^*(a \cup \varepsilon)b^*$

(iii) "Exactly one occurrence" (overlapping counted, so $aaaa$ has two) means exactly one maximal a-block of length exactly 3 and all other a-blocks of length ≤ 2. Before the block: no aaa and ending in b or empty $= (b \cup ab \cup aab)^*$; after: no aaa and starting with b or empty $= (b \cup ba \cup baa)^*$.

Answer. $(b \cup ab \cup aab)^*\, aaa\, (b \cup ba \cup baa)^*$

(All three verified by enumeration up to length 10.)

January 2020 Term • Q2(c) Source M36 MARKS 10

Write a regular expression for the strings NOT belonging to the language of the DFA in 2(a) (the DFA accepting strings ending in 00 or 11).

SOLUTION

$L = (0 \cup 1)^*(00 \cup 11)$. Its complement: strings of length < 2, or whose last two symbols differ.

Answer. $\varepsilon \cup 0 \cup 1 \cup (0 \cup 1)^*(01 \cup 10)$

(Equivalently: make q_0, q_1, q_2 accepting and q_3, q_4 non-accepting in the given DFA and convert by GNFA state ripping.)

January 2020 Term • Q3(a) Source M37 MARKS 10

Evaluate each regular expression, showing detailed computations: (i) $\emptyset^*\emptyset^*\emptyset^*\emptyset^*$; (ii) $\emptyset^* \cup \{\varepsilon\}$; (iii) $(\emptyset^* \cup 1)\{0,1\}$; (iv) $(11)^*0(00)^*$; (v) $(\emptyset^* \cup 0)\{0,1,11\}$.

SOLUTION

Recall $\emptyset^* = \{\varepsilon\}$ and $\{\varepsilon\}L = L$.

(i) $\{\varepsilon\}\{\varepsilon\}\{\varepsilon\}\{\varepsilon\} = \{\varepsilon\}$.

(ii) $\{\varepsilon\} \cup \{\varepsilon\} = \{\varepsilon\}$.

(iii) $(\{\varepsilon\} \cup \{1\})\{0,1\} = \{\varepsilon, 1\}\{0,1\} = \{0, 1, 10, 11\}$.

(iv) $\{1^{2i}\,0\,0^{2j} \mid i, j \geq 0\} = \{1^{2i}0^{2j+1}\}$: an even number of 1's followed by an odd number of 0's, e.g. 0, 110, 000, 11000.

(v) $\{\varepsilon, 0\}\{0,1,11\} = \{0, 1, 11, 00, 01, 011\}$.

January 2020 Term • Q4(a) Source M37 MARKS 10

Describe in English the language of $(1 + \varepsilon)(00^*1)^*0^*$.

SOLUTION

An optional leading 1, then blocks each consisting of one or more 0's followed by a single 1, then any trailing 0's. Thus every 1 except possibly the first symbol is immediately preceded by a 0, i.e. no two 1's are adjacent; conversely every string without 11 splits this way.

Answer. All binary strings with no two consecutive 1's.

2017–2018 • Q4(b) Source M33 **MARKS 20**

Find regular expressions for: (i) $\{w \in \{a,b,c\}^* \mid \text{each alphabet symbol occurs at least once}\}$; (ii) $\{a^n b^m \mid n \geq 3,\ m \text{ odd}\}$; (iii) $\{a^n b^m \mid n+m \text{ odd}\}$; (iv) $\{vwv \mid v,w \in \{a,b\}^*,\ |v| = 2\}$; (v) $\{w \in \{a,b\}^* \mid n_a(w) \bmod 3 = 0\}$.

SOLUTION

Let $\Sigma = (a \cup b \cup c)$.

(i) Split on the order of first occurrences (6 permutations):

$$\Sigma^* a \Sigma^* b \Sigma^* c \Sigma^* \cup \Sigma^* a \Sigma^* c \Sigma^* b \Sigma^* \cup \Sigma^* b \Sigma^* a \Sigma^* c \Sigma^* \\ \cup \Sigma^* b \Sigma^* c \Sigma^* a \Sigma^* \cup \Sigma^* c \Sigma^* a \Sigma^* b \Sigma^* \cup \Sigma^* c \Sigma^* b \Sigma^* a \Sigma^*$$

(ii) $aaaa^*\, b(bb)^*$

(iii) n odd & m even, or n even & m odd: $a(aa)^*(bb)^* \cup (aa)^* b(bb)^*$

(iv) $v \in \{aa, ab, ba, bb\}$: $aa(a \cup b)^* aa \cup ab(a \cup b)^* ab \cup ba(a \cup b)^* ba \cup bb(a \cup b)^* bb$

(v) $b^*(ab^*ab^*ab^*)^*$

2016–2017 • Q3(b) Source M29 **MARKS 4 + 4 + 7 = 15**

Write regular expressions for (i) alternating 0's and 1's; (ii) strings over $\{a,b,c\}$ containing at least one a and at least one b; (iii) binary strings in which every pair of adjacent 0's occurs before any pair of adjacent 1's.

SOLUTION

(i) $(\varepsilon \cup 1)(01)^*(\varepsilon \cup 0)$ (covers $\varepsilon, 0, 1, 01, 10, 010, 101, \ldots$; same answer as 2006–07 Q1(b)).

(ii) $\Sigma^* a \Sigma^* b \Sigma^* \cup \Sigma^* b \Sigma^* a \Sigma^*$ with $\Sigma = (a \cup b \cup c)$.

(iii) No 00 may occur after a 11. Cut the string just after the first symbol of the first 11: the left part has no 11, the right part has no 00; conversely gluing any such parts never creates a 00 after a 11. With "no 11" $= (0 \cup 10)^*(\varepsilon \cup 1)$ and "no 00" $= (1 \cup 01)^*(\varepsilon \cup 0)$:

Answer. $(0 \cup 10)^*(\varepsilon \cup 1)(1 \cup 01)^*(\varepsilon \cup 0)$

2015–2016 • Q3(a) Source M25 • regex part **MARKS part of 18**

Compute a regular expression for each binary language: (i) begins with 1 and ends with 0; (ii) length at least 3 and third symbol 0; (iii) an even number of 0's or exactly two 1's; (iv) every string except 11 and 111; (v) at least two 0's and at most one 1.

SOLUTION

(The DFAs asked for in the same question are in the DFA part of 2015–2016 Q3(a), Topic 02.)

(i) $1(0 \cup 1)^*0$.

(ii) $(0 \cup 1)(0 \cup 1)0(0 \cup 1)^*$ — any two symbols, then 0, then anything.

(iii) $(1^*01^*0)^*1^* \cup 0^*10^*10^*$ — the first term has the 0's in pairs (even number of 0's), the second has exactly two 1's.

(iv) $\varepsilon \cup 0 \cup 1 \cup 0(0 \cup 1) \cup 10 \cup 0(0 \cup 1)(0 \cup 1) \cup 10(0 \cup 1) \cup 110 \cup (0 \cup 1)^4(0 \cup 1)^*$ — list all strings of length ≤ 3 except 11 and 111, then every string of length ≥ 4.

(v) $000^*(\varepsilon \cup 10^*) \cup 0100^* \cup 1000^*$ — no 1 and at least two 0's, or a single 1 with at least two 0's before it, exactly one before it (then at least one after), or none before it (then at least two after).

(All five regular expressions were cross-checked by enumeration on all strings up to length 10.)

2014–2015 • Q3(b) Source M22 **MARKS 4 + 3 + 3 = 10**

Give regular expressions for: (i) decimal numbers divisible by 5, over $\{0, \ldots, 9\}$, with no leading 0 except standalone 0; (ii) binary strings with an odd number of 0's (any number of 1's) or an odd number of 1's (any number of 0's); (iii) binary strings containing no embedded 101.

SOLUTION

Let $D = (0 \cup 1 \cup \cdots \cup 9)$ and $N = (1 \cup 2 \cup \cdots \cup 9)$.

(i) A number is divisible by 5 iff its last digit is 0 or 5:

Answer. $0 \cup 5 \cup N\,D^*(0 \cup 5)$

(ii) Odd number of 0's: $1^*0(1^*01^*0)^*1^*$; odd number of 1's: $0^*1(0^*10^*1)^*0^*$:

Answer. $1^*0(1^*01^*0)^*1^* \cup 0^*1(0^*10^*1)^*0^*$

(iii) No 101 means every 0-block lying between two 1's has length ≥ 2:

Answer. $0^*(11^*000^*)^*(\varepsilon \cup 11^*0^*)$

2013–2014 • Q5(b) Source M20 **MARKS 5**

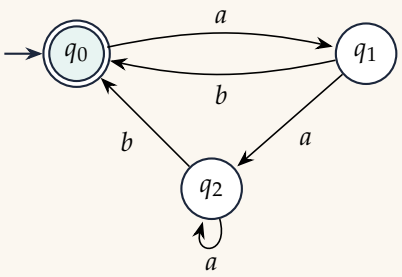
Give a counterexample to $(R_1 \cup R_2)^* = R_1^* \cup R_2^*$, where R_1 and R_2 are regular expressions.

SOLUTION

Take $R_1 = a$, $R_2 = b$. Then $ab \in (a \cup b)^*$, but $ab \notin a^* \cup b^*$ (strings there use only one letter). So the equality fails; in general only $R_1^* \cup R_2^* \subseteq (R_1 \cup R_2)^*$ holds.

2013–2014 • Q6(c) Source M20 **MARKS 8**

Construct a regular expression for the given NFA.



SOLUTION

GNFA. Add a new start $q_s \xrightarrow{\varepsilon} q_0$ and a new accept state q_a with $q_0 \xrightarrow{\varepsilon} q_a$ (q_0 is the only accepting state). Labels: $R(q_0,q_1) = a$, $R(q_1,q_0) = b$, $R(q_1,q_2) = a$, $R(q_2,q_2) = a$, $R(q_2,q_0) = b$; every other label is $\emptyset$. We use $R_{\text{new}}(q_i,q_j) = R(q_i,q_j) \cup R(q_i,q_{rip})R(q_{rip},q_{rip})^*R(q_{rip},q_j)$.
Rip q_2. The only path through q_2 is $q_1 \to q_2 \to q_0$:

$$R_{\text{new}}(q_1,q_0) = b \cup a\,a^*\,b.$$

Rip q_1. The only path through q_1 is $q_0 \to q_1 \to q_0$ (with $R(q_1,q_1) = \emptyset$, $\emptyset^* = \varepsilon$):

$$R_{\text{new}}(q_0,q_0) = \emptyset \cup a\,(b \cup aa^*b) = a(b \cup aa^*b).$$

Rip q_0. Path $q_s \to q_0 \to q_a$: $R(q_s,q_a) = \varepsilon\,\big(a(b \cup aa^*b)\big)^*\varepsilon$. Since $a(b \cup aa^*b) = a(\varepsilon \cup aa^*)b = aa^*b$,

Answer. $\big(a(b \cup aa^*b)\big)^* = (aa^*b)^*$: empty, or strings that start with a, end with b, and contain no bb.
(Checked against the NFA on all strings of length ≤ 10.)

2012–2013 • Q6(c) Source M18 MARKS 3

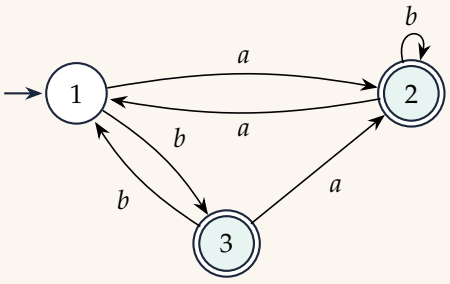
Write a regular expression for binary strings with even parity, meaning an even number of 1's.

SOLUTION

Answer. $0^*(10^*10^*)^*$
The 1's come in pairs, with any 0's in between and before.

2012–2013 • Q7(b) Source M18 MARKS $7\frac{1}{3}$

Convert the given DFA into a regular expression.



SOLUTION

GNFA. Add a new start $q_s \xrightarrow{\varepsilon} 1$ and a new accept state q_a with $2 \xrightarrow{\varepsilon} q_a$, $3 \xrightarrow{\varepsilon} q_a$. Labels: $R(1,2) = a$, $R(1,3) = b$, $R(2,1) = a$, $R(2,2) = b$, $R(3,1) = b$, $R(3,2) = a$; every other label is $\emptyset$ (in particular $R(1,1) = R(3,3) = \emptyset$). Rip with $R_{\text{new}}(q_i, q_j) = R(q_i, q_j) \cup R(q_i, q_{rip})R(q_{rip}, q_{rip})^* R(q_{rip}, q_j)$.
Rip 3. State 3 is entered only from 1 (on b) and leaves to 1, 2 and q_a; $R(3,3)^* = \emptyset^* = \varepsilon$:

$$R_{\text{new}}(1,1) = \emptyset \cup b\,b = bb, \qquad R_{\text{new}}(1,2) = a \cup ba, \qquad R_{\text{new}}(1,q_a) = \emptyset \cup b\,\varepsilon = b.$$

Rip 2. State 2 is entered only from 1 (label $a \cup ba$), has loop b, and leaves to 1 and q_a:

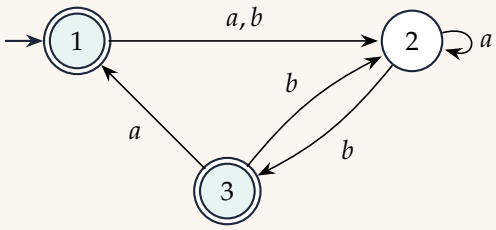
$$R_{\text{new}}(1,1) = bb \cup (a \cup ba)b^*a, \qquad R_{\text{new}}(1,q_a) = b \cup (a \cup ba)b^*.$$

Rip 1. Path $q_s \to 1 \to q_a$ with the loop on 1:

Answer. $L = \big(bb \cup (a \cup ba)b^*a\big)^* \big(b \cup (a \cup ba)b^*\big)$
(Checked against the DFA on all strings of length ≤ 10.)

2011–2012 • Q7(c) Source M16 MARKS $7\frac{1}{3}$

Derive a regular expression for the given finite automaton.



SOLUTION

GNFA. Add a new start $q_s \xrightarrow{\varepsilon} 1$ and a new accept state q_a with $1 \xrightarrow{\varepsilon} q_a$, $3 \xrightarrow{\varepsilon} q_a$. Labels: $R(1,2) = a \cup b$ (parallel edges combined), $R(2,2) = a$, $R(2,3) = b$, $R(3,2) = b$, $R(3,1) = a$; every other label is $\emptyset$. Rip with $R_{\text{new}}(q_i, q_j) = R(q_i, q_j) \cup R(q_i, q_{rip})R(q_{rip}, q_{rip})^* R(q_{rip}, q_j)$.
Rip 2. State 2 is entered from 1 and 3, has loop a, and leaves only to 3:

$$R_{\text{new}}(1,3) = \emptyset \cup (a \cup b)a^*b, \qquad R_{\text{new}}(3,3) = \emptyset \cup b\,a^*b = ba^*b.$$

Rip 3. State 3 is entered only from 1, has loop ba^*b, and leaves to 1 and q_a:

$$R_{\text{new}}(1,1) = \emptyset \cup (a \cup b)a^*b(ba^*b)^*a, \qquad R_{\text{new}}(1,q_a) = \varepsilon \cup (a \cup b)a^*b(ba^*b)^*.$$

Rip 1. Path $q_s \to 1 \to q_a$ with the loop on 1:

Answer. $L = \big((a \cup b)a^*b(ba^*b)^*a\big)^* \big(\varepsilon \cup (a \cup b)a^*b(ba^*b)^*\big)$
(Checked against the automaton on all strings of length ≤ 10.)

2010–2011 • Q4 Source M11 • regex part MARKS 3

Give a regular expression for the language of binary strings not containing substring 110.

SOLUTION

Use the DFA of the DFA part of 2010–2011 Q4 (Topic 02): once 11 has occurred (q_2), no 0 may ever follow. Before that, the machine moves between q_0 and q_1, i.e. it reads blocks 0 and 10 (every 1 is immediately followed by a 0). So the string is a sequence of blocks 0, 10 followed by a final block of 1's:

Answer. $(0 \cup 10)^*1^*$

(Checked by enumeration against "does not contain 110" on all strings up to length 12.)

2009–2010 • Q4 Source M08 **MARKS 10**

Let $L_1 = 1(1+0)^*000$ and $L_2 = (1(01)^*0)^*$. Determine, with reasoning, whether each belongs to $L_1 \circ L_2$: (a) 1000; (b) 1000101010; (c) 10000111000100000.

SOLUTION

L_1 = strings that start with 1 and end with 000 (length ≥ 4). Each block $1(01)^k0 = (10)^{k+1}$, so $L_2 = (10)^*$. Hence $w \in L_1L_2$ iff $w = xy$ with $x \in L_1$ and $y \in (10)^*$.

(a) $1000 = 1000 \cdot \varepsilon$; $1000 \in L_1$, $\varepsilon \in L_2$. **Yes.**

(b) $1000101010 = 1000 \cdot 101010$; $101010 = (10)^3 \in L_2$. **Yes.**

(c) The string ends in 00, so y cannot be a non-empty member of $(10)^*$ (which ends in 10); take $y = \varepsilon$. Then $x = 10000111000100000$ starts with 1 and ends with 000, so $x \in L_1$. **Yes.**

2008–2009 • Q7(b) Source M07 **MARKS 5**

Write regular expressions for: (i) strings over $\{a, b, c\}$ containing at least one a and at least one c; (ii) binary strings with no two consecutive 0's or no two consecutive 1's.

SOLUTION

(i) $\Sigma^*a\Sigma^*c\Sigma^* \cup \Sigma^*c\Sigma^*a\Sigma^*$, $\Sigma = (a \cup b \cup c)$.

(ii) No 00: $(1 \cup 01)^*(\varepsilon \cup 0)$; no 11: $(0 \cup 10)^*(\varepsilon \cup 1)$.

Answer. $(1 \cup 01)^*(\varepsilon \cup 0) \cup (0 \cup 10)^*(\varepsilon \cup 1)$

2007–2008 • Q2(b) Source M04 **MARKS 8**

For $\Sigma = \{0, 1\}$, which languages are represented by $(0 \cup 1)^*$ and by $(0\Sigma^*) \cup (\Sigma^*1)$?

SOLUTION

$(0 \cup 1)^* = \Sigma^*$: **all** binary strings, including ε. $(0\Sigma^*) \cup (\Sigma^*1)$: strings that **begin with 0 or end with 1** (e.g. 0, 01, 1, 101; but not ε, 10, 110).

2007–2008 • Q4(c) Source M04 **MARKS 5 $\frac{1}{3}$**

Regular expressions define exactly the same languages as the various forms of finite automata. Why, then, do we need regular expressions?

SOLUTION

Equal power does not mean equal usefulness. Regular expressions are:

- **Declarative and compact** — they describe *what* the strings look like (e.g. `[0-9]+(\.[0-9]+)?`) instead of a state graph; much easier for people to write, read and type in one line.
- **Algebraic** — they obey identities ($R \cup R = R$, $(R^*)^* = R^*$, …) that allow simplification and equational reasoning.
- **The input language of tools** — `grep`, `sed`, editors, lexer generators (`lex`/`flex`), input validation in programs. The tool converts the expression to an NFA/DFA internally.
- **Composable** — union, concatenation and star build big patterns from small ones directly.

Automata are the *implementation* model; regular expressions are the *specification* language. Having both, plus the conversions, gives the best of each.

2006–2007 • Q1(b) Source M01 **MARKS 8**

Develop a regular expression for the set of strings consisting of alternating 0's and 1's.

SOLUTION

An alternating string is $(01)^*$ or $(10)^*$, optionally with one extra symbol: $(01)^* \cup (10)^* \cup 0(10)^* \cup 1(01)^*$. Factor out the optional first 1 and last 0:

Answer. $(\varepsilon \cup 1)(01)^*(\varepsilon \cup 0)$

(Same answer as 2016–17 Q3(b)(i).)

2006–2007 • Q1(c) Source M01 **MARKS $5\frac{1}{3}$**

Explain: '$\emptyset$ and ε are languages over any alphabet.' Can these two be called equivalent? Why?

SOLUTION

A language over Σ is any subset of Σ^*. The empty set $\emptyset$ is a subset of every set, so it is a language over every Σ. The empty string ε belongs to Σ^* for every Σ (it uses no symbols), so $\{\varepsilon\}$ (usually just written ε in expressions) is also a language over every alphabet.

They are **not** equivalent:

- $|\emptyset| = 0$ but $|\{\varepsilon\}| = 1$.
- $\emptyset \circ L = \emptyset$, whereas $\{\varepsilon\} \circ L = L$ ($\emptyset$ is the "zero", $\{\varepsilon\}$ the "one" of concatenation).
- An automaton for $\emptyset$ accepts nothing; one for $\{\varepsilon\}$ accepts the empty input.

They are related only by $\emptyset^* = \{\varepsilon\}^* = \{\varepsilon\}$.

2006–2007 • Q4(b) Source M01–M02 **MARKS 8**

Describe the language represented by each regular expression over $\Sigma = \{0,1\}$: (i) 0^*10^*; (ii) $\Sigma^*1\Sigma^*$; (iii) $1^*(01^*)^*$; (iv) $(\Sigma\Sigma\Sigma)^*$; (v) $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1$; (vi) $01 \cup 10$; (vii) $(0 \cup \varepsilon)1^*$; (viii) $(01 \cup 010)^*$.

SOLUTION

(i) Exactly one 1.

(ii) At least one 1.

(iii) *All* binary strings (1^* then each 0 followed by any 1's) $= \Sigma^*$.

(iv) Strings whose length is a multiple of 3.

(v) Non-empty strings that start and end with the same symbol.

(vi) The two strings $\{01, 10\}$.

(vii) $01^* \cup 1^*$: any number of 1's, optionally preceded by a single 0.

(viii) Concatenations of 01 and 010: ε, or strings that start with 01, in which every later 1 is preceded by exactly one or two 0's, with no 11, and which end in 1 or in a single 0 after the last 1 (e.g. 01, 010, 0101, 01001, 010010).

05

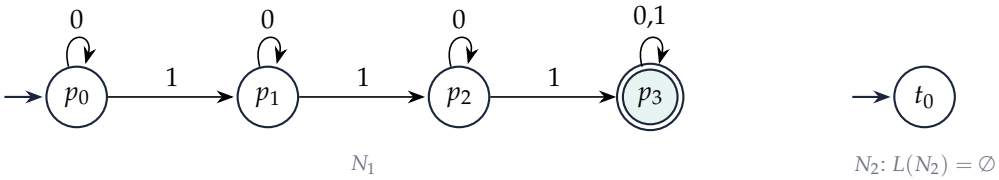
Regular-language closure and equivalence

2023–2024 • Q1(c) Source N01 **MARKS 10**

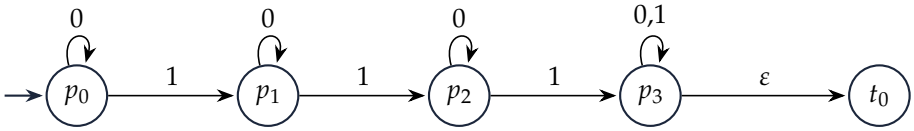
Draw NFAs for the concatenation of two languages over $\{0,1\}$: (i) strings containing at least three 1's and (ii) the empty set. Begin with the individual NFAs and apply the construction used to prove regular-language closure under concatenation.

SOLUTION

Individual NFAs. N_1 (at least three 1's) and N_2 (the empty set: one non-accepting start state, no transitions):



Concatenation construction. Start state = start of N_1; add an ε-edge from every accepting state of N_1 to the start of N_2; the accepting states of N are those of N_2 only (the accepting states of N_1 lose their status).



N has **no accepting state**, so $L(N) = \emptyset$ — as it must be, since $L \circ \emptyset = \emptyset$ for every L (there is no string of $\emptyset$ to append).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q3(b) Source N02 **MARKS 13**

In proving closure under star, someone proposes marking the old start state accepting, adding no new start state, and leaving the rest of the construction unchanged. Explain carefully why this is a bad idea and give a counterexample.

2016–2017 • Q4(a) Source M30 **MARKS 10**

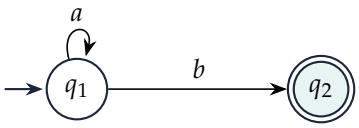
In the usual NFA construction for A^*, add a new start state that is also accepting, plus the required ε-edges. Hastif Hasty proposes instead marking the original start state accepting and keeping the other new edges where applicable. Explain clearly when this proposed construction fails.

SOLUTION

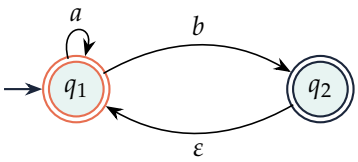
Correct construction. Given N_1 for A: add a new start state q_0 which is accepting, $q_0 \xrightarrow{\varepsilon} q_1$ (old start), and ε-edges from every accepting state back to q_1. The new state exists only to accept ε and has *no incoming edges*.

Proposal. Make the old start q_1 accepting (to get ε) and keep the back ε-edges $F_1 \to q_1$. The danger: q_1 *may have incoming edges inside* N_1. Then a partial computation that returns to q_1 without completing a word of A is now accepted.

Counterexample. $A = a^*b$, with N_1: $q_1 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_2$, q_2 accepting.



N_1: $L = a^*b$



proposed: accepts a, aa, ...

$A^* = (a^*b)^*$ consists of ε and strings ending in b. The proposed machine accepts a (loop on the now-accepting q_1), but $a \notin A^*$. So the construction is wrong.

When does it fail / work? It is guaranteed safe when the old start state has *no incoming transitions* (no path returns to it except via the new ε-edges). If q_1 has incoming transitions it can fail: any string that brings N_1 back to q_1 without finishing a word of A (like a above) is wrongly accepted. Adding a fresh start state avoids this in every case.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2022–2023 • Q3(b) Source M51 MARKS 12

Swapping accepting and nonaccepting states of a DFA for L yields a DFA for its complement. Does this always work for an NFA? Give an example.

2021–2022 • Q1(c) Source M44 MARKS 6 + 3 = 9

Explain why swapping accepting and nonaccepting states of a DFA recognizing C gives a DFA for the complement of C. Does the same method work for NFAs? Explain with examples.

2017–2018 • Q2(a) Source M32 MARKS 10

Given a DFA D for L over Σ, how can you obtain a DFA for $\Sigma^* - L$? Explain why the method works and when it would fail. Does it work for NFAs? Explain.

2015–2016 • Q3(b) Source M25 MARKS $5\frac{1}{3}$

Swapping accepting and nonaccepting states in a DFA gives its complement. Give an NFA example for which the same swap does not recognize the complement.

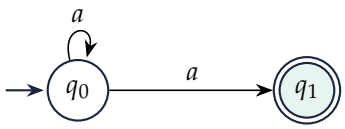
SOLUTION

Method. From $D = (Q, \Sigma, \delta, q_0, F)$ build $\overline{D} = (Q, \Sigma, \delta, q_0, Q - F)$.

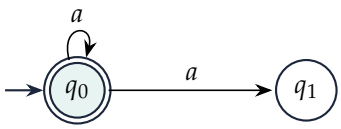
Why it works for DFAs. δ is a total function, so on every input w there is *exactly one* run and it ends in exactly one state $\hat{\delta}(q_0, w)$. Then

$$w \in L(\overline{D}) \iff \hat{\delta}(q_0, w) \in Q - F \iff \hat{\delta}(q_0, w) \notin F \iff w \notin L(D).$$

When it fails (even for "DFAs"). (1) If the DFA is *incomplete* (some transitions missing, implicitly rejecting), strings that "fall off" are rejected both before and after the swap. Fix: first add an explicit dead state receiving all missing moves (it becomes accepting after the swap). (2) If D is over a smaller alphabet than Σ, add the missing symbols (to the dead state) first.
NFAs: it does not work in general. An NFA accepts if *some* run ends in F. After swapping, it accepts if *some* run ends outside F — that is not the same as "*no* run ends in F". A string can have both kinds of runs and then is accepted by both machines.
Example. Over $\{a\}$: $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{a} q_1$, $F = \{q_1\}$; $L(N) = a^+$.



N: a^+

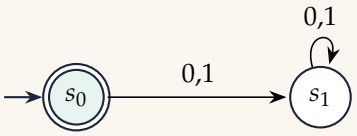


swapped: a^* (not $\{\varepsilon\}$)

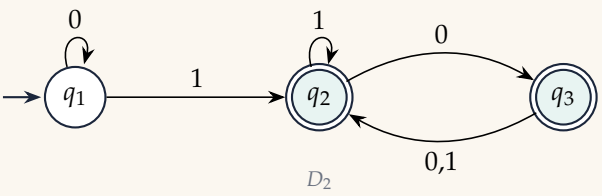
The complement of a^+ is $\{\varepsilon\}$, but the swapped NFA accepts a^* (e.g. a via the run staying in q_0). Missing transitions cause the same problem: dead branches are rejected in both versions.
Correct way for an NFA: convert it to a complete DFA by the subset construction, *then* swap.

2022–2023 • Q4(a) Source M51 **MARKS 10**

Using basic principles, construct a DFA D for $L(D_1) \cup L(D_2)$, where D_1 and D_2 are shown. (i) Describe $L(D_1)$ in plain English. (ii) Draw the start state of D and two states reachable from it.



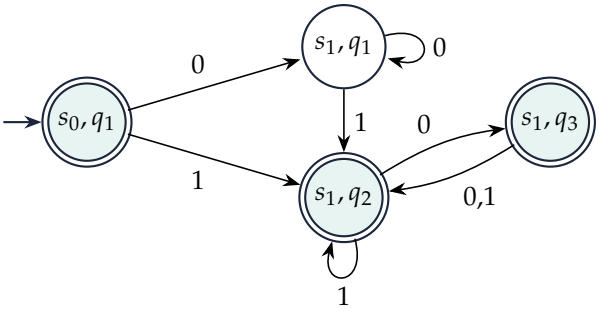
D_1



D_2

SOLUTION

(i) D_1 accepts only in s_0 and leaves it on any symbol forever, so $L(D_1) = \{\varepsilon\}$ — only the empty string. (For reference, $L(D_2)$ = strings containing at least one 1.)
(ii) Product construction. $Q = Q_1 \times Q_2$, start (s_0, q_1), $\delta((r,s),x) = (\delta_1(r,x), \delta_2(s,x))$, and $F = \{(r,s) \mid r \in F_1 \text{ or } s \in F_2\}$ for union.



The start state (s_0, q_1) is accepting (because $s_0 \in F_1$); the two states reachable in one step are (s_1, q_1) (non-accepting) and (s_1, q_2) (accepting). The complete D has the 4 reachable states shown and recognises $\{\varepsilon\} \cup \{w \mid w \text{ contains a } 1\}$.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q4(b) Source M46 **MARKS 12**

Given NFAs $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ for L_1 and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ for L_2, construct $N = (Q, \Sigma, \delta, q_0, F)$ for $L_1 \cup L_2$. Explain the reasons for: (i) choosing its start state; (ii) choosing its accepting states; (iii) each case for transitions from the new start state and their possible destinations.

2019–2020 • Q3(c) Source M41 **MARKS 15**

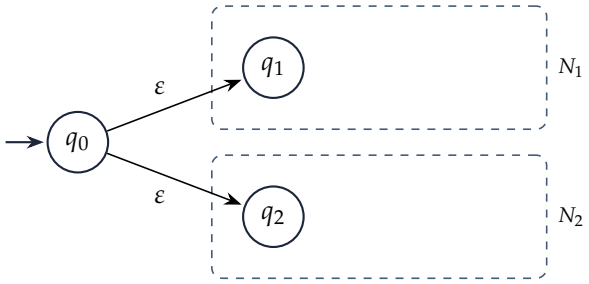
NFAs N_1 and N_2 recognize L_1 and L_2. To construct $N = (Q, \Sigma, \delta, q_0, F)$ recognizing $L_1 \cup L_2$, write the expression for δ and explain each part with justification.

SOLUTION

Assume $Q_1 \cap Q_2 = \emptyset$ (rename if necessary).

$$Q = \{q_0\} \cup Q_1 \cup Q_2, \qquad F = F_1 \cup F_2,$$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \\ \delta_2(q,a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0,\ a = \varepsilon \\ \emptyset & q = q_0,\ a \neq \varepsilon \end{cases} \qquad (a \in \Sigma_\varepsilon).$$



(i) Start state. A *new* state q_0 is needed because the machine must be able to begin either computation; we cannot simply pick q_1 or q_2. Being fresh, it has no incoming edges, so it

cannot introduce unintended paths (and it does not disturb the internal behaviour of N_1 or N_2).

(ii) Accepting states $F = F_1 \cup F_2$. A string is in $L_1 \cup L_2$ iff it is accepted by N_1 or by N_2, i.e. iff some branch ends in F_1 or in F_2. q_0 is not accepting; if $\varepsilon \in L_1 \cup L_2$ it is still accepted via the ε-move into q_1 or q_2.

(iii) Transitions. For $q \in Q_1$ (resp. Q_2) we copy δ_1 (resp. δ_2), so each sub-machine runs unchanged. From q_0: on ε go to *both* q_1 and q_2 — nondeterministically guessing which language w belongs to, without consuming input; on any real symbol $a \neq \varepsilon$ there is no move ($\emptyset$), because consuming a symbol at q_0 would skip the first symbol of w and corrupt both simulations.

Correctness. $w \in L(N) \iff$ some run $q_0 \xrightarrow{\varepsilon} q_i \rightsquigarrow F_i \iff w \in L(N_1) \cup L(N_2)$.

January 2020 Term • Q3(b) Source M37 MARKS 20

NFA $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognizes L_1. Construct $N = (Q, \Sigma, \delta, q_0, F)$ recognizing L_1^*. For $q \in Q$ and $a \in \Sigma_\varepsilon$, δ is defined by: $\delta_1(q,a)$ if $q \in Q_1, q \notin F_1$; $\delta_1(q,a)$ if $q \in F_1, a \neq \varepsilon$; $\delta_1(q,a) \cup \{q_1\}$ if $q \in F_1, a = \varepsilon$; $\{q_1\}$ if $q = q_0, a = \varepsilon$; $\emptyset$ if $q = q_0, a \neq \varepsilon$. Explain in plain English the purpose of each condition.

SOLUTION

Here $Q = \{q_0\} \cup Q_1$, the new start q_0, and $F = \{q_0\} \cup F_1$.

(i) $q \in Q_1, q \notin F_1$**:** $\delta_1(q,a)$**.** Inside a piece that is not yet finished, behave exactly like N_1 — we are in the middle of reading one word of L_1.

(ii) $q \in F_1, a \neq \varepsilon$**:** $\delta_1(q,a)$**.** At an accepting state, N_1 may still continue reading (the current word of L_1 might be a prefix of a longer word of L_1), so its ordinary moves are kept.

(iii) $q \in F_1, a = \varepsilon$**:** $\delta_1(q,\varepsilon) \cup \{q_1\}$**.** Having completed one word of L_1, the machine may (without reading input) jump back to N_1's start to begin the *next* word — this is the "repeat" of the star. The original ε-moves of q are kept too.

(iv) $q = q_0, a = \varepsilon$**:** $\{q_1\}$**.** The new start state immediately enters N_1 to read the first piece. Because q_0 is accepting, zero pieces (ε) are accepted — required since $\varepsilon \in L_1^*$.

(v) $q = q_0, a \neq \varepsilon$**:** $\emptyset$**.** q_0 consumes no input; it exists only to accept ε and to hand over to q_1. Having no other edges (and no incoming edges) guarantees it adds no unintended strings — which is why we do not simply make q_1 accepting (see the counterexample in 2023–24 Q3(b) above).

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2016–2017 • Q4(b) Source M30 MARKS 13

Prove regular-language closure under concatenation.

2015–2016 • Q2(b) Source M25 MARKS 15

Prove that regular languages are closed under regular operations.

2012–2013 • Q5(c) Source M18 **MARKS 10**

Define a regular language and prove closure under (i) star and (ii) concatenation.

2011–2012 • Q5(b) Source M15 **MARKS 12**

Define a regular language. Prove closure under (i) star and (ii) concatenation.

SOLUTION

SCOPE *2016–2017 Q4(b) asks only for concatenation; 2012–2013 Q5(c) and 2011–2012 Q5(b) ask for the definition, star and concatenation; 2015–2016 Q2(b) asks for all three regular operations (union, concatenation, star).*

Definition. A language is *regular* if some finite automaton (DFA, equivalently NFA) recognises it; equivalently, if it is described by a regular expression. The regular operations are union, concatenation and star. Because NFAs and DFAs are equivalent, it suffices to build an *NFA* in each case. Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognise A_1 and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognise A_2, with disjoint state sets.

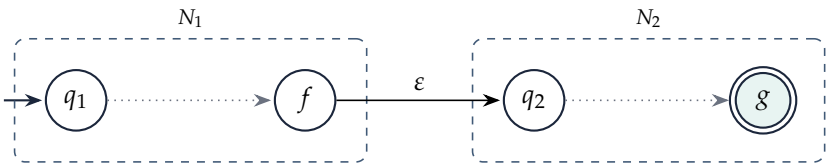
Union $A_1 \cup A_2$

New start q_0 with ε-edges to q_1, q_2; $F = F_1 \cup F_2$ (full δ in the previous solution). Some branch accepts iff N_1 or N_2 accepts.

Concatenation $A_1 \circ A_2$

$N = (Q_1 \cup Q_2, \Sigma, \delta, q_1, F_2)$ with

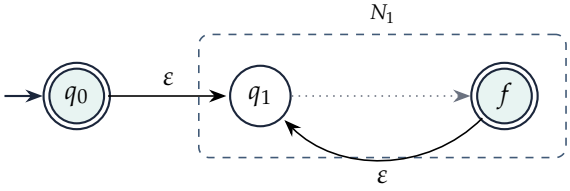
$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 - F_1 \\ \delta_1(q,a) & q \in F_1,\ a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1,\ a = \varepsilon \\ \delta_2(q,a) & q \in Q_2. \end{cases}$$



Proof. ($A_1A_2 \subseteq L(N)$) If $w = xy$, $x \in A_1$, $y \in A_2$: N reads x along an accepting run of N_1 ending in some $f \in F_1$, takes $f \xrightarrow{\varepsilon} q_2$, then reads y along an accepting run of N_2. ($L(N) \subseteq A_1A_2$) Every accepting run of N ends in $F_2 \subseteq Q_2$ and starts in Q_1; the only way from Q_1 into Q_2 is an ε-edge $f \to q_2$ with $f \in F_1$, taken exactly once. The input read before it is accepted by N_1, the input after it by N_2. Hence $L(N) = A_1 \circ A_2$.

Star A_1^*

$N = (\{q_0\} \cup Q_1, \Sigma, \delta, q_0, \{q_0\} \cup F_1)$ with a new accepting start $q_0 \xrightarrow{\varepsilon} q_1$, and ε-edges from every $f \in F_1$ back to q_1 (the five-case δ written out in January 2020 Q3(b) above).



Proof. ($A_1^* \subseteq L(N)$) ε is accepted at q_0. If $w = x_1 \cdots x_k$ with $x_i \in A_1$, $k \geq 1$: go $q_0 \to q_1$, read x_1 to some $f \in F_1$, jump back to q_1, read $x_2, \ldots$, end in F_1. ($L(N) \subseteq A_1^*$) An accepting run either stops at q_0 (input ε) or enters q_1 and ends in F_1; cutting it at each use of a back-edge $f \to q_1$ splits the input into pieces each accepted by N_1, so $w \in A_1^*$. The new start state is essential (it has no incoming edges; see the counterexample earlier in this topic).
Since each construction yields an NFA, $A_1 \cup A_2$, $A_1 A_2$ and A_1^* are regular. ■

2013–2014 • Q5(a) Source M20 **MARKS 8**

Prove regular-language closure under difference: $L_1 - L_2 = \{w \mid w \in L_1 \text{ and } w \notin L_2\}$.

SOLUTION

Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ be complete DFAs for L_1, L_2. Build the product DFA

$$M = (Q_1 \times Q_2,\ \Sigma,\ \delta,\ (s_1, s_2),\ F_1 \times (Q_2 - F_2)), \qquad \delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a)).$$

By induction on $|w|$, $\hat{\delta}((s_1,s_2),w) = (\hat{\delta}_1(s_1,w), \hat{\delta}_2(s_2,w))$: M runs both machines in parallel. So M accepts w iff $\hat{\delta}_1(s_1,w) \in F_1$ and $\hat{\delta}_2(s_2,w) \notin F_2$, i.e. iff $w \in L_1$ and $w \notin L_2$. Thus $L_1 - L_2$ is regular. (Alternatively: $L_1 - L_2 = L_1 \cap \overline{L_2} = \overline{\overline{L_1} \cup L_2}$, and regular languages are closed under complement and union.) ■

2013–2014 • Q8(a) Source M21 **MARKS 7**

Suppose DFAs $M_1 = \langle \Sigma, Q_1, q_0^1, \delta_1, F_1 \rangle$ and $M_2 = \langle \Sigma, Q_2, q_0^2, \delta_2, F_2 \rangle$ recognize L_1 and L_2. Describe how to construct a DFA recognizing $L_1 \cup L_2$.

SOLUTION

Product construction (no nondeterminism needed):

$$M = \langle \Sigma,\ Q_1 \times Q_2,\ (q_0^1, q_0^2),\ \delta,\ F \rangle, \quad \delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a)),$$

$$F = \{(p,q) \mid p \in F_1 \text{ or } q \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2).$$

M simulates both DFAs simultaneously; after reading w it is in $(\hat{\delta}_1(q_0^1,w), \hat{\delta}_2(q_0^2,w))$, which is in F iff $w \in L_1$ or $w \in L_2$. It has $|Q_1| \cdot |Q_2|$ states (only reachable ones need be drawn). Choosing $F = F_1 \times F_2$ instead gives intersection. See 2022–23 Q4(a) above for a worked example.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2013–2014 • Q8(b) Source M21 **MARKS $10\frac{1}{3}$**

Prove that every NFA has an equivalent DFA.

2010–2011 • Q1 Source M11 MARKS 10

A DFA has been constructed from an NFA using subset construction. Will the DFA accept/reject the same language as the NFA? Explain with justification.

2007–2008 • Q2(a) Source M04 MARKS 10

Prove that subset construction preserves the language: if $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is constructed from $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, then $L(D) = L(N)$.

2006–2007 • Q2(a) Source M01 MARKS 10

Prove: if $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is the DFA constructed from NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ by subset construction, then $L(D) = L(N)$.

SOLUTION

Construction (for an NFA without ε-moves; see the remark for ε-NFAs):

$$Q_D = \mathcal{P}(Q_N), \quad \text{start } \{q_0\}, \quad \delta_D(S,a) = \bigcup_{p \in S} \delta_N(p,a), \quad F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}.$$

δ_D is a total function from (set, symbol) to a set, so D is a genuine DFA.
Claim. For every $w \in \Sigma^*$: $\hat{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w)$.
Proof by induction on $|w|$. *Basis* $w = \varepsilon$: $\hat{\delta}_D(\{q_0\}, \varepsilon) = \{q_0\} = \hat{\delta}_N(q_0, \varepsilon)$. *Induction* $w = xa$: assume $\hat{\delta}_D(\{q_0\}, x) = \hat{\delta}_N(q_0, x) = \{p_1, \ldots, p_k\}$. Then

$$\hat{\delta}_D(\{q_0\}, xa) = \delta_D\big(\hat{\delta}_D(\{q_0\}, x), a\big) = \delta_D(\{p_1, \ldots, p_k\}, a) = \bigcup_{i=1}^{k} \delta_N(p_i, a) = \hat{\delta}_N(q_0, xa),$$

the last equality being the definition of $\hat{\delta}_N$. □
Conclusion.

$$w \in L(D) \iff \hat{\delta}_D(\{q_0\}, w) \in F_D \iff \hat{\delta}_N(q_0, w) \cap F_N \neq \emptyset \iff w \in L(N).$$

So yes — the DFA accepts exactly the strings the NFA accepts and rejects exactly those it rejects; every NFA has an equivalent DFA (with at most $2^{|Q_N|}$ states). ■
Remark (ε-NFA). Proceed in two steps. (1) *Remove the ε-moves*: keep the same states and start q_0, and set $\delta'(q,a) = E\big(\bigcup_{r \in E(q)} \delta_N(r,a)\big)$ (free moves, read a, free moves again) and $F' = \{q \mid E(q) \cap F_N \neq \emptyset\}$; the resulting NFA has no ε-moves and recognises the same language. (2) Apply the subset construction above to it, starting from $\{q_0\}$ and keeping only the reachable subsets.

2013–2014 • Q8(c) Source M21 MARKS 6

Disprove by counterexample: every subset of a regular language is regular.

SOLUTION

$\Sigma^* = \{0,1\}^*$ is regular (one-state DFA). Its subset $\{0^n 1^n \mid n \geq 0\}$ is not regular (pumping lemma: let p be the pumping length and $s = 0^p 1^p \in L$, $|s| \geq p$; for any split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$, since $|xy| \leq p$ we have $y = 0^k$ with $k \geq 1$, so $xy^2z = 0^{p+k}1^p \notin L$). Hence a subset of a

regular language need not be regular.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q8(b) Source M18 **MARKS 12 1/3**

Prove that a set is regular if and only if it can be described by a regular expression.

2008–2009 • Q7(a) Source M07 **MARKS 14**

Prove that if $L = L(A)$ for a DFA A, then there is a regular expression R with $L = L(R)$.

SOLUTION

SCOPE *2008–2009 Q7(a) asks only for the ($\Rightarrow$) direction (second part).*

($\Leftarrow$) Regular expression $\Rightarrow$ automaton

Induction on the structure of R. Base cases: $\emptyset$ (one non-accepting state), ε (one accepting state), a (two states $s \xrightarrow{a} f$). Inductive cases $R_1 \cup R_2$, R_1R_2, R_1^*: by the induction hypothesis there are NFAs for R_1, R_2, and the union, concatenation and star constructions above give an NFA for R. Hence $L(R)$ is regular.

($\Rightarrow$) DFA $\Rightarrow$ regular expression (GNFA state ripping)

A GNFA is an NFA whose arrows are labelled by regular expressions; it moves from q_i to q_j by reading a block of input in $L(R(q_i, q_j))$. Turn the DFA A into a GNFA G: add a new start state q_s with $q_s \xrightarrow{\varepsilon} q_0$ and a new accept state q_a with $f \xrightarrow{\varepsilon} q_a$ for each $f \in F$; parallel edges a, b become one label $a \cup b$; missing edges get the label $\emptyset$. Clearly $L(G) = L(A)$.

CONVERT(G): let k be the number of states of G. If $k = 2$, return $R(q_s, q_a)$. If $k > 2$, choose $q_{rip} \notin \{q_s, q_a\}$, delete it, and for all $q_i \neq q_a$, $q_j \neq q_s$ set

$$R_{\text{new}}(q_i, q_j) = R(q_i, q_j) \cup R(q_i, q_{rip})\, R(q_{rip}, q_{rip})^*\, R(q_{rip}, q_j);$$

call the result G' ($k-1$ states) and return CONVERT(G').

Claim: $L(G') = L(G)$. If G accepts w along $q_s, r_1, \ldots, r_m, q_a$, each step avoiding q_{rip} is read by the first term of the new label, and each maximal block $q_i, q_{rip}, \ldots, q_{rip}, q_j$ is read by the second term; so G' accepts w. Conversely each step $q_i \to q_j$ of G' reads a string in $R(q_i, q_j)$ (a direct step of G) or in $R(q_i, q_{rip})R(q_{rip}, q_{rip})^*R(q_{rip}, q_j)$ (a path of G through q_{rip}); so G accepts w. By induction on k, CONVERT(G) returns R with $L(R) = L(G) = L(A)$.

Thus every DFA language has a regular expression. Combining with NFA$\leftrightarrow$DFA equivalence: a set is regular $\iff$ it is denoted by a regular expression. ■

(Worked examples of state ripping: 2013–14 Q6(c), 2012–13 Q7(b) and 2011–12 Q7(c) in Topic 04.)

06

Regular pumping lemma and nonregular languages

NOTE **Throughout: Pumping Lemma (PL). If L is regular, there is a number p (the pumping length) such that every $s \in L$ with $|s| \geq p$ can be written $s = xyz$ with (1) $|y| > 0$, (2) $|xy| \leq p$, (3) $xy^iz \in L$ for all $i \geq 0$. To show L is *not* regular we play against an adversary: assume L is regular and let p be its pumping length; *we* choose $s \in L$ (in terms of p) with $|s| \geq p$; the *adversary* chooses any split $s = xyz$ obeying (1),(2); *we* exhibit one i with $xy^iz \notin L$ — a contradiction, so L is not regular.**

2023–2024 • Q2(a) Source N01 **MARKS 15**

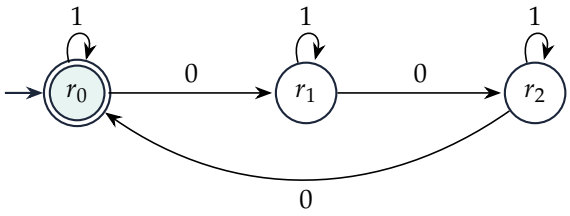
Justify why the regular pumping lemma requires $|xy| \leq p$ in $w = xyz$. Explain clearly with an example language and its finite automaton.

SOLUTION

Where the condition comes from. Let p be the number of states of a DFA for L and $s \in L$ with $|s| \geq p$. The run on s visits $p+1$ states $r_0, r_1, \ldots, r_p$ while reading just the *first* p *symbols*. By the pigeonhole principle two of these coincide, $r_j = r_k$ with $0 \leq j < k \leq p$. Take $x = s_1 \cdots s_j$, $y = s_{j+1} \cdots s_k$ (the loop), z = the rest. Because the repetition is found inside the first p symbols, $|xy| = k \leq p$ *automatically* — it costs nothing to state it, and it is true.

Why we need it. It pins the pumpable part y near the *start* of s, so in a proof we know what y consists of. Example $L = \{0^n1^n\}$: assume L is regular and let p be its pumping length; choose $s = 0^p1^p \in L$, $|s| = 2p \geq p$. Because $|xy| \leq p$, every split has $y = 0^k$ with $k \geq 1$, and $xy^2z = 0^{p+k}1^p \notin L$ — a contradiction, so L is not regular. Without $|xy| \leq p$ the adversary could choose $y = 0^m1^m$ or y inside the 1's, and we would have to analyse many cases; and for some languages, where a later portion of s genuinely can be pumped, no contradiction could be reached at all.

Example automaton. $L = \{w \in \{0,1\}^* \mid \#_0(w) \equiv 0 \pmod 3\}$, DFA with $p = 3$ states:



For $s = 10010 \in L$ (three 0's, $|s| = 5 \geq 3$): the first $p = 3$ symbols give the run $r_0 \xrightarrow{1} r_0 \xrightarrow{0} r_1 \xrightarrow{0} r_2$; already r_0 repeats after the first symbol, so $x = \varepsilon$, $y = 1$, $z = 0010$, and $|xy| = 1 \leq 3$. Pumping y goes around the loop and stays in L. A loop is *always* found within the first p symbols, which is precisely $|xy| \leq p$.

2023–2024 • Q4(c) Source N02 **MARKS 13**

Use the regular pumping lemma to show that $L = \{a^{n!} \mid n \geq 1\}$ is not regular.

SOLUTION

Assume L is regular and let p be its pumping length. Let $m = \max(p, 2)$ and $s = a^{m!} \in L$; $|s| = m! \geq m \geq p$. For any split, $y = a^k$ with $1 \leq k \leq |xy| \leq p \leq m$. Pump up ($i = 2$): $|xy^2z| = m! + k$. Now

$$m! < m! + k \leq m! + m < m! + m \cdot m! = (m+1)!,$$

(using $m \geq 2$, so $m < m \cdot m!$). Thus $m! + k$ lies strictly between two consecutive factorials and is not a factorial, so $xy^2z \notin L$. This contradicts the PL; L is not regular. ■

2022–2023 • Q2(b) Source M50 **MARKS 15**

For $L = \{w \mid w = a^{2+3k} \text{ or } w = b^{10+12k},\ k \geq 0\}$ over $\{a, b\}$, argue carefully using the pumping lemma whether 3, 5, 9, or 24 is a pumping length. Find and analyze the minimum pumping length.

SOLUTION

$L = \{a^2, a^5, a^8, \dots\} \cup \{b^{10}, b^{22}, b^{34}, \dots\}$. A number p is a pumping length if *every* $s \in L$ with $|s| \geq p$ has a split satisfying (1)–(3).

***a*-strings.** For $s = a^{2+3k}$, $y = a^j$. Pumping up keeps the length $\equiv 2 \pmod 3$ only if $3 \mid j$; pumping down needs $2 + 3k - j \geq 2$. So we need $y = a^3$ (or $a^6, \dots$) and $k \geq 1$. With $x = \varepsilon$, $y = aaa$: $|xy| = 3$. This works for every a^{2+3k}, $k \geq 1$, whenever $p \geq 3$; and a^2 ($k = 0$) is unpumpable but is exempt as soon as $p \geq 3$.

***b*-strings.** For $s = b^{10+12k}$, $y = b^j$ must have $12 \mid j$, and pumping down needs $k \geq 1$. Hence (a) b^{10} can *never* be pumped, so we need $p > 10$; (b) y has length ≥ 12, so $|xy| \leq p$ forces $p \geq 12$. Conversely, if $p \geq 12$, every b-string with $|s| \geq p$ has $k \geq 1$ and the split $x = \varepsilon$, $y = b^{12}$ works: $xy^iz = b^{10+12(k-1+i)} \in L$ for all $i \geq 0$.

p	verdict
3	× $s = b^{10}$ (length $10 \geq 3$) cannot be pumped: $b^{10+j} \notin L$ for $1 \leq j \leq 3$
5	× same witness b^{10}
9	× same witness b^{10}
24	✓ a-strings: $y = a^3$; b-strings of length ≥ 24 are $b^{34}, b^{46}, \dots$: $y = b^{12}$, $\lvert xy\rvert = 12 \leq 24$

Minimum. $p = 11$ fails: $s = b^{22}$ has length ≥ 11 but its y would need length 12 (or 24, …) > 11. $p = 12$ works by the analysis above.

Answer. 3, 5, 9 are not pumping lengths; 24 is. The minimum pumping length is 12.

2021–2022 • Q1(d) Source M44–M45 **MARKS 9**

Comment on the acceptability of these arguments by B. L. Perfunctory (BLP): (i) Assume binary palindromes are regular with pumping length p. Let $w = 00011000$. Since w is a palindrome, there are x, y, z satisfying the lemma. xy lies entirely in the initial 0^p, so x and y contain only zeros. Pumping down gives xz, which cannot be a palindrome, so the language is not regular. (ii) Assume equal-count binary strings are regular. Choose $w = 0^{p/4}1^{p/4}0^{p/4}1^{p/4} = xyz$. y lies entirely in the first zeros. For any permissible length of y, xy^2z has unequal counts and is outside the language; hence the language is not regular. (iii) Assume $\{0^{2^n} \mid n \geq 0\}$ is regular. Choose

$w = 0^{2p}$, $x = \varepsilon$, $y = 0$, $z = 0^{2p-1}$. Taking $i = 0$ gives $xy^iz = 0^{2p-1}$, whose length is odd; hence the language is not regular.

SOLUTION

(i) Not acceptable. BLP's string 00011000 is a *fixed* string; it must depend on p. If $p > 8$ the lemma says nothing about it (it is shorter than p), and "the initial 0^p" does not exist (it starts with only three 0's). The conclusion is true, but the proof is wrong. Fix: let p be the pumping length and $s = 0^p10^p$ (a palindrome, $|s| = 2p + 1 \geq p$); every split has $y = 0^k$, $k \geq 1$, and $xy^0z = 0^{p-k}10^p$ is not a palindrome — contradiction.

(ii) Not acceptable. BLP's string has length exactly p, so $|xy| \leq p$ allows y *anywhere* in it, not just in the first block. The adversary may pick $y = 01$ (straddling the first two blocks); then xy^iz keeps equal counts for all i and no contradiction arises. Fix: $s = 0^p1^p$ ($|s| = 2p \geq p$), forcing $y = 0^k$; then $xy^2z = 0^{p+k}1^p$ has unequal counts.

(iii) Not acceptable. The prover may not choose x, y, z; the lemma only promises that *some* split works. Here the adversary chooses $y = 00$ (legal whenever $p \geq 2$), and then $xy^iz = 0^{2p-2+2i} \in L$ for all i. In fact the language $(00)^*$ *is regular*, so no valid proof exists.

2021–2022 • Q4(a) Source M46 **MARKS 4**

Why is the regular pumping lemma useful for 'negative proofs' only?

SOLUTION

The lemma is an implication: *regular* ⇒ *pumpable*. Its contrapositive, *not pumpable* ⇒ *not regular*, is what we use. The converse is false: some nonregular languages satisfy the pumping property, e.g.

$$L = \{a^ib^jc^k \mid i, j, k \geq 0,\ i = 1 \Rightarrow j = k\}$$

is nonregular, yet every long string can be pumped with $p = 2$ (pump the first symbol unless $i = 2$, where pump the first two). So showing that a language *can* be pumped proves nothing; regularity must be shown positively by giving a DFA/NFA/regular expression or using closure properties.

2019–2020 • Q1(a) Source M40 **MARKS 5**

For $L = \{a^n \mid n \text{ is prime}\}$, a pumping-lemma proof starts: 'Given p, let $w = a^p \ldots$', where p is the pumping length. Comment on the acceptability of this start.

SOLUTION

Not acceptable as written: the pumping length p is *any* number supplied by the lemma, not necessarily prime (e.g. $p = 4$), and then $a^p \notin L$, so the lemma does not apply to it. The chosen string must belong to L. Correct start: assume L is regular and let p be its pumping length; choose a prime $q > p$ (infinitely many primes exist) and $s = a^q \in L$, $|s| = q \geq p$. Every split has $y = a^k$, $1 \leq k \leq p$; pump $i = q + 1$: $|xy^{q+1}z| = q + qk = q(1 + k)$, a product of two numbers ≥ 2, hence not prime, so $xy^{q+1}z \notin L$ — contradiction, L is not regular. (If p happens to be prime, $s = a^p$ is fine, but the proof cannot assume that.)

2019–2020 • Q4(c) Source M41 **MARKS 15**

Use the pumping lemma to prove $L = \{a^{2n}b^{3n}a^n \mid n \geq 0\}$ is not regular.

SOLUTION

Assume L is regular and let p be its pumping length. Take $s = a^{2p}b^{3p}a^p \in L$ ($n = p$), $|s| = 6p \geq p$. Let $s = xyz$ be any split with $|y| > 0$, $|xy| \leq p$. Since $|xy| \leq p \leq 2p$, x and y lie inside the first block: $y = a^k$, $k \geq 1$. Pump up:

$$xy^2z = a^{2p+k}b^{3p}a^p.$$

For this to be in L, the last block forces $n = p$, so the first block would have to be a^{2p} — but it is a^{2p+k}. Hence $xy^2z \notin L$, contradicting the lemma. L is not regular. ■

January 2020 Term • Q1(b) Source M36 **MARKS 12**

Surcharged Skye gives these pumping-lemma arguments. Explain clearly what, if anything, is wrong: (i) $L_1 = \{ww \mid w \in \{0,1\}^*\}$. Choose 0^p0^p, $x = \varepsilon$, $y = (00)^k$, $k \leq p/2$. Since $xy^iz \in L_1$ for all $i \geq 0$, L_1 is regular. (ii) $L_2 = \{w \in \{0,1\}^* \mid n_0(w) = 3n_1(w)\}$. Choose $0^{3p}1^p$ and $y = 0001$. Since $xy^iz \in L_2$ for all $i \geq 0$, we have failed to show that L_2 contradicts the lemma.

SOLUTION

(i) Wrong logic. The PL gives a *necessary* condition for regularity; showing that one string (or even every string) can be pumped never proves a language regular. Also, the prover does not get to choose y. In fact L_1 is *not* regular: let p be the pumping length and take $s = 0^p10^p1 \in L_1$, $|s| = 2p + 2 \geq p$; every split has $y = 0^k$ ($k \geq 1$) in the first block, and $xy^2z = 0^{p+k}10^p1 \notin L_1$ (a square uu ending in 1 with two 1's needs u to contain exactly one 1, so both halves would be $0^{p+k}1 = 0^p1$, impossible). Contradiction.

(ii) Illegal split. With $s = 0^{3p}1^p$ and $|xy| \leq p$, y must lie within the first p symbols, which are all 0's — $y = 0001$ is not a legal choice (the first 1 is at position $3p + 1$). Moreover the prover must handle *every* legal y, not pick a convenient one. Correct argument: assume L_2 regular with pumping length p; $s = 0^{3p}1^p \in L_2$, $|s| = 4p \geq p$; every legal split has $y = 0^k$, $k \geq 1$; then $xy^2z = 0^{3p+k}1^p$ has $n_0 = 3p + k \neq 3p = 3n_1$, so $xy^2z \notin L_2$ — contradiction, and L_2 is not regular.

2017–2018 • Q3(b) Source M32–M33 **MARKS 17**

Surcharged Skye gives these pumping-lemma arguments. Explain clearly what, if anything, is wrong in each: (i) $L_1 = \{ww \mid w \in \{0,1\}^*\}$. Choose 0^p0^p, $x = \varepsilon$, $y = (00)^k$ with $k \leq p/2$. Since $xy^iz \in L_1$ for every $i \geq 0$, L_1 is regular. (ii) $L_2 = \{w \in \{0,1\}^* \mid n_0(w) = 2n_1(w)\}$. Choose $0^{2p}1^p$ and $y = 01$. Since $xy^iz \in L_2$ for every $i \geq 0$, we have failed to show that L_2 contradicts the lemma. (iii) $L_3 = \{0^n1^n0^n \mid n \geq 0\}$. Choose $0^k1^k0^k$ with $k = \lfloor p/3 \rfloor$. Since $xy^iz \notin L_3$ for every $i \geq 0$ and all possible y, L_3 is not regular.

SOLUTION

(i) Same error as above: pumpability never implies regularity, and y is not ours to choose. L_1 is in fact nonregular (use $s = 0^p10^p1$, $|s| = 2p + 2 \geq p$, exactly as above).

(ii) $y = 01$ is not a legal split: for $s = 0^{2p}1^p \in L_2$ ($|s| = 3p \geq p$), $|xy| \leq p$ places y inside the leading 0^{2p}, so $y = 0^k$, $k \geq 1$. Then $xy^2z = 0^{2p+k}1^p \notin L_2$ (since $2p + k \neq 2p$) — the correct proof that L_2 is not regular. Skye's "failure" comes only from an illegal (and self-chosen) y.

(iii) Two errors. (a) Skye's string has length $3\lfloor p/3 \rfloor \leq p$ — possibly *shorter* than p, and the lemma promises nothing for such a string; the chosen s must satisfy $|s| \geq p$. (b) "$xy^iz \notin L_3$ for every $i \geq 0$" is false — for $i = 1$, $xyz = w \in L_3$; we only need *some* i. The conclusion is true, but the proof must use, e.g., $s = 0^p1^p0^p \in L_3$ ($|s| = 3p \geq p$): then $y = 0^k$, $k \geq 1$, and $xy^0z = 0^{p-k}1^p0^p \notin L_3$ — contradiction.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2016–2017 • Q4(c) Source M30 **MARKS 12**

Use the pumping lemma to show that strings of 1's of prime length do not form a regular language.

2009–2010 • Q5 Source M08 **MARKS 10**

Use the pumping lemma to determine whether strings of 1's whose lengths are prime form a regular language.

2007–2008 • Q3(a) Source M04 **MARKS 10**

Use the pumping lemma to determine whether the language of all strings of 1's whose lengths are prime is regular.

SOLUTION

$L = \{1^q \mid q \text{ prime}\}$. Assume L is regular and let p be its pumping length. Choose a prime $q \geq p + 2$ (possible since there are infinitely many primes) and $s = 1^q \in L$, $|s| = q \geq p$. Any legal split has $y = 1^k$ with $1 \leq k \leq p$, so $|xz| = q - k \geq 2$. Pump with $i = q - k$:

$$|xy^{q-k}z| = (q - k) + (q - k)k = (q - k)(1 + k).$$

Both factors are ≥ 2 ($q - k \geq 2$, $1 + k \geq 2$), so the length is composite and $xy^{q-k}z \notin L$. This contradicts the PL, so L is not regular.

Answer. Not regular.

(Alternative: pump $i = q + 1$ to get length $q(1 + k)$.)

2015–2016 • Q4(c) Source M26 **MARKS 6**

Use the pumping lemma to prove that $A = \{0^n1^n2^n \mid n \geq 0\}$ and $B = \{0^i1^j \mid i > j\}$ are not regular.

SOLUTION

A. Assume A is regular and let p be its pumping length. Choose $s = 0^p1^p2^p \in A$, $|s| = 3p \geq p$. Any split $s = xyz$ with $|y| > 0$, $|xy| \leq p$ has $y = 0^k$ ($k \geq 1$). Pump $i = 2$: $xy^2z = 0^{p+k}1^p2^p \notin A$.

Contradiction; A is not regular.
B. Assume B is regular and let p be its pumping length. Choose $s = 0^{p+1}1^p \in B$, $|s| = 2p+1 \geq p$; again $y = 0^k$, $1 \leq k \leq p$. Pump *down* ($i = 0$): $xy^0z = 0^{p+1-k}1^p$ and $p+1-k \leq p$, so the number of 0's is not greater than the number of 1's: $xz \notin B$. Contradiction; B is not regular. (Pumping up would not work here — it only adds 0's.)

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2014–2015 • Q2(a) Source M22 **MARKS 10**

Use the regular pumping lemma to determine whether $\{1^{n^2} \mid n \geq 0\}$ is regular.

2012–2013 • Q5(b) Source M18 **MARKS $5\frac{1}{3}$**

Prove that $\{0^n \mid n \text{ is a perfect square}\}$ is not regular.

SOLUTION

(Shown for symbol 1; the proof for 0 is identical.) Assume L is regular and let p be its pumping length. Choose $s = 1^{p^2} \in L$, $|s| = p^2 \geq p$. Any split $s = xyz$ with $|y| > 0$, $|xy| \leq p$: $y = 1^k$, $1 \leq k \leq p$. Pump $i = 2$: Then

$$p^2 < |xy^2z| = p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2.$$

$p^2 + k$ lies strictly between consecutive squares, so it is not a perfect square: $xy^2z \notin L$. Contradiction.

Answer. Not regular.

REPEATED ×6 Same question in 6 papers — stacked below, solved once.

2014–2015 • Q4(b) Source M22 **MARKS 12**

State and prove the regular pumping lemma. Explain the proof idea before giving the actual proof.

2013–2014 • Q6(a) Source M20 **MARKS $7\frac{1}{3}$**

State and prove the pumping lemma for regular languages.

2012–2013 • Q6(b) Source M18 **MARKS $12\frac{1}{3}$**

Is there a technique to test whether a given L is regular? State and prove it.

2011–2012 • Q6(a) Source M15 **MARKS 13 $\frac{1}{3}$**

Is there a technique to test whether a given language is regular? State and prove it.

2010–2011 • Q5 Source M11 **MARKS 10**

State and prove the pumping lemma.

2008–2009 • Q5(a) Source M07 **MARKS 10**

Let L be regular. Prove that there is a constant n depending on L such that every $w \in L$ with $|w| \geq n$ can be written $w = xyz$, where $y \neq \varepsilon$, $|xy| \leq n$, and $xy^k z \in L$ for all $k \geq 0$.

SOLUTION

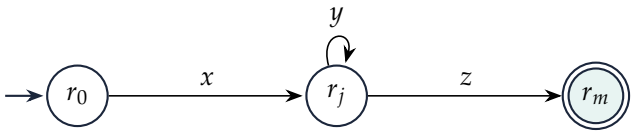
The technique. The pumping lemma is the standard tool to show a language is *not* regular (it gives a necessary condition; regularity itself is shown by building an automaton or regular expression).
Statement. If L is regular, there is a number p (the pumping length) such that every $s \in L$ with $|s| \geq p$ can be divided as $s = xyz$ where

$$(1)\ |y| > 0, \qquad (2)\ |xy| \leq p, \qquad (3)\ xy^i z \in L \ \text{ for every } i \geq 0.$$

(In the 2008–09 wording the constant is $n = p$, the string is $w = s$ and the exponent is $k = i$.)
Proof idea. A DFA has finitely many states. A long input forces the run to revisit some state; the part of the input read between the two visits is a loop that can be traversed any number of times (or skipped) without affecting acceptance.
Proof. Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA for L and $p = |Q|$. Let $s = s_1 s_2 \cdots s_m \in L$ with $m \geq p$, and let $r_0 = q_0$, $r_t = \delta(r_{t-1}, s_t)$ be the run on s; it is a sequence of $m + 1$ states with $r_m \in F$. Among the first $p + 1$ states $r_0, \ldots, r_p$ two must be equal (pigeonhole: only p distinct states). Let $r_j = r_k$ with $0 \leq j < k \leq p$. Put

$$x = s_1 \cdots s_j, \qquad y = s_{j+1} \cdots s_k, \qquad z = s_{k+1} \cdots s_m.$$

Then x takes M from r_0 to r_j, y from r_j to $r_k = r_j$, and z from r_k to $r_m \in F$.



- (1) $|y| = k - j \geq 1$ since $j < k$.
- (2) $|xy| = k \leq p$.
- (3) For any $i \geq 0$, $xy^i z$ takes M from r_0 to r_j, then around the y-loop i times back to r_j, then along z to $r_m \in F$. Hence $xy^i z \in L$.

■

Using it (the test). To show L is not regular: assume it is, let p be the pumping length, pick a suitable $s \in L$ (in terms of p) with $|s| \geq p$, show that *every* split satisfying (1) and (2) fails (3) for some i, and conclude. Example: $\{0^n 1^n\}$, $s = 0^p 1^p$, $|s| = 2p \geq p$, $y = 0^k$, $xy^2 z \notin L$.

2013–2014 • Q6(b) Source M20 **MARKS 8**

Over $\{(,)\}$, let L be all properly nested parentheses. It includes $()()$, $((()))$, $(()()((()())))$, and ε, but excludes $)($ and $((((.$ Use the pumping lemma to show L is not regular.

SOLUTION

Assume L is regular and let p be its pumping length. Choose $s = (^p)^p \in L$, $|s| = 2p \geq p$. For any split $s = xyz$ with $|y| > 0$, $|xy| \leq p$, we get $y = (^k$ with $k \geq 1$. Pump $i = 2$: $xy^2z = (^{p+k})^p$ has more left than right parentheses, so it is not balanced: $xy^2z \notin L$. Contradiction; L is not regular. ■

2012–2013 • Q8(c) Source M18 **MARKS 6**

Prove that $\{0^n10^n \mid n > 1\}$ is not regular.

SOLUTION

Let $L = \{0^n10^n \mid n > 1\}$. Assume L is regular and let p be its pumping length. Let $m = \max(p, 2)$ (so that $m > 1$) and choose $s = 0^m10^m \in L$, $|s| = 2m + 1 \geq p$. For any split $s = xyz$ with $|y| > 0$ and $|xy| \leq p \leq m$, both x and y lie in the first block, so $y = 0^k$ with $k \geq 1$. Pump $i = 2$: $xy^2z = 0^{m+k}10^m$. Its single 1 separates blocks of lengths $m + k \neq m$, so $xy^2z \notin L$. This contradicts the PL; L is not regular. ■

2011–2012 • Q6(b) Source M15 **MARKS 10**

Prove that the following are not regular: (i) $\{www \mid w \in \{0,1\}^*\}$; (ii) $\{1^{n^2} \mid n \geq 0\}$.

SOLUTION

(i) Assume the language is regular and let p be its pumping length. Choose $s = 0^p1\,0^p1\,0^p1 \in L$ (take $w = 0^p1$), $|s| = 3p + 3 \geq p$. Since $|xy| \leq p$, every split has $y = 0^k$, $k \geq 1$, in the first block. Pump down ($i = 0$): $s' = xz = 0^{p-k}1\,0^p1\,0^p1$. If $s' = uuu$, each u contains exactly one 1 (there are three 1's) and u ends with 1 (since s' does), so u is the first segment $0^{p-k}1$ and also the second segment 0^p1 — impossible since $k \geq 1$. So $xz \notin L$, a contradiction; the language is not regular.
(ii) Assume the language is regular and let p be its pumping length. Choose $s = 1^{p^2} \in L$, $|s| = p^2 \geq p$. Every split has $y = 1^k$, $1 \leq k \leq p$. Pump $i = 2$: $p^2 < |xy^2z| = p^2 + k \leq p^2 + p < (p+1)^2$, so the length is not a perfect square and $xy^2z \notin L$. Contradiction; not regular.

2010–2011 • Q6 Source M11 **MARKS 10**

Using the pumping lemma, determine whether $L = \{0^n1^m0^n \mid m, n \geq 0\}$ is regular.

SOLUTION

Assume L is regular and let p be its pumping length. Choose $s = 0^p1\,0^p \in L$ ($n = p$, $m = 1$), $|s| = 2p + 1 \geq p$. For any split $s = xyz$ with $|y| > 0$, $|xy| \leq p$ gives $y = 0^k$, $k \geq 1$, inside the first block. Pump $i = 2$: $xy^2z = 0^{p+k}10^p$: the 0-blocks around the single 1 differ, and no other split into $0^n1^m0^n$ exists (the 1-block is unique), so $xy^2z \notin L$. This contradicts the PL.

Answer. L is not regular.

2008–2009 • Q5(b) Source M07 **MARKS 8**

Prove that $\{0^n 1 0^n \mid n \geq 1\}$ is not regular.

SOLUTION

Same argument as 2012–2013 Q8(c) above; the only difference is the lower bound $n \geq 1$, so the adjustment $m = \max(p, 2)$ is not needed. Assume $L = \{0^n 1 0^n \mid n \geq 1\}$ is regular and let p be its pumping length. Choose $s = 0^p 1 0^p$; since $p \geq 1$, $s \in L$, and $|s| = 2p + 1 \geq p$. Every split with $|y| > 0$, $|xy| \leq p$ has $y = 0^k$, $k \geq 1$, inside the first block. Pump $i = 2$: $xy^2z = 0^{p+k} 1 0^p \notin L$ (unequal blocks around the single 1). Contradiction; L is not regular. ■

2006–2007 • Q3(a) Source M01 **MARKS 10**

Using the pumping lemma, show that the language of all strings with equal numbers of 0's and 1's, in any order, is not regular.

SOLUTION

Assume $L = \{w \mid n_0(w) = n_1(w)\}$ is regular and let p be its pumping length. Choose $s = 0^p 1^p \in L$, $|s| = 2p \geq p$. Any split $s = xyz$ with $|y| > 0$, $|xy| \leq p$ has $y = 0^k$ with $k \geq 1$, and $xy^2z = 0^{p+k} 1^p$ has more 0's than 1's: $\notin L$. Contradiction; L is not regular. ■
(Alternative: if L were regular, so would be $L \cap 0^*1^* = \{0^n 1^n\}$, which is not.)

07

CFG construction, derivations and parse trees

NOTE Grammars marked "verified" were checked by generating every string up to length 7–9 and comparing with the definition of the language.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q3(c) Source N02 **MARKS 7**

Why are CFGs called 'context-free'? Explain with examples.

2021–2022 • Q1(e) Source M45 **MARKS 3**

Why are context-free grammars called 'context-free'?

SOLUTION

Every production of a CFG has the form $A \to \alpha$ with a *single variable* on the left. Whenever A occurs in a sentential form it may be replaced by α *regardless of the symbols around it* — its context. E.g. with $S \to aSb \mid \varepsilon$, the S in $aaSbb$ is rewritten exactly as the S in S; the surrounding $aa \ldots bb$ is irrelevant.

Contrast with *context-sensitive* grammars, whose rules look like $\alpha A \beta \to \alpha \gamma \beta$: A may be rewritten only when surrounded by α and β. Example rule $bB \to bb$ ("B becomes b only after a b"), used in the grammar for $\{a^n b^n c^n\}$, which no CFG can generate.

Because replacements are independent of context, each variable derives its substring independently — which is exactly why derivations can be drawn as parse *trees*.

2022–2023 • Q5(i–ii) Source M52 **MARKS (i) 8; (ii) 7**

Give CFGs for (i) balanced parentheses over $\{(,)\}$; examples $(()())$, $((()))$, ε are included, but $((())$, $()))$, $)()($ are excluded; (ii) strings with at least two 1's over $\{0,1\}$.

SOLUTION

(i) $S \to (S)S \mid \varepsilon$. A non-empty balanced string starts with "(" whose matching ")" encloses a balanced string, followed by another balanced string. E.g. $(()())$: $S \Rightarrow (S)S \Rightarrow (S) \Rightarrow ((S)S) \Rightarrow (()S) \Rightarrow (()(S)S) \Rightarrow (()()S) \Rightarrow (()())$.

(ii) $S \to A1A1A, \quad A \to 0A \mid 1A \mid \varepsilon$. ($A$ generates any string, so the two explicit 1's guarantee at least two.)

2021–2022 • Q2(d) Source M45 **MARKS 15**

Design a CFG for $\{a^i b^j c^k \mid i \neq j+k,\ i,j,k \geq 0\}$. Explain the design idea before giving rules; rules alone are insufficient.

SOLUTION

Idea. When $i = j + k$, each c can be matched with an a from the outside in, and then each b with an a: $a^k\,(a^j b^j)\,c^k$. $i \neq j + k$ means either some a's are left unmatched ($i > j + k$) or some b's/c's are left unmatched ($i < j + k$). Generate the matched pairs with recursive rules and force at least one unmatched symbol in the middle.

- $i > j + k$: $a^k\,a^j\,(a^+)\,b^j\,c^k$.
- $i < j + k$, case $i \leq k$: $a^i\,(b^j c^{k-i})\,c^i$ where the middle b^*c^* is non-empty and contains no a's.
- $i < j + k$, case $i > k$: $a^k\,(a^m b^{m+t})\,c^k$ with $t \geq 1$ extra b's.

Grammar (verified):

$$\begin{array}{lll} S & \to & P \mid Q \\ P & \to & aPc \mid T \qquad T \to aTb \mid A \qquad A \to aA \mid a \qquad (i > j + k) \\ Q & \to & aQc \mid R \mid BC \qquad R \to aRb \mid bB \qquad (i < j + k) \\ B & \to & bB \mid \varepsilon \qquad C \to cC \mid c \end{array}$$

P wraps $a \ldots c$ pairs around T, which wraps $a \ldots b$ pairs around at least one extra a. Q wraps $a \ldots c$ pairs around either R (pairs $a \ldots b$ around at least one extra b) or $BC = b^*c^+$ (extra c's, with any b's). Example: abc ($1 < 2$): $S \Rightarrow Q \Rightarrow aQc \Rightarrow aRc \Rightarrow abBc \Rightarrow abc$.

2021–2022 • Q4(c) Source M46 MARKS 3 + 6 = 9

For $\{a^i b^j c^k \mid i = j \text{ or } j = k;\ i, j, k \geq 0\}$, A. B. Sloppy proposes: (i) $S \to aSb \mid bSc \mid aS \mid Sc \mid \varepsilon$. (ii) $S_0 \to S_1 \mid S_3$; $S_1 \to aS_1bS_2c \mid \varepsilon$; $S_2 \to S_2c \mid \varepsilon$; $S_3 \to aS_4bS_3c \mid \varepsilon$; $S_4 \to aS_4 \mid \varepsilon$. Do these solutions have flaws? Explain clearly.

SOLUTION

(i) Flawed. It over-generates in two ways. Order: $S \Rightarrow bSc \Rightarrow baSbc \Rightarrow babc$, not even of the form $a^*b^*c^*$. Counts: $S \Rightarrow aS \Rightarrow aSc \Rightarrow ac$ has $i = 1, j = 0, k = 1$, so $i \neq j$ and $j \neq k$.
(ii) Flawed. $S_1 \to aS_1bS_2c$ puts c's *between* b's: $S_1 \Rightarrow aS_1bS_2c \Rightarrow a(abc)bc = aabcbc$ (taking the inner $S_1 \Rightarrow abc$, $S_2 \Rightarrow \varepsilon$), not of the form $a^*b^*c^*$. S_3 has the same problem (and S_4 adds a's inside). Neither branch keeps the blocks in order.
Correct grammar (verified):

$$S \to XC \mid AY, \quad X \to aXb \mid \varepsilon, \quad C \to cC \mid \varepsilon, \quad A \to aA \mid \varepsilon, \quad Y \to bYc \mid \varepsilon.$$

XC gives $a^n b^n c^*$ ($i = j$); AY gives $a^* b^n c^n$ ($j = k$).

2019–2020 • Q5(i–ii) Source M41 MARKS (i) 7; (ii) 8

Give CFGs for (i) balanced parentheses over $\{(,)\}$, including $(()())$, $((()))$, and ε, excluding $(((()$, $()))$, and $)()($; (ii) binary strings with equal numbers of 0's and 1's.

SOLUTION

(i) $S \to (S)S \mid \varepsilon$ (same as 2022–2023 Q5(i) above). A non-empty balanced string starts with "(" whose matching ")" encloses a balanced string, followed by another balanced string.
(ii) (verified)

$$S \to 0S1S \mid 1S0S \mid \varepsilon.$$

Soundness. Every non-ε rule introduces one 0 and one 1, so every generated string has equally many 0's and 1's.
Completeness. Let $w \neq \varepsilon$ have equally many 0's and 1's, and suppose it starts with 0. Scan from left to right until the first point where the prefix has equally many 0's and 1's; that point is reached at a 1. Thus $w = 0\,x\,1\,y$ where x and y both have equally many 0's and 1's (and are shorter than w), so by induction $S \Rightarrow^* x$ and $S \Rightarrow^* y$, and $S \Rightarrow 0S1S \Rightarrow^* 0x1y = w$. If w starts with 1, use $S \to 1S0S$ symmetrically.

January 2020 Term • Q5(a–b) Source M38 — MARKS (a) 7; (b) 8

Give CFGs over $\{0,1\}$ for (a) strings with at least one 0 and at least one 1; (b) strings with equal numbers of 0's and 1's.

SOLUTION

(a) (verified) $S \to A0A1A \mid A1A0A, \quad A \to 0A \mid 1A \mid \varepsilon$.
A generates any binary string; the two explicit symbols guarantee a 0 and a 1, the 0 coming either before ($A0A1A$) or after ($A1A0A$) the 1.
(b) Same grammar as 2019–2020 Q5(ii) above: $S \to 0S1S \mid 1S0S \mid \varepsilon$ (see there for why it generates exactly these strings).

2017–2018 • Q5(a–b) Source M34 — MARKS (a) 7; (b) 8

Provide CFGs over $\{0,1\}$ for (a) equal numbers of 0's and 1's; (b) at least three 1's.

SOLUTION

(a) Same grammar as 2019–2020 Q5(ii) above: $S \to 0S1S \mid 1S0S \mid \varepsilon$.
(b) (verified) $S \to A1A1A1A, \quad A \to 0A \mid 1A \mid \varepsilon$. ($A$ generates any string, so the three explicit 1's guarantee at least three.)

2016–2017 • Q5(a) Source M30 — MARKS 5 × 3 = 15

Give only CFG production rules over $\{0,1\}$ for (i) odd-length strings; (ii) strings with at least three 1's; (iii) odd-length strings with middle symbol 0.

SOLUTION

(i) $S \to XXS \mid X, \quad X \to 0 \mid 1$.

(ii) $S \to A1A1A1A, \quad A \to 0A \mid 1A \mid \varepsilon$.

(iii) $S \to XSX \mid 0, \quad X \to 0 \mid 1$.

2016–2017 • Q8(b) Source M31 — MARKS 10

Give only CFG production rules for $\{w \in \{0,1\}^* \mid w = w^R \text{ and } |w| \text{ is divisible by } 4\}$.

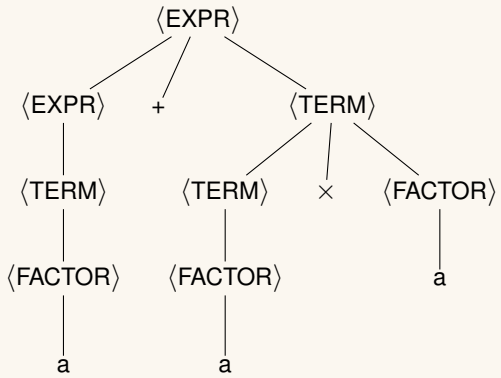
SOLUTION

Such a palindrome is uu^R with $|u|$ even, so peel two symbols from each end at a time:

$$S \to 00S00 \mid 01S10 \mid 10S01 \mid 11S11 \mid \varepsilon.$$

2016–2017 • Q8(c) Source M31 **MARKS 10**

Give only CFG production rules that generate the shown parse tree.



SOLUTION

Read each interior node with its children:

$$\begin{aligned} \langle \text{EXPR} \rangle &\to \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle &\to \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\ \langle \text{FACTOR} \rangle &\to a \ \ (\mid (\langle \text{EXPR} \rangle) \text{ optionally}) \end{aligned}$$

2015–2016 • Q4(a) Source M25–M26 **MARKS $7\frac{1}{3}$**

For G with start R and rules $R \to XRX \mid S$; $S \to aTb \mid bTa$; $T \to XTX \mid X \mid \varepsilon$; $X \to a \mid b$: (i) give the variables; (ii) give the terminals; (iii) give the start variable; (iv) give three strings in $L(G)$; (v) give three strings outside $L(G)$; (vi) describe $L(G)$; (vii) True or False: S1: $T \Rightarrow aba$; S2: $T \Rightarrow^* aba$; S3: $T \Rightarrow T$; S4: $T \Rightarrow^* T$; S5: $XXX \Rightarrow^* aba$; S6: $X \Rightarrow^* aba$; S7: $T \Rightarrow^* XX$; S8: $T \Rightarrow^* XXX$; S9: $S \Rightarrow^* \varepsilon$.

SOLUTION

(i) $\{R, S, T, X\}$. (ii) $\{a, b\}$. (iii) R. (iv) ab, ba, aab. (v) ε, a, aba.
(vi) T generates every string over $\{a, b\}$. S produces a string whose first and last symbols differ; R then adds equally many arbitrary symbols on both sides. So a string is generated iff some pair of positions symmetric about the centre holds different symbols, i.e.

Answer. $L(G)$ = all strings over $\{a, b\}$ that are **not palindromes**.

(vii)

S1	False	$\Rightarrow$ is one step; T's one-step results are XTX, X, ε.
S2	True	$T \Rightarrow XTX \Rightarrow XXX \Rightarrow^* aba$.
S3	False	no rule $T \to T$.
S4	True	$\Rightarrow^*$ allows zero steps.
S5	True	$X \Rightarrow a$, $X \Rightarrow b$, $X \Rightarrow a$.
S6	False	X derives only a or b.
S7	True	$T \Rightarrow XTX \Rightarrow XX$ (using $T \to \varepsilon$).
S8	True	$T \Rightarrow XTX \Rightarrow XXX$ (using $T \to X$).
S9	False	every S-derivation contains $a \ldots b$ or $b \ldots a$.

2015–2016 • Q4(b) Source M26 **MARKS 10**

Find CFGs for (i) binary strings containing at least three 1's; (ii) odd-length binary strings with middle 0; (iii) binary strings with more 0's than 1's; (iv) $\{w\#x \mid w^R \text{ is a substring of } x,\ w, x \in \{0,1\}^*\}$.

SOLUTION

(i) $S \to A1A1A1A$, $A \to 0A \mid 1A \mid \varepsilon$.

(ii) $S \to XSX \mid 0$, $X \to 0 \mid 1$.

(iii) (verified) $S \to T0T,\ T \to TT \mid 0T1 \mid 1T0 \mid 0 \mid \varepsilon$.
T generates exactly the strings with #0 $\geq$ #1 (every rule keeps this). If #0 $>$ #1, cut w after the *last* prefix with balance 0; the next symbol must be a 0 and the rest has #0 $\geq$ #1, so $w \in T\,0\,T$.

(iv) (verified) $S \to TV,\ T \to 0T0 \mid 1T1 \mid \#V,\ V \to 0V \mid 1V \mid \varepsilon$.
T generates $w\#u\,w^R$ (each outer pair puts a symbol of w on the left and the same symbol on the right, reversed order); V appends the remaining part of x. So $x = u\,w^R\,v$.

2014–2015 • Q5(a) Source M23 **MARKS 7**

What language is generated by $S \to abScB \mid \varepsilon$; $B \to bB \mid b$?

SOLUTION

Applying $S \to abScB$ n times and then $S \to \varepsilon$: $S \Rightarrow^* (ab)^n(cB)^n$ (the cB's pile up on the right in the same order). Each B derives b^m, $m \geq 1$.

Answer. $L = \{(ab)^n\, cb^{m_1}\, cb^{m_2} \cdots cb^{m_n} \mid n \geq 0,\ m_1, \ldots, m_n \geq 1\}$ — n copies of ab, followed by exactly n blocks each consisting of a c and at least one b.

E.g. ε, $abcb$, $ababcbbcb$.

2014–2015 • Q5(b) Source M23 **MARKS 7**

Construct CFGs over $\{0,1\}$ for (i) strings starting and ending with the same symbol; (ii) odd-length strings with middle symbol 0.

SOLUTION

(i) $S \to 0A0 \mid 1A1 \mid 0 \mid 1$, $A \to 0A \mid 1A \mid \varepsilon$.

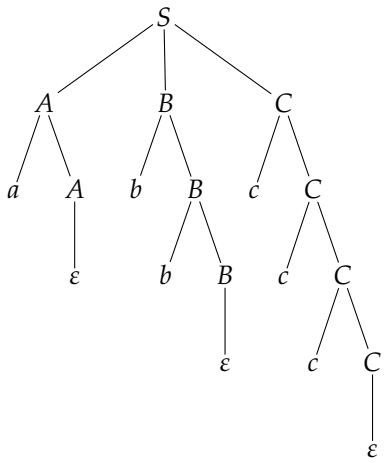
(ii) $S \to XSX \mid 0$, $X \to 0 \mid 1$.

2013–2014 • Q7(b) Source M21 **MARKS $8\frac{1}{3}$**

Construct a CFG recognizing $\{a^i b^j c^k \mid i, j, k \geq 0\}$. Draw a parse tree for ab^2c^3.

SOLUTION

$S \to ABC$, $A \to aA \mid \varepsilon$, $B \to bB \mid \varepsilon$, $C \to cC \mid \varepsilon$. Parse tree of $abbccc$:



REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q7(a) Source M18 **MARKS 8**

Design a CFG for $0^*1(0+1)^*$. Give leftmost and rightmost derivations of 00101.

2008–2009 • Q8(b) Source M07 **MARKS 9**

Design a CFG G for the regular expression $0^*1(0+1)^*$. Give leftmost and rightmost derivations of 00101 using G.

SOLUTION

$$S \to A1B, \qquad A \to 0A \mid \varepsilon, \qquad B \to 0B \mid 1B \mid \varepsilon.$$

Leftmost: $S \Rightarrow A1B \Rightarrow 0A1B \Rightarrow 00A1B \Rightarrow 001B \Rightarrow 0010B \Rightarrow 00101B \Rightarrow 00101$.
Rightmost: $S \Rightarrow A1B \Rightarrow A10B \Rightarrow A101B \Rightarrow A101 \Rightarrow 0A101 \Rightarrow 00A101 \Rightarrow 00101$.

2011–2012 • Q8(b) Source M16 **MARKS 10**

Design CFGs for (i) $\{0^n1^n \mid n \geq 0\} \cup \{1^n0^n \mid n \geq 0\}$; (ii) binary strings containing at least three 1's.

SOLUTION

(i) $S \to A \mid B$, $A \to 0A1 \mid \varepsilon$, $B \to 1B0 \mid \varepsilon$.

(ii) $S \to A1A1A1A$, $A \to 0A \mid 1A \mid \varepsilon$.

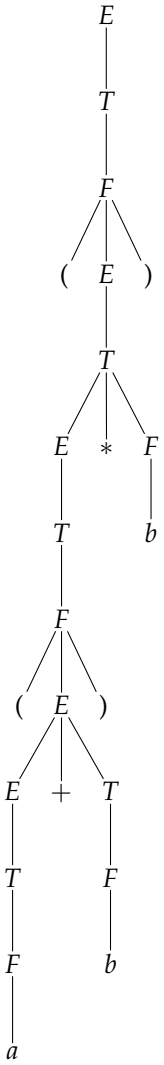
2011–2012 • Q8(c) Source M16 **MARKS $7\frac{1}{3}$**

For the CFG $(\{E,T,F\},\{a,b,+,*,(,)\},R,E)$, with rules $E \to E+T \mid T$; $T \to E*F \mid F$; $F \to (E) \mid a \mid b$, show a derivation and parse tree for $((a+b)*b)$.

SOLUTION

(Using $T \to E*F$ as printed.)

$$\begin{aligned} E &\Rightarrow T \Rightarrow F \Rightarrow (E) \Rightarrow (T) \Rightarrow (E*F) \Rightarrow (T*F) \Rightarrow (F*F) \Rightarrow ((E)*F) \\ &\Rightarrow ((E+T)*F) \Rightarrow ((T+T)*F) \Rightarrow ((F+T)*F) \Rightarrow ((a+T)*F) \\ &\Rightarrow ((a+F)*F) \Rightarrow ((a+b)*F) \Rightarrow ((a+b)*b). \end{aligned}$$



2010–2011 • Q7 Source M12 **MARKS 10**

For $G=(V,\Sigma,P,S)$ with $V=\{a,b,S,A\}$, $\Sigma=\{a,b\}$, and $P=\{S \to AA,\ A \to AAA,\ A \to a,\ A \to bA,\ A \to Ab\}$, explain a derivation of $b^m a b^n a b^p$ for arbitrary $m,n,p>0$.

SOLUTION

Split the target as $(b^m a b^n)(a b^p)$ and derive each half from one A.

- First A: apply $A \to bA$ m times, then $A \to Ab$ n times, then $A \to a$: $A \Rightarrow^m b^m A \Rightarrow^n b^m A b^n \Rightarrow b^m a b^n$.
- Second A: apply $A \to Ab$ p times, then $A \to a$: $A \Rightarrow^p A b^p \Rightarrow a b^p$.

Leftmost derivation:

$$S \Rightarrow AA \overset{m}{\Rightarrow} b^m AA \overset{n}{\Rightarrow} b^m A b^n A \Rightarrow b^m a b^n A \overset{p}{\Rightarrow} b^m a b^n A b^p \Rightarrow b^m a b^n a b^p,$$

a total of $m + n + p + 3$ steps. (The rule $A \to AAA$ is not needed.)

2010–2011 • Q8 Source M12 **MARKS 10**

For $\Sigma = \{a, b, (,), \cup, *, \emptyset\}$, use the basic and inductive rules for regular expressions to construct a CFG generating exactly the strings in Σ^* that are regular expressions over $\{a, b\}$.

SOLUTION

Basic rules: a, b, $\emptyset$ are regular expressions (ε is not in Σ; it is expressible as $\emptyset^*$). Inductive rules: if R_1, R_2 are regular expressions, so are $(R_1 \cup R_2)$, $(R_1 R_2)$ and (R_1^*). Translating rule by rule (fully parenthesised syntax):

$$R \to a \mid b \mid \emptyset \mid (R \cup R) \mid (RR) \mid (R^*).$$

If the usual precedence conventions (star > concatenation > union) with optional parentheses are intended:

$$R \to R \cup C \mid C, \qquad C \to CK \mid K, \qquad K \to K^* \mid (R) \mid a \mid b \mid \emptyset.$$

2010–2011 • Q9(a–d) Source M12 **MARKS 10 (shared)**

Give CFGs for: (a) $\{0^n 1^m 0^m 1^n \mid n, m \geq 0\}$; (b) $\{a^n b^m c^k \mid n, m, k \geq 0 \text{ and } n = m + k\}$; (c) $\{a^n b^m c^k \mid n, m, k \geq 0 \text{ and } n = 2m + 3k\}$; (d) $\{a^n b^m \mid 0 \geq n \geq m \geq 2n\}$.

SOLUTION

(a) $S \to 0S1 \mid A$, $A \to 1A0 \mid \varepsilon$.

(b) $a^{m+k} b^m c^k = a^k (a^m b^m) c^k$: $S \to aSc \mid T$, $T \to aTb \mid \varepsilon$.

(c) $a^{2m+3k} b^m c^k = a^{3k} (a^{2m} b^m) c^k$: $S \to aaaSc \mid T$, $T \to aaTb \mid \varepsilon$.

(d) As printed, $0 \geq n \geq m \geq 2n$ forces $n = m = 0$, so $L = \{\varepsilon\}$ and $S \to \varepsilon$. The intended condition is surely $0 \leq n \leq m \leq 2n$: each a pairs with one or two b's:

Answer. $S \to aSb \mid aSbb \mid \varepsilon$ (verified for $n \leq m \leq 2n$).

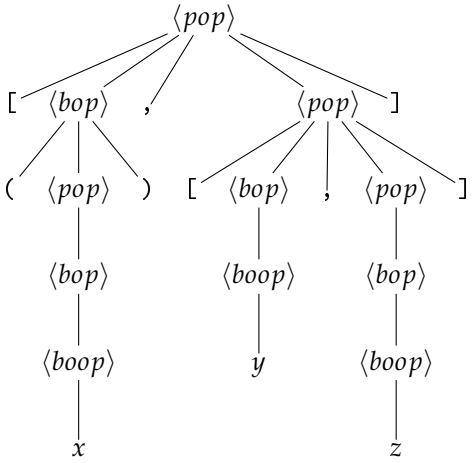
2009–2010 • Q6 Source M08–M09 **MARKS 10**

For the grammar below: (a) identify terminal and nonterminal symbols; (b) identify the start symbol and explain why; (c) draw a parse tree for the sentence printed in the source. $\langle pop \rangle \to [\langle bop \rangle, \langle pop \rangle] \mid \langle bop \rangle$; $\langle bop \rangle \to \langle boop \rangle \mid (\langle pop \rangle)$; $\langle boop \rangle \to x \mid y \mid z$.

SOLUTION

(a) Terminals: [ ] , ( ) x y z. Nonterminals: $\langle pop \rangle$, $\langle bop \rangle$, $\langle boop \rangle$.
(b) $\langle pop \rangle$. It is the most general category: every $\langle boop \rangle$ is a $\langle bop \rangle$ and every $\langle bop \rangle$ is a $\langle pop \rangle$, and only $\langle pop \rangle$ can produce the outermost list form $[\ldots,\ldots]$. Starting from $\langle bop \rangle$ or $\langle boop \rangle$ would miss sentences such as $[x, y]$.
(c)

NOTE **The sentence in the scan is crossed out. We use the representative sentence $[(x), [y, z]]$.**



2009–2010 • Q7 Source M09 MARKS 10

Give a grammar for palindromes over $\{0, 1\}$. Modify it for alphabet $\{0, 1, a\}$ with length restricted to even lengths.

SOLUTION

All palindromes over $\{0, 1\}$: $P \to 0P0 \mid 1P1 \mid 0 \mid 1 \mid \varepsilon$.
Even-length palindromes over $\{0, 1, a\}$ (drop the odd-length centres 0, 1, a): $P \to 0P0 \mid 1P1 \mid aPa \mid \varepsilon$.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2009–2010 • Q8 Source M09 MARKS 10

With examples, discuss recursive inference versus derivation for context-free grammars.

2007–2008 • Q4(b) Source M04 MARKS 8

With examples, discuss the differences and similarities between recursive inference and derivation for context-free grammars.

SOLUTION

Both are ways to decide that a string w is in the language of a variable A.
Recursive inference (bottom-up). Start from the bodies: if $A \to X_1 X_2 \cdots X_k$ and we already know

$w_i \in L(X_i)$ for each variable X_i (terminals stand for themselves), we *infer* $w_1 \cdots w_k \in L(A)$. Facts are built from small strings to large.

Derivation (top-down). Start from A and repeatedly replace a variable by one of its bodies: $A \Rightarrow \alpha_1 \Rightarrow \cdots \Rightarrow w$.

Example. $E \to I \mid E+E \mid E*E \mid (E)$, $I \to a \mid b$, string $a*(a+b)$.

	Recursive inference	Derivation
1	$a \in L(I)$ (rule $I \to a$)	$E \Rightarrow E*E$
2	$b \in L(I)$	$\Rightarrow I*E$
3	$a, b \in L(E)$ $(E \to I)$	$\Rightarrow a*E$
4	$a+b \in L(E)$ $(E \to E+E)$	$\Rightarrow a*(E)$
5	$(a+b) \in L(E)$ $(E \to (E))$	$\Rightarrow a*(E+E) \Rightarrow^* a*(a+b)$
6	$a*(a+b) \in L(E)$ $(E \to E*E)$	

Similarities. They establish the same facts: $w \in L(A)$ by recursive inference $\iff$ $A \Rightarrow^* w$ $\iff$ $A \Rightarrow^*_{lm} w$ $\iff$ $A \Rightarrow^*_{rm} w$ $\iff$ there is a parse tree with root A and yield w. Both use the same productions; one is the other read backwards.

Differences. Direction (bottom-up vs top-down); inference manipulates *facts about terminal strings*, derivation manipulates *sentential forms* containing variables; inference corresponds to how parsers like CYK work, derivation to how strings are generated; a derivation has an order (leftmost/rightmost), inference only a dependency structure (a tree).

2009–2010 • Q9 Source M09 MARKS 10

Give a CFG G for $L = \{w \in \{a,b\}^* \mid w \text{ has equal numbers of } a\text{'s and } b\text{'s}\}$. Show a leftmost derivation of $aabbab$.

SOLUTION

The grammar of 2019–2020 Q5(ii) above with $0 \mapsto a$, $1 \mapsto b$ (same argument):

$$G: \quad S \to aSbS \mid bSaS \mid \varepsilon.$$

Leftmost derivation of $aabbab$:

$$S \Rightarrow aSbS \Rightarrow aaSbSbS \Rightarrow aabSbS \Rightarrow aabbS \Rightarrow aabbaSbS \Rightarrow aabbabS \Rightarrow aabbab.$$

2007–2008 • Q1(a) Source M04 MARKS 10

Explain the properties of parse trees with examples.

SOLUTION

For a CFG $G = (V, \Sigma, R, S)$ a parse tree satisfies:

(i) The root is labelled by the start variable S.

(ii) Every internal node is labelled by a variable.

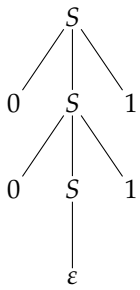
(iii) Every leaf is labelled by a terminal or by ε.

(iv) If an internal node labelled A has children labelled, from left to right, $X_1, X_2, \ldots, X_k$, then $A \to X_1 X_2 \cdots X_k$ is a production in R. Special case: if the node uses $A \to \varepsilon$, it has a single child labelled ε.

Further properties:

- The **yield** is obtained by reading the leaves from left to right, ignoring any ε-leaf; it is a string of $L(G)$.
- For every variable A and terminal string w, the following are equivalent: $A \Rightarrow^* w$; $A \Rightarrow^*_{lm} w$; $A \Rightarrow^*_{rm} w$; there is a parse tree with root A and yield w. Each parse tree corresponds to exactly one leftmost (and one rightmost) derivation, so a string with two different parse trees shows the grammar is ambiguous.
- A sufficiently long yield needs a sufficiently tall tree; since there are finitely many variables, some variable then repeats on a root-to-leaf path — the idea behind the CFL pumping lemma.

Example. $S \to 0S1 \mid \varepsilon$, string 0011:



Interior nodes are S; leaves $0, 0, \varepsilon, 1, 1$; yield 0011; it corresponds to the unique derivation $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$.

2007–2008 • Q4(a) Source M04 MARKS 10

Describe a grammar for the palindromes over $\{0,1\}$. Prove its validity by induction.

SOLUTION

G_{pal}: $P \to \varepsilon \mid 0 \mid 1 \mid 0P0 \mid 1P1$. **Theorem.** $L(G_{pal}) = \{w \in \{0,1\}^* \mid w = w^R\}$.
($\supseteq$) Every palindrome is generated, by induction on $|w|$. Basis: $|w| \leq 1$: $w \in \{\varepsilon, 0, 1\}$, generated in one step. Induction: $|w| \geq 2$ and $w = w^R$; then w begins and ends with the same symbol, $w = 0x0$ or $w = 1x1$, and $x = x^R$ (reverse both sides). By the hypothesis $P \Rightarrow^* x$, so $P \Rightarrow 0P0 \Rightarrow^* 0x0 = w$ (similarly for 1).
($\subseteq$) Everything generated is a palindrome, by induction on the number of derivation steps. Basis: one step yields ε, 0 or 1 — palindromes. Induction: an $(n+1)$-step derivation starts $P \Rightarrow 0P0$ (or $1P1$) and then $P \Rightarrow^n x$; by hypothesis $x = x^R$, so $(0x0)^R = 0x^R0 = 0x0$. ■

2006–2007 • Q5 Source M02 MARKS 10

Define a context-free grammar. Design a CFG for $\{a^m b^n \mid m > n,\ n \geq 1\}$.

SOLUTION

A context-free grammar is a 4-tuple $G = (V, \Sigma, R, S)$: V a finite set of variables (nonterminals), Σ a finite set of terminals disjoint from V, R a finite set of productions $A \to \alpha$ with $A \in V$, $\alpha \in (V \cup \Sigma)^*$, and $S \in V$ the start variable. $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$.
Grammar (verified): $S \to aSb \mid aTb, \quad T \to aT \mid a$. aTb gives $a^{t+1}b$ with $t \geq 1$ extra a's, and aSb adds matched pairs, so $m = n + t > n \geq 1$.

2006–2007 • Q6 Source M02 MARKS 10

Define a parse tree and its yield. For the grammar with start symbol D, derive (i) `bin_ab00=101,b1,b1;` by leftmost derivation and (ii) `bin_a=10,b=1;` by rightmost derivation. $D \to TL;\quad T \to \texttt{bin_};\quad L \to L,V \mid V;\quad V \to I \mid I{=}N;\quad I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1;\quad N \to 1 \mid N1 \mid N0.$

SOLUTION

Parse tree: a tree whose root is the start symbol, interior nodes are variables, each interior node with its children forms a production, leaves are terminals or ε (an ε-leaf being the only child of its parent). **Yield:** the concatenation of the leaves from left to right, ignoring ε-leaves — the string the tree derives. (See 2007–08 Q1(a) above.)

(i) Leftmost (the semicolon is part of $D \to TL;$):

$$D \Rightarrow TL; \Rightarrow \texttt{bin_}L; \Rightarrow \texttt{bin_}L,V; \Rightarrow \texttt{bin_}L,V,V; \Rightarrow \texttt{bin_}V,V,V;$$
$$\Rightarrow \texttt{bin_}I{=}N,V,V; \Rightarrow \texttt{bin_}I0{=}N,V,V; \Rightarrow \texttt{bin_}I00{=}N,V,V; \Rightarrow \texttt{bin_}Ib00{=}N,V,V;$$
$$\Rightarrow \texttt{bin_}ab00{=}N,V,V; \Rightarrow \texttt{bin_}ab00{=}N1,V,V; \Rightarrow \texttt{bin_}ab00{=}N01,V,V; \Rightarrow \texttt{bin_}ab00{=}101,V,V;$$
$$\Rightarrow \texttt{bin_}ab00{=}101,I,V; \Rightarrow \texttt{bin_}ab00{=}101,I1,V; \Rightarrow \texttt{bin_}ab00{=}101,b1,V;$$
$$\Rightarrow \texttt{bin_}ab00{=}101,b1,I; \Rightarrow \texttt{bin_}ab00{=}101,b1,I1; \Rightarrow \texttt{bin_}ab00{=}101,b1,b1;$$

(ii) Rightmost:

$$D \Rightarrow TL; \Rightarrow TL,V; \Rightarrow TL,I{=}N; \Rightarrow TL,I{=}1; \Rightarrow TL,b{=}1; \Rightarrow TV,b{=}1;$$
$$\Rightarrow TI{=}N,b{=}1; \Rightarrow TI{=}N0,b{=}1; \Rightarrow TI{=}10,b{=}1; \Rightarrow Ta{=}10,b{=}1; \Rightarrow \texttt{bin_}a{=}10,b{=}1;$$

08

Ambiguous grammars and inherent ambiguity

2023–2024 • Q5 Source N03 **MARKS 10 + 5 = 15**

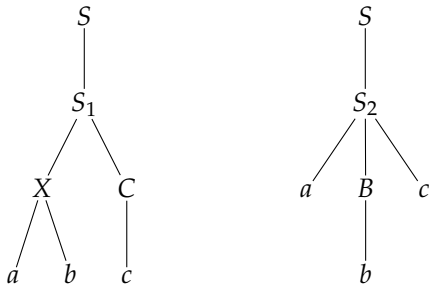
Design a CFG for $\{a^i b^j c^k \mid (i = j \text{ or } i = k),\ i, j, k \geq 1\}$. Show ambiguity using two different parse trees for some string.

SOLUTION

Two sub-grammars, one per condition:

$$\begin{array}{llll} S \to S_1 \mid S_2 & & & \\ S_1 \to XC & X \to aXb \mid ab & C \to cC \mid c & (a^n b^n c^k,\ i = j) \\ S_2 \to aS_2c \mid aBc & B \to bB \mid b & & (a^n b^j c^n,\ i = k) \end{array}$$

A string with $i = j = k$ is produced by both sub-grammars. For abc:



Two distinct parse trees for the same string ⇒ the grammar is ambiguous. (In fact the language is inherently ambiguous: every grammar must generate $a^n b^n c^n$ in two ways for infinitely many n.)

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q6 Source M52 **MARKS 10 + 5 = 15**

Give a CFG for $\{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$. Show ambiguity using two parse trees for some string.

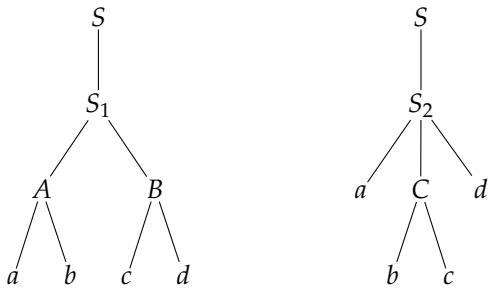
January 2020 Term • Q6 Source M38 **MARKS 10 + 5 = 15**

Design a CFG over $\{a, b, c, d\}$ for $\{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$. Demonstrate ambiguity with two parse trees for some string.

SOLUTION

$$S \to S_1 \mid S_2$$
$$S_1 \to AB \qquad A \to aAb \mid ab \quad B \to cBd \mid cd$$
$$S_2 \to aS_2d \mid aCd \quad C \to bCc \mid bc$$

S_1 generates $a^n b^n c^m d^m$ (two independent matched blocks); S_2 generates $a^n(b^m c^m)d^n$ (nested). Strings with $n = m$, e.g. $abcd$, belong to both:



Hence the grammar is ambiguous. (This language is the textbook example of an *inherently* ambiguous CFL; see 2008–09 Q8(a) below.)

2021–2022 • Q4(d) Source M47 — MARKS 10

For a grammar of an infinite language and string s in it, comment with reasons on ambiguity of both s and the grammar when: (i) s has one parse tree but other strings have multiple trees; (ii) s and every other string have one tree; (iii) s has more than one leftmost derivation.

SOLUTION

A grammar is ambiguous iff *some* string of its language has two or more parse trees (equivalently two leftmost derivations).

(i) s itself is unambiguously derived (one tree), but the **grammar is ambiguous** because some other string has multiple trees. Ambiguity is a property of the grammar witnessed by any single string.

(ii) s is unambiguous and, since *no* string has two trees, the **grammar is unambiguous**.

(iii) Leftmost derivations correspond one-to-one with parse trees, so s has more than one parse tree: s is ambiguously derived and therefore the **grammar is ambiguous**. (The language may or may not be inherently ambiguous — another grammar might be unambiguous.)

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q6 Source M47 — MARKS 6 + 9 = 15

Define ambiguous grammar and inherently ambiguous language. Determine whether this grammar is ambiguous: $S \to AB \mid C$; $A \to aAb \mid ab$; $B \to cB \mid c$; $C \to aCc \mid aDc$; $D \to bD \mid b$.

2019–2020 • Q6 Source M41 — MARKS 5 + 10 = 15

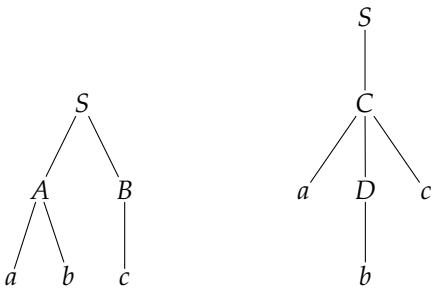
Define ambiguous grammar and inherently ambiguous language. Show ambiguity of the same grammar: $S \to AB \mid C$; $A \to aAb \mid ab$; $B \to cB \mid c$; $C \to aCc \mid aDc$; $D \to bD \mid b$.

SOLUTION

Definitions. A CFG G is *ambiguous* if some $w \in L(G)$ has two (or more) distinct parse trees — equivalently two distinct leftmost derivations. A CFL L is *inherently ambiguous* if *every* CFG generating L is ambiguous.

The grammar. AB generates $a^n b^n c^m$ $(n, m \geq 1)$; C generates $a^n b^m c^n$ $(n, m \geq 1)$. Any $a^n b^n c^n$ is generated both ways. For abc:

$$S \Rightarrow AB \Rightarrow abB \Rightarrow abc \qquad \text{and} \qquad S \Rightarrow C \Rightarrow aDc \Rightarrow abc.$$



Two leftmost derivations / parse trees ⇒ **ambiguous**. (Its language $\{a^n b^n c^m\} \cup \{a^n b^m c^n\}$ is in fact inherently ambiguous.)

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2017–2018 • Q6(a–b) Source M34 MARKS (a) 7; (b) 8

(a) Show ambiguity by giving two parse trees for a string in $E \to E + E \mid E \times E \mid (E) \mid a$. (b) Remove the ambiguity.

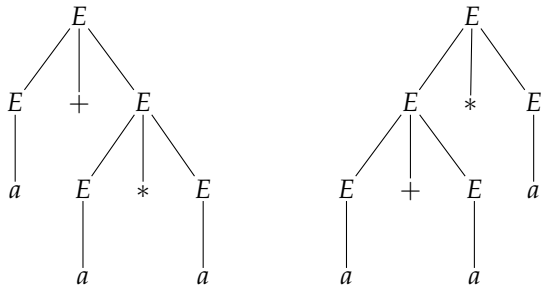
2012–2013 • Q5(a) Source M18 MARKS 8

For $E \to E + E \mid E * E \mid (E) \mid a$, determine whether the grammar is ambiguous. Explain using $a + a * a$.

SOLUTION

SCOPE *2012–2013 Q5(a) asks only whether the grammar is ambiguous, using $a + a * a$ (first paragraph).*

Ambiguous. $a + a * a$ has two parse trees (write $\times$ or $*$ alike):

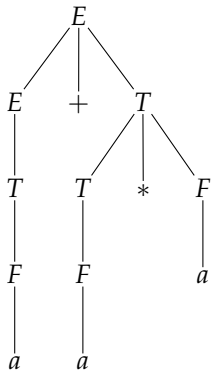


Leftmost derivations: $E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a$ and $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a$. The first tree means $a + (a * a)$, the second $(a + a) * a$ — different meanings.

Sources of ambiguity: no precedence between $+$ and $*$, and no associativity ($a+a+a$ also has two trees).
Removing it. Introduce one variable per precedence level, and make each operator left-associative by putting the recursion on the left:

$$\begin{aligned} E &\to E+T \mid T \\ T &\to T*F \mid F \\ F &\to (E) \mid a \end{aligned}$$

Now $a+a*a$ has the single tree corresponding to $a+(a*a)$:



A $+$ can only appear at the E level, above any $*$, so $*$ binds tighter; and in $E \to E+T$ the right operand cannot contain an unparenthesised $+$, so $a+a+a$ groups as $(a+a)+a$.

2014–2015 • Q6(a) Source M23 MARKS 7

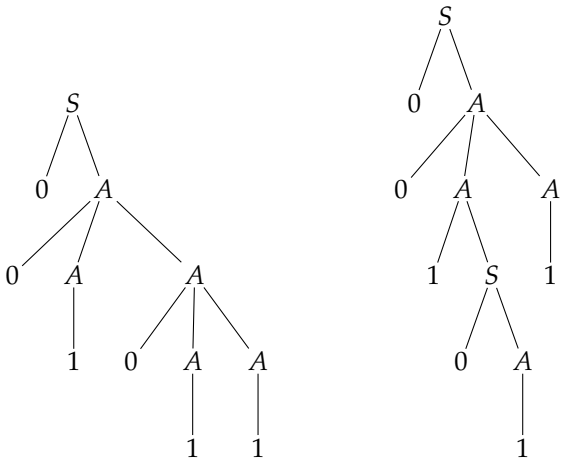
Explain why this CFG is ambiguous: $S \to 0A \mid 1B$; $A \to 0AA \mid 1S \mid 1$; $B \to 1BB \mid 0S \mid 0$.

SOLUTION

(A generates strings with one more 1 than 0's, B one more 0; S generates non-empty strings with equally many 0's and 1's.) The string 001011 has two leftmost derivations (found by exhaustive search; it is the shortest ambiguous string):

$$S \Rightarrow 0A \Rightarrow 00AA \Rightarrow 001A \Rightarrow 0010AA \Rightarrow 00101A \Rightarrow 001011$$
$$S \Rightarrow 0A \Rightarrow 00AA \Rightarrow 001SA \Rightarrow 0010AA \Rightarrow 00101A \Rightarrow 001011$$

In the first, the third symbol comes from $A \to 1$; in the second from $A \to 1S$.



Two parse trees ⇒ ambiguous. The cause: a 1 that "closes" an A can either end it ($A \to 1$) or start a new balanced block ($A \to 1S$), and the grammar does not fix which.

2012–2013 • Q6(a) Source M18 **MARKS 8**

Let $L = \{a^n b^n c^m \mid n \geq 1, m \geq 1\} \cup \{a^n b^m c^m \mid n \geq 1, m \geq 1\}$ be context-free. Explain why all grammars for L are ambiguous.

SOLUTION

L is *inherently ambiguous*: every CFG for it is ambiguous.

Intuition (as on the slides). Strings $a^n b^n c^n$ belong to *both* parts of the union. A grammar for the first part must match a's with b's (structure $(a^n b^n)\, c^m$); a grammar for the second part must match b's with c's (structure $a^n\, (b^m c^m)$). A CFG matches symbols only by wrapping them around a common sub-derivation, so it cannot do both comparisons in one tree; it has to "decide" which pair of counts to compare, and for $a^n b^n c^n$ both decisions succeed — giving two parse trees.

Sketch of the proof. Let G be any CFG for L and p a large constant depending on G; put $N = p + p!$.

(i) Take $w_1 = a^p b^p c^{p+p!} \in L$ and look at a parse tree of w_1. A pumping argument in which the pumped pieces are forced to contain a's gives a variable A with $A \Rightarrow^* a^k A b^k$ ($1 \leq k \leq p$) inside this tree: the pumped strings must stay in L for *every* number of repetitions, and $\#b = \#c$ can hold for at most one of them, so $\#a = \#b$ must be kept — the same number of b's is pumped together with the a's. Since k divides $p!$, repeating this sub-derivation $p!/k$ more times gives a parse tree T_1 of $a^N b^N c^N$ containing an A-node whose subtree yields only a's and b's and can still be pumped by $a^k \ldots b^k$.

(ii) Symmetrically, from $w_2 = a^{p+p!} b^p c^p$ (forcing c's into the pumped pieces) we get a parse tree T_2 of the *same* string $a^N b^N c^N$ containing a B-node with $B \Rightarrow^* b^{k'} B c^{k'}$, whose subtree yields only b's and c's.

(iii) $T_1 \neq T_2$. Otherwise one tree has both nodes. Their yields are either nested or disjoint; the A-yield contains a's (the B-yield does not) and the B-yield contains c's (the A-yield does not), so they are disjoint and can be pumped independently. Pumping both once gives $a^{N+k} b^{N+k+k'} c^{N+k'}$, which has neither $\#a = \#b$ nor $\#b = \#c$ — not in L, a contradiction.

So $a^N b^N c^N$ has two different parse trees in every grammar for L.

NOTE The ordinary pumping lemma is not enough for steps 1 and 2: it does not let us force the pumped pieces into the a's (pumping only c's in w_1 keeps the string in L). The rigorous proof uses *Ogden's lemma* (a pumping lemma in which chosen positions are "marked" and the pumped pieces must contain a marked position; Hopcroft–Motwani–Ullman §5.4.4 and §7.2). This goes beyond the slides, which give only the intuition above.

2011–2012 • Q8(a) Source M16 **MARKS 6**

Formally define a CFG. Define an ambiguous grammar and give an example.

SOLUTION

A CFG is a 4-tuple $G = (V, \Sigma, R, S)$: V a finite set of variables; Σ a finite set of terminals disjoint from V; R a finite set of rules $A \to w$ with $A \in V$, $w \in (V \cup \Sigma)^*$; $S \in V$ the start variable. $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$.
G is *ambiguous* if some string in $L(G)$ has two different parse trees (two leftmost derivations). Example: $E \to E + E \mid E * E \mid a$; the string $a + a * a$ has the trees for $(a + a) * a$ and $a + (a * a)$ shown above.

2008–2009 • Q8(a) Source M07 **MARKS 9**

Let $L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$ be context-free. Explain why all grammars for L are ambiguous.

SOLUTION

This is the slides' standard example of an inherently ambiguous language; the reasoning is the same as in 2012–2013 Q6(a) above.
Intuition. Strings $a^n b^n c^n d^n$ belong to both parts. The first part needs the side-by-side structure $(a^n b^n)(c^m d^m)$ (a's matched with b's, c's with d's), the second the nested structure $a^n (b^m c^m) d^n$ (a's with d's around b's with c's), as in the grammar $S \to AB \mid C$, $A \to aAb \mid ab$, $B \to cBd \mid cd$, $C \to aCd \mid aDd$, $D \to bDc \mid bc$ (2022–2023 Q6 above). For $a^n b^n c^n d^n$ both structures apply, so it gets two parse trees.
Sketch. With $N = p + p!$ as before: from $a^p b^p c^{p+p!} d^{p+p!}$ (pumped pieces forced into the a's) we obtain a tree of $a^N b^N c^N d^N$ with a pumpable node $A \Rightarrow^* a^k A b^k$; from $a^{p+p!} b^p c^p d^{p+p!}$ (pumped pieces forced into the b's) a tree of the same string with a pumpable node $B \Rightarrow^* b^{k'} B c^{k'}$. If the two trees were equal, the two nodes would have disjoint yields (one contains a's, the other c's), and pumping both would give $a^{N+k} b^{N+k+k'} c^{N+k'} d^N \notin L$ (neither $\#a = \#b$ nor $\#a = \#d$). Hence two different parse trees, in every grammar for L.

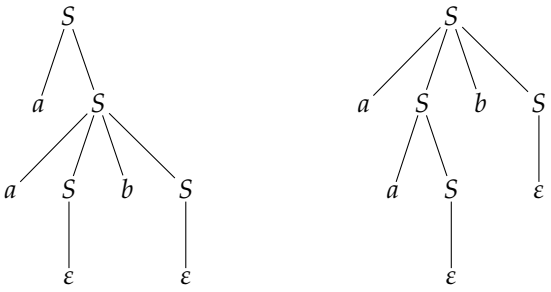
NOTE As in 2012–2013 Q6(a): forcing the pumped pieces into chosen positions needs Ogden's lemma, which goes beyond the slides; the slides give only the intuition.

2008–2009 • Q8(c) Source M07 **MARKS $5\frac{1}{3}$**

For $S \to aS \mid aSbS \mid \varepsilon$, show that aab has two (i) parse trees, (ii) leftmost derivations, and (iii) rightmost derivations.

SOLUTION

The question is which a is matched with the b.



(ii) Leftmost: (1) $S \Rightarrow aS \Rightarrow aaSbS \Rightarrow aabS \Rightarrow aab$; (2) $S \Rightarrow aSbS \Rightarrow aaSbS \Rightarrow aabS \Rightarrow aab$.
(iii) Rightmost: (1) $S \Rightarrow aS \Rightarrow aaSbS \Rightarrow aaSb \Rightarrow aab$; (2) $S \Rightarrow aSbS \Rightarrow aSb \Rightarrow aaSb \Rightarrow aab$.

2007–2008 • Q3(b) Source M04 **MARKS 8**

What is ambiguity in grammars? How do we remove ambiguity? What is inherent ambiguity?

SOLUTION

Ambiguity: a CFG is ambiguous if some string has more than one parse tree (equivalently more than one leftmost or rightmost derivation). It is harmful in compilers because different trees give different meanings, e.g. $a + a * a$ as $(a + a) * a$ or $a + (a * a)$.
Removing it (no general algorithm exists — ambiguity is undecidable — but standard techniques work in practice):

- *Precedence*: one variable per precedence level ($E \to E + T \mid T$, $T \to T * F \mid F$, $F \to (E) \mid a$).
- *Associativity*: put the recursion on the left for left-associative operators (on the right for right-associative ones).
- *Dangling else*: distinguish "matched" and "unmatched" statements so each `else` attaches to the nearest `if`.
- Rewrite to remove redundant ways of generating the same substring (e.g. $S \to SS \mid (S) \mid \varepsilon \to S \to (S)S \mid \varepsilon$).

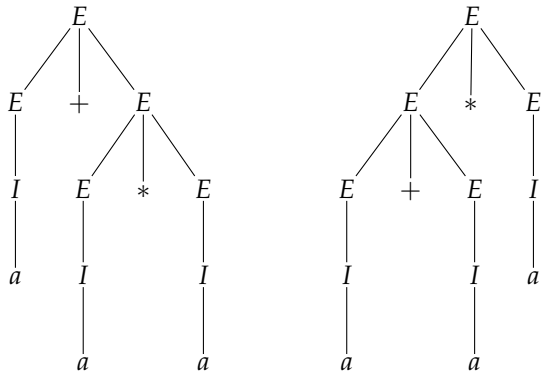
Inherent ambiguity: a context-free *language* is inherently ambiguous if every grammar for it is ambiguous, so ambiguity cannot be removed at all. Example: $\{a^n b^n c^m d^m\} \cup \{a^n b^m c^m d^n\}$ $(n, m \geq 1)$.

2006–2007 • Q7 Source M02 **MARKS 10**

Show that this grammar is ambiguous and remove its ambiguity: $E \to E + E \mid E * E \mid (E) \mid I$; $I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$.

SOLUTION

Ambiguous. Here identifiers are generated by I (a, b, ab, $a01$, …). The string $a + a * a$ has two parse trees:



equivalently two leftmost derivations

$$E \Rightarrow E+E \Rightarrow I+E \Rightarrow a+E \Rightarrow a+E*E \Rightarrow a+I*E \Rightarrow a+a*E \Rightarrow a+a*I \Rightarrow a+a*a,$$
$$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow I+E*E \Rightarrow a+E*E \Rightarrow a+I*E \Rightarrow a+a*E$$
$$\Rightarrow a+a*I \Rightarrow a+a*a,$$

meaning $a+(a*a)$ and $(a+a)*a$ respectively. As in 2017–2018 Q6 above, the causes are the missing precedence between $+$ and $*$ and the missing associativity.
Removing it. Same layering as in 2017–2018 Q6(b) above, with identifiers as the basic factors:

$$\begin{aligned} E &\rightarrow E+T \mid T \\ T &\rightarrow T*F \mid F \\ F &\rightarrow (E) \mid I \\ I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \end{aligned}$$

$*$ is forced below $+$ in every tree and both operators group to the left, so $a+a*a$ now has the single tree for $a+(a*a)$. The identifier part I is unambiguous already: an identifier is built left to right, one symbol per step, in exactly one way.

09

Grammar simplification, CNF and CYK

NOTE **CNF and CYK recap (Atif sir's slides).** ***CNF:*** **every production is $A \to BC$ (A, B, C variables) or $A \to a$ (a a terminal), and there are no useless symbols; every non-empty CFL without ε has such a grammar (if $\varepsilon \in L$, convert for $L - \{\varepsilon\}$ and add $S \to \varepsilon$).** ***Conversion order:*** **(i) eliminate ε-productions; (ii) eliminate unit productions; (iii) eliminate useless symbols (non-generating, then non-reachable); (iv) (a) make all bodies of length ≥ 2 consist only of variables, (b) break bodies of length ≥ 3 into a cascade of two-variable bodies.**
CYK: **for $w = a_1 a_2 \cdots a_n$, X_{ij} ($i \leq j$) is the set of variables A with $A \Rightarrow^* a_i a_{i+1} \cdots a_j$. $X_{ii} = \{A \mid A \to a_i\}$. For $i < j$: $A \in X_{ij}$ iff there are k with $i \leq k < j$ and a production $A \to BC$ with $B \in X_{ik}$, $C \in X_{k+1,j}$. $w \in L(G)$ iff $S \in X_{1n}$. In the tables, as on the slides, the bottom row lists $X_{11}, \ldots, X_{nn}$, the next row $X_{12}, X_{23}, \ldots$, and so on up to X_{1n} at the top; S is shown in bold.**

2023–2024 • Q6(a–b) Source N03 **MARKS (a) 10; (b) 5**

For the CNF grammar below: (a) use CYK to test 0010, showing the table; (b) describe $L(G)$ in simple English. $S \to BZ \mid CY \mid ZZ \mid YY$; $B \to ZA$; $C \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Z \to 0$; $Y \to 1$.

SOLUTION

(a) CYK for 0010

Here $a_1 a_2 a_3 a_4 = 0010$. Bottom row: 0 is produced by A, Z and 1 by A, Y.

$X_{i,i+3}$	$\{A, B, \mathbf{S}\}$			
$X_{i,i+2}$	$\{A, B\}$	$\{A, B, \mathbf{S}\}$		
$X_{i,i+1}$	$\{A, B, \mathbf{S}\}$	$\{A, B\}$	$\{A, C\}$	
X_{ii}	$\{A, Z\}$	$\{A, Z\}$	$\{A, Y\}$	$\{A, Z\}$
	0	0	1	0

Sample entries: X_{12} ("00"): $Z \in X_{11}$, $A \in X_{22}$ give $A \to ZA$ and $B \to ZA$; $Z \in X_{11}$, $Z \in X_{22}$ give $S \to ZZ$; so $X_{12} = \{A, B, S\}$. X_{14}: the split $k = 3$ ("001" | "0") has $B \in X_{13}$, $Z \in X_{44}$, so $S \to BZ$ puts S in X_{14}. Since $S \in X_{14}$: $0010 \in L(G)$ ✓ (it starts and ends with 0).

(b) The language

$A \to ZA \mid YA \mid 0 \mid 1$ makes A generate every non-empty binary string. Hence $B = ZA$ generates $0\{0,1\}^+$ and $C = YA$ generates $1\{0,1\}^+$. So

$$S \to BZ:\ 0\,x\,0\ (|x| \geq 1), \quad S \to ZZ:\ 00, \quad S \to CY:\ 1\,x\,1, \quad S \to YY:\ 11.$$

Answer. $L(G)$ = binary strings of length at least 2 whose first and last symbols are the same.

2022–2023 • Q7 Source M52 **MARKS 15**

For this CNF grammar, use CYK to determine whether $0100 \in L(G)$, showing the table: $S \to AB \mid AY \mid YA \mid 1$; $A \to ZA \mid YA \mid 0 \mid 1$; $B \to YA$; $Z \to 0$; $Y \to 1$.

SOLUTION

(A = all non-empty strings, $B = 1\{0,1\}^+$, so $L(G)$ = all strings containing at least one 1.) Here $a_1 \cdots a_4 = 0100$.

$X_{i,i+3}$	$\{A, \mathbf{S}\}$			
$X_{i,i+2}$	$\{A, \mathbf{S}\}$	$\{A, B, \mathbf{S}\}$		
$X_{i,i+1}$	$\{A, \mathbf{S}\}$	$\{A, B, \mathbf{S}\}$	$\{A\}$	
X_{ii}	$\{A, Z\}$	$\{A, \mathbf{S}, Y\}$	$\{A, Z\}$	$\{A, Z\}$
	0	1	0	0

Top cell $X_{14} = \{A, S\}$ contains S (e.g. the split $k = 1$, "0" | "100": $A \in X_{11}$, $B \in X_{24}$, rule $S \to AB$). $0100 \in L(G)$.

2021–2022 • Q5 Source M47 MARKS 15

Convert to CNF, showing conversion steps: $S \to A1A \mid B$; $A \to 0A \mid 1A \mid \varepsilon$; $B \to 1$.

SOLUTION

We follow the order of the slides: ε-productions, unit productions, useless symbols, then the CNF steps.

(i) Eliminate ε-productions. Nullable: A ($A \to \varepsilon$). S and B are not nullable (every body of S contains 1 or B, and $B \to 1$). For each body, add every version with occurrences of A deleted, then drop $A \to \varepsilon$:

$$S \to A1A \mid 1A \mid A1 \mid 1 \mid B, \qquad A \to 0A \mid 1A \mid 0 \mid 1, \qquad B \to 1.$$

S is not nullable, so $\varepsilon \notin L$ and no $S \to \varepsilon$ is needed.

(ii) Eliminate unit productions. The only unit production is $S \to B$. Unit pairs: (S, S), (A, A), (B, B) and (S, B). For (S, B) add $S \to 1$ (the non-unit body of B; already present), then remove $S \to B$:

$$S \to A1A \mid 1A \mid A1 \mid 1, \qquad A \to 0A \mid 1A \mid 0 \mid 1, \qquad B \to 1.$$

(iii) Eliminate useless symbols. Generating: $0, 1, A, S, B$ (all). Reachable from S: $S, A, 0, 1$; B is no longer reachable, so $B \to 1$ is deleted.

(iv)(a) Bodies of length ≥ 2 only of variables. Introduce $Y \to 1$, $Z \to 0$:

$$S \to AYA \mid YA \mid AY \mid 1, \qquad A \to ZA \mid YA \mid 0 \mid 1.$$

(iv)(b) Cascade bodies of length ≥ 3. $S \to AYA$ becomes $S \to AC_1$, $C_1 \to YA$.

Answer. $S \to AC_1 \mid YA \mid AY \mid 1$; $C_1 \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Y \to 1$; $Z \to 0$.

(The language is "strings containing at least one 1". Renaming C_1 as B gives exactly the grammar of 2022–2023 Q7 above.)

2019–2020 • Q7 Source M42 MARKS 10 + 5 = 15

For this CNF grammar with start S, use CYK to test 01100, showing the table. Describe $L(G)$ informally. $S \to BZ \mid CY \mid ZZ \mid YY$; $B \to ZA$; $C \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Z \to 0$; $Y \to 1$.

SOLUTION

Same grammar as 2023–2024 Q6 above.

CYK for 01100 ($a_1 \cdots a_5 = 01100$):

$X_{i,i+4}$	$\{A, B, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, B, \mathbf{S}\}$	$\{A, C\}$			
$X_{i,i+2}$	$\{A, B\}$	$\{A, C\}$	$\{A, C\}$		
$X_{i,i+1}$	$\{A, B\}$	$\{A, C, \mathbf{S}\}$	$\{A, C\}$	$\{A, B, \mathbf{S}\}$	
X_{ii}	$\{A, Z\}$	$\{A, Y\}$	$\{A, Y\}$	$\{A, Z\}$	$\{A, Z\}$
	0	1	1	0	0

Sample entries: X_{23} ("11"): $Y \in X_{22}$, $Y \in X_{33}$ give $S \to YY$; $Y \in X_{22}$, $A \in X_{33}$ give $A \to YA$, $C \to YA$; so $X_{23} = \{A, C, S\}$. X_{15}: the split $k = 4$ ("0110" | "0") has $B \in X_{14}$, $Z \in X_{55}$, so $S \to BZ$ gives $S \in X_{15}$. $01100 \in L(G)$ ✓.

Language (derived in 2023–2024 Q6(b) above):

Answer. $L(G)$ = binary strings of length at least 2 that begin and end with the same symbol.

January 2020 Term • Q7 Source M38 **MARKS 15**

For the following CNF grammar with start S, determine whether $10011 \in L(G)$ by CYK, showing the table. $S \to BZ \mid CY \mid ZZ \mid YY$; $B \to ZA$; $C \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Z \to 0$; $Y \to 1$.

SOLUTION

Same grammar as 2023–2024 Q6 above; $a_1 \cdots a_5 = 10011$.

$X_{i,i+4}$	$\{A, C, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, C, \mathbf{S}\}$	$\{A, B\}$			
$X_{i,i+2}$	$\{A, C\}$	$\{A, B\}$	$\{A, B\}$		
$X_{i,i+1}$	$\{A, C\}$	$\{A, B, \mathbf{S}\}$	$\{A, B\}$	$\{A, C, \mathbf{S}\}$	
X_{ii}	$\{A, Y\}$	$\{A, Z\}$	$\{A, Z\}$	$\{A, Y\}$	$\{A, Y\}$
	1	0	0	1	1

Sample entries: X_{12} ("10"): $Y \in X_{11}$, $A \in X_{22}$ give $A \to YA$, $C \to YA$; no body YZ exists, so $S \notin X_{12}$. X_{45} ("11"): $S \to YY$ applies. X_{15}: the split $k = 4$ ("1001" | "1") has $C \in X_{14}$, $Y \in X_{55}$, so $S \to CY$ gives $S \in X_{15}$. $10011 \in L(G)$ ✓ (it begins and ends with 1).

2017–2018 • Q7(a–b) Source M34 **MARKS (a) 8; (b) 7**

(a) Convert $S \to A1A \mid B$; $A \to 0A \mid 1A \mid \varepsilon$; $D \to 00$ into CNF, showing all steps (0 and 1 are terminals). (b) Use CYK to determine whether 01101 belongs to the resulting grammar.

SOLUTION

NOTE B **has no productions and** D **is never used. We follow the grammar exactly as printed:** B **is non-generating and** D **is unreachable, so both disappear. (If** B **was meant to be** D**, step (ii) adds** $S \to 00$**, i.e.** $S \to ZZ$ **at the end; this does not affect part (b), since** $01101 \neq 00$**.)**

(a) Order of the slides:

(i) *ε-productions.* Nullable: A only. $S \to A1A \mid 1A \mid A1 \mid 1 \mid B$; $A \to 0A \mid 1A \mid 0 \mid 1$; $D \to 00$. (S is not nullable, so no $S \to \varepsilon$.)

(ii) *Unit productions.* $S \to B$ is a unit production; the unit pair (S, B) adds nothing because B has no bodies. Removing $S \to B$: $S \to A1A \mid 1A \mid A1 \mid 1$.

(iii) *Useless symbols.* B is non-generating (no productions) and no longer occurs; D is generating but not reachable from S, so $D \to 00$ is deleted. Left: $S \to A1A \mid 1A \mid A1 \mid 1$; $A \to 0A \mid 1A \mid 0 \mid 1$.

(iv) *CNF steps.* (a) $Y \to 1$, $Z \to 0$: $S \to AYA \mid YA \mid AY \mid 1$; $A \to ZA \mid YA \mid 0 \mid 1$. (b) $S \to AYA$ becomes $S \to AC_1$, $C_1 \to YA$.

Answer. $S \to AC_1 \mid YA \mid AY \mid 1;\ C_1 \to YA;\ A \to ZA \mid YA \mid 0 \mid 1;\ Y \to 1;\ Z \to 0.$

(b) CYK on 01101 ($a_1 \cdots a_5 = 01101$; 1 is produced by A, S, Y, 0 by A, Z):

$X_{i,i+4}$	$\{A, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$			
$X_{i,i+2}$	$\{A, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$		
$X_{i,i+1}$	$\{A, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$	$\{A, \mathbf{S}\}$	
X_{ii}	$\{A, Z\}$	$\{A, \mathbf{S}, Y\}$	$\{A, \mathbf{S}, Y\}$	$\{A, Z\}$	$\{A, \mathbf{S}, Y\}$
	0	1	1	0	1

$S \in X_{15}$ (e.g. split $k = 1$: $A \in X_{11}$, $C_1 \in X_{25}$, rule $S \to AC_1$), so $01101 \in L(G)$ (as expected: it contains a 1).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q6(b) Source M31 **MARKS 10**

Convert $A \to BAB \mid B \mid \varepsilon$; $B \to 00 \mid \varepsilon$ to an equivalent CFG in CNF, showing the steps.

2014–2015 • Q5(c) Source M23 **MARKS $9\frac{1}{3}$**

What is CNF? Convert $A \to BAB \mid B\varepsilon$; $B \to 00 \mid \varepsilon$ to an equivalent CFG in CNF.

SOLUTION

CNF. A grammar is in Chomsky normal form if every production is of one of the forms $A \to BC$ (A, B, C variables) or $A \to a$ (A a variable, a a terminal), and it has no useless symbols. Every non-empty CFL without ε has a CNF grammar; if $\varepsilon \in L$, we convert for $L - \{\varepsilon\}$ and add $S \to \varepsilon$ for the start symbol.

NOTE "$B\varepsilon$" **in the 2014–15 paper is read as "$B \mid \varepsilon$" (and anyway $B\varepsilon = B$); both papers then have the same grammar. The start variable is A.**

(i) Eliminate ε-productions. Nullable: B ($B \to \varepsilon$) and A ($A \to \varepsilon$). From $A \to BAB$ add every version with nullable occurrences deleted: AB, BA, BB, B, and A (a trivial $A \to A$, dropped). Dropping the ε-productions:

$$A \to BAB \mid AB \mid BA \mid BB \mid B, \qquad B \to 00.$$

The start variable A is nullable, so $\varepsilon \in L$; following the slides, the ε-production for the start symbol is added back at the end.

(ii) Eliminate unit productions. Unit pair (A, B): add $A \to 00$, remove $A \to B$:

$$A \to BAB \mid AB \mid BA \mid BB \mid 00, \qquad B \to 00.$$

(iii) Eliminate useless symbols. A and B are generating ($\to 00$) and reachable; nothing is removed.
(iv)(a) $Z \to 0$; replace 00 by ZZ. **(iv)(b)** $A \to BAB$ becomes $A \to BC_1$, $C_1 \to AB$. This gives a CNF grammar for $L - \{\varepsilon\}$: $A \to BC_1 \mid AB \mid BA \mid BB \mid ZZ$, $C_1 \to AB$, $B \to ZZ$, $Z \to 0$.
Adding ε. The slides add "$S \to \varepsilon$". Since the start variable A occurs in bodies (AB, BA, $C_1 \to AB$), adding $A \to \varepsilon$ directly would put ε back inside those bodies; so we use a fresh start variable S_0 that has all of A's bodies and $S_0 \to \varepsilon$:

Answer. $S_0 \to BC_1 \mid AB \mid BA \mid BB \mid ZZ \mid \varepsilon$; $A \to BC_1 \mid AB \mid BA \mid BB \mid ZZ$; $C_1 \to AB$; $B \to ZZ$; $Z \to 0$.
(Both grammars generate $(00)^*$.)

2008–2009 • Q2(a–c) Source M06 **MARKS (a) $7\frac{1}{3}$; (b) 8; (c) 8**

For the grammar $S \to ASB \mid \varepsilon$; $A \to aAS \mid a$; $B \to SbS \mid A \mid bb$: (a) eliminate ε-productions; (b) eliminate unit productions; (c) eliminate useless symbols.

SOLUTION

(a) ε-productions. Nullable symbols: S (directly). A is not nullable (every body contains a); B is not nullable (SbS contains b, A is not nullable, bb). For each rule add the versions with occurrences of S removed, and drop $S \to \varepsilon$:

$$\begin{aligned} S &\to ASB \mid AB \\ A &\to aAS \mid aA \mid a \\ B &\to SbS \mid bS \mid Sb \mid b \mid A \mid bb \end{aligned}$$

This grammar generates $L - \{\varepsilon\}$. Since S is nullable, $\varepsilon \in L$; following the slides, $S \to \varepsilon$ is added to the final grammar.
(b) Unit productions. The only unit production is $B \to A$; non-trivial unit pair (B, A). Replace $B \to A$ by A's bodies:

$$\begin{aligned} S &\to ASB \mid AB \\ A &\to aAS \mid aA \mid a \\ B &\to SbS \mid bS \mid Sb \mid b \mid bb \mid aAS \mid aA \mid a \end{aligned}$$

(c) Useless symbols. *Generating:* a, b; A ($\to a$), B ($\to b$), S ($\to AB$). *Reachable* from S: A, B (via $S \to AB$), then a, b. Every symbol is both generating and reachable, so **no symbol is useless**; the grammar of (b) is final, together with $S \to \varepsilon$ for the empty string.

2007–2008 • Q13 Source M05 **MARKS 10**

What is Chomsky normal form (CNF)? For the CNF grammar below, use CYK to determine whether $abaab \in L(G)$: $S \to AB \mid BC$; $A \to BA \mid a$; $B \to CC \mid b$; $C \to AB \mid a$.

SOLUTION

CNF: every production is $A \to BC$ (A, B, C variables) or $A \to a$ (a a terminal), and there are no useless symbols. Every non-empty CFL without ε has a CNF grammar. In CNF every derivation of

a length-n string takes exactly $2n-1$ steps and parse trees are binary, which is what CYK exploits. Here $a_1 \cdots a_5 = abaab$. Bottom row: a is produced by A, C; b by B.

$X_{i,i+4}$	$\{C, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, \mathbf{S}\}$	$\emptyset$			
$X_{i,i+2}$	$\{B\}$	$\emptyset$	$\{B\}$		
$X_{i,i+1}$	$\{C, \mathbf{S}\}$	$\{A, \mathbf{S}\}$	$\{B\}$	$\{C, \mathbf{S}\}$	
X_{ii}	$\{A, C\}$	$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$
	a	b	a	a	b

Example entries: X_{12} ("ab"): $A \in X_{11}$, $B \in X_{22}$ match $S \to AB$ and $C \to AB$, so $X_{12} = \{C, S\}$. X_{24} ("baa"): the split $k = 2$ gives $X_{22} \cdot X_{34} = \{B\} \cdot \{B\}$ — no rule with body BB; the split $k = 3$ gives $X_{23} \cdot X_{44} = \{A, S\} \cdot \{A, C\}$ — no rule with body AA, AC, SA, SC; so $X_{24} = \emptyset$. Top cell $X_{15} = \{C, S\}$: the split $k = 4$ ("$abaa$" | "b") gives $A \in X_{14}$ and $B \in X_{55}$, so $S \to AB$ (and $C \to AB$) apply.

Answer. $S \in X_{15}$, so $abaab \in L(G)$.

10

Pushdown automata and CFG–PDA equivalence

NOTE **Atif sir's (Hopcroft) notation. A PDA is $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$; initially the stack holds only the start symbol Z_0, which marks the bottom. An arc from q to p labelled $a, X/\gamma$ means $(p, \gamma) \in \delta(q, a, X)$: read a (nothing if $a = \varepsilon$) with X on top of the stack, and replace X by the string γ (its leftmost symbol becomes the new top). So $a, X/X$ leaves the stack unchanged, $a, X/\varepsilon$ pops X and $a, X/AX$ pushes A. Several labels stacked on one arc are several transitions between the same two states. An ID (q, w, γ) lists the state, the remaining input and the stack (top at the left, Z_0 at the bottom); $(q, aw, X\beta) \vdash (p, w, \gamma\beta)$ whenever $(p, \gamma) \in \delta(q, a, X)$. Acceptance is by final state unless stated otherwise.**

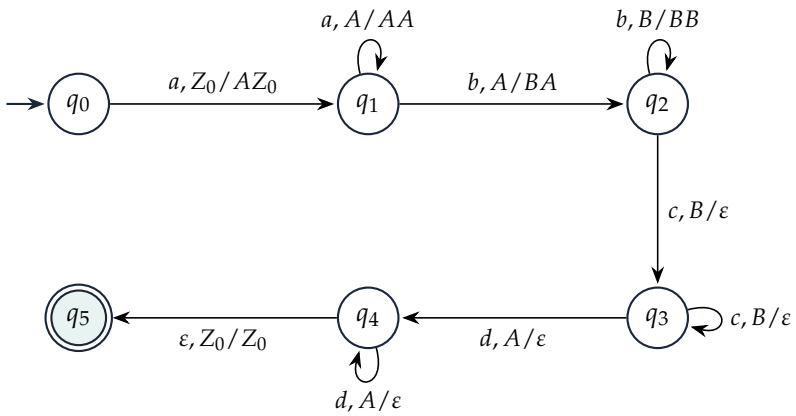
2023–2024 • Q8 Source N03 **MARKS 15**

Design a PDA for $\{a^n b^m c^m d^n \mid n, m \geq 1\}$. Draw its transition diagram.

SOLUTION

Push an A per a and a B per b; pop the B's with c's, then the A's with d's. Because the B's sit above the A's, reading c with A on top means too many c's, and reading d with B on top means too few c's; neither situation has a move, so that branch dies. The first a $(a, Z_0/AZ_0)$ and the first b $(b, A/BA)$ have their own arcs, which enforces $n, m \geq 1$; the last arc needs Z_0 on top, i.e. every A has been matched by a d.

$$P = (\{q_0, \ldots, q_5\}, \{a, b, c, d\}, \{A, B, Z_0\}, \delta, q_0, Z_0, \{q_5\})$$



E.g. $aabcdd$: $(q_0, aabcdd, Z_0) \vdash (q_1, abcdd, AZ_0) \vdash (q_1, bcdd, AAZ_0) \vdash (q_2, cdd, BAAZ_0) \vdash (q_3, dd, AAZ_0) \vdash (q_4, d, AZ_0) \vdash (q_4, \varepsilon, Z_0) \vdash (q_5, \varepsilon, Z_0)$ — accepted.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q8 Source M52 **MARKS 15**

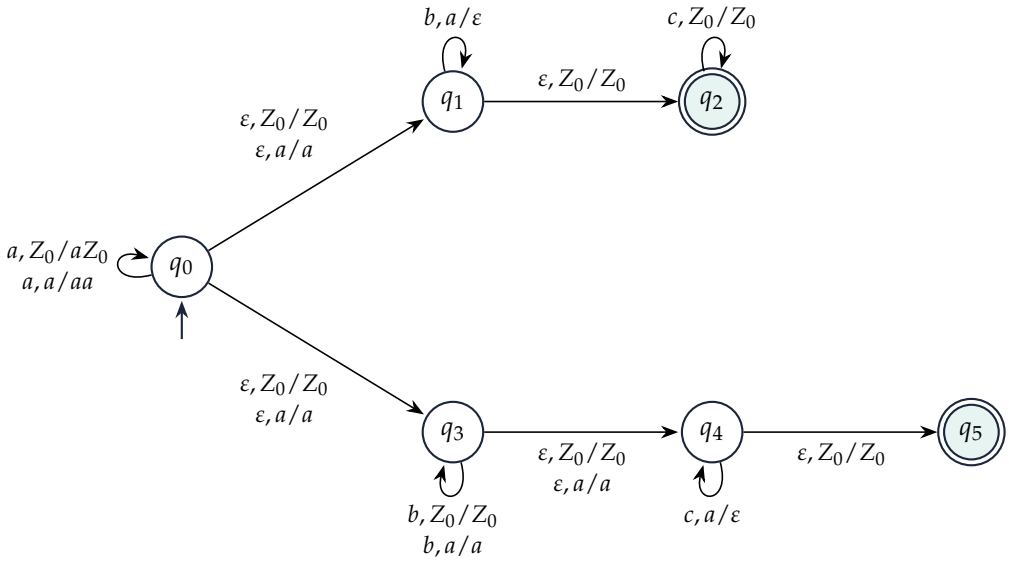
Design a PDA for $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i = j \text{ or } i = k)\}$. Draw its transition diagram. Recall PDAs are nondeterministic.

January 2020 Term • Q8 Source M38 MARKS 15

Design a PDA for $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i = j \text{ or } i = k)\}$. Draw the transition diagram.

SOLUTION

Push the a's, then *guess* which count to compare them with. Upper branch: pop one a per b, check that Z_0 is on top again, then read the c's without touching the stack. Lower branch: read the b's without touching the stack, then pop one a per c and check Z_0.



$P = (\{q_0, \ldots, q_5\}, \{a, b, c\}, \{a, Z_0\}, \delta, q_0, Z_0, \{q_2, q_5\})$. If $i = j$ some computation takes the upper branch and accepts; if $i = k$ the lower branch accepts; otherwise every branch gets stuck or ends in a non-accepting state. Nondeterminism is essential here (the language is not deterministic context-free).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q7 Source M47 MARKS 15

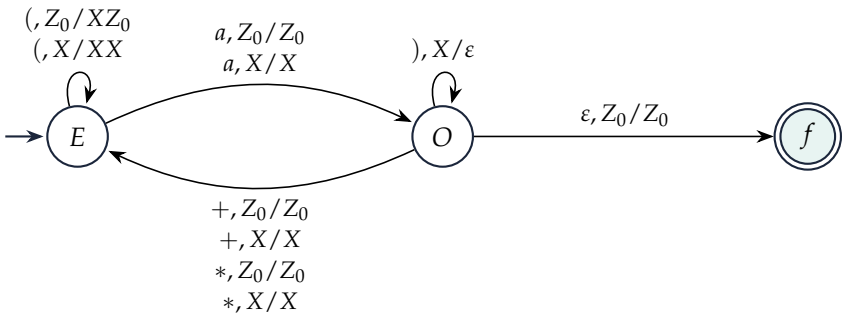
Construct a PDA for legal arithmetic expressions over $\{a, +, *, (,)\}$. Give its transition diagram. Valid: $a + a * a$, $((a + a) * (a + a)) + (a)$. Invalid: $aa * a$, $a{+}{+}a$, $+a$, $(a + a))$.

2019–2020 • Q8 Source M42 MARKS 15

Design a PDA for legal arithmetic expressions over $\{a, +, *, (,)\}$. Give the transition diagram. (Same examples.)

SOLUTION

Idea. The finite control remembers whether we expect an *operand* (state E: start of input, after "(" or after an operator) or an *operator* (state O: after a or ")"). The stack counts open parentheses with a symbol X above Z_0.



$P = (\{E, O, f\}, \{a, +, *, (,)\}, \{X, Z_0\}, \delta, E, Z_0, \{f\})$.

- Adjacent variables ($aa*a$): in O there is no move on $a \Rightarrow$ reject.
- Adjacent operators ($a{+}{+}a$): in E there is no move on $+$ or $*$.
- Initial operator ($+a$): the start state E has no move on $+$.
- Unbalanced ($(a+a))$): the second ")" finds Z_0 on top, and there is no move on ")" with Z_0 on top. A missing ")" leaves an X above Z_0, so the arc $O \to f$ (which needs Z_0 on top) is impossible.

Trace of $(a)+a$: $(E,\,(a)+a,\,Z_0) \vdash (E,\,a)+a,\,XZ_0) \vdash (O,\,)+a,\,XZ_0) \vdash (O,\,+a,\,Z_0) \vdash (E,\,a,\,Z_0) \vdash (O,\,\varepsilon,\,Z_0) \vdash (f,\,\varepsilon,\,Z_0)$ — accepted.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2019–2020 • Q10(a) Source M42 — **MARKS 8**

Show that languages accepted by final state by some PDA are exactly those accepted by empty stack by some PDA.

2015–2016 • Q6(b) Source M27 — **MARKS $10\frac{1}{3}$**

Given a PDA P_F accepting L by final state, can you always construct a PDA P_N accepting L by empty stack? If yes, give the construction and definitions of both P_F and P_N; otherwise justify.

2007–2008 • Q6 Source M05 — **MARKS 10**

Justify: a language has a PDA accepting by final state if and only if it has a PDA accepting by empty stack.

SOLUTION

SCOPE *2015–2016 Q6(b) asks only for the definitions and the direction $P_F \to P_N$ (the first two parts below); 2019–2020 Q10(a) and 2007–2008 Q6 ask for both directions.*

Definitions. A PDA is $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. *Final-state* acceptance: $L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \alpha),\ q \in F\}$. *Empty-stack* acceptance: $N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon)\}$ (any q).

Yes: from P_F (final state) to P_N (empty stack)

Given $P_F = (Q, \Sigma, \Gamma, \delta_F, q_0, Z_0, F)$ with $L = L(P_F)$, let $P_N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, p_0, X_0)$:

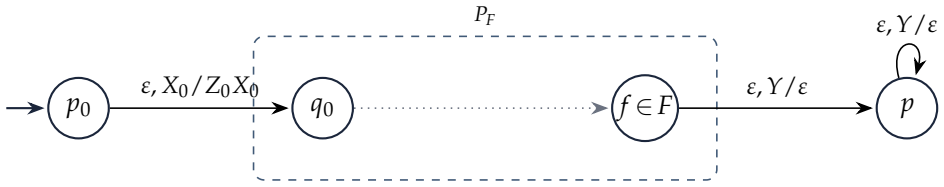
(i) $\delta_N(p_0, \varepsilon, X_0) = \{(q_0, Z_0X_0)\}$ — put P_F's start symbol above a new bottom marker X_0. (X_0 prevents P_N from accidentally emptying its stack when P_F empties its own stack without

accepting.)

(ii) All moves of P_F are copied: $\delta_N(q,a,Y) \supseteq \delta_F(q,a,Y)$.

(iii) For every $q \in F$ and $Y \in \Gamma \cup \{X_0\}$: $(p,\varepsilon) \in \delta_N(q,\varepsilon,Y)$ — when P_F is in an accepting state, P_N may start emptying.

(iv) For every $Y \in \Gamma \cup \{X_0\}$: $\delta_N(p,\varepsilon,Y) = \{(p,\varepsilon)\}$ — pop everything.



Correctness. If $w \in L(P_F)$: $(p_0,w,X_0) \vdash (q_0,w,Z_0X_0) \vdash^* (q,\varepsilon,\alpha X_0)$ with $q \in F$, then $\vdash (p,\varepsilon,\alpha X_0) \vdash^* (p,\varepsilon,\varepsilon)$. Conversely, P_N can empty its stack only by popping X_0, which happens only in p, entered only from an accepting state of P_F after simulating it on w; so $w \in L(P_F)$. Hence $N(P_N) = L$.

Conversely: from P_N (empty stack) to P_F (final state)

Given $P_N = (Q,\Sigma,\Gamma,\delta_N,q_0,Z_0)$, let $P_F = (Q \cup \{p_0,p_f\},\Sigma,\Gamma \cup \{X_0\},\delta_F,p_0,X_0,\{p_f\})$: $\delta_F(p_0,\varepsilon,X_0) = \{(q_0,Z_0X_0)\}$; copy all of δ_N; and for every $q \in Q$: $(p_f,\varepsilon) \in \delta_F(q,\varepsilon,X_0)$. Seeing X_0 on top means P_N has just emptied its own stack, so P_F moves to the accepting state p_f. *Correctness.* If $(q_0,w,Z_0) \vdash_{P_N}^* (q,\varepsilon,\varepsilon)$, then $(p_0,w,X_0) \vdash (q_0,w,Z_0X_0) \vdash^* (q,\varepsilon,X_0) \vdash (p_f,\varepsilon,\varepsilon)$; conversely p_f is entered only when X_0 is exposed, i.e. after P_N has emptied its stack. Thus $L(P_F) = N(P_N)$.

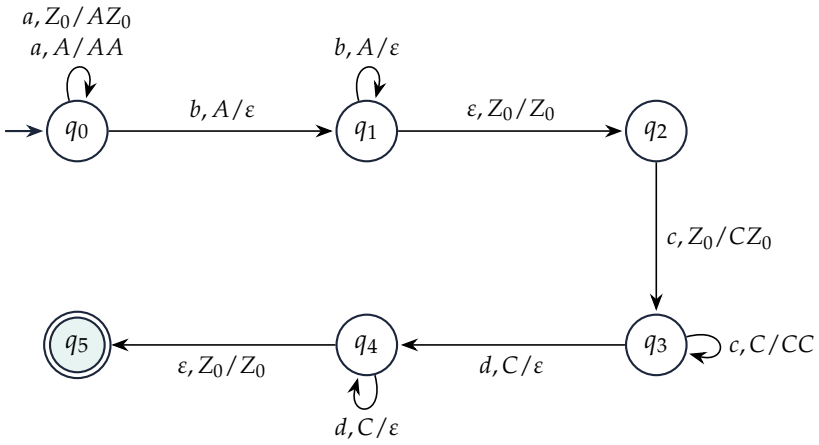
Answer. Both acceptance modes define exactly the same class of languages (the CFLs).

2017–2018 • Q8(a) Source M34 **MARKS 15**

Design a PDA recognizing $L = \{a^n b^n c^m d^m \mid n \geq 1,\ m \geq 1\}$. Draw its transition diagram.

SOLUTION

Match a's with b's, check that the stack is back at Z_0, then match c's with d's.



- $q_0 \to q_1$ needs an A on top, so at least one a was read ($n \geq 1$); in q_1 every b pops an A.
- $q_1 \xrightarrow{\varepsilon, Z_0/Z_0} q_2$ (stack unchanged) is possible only when Z_0 is on top, i.e. all A's are gone: $\#a = \#b$. A b read with Z_0 on top has no move (too many b's).

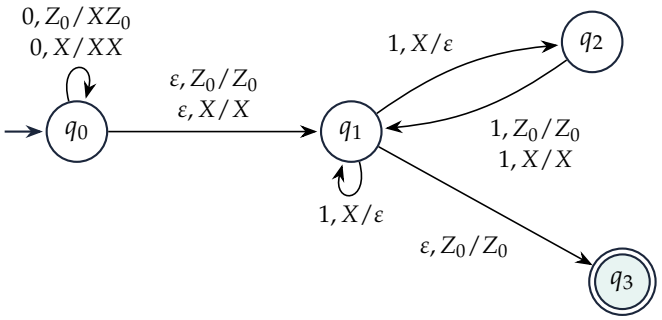
- $q_2 \to q_3$ forces at least one c; every c pushes a C, every d pops one; $q_4 \to q_5$ needs Z_0 on top, i.e. $\#c = \#d$.

2016–2017 • Q6(a) Source M30 **MARKS 10 + 5 = 15**

Design a PDA for $L = \{0^n 1^m \mid n \le m \le 2n\}$. Give the transition diagram only, and show acceptance of 00111 and rejection of 0011111.

SOLUTION

Push one X per 0. Each X is then popped by *one or two* 1's (nondeterministic choice): the loop at q_1 pops an X with one 1, the detour $q_1 \to q_2 \to q_1$ pops an X with one 1 and then reads one more 1. Hence the number of 1's is between n and $2n$.



$P = (\{q_0, q_1, q_2, q_3\}, \{0,1\}, \{X, Z_0\}, \delta, q_0, Z_0, \{q_3\})$.
Accepting 00111 ($n = 2$, $m = 3$): $(q_0, 00111, Z_0) \vdash (q_0, 0111, XZ_0) \vdash (q_0, 111, XXZ_0) \vdash (q_1, 111, XXZ_0) \vdash (q_1, 11, XZ_0)$ (one 1 for the first X) $\vdash (q_2, 1, Z_0) \vdash (q_1, \varepsilon, Z_0)$ (two 1's for the second X) $\vdash (q_3, \varepsilon, Z_0)$ — accepted.
Rejecting 0011111 ($m = 5 > 4$): 0's are read only in q_0, so at most two X's are ever pushed. In q_1 a 1 can be read only by popping an X, and the arc $q_2 \to q_1$ reads one extra 1 only once after each pop that entered q_2. So every X accounts for at most two 1's and at most four 1's can be read; e.g. the longest branch is $(q_0, 0011111, Z_0) \vdash (q_0, 011111, XZ_0) \vdash (q_0, 11111, XXZ_0) \vdash (q_1, 11111, XXZ_0) \vdash (q_2, 1111, XZ_0) \vdash (q_1, 111, XZ_0) \vdash (q_2, 11, Z_0) \vdash (q_1, 1, Z_0) \vdash (q_3, 1, Z_0)$, which halts with a 1 unread. Every branch halts with unread input, so no branch accepts.

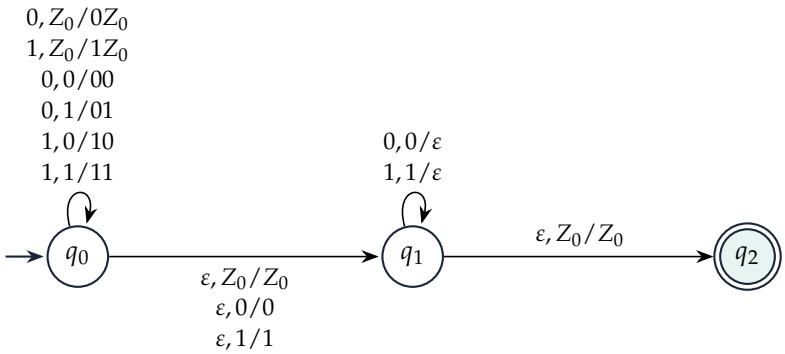
2016–2017 • Q7(b) Source M31 **MARKS 10**

Give a PDA recognizing $\{ww^R \mid w \in \{0,1\}^*\}$, where w^R is w written backward.

SOLUTION

(The lecture's PDA for L_{wwr}.) In q_0 push w; at any time guess that the middle has been reached and move to q_1 without touching the stack; in q_1 pop while matching w^R; when Z_0 is exposed, move to the accepting state q_2.

$$P = (\{q_0, q_1, q_2\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \{q_2\})$$

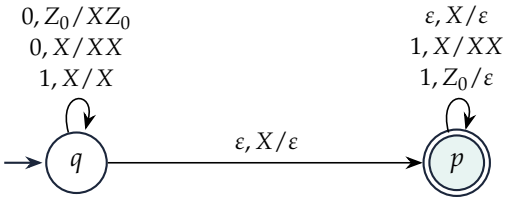


2015–2016 • Q5(a–b) Source M26–M27 **MARKS (a) 8$\frac{1}{3}$; (b) 15**

For $P = (\{q, p\}, \{0, 1\}, \{Z_0, X\}, \delta, q, Z_0, \{p\})$: (a) draw a transition diagram using $\delta(q, 0, Z_0) = \{(q, XZ_0)\}$; $\delta(q, 0, X) = \{(q, XX)\}$; $\delta(q, 1, X) = \{(q, X)\}$; $\delta(q, \varepsilon, X) = \{(p, \varepsilon)\}$; $\delta(p, \varepsilon, X) = \{(p, \varepsilon)\}$; $\delta(p, 1, X) = \{(p, XX)\}$; $\delta(p, 1, Z_0) = \{(p, \varepsilon)\}$. (b) Starting at ID (q, w, Z_0), show all reachable IDs for (i) 0011; (ii) 0101; (iii) 1010.

SOLUTION

(a) Transition diagram



(b) Reachable IDs

Computed exhaustively (breadth-first): row k lists the IDs first reached after exactly k moves (stack top on the left). IDs with an empty stack and input left, and IDs such as $(p, 01, Z_0)$ with no applicable move, are dead ends.

moves	(i) 0011	(ii) 0101	(iii) 1010
0	$(q, 0011, Z_0)$	$(q, 0101, Z_0)$	$(q, 1010, Z_0)$
1	$(q, 011, XZ_0)$	$(q, 101, XZ_0)$	*no move*: $\delta(q, 1, Z_0)$
2	$(q, 11, XXZ_0)$, $(p, 011, Z_0)$	$(q, 01, XZ_0)$, $(p, 101, Z_0)$	and $\delta(q, \varepsilon, Z_0)$
3	$(q, 1, XXZ_0)$, $(p, 11, XZ_0)$	$(q, 1, XXZ_0)$, $(p, 01, Z_0)$, $(p, 01, \varepsilon)$	are undefined.
4	(q, ε, XXZ_0), $(p, 1, XXZ_0)$, $(p, 1, XZ_0)$, $(p, 11, Z_0)$	(q, ε, XXZ_0), $(p, 1, XZ_0)$	
5	$(p, \varepsilon, XXXZ_0)$, (p, ε, XXZ_0), (p, ε, XZ_0), $(p, 1, Z_0)$, $(p, 1, \varepsilon)$	(p, ε, XXZ_0), (p, ε, XZ_0), $(p, 1, Z_0)$	
6	(p, ε, Z_0), $(p, \varepsilon, \varepsilon)$	(p, ε, Z_0), $(p, \varepsilon, \varepsilon)$	
	accepted (e.g. (p, ε, XZ_0))	**accepted**	**rejected**

A sample accepting path for 0011: $(q, 0011, Z_0) \vdash (q, 011, XZ_0) \vdash (q, 11, XXZ_0) \vdash (q, 1, XXZ_0) \vdash (p, 1, XZ_0) \vdash (p, \varepsilon, XXZ_0)$, ending in the final state p with all input read.

2015–2016 • Q6(a) Source M27 **MARKS 13**

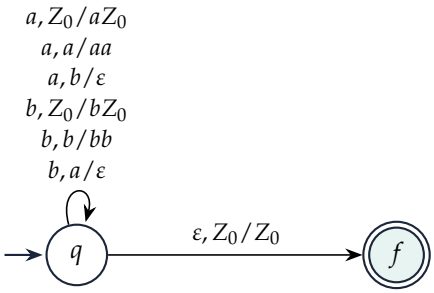
Define a PDA accepting $\{w \in \{a,b\}^* \mid w \text{ has the same number of } a\text{'s and } b\text{'s}\}$.

SOLUTION

(The lecture's example.) The stack holds the current surplus above the bottom marker Z_0: a string of a's if more a's than b's have been read, a string of b's if more b's. An a either cancels a b or pushes an a; symmetrically for b.

$$P = (\{q,f\},\{a,b\},\{a,b,Z_0\},\delta,q,Z_0,\{f\})$$

$$\begin{aligned}
&\delta(q,a,Z_0) = \{(q,aZ_0)\} && \delta(q,b,Z_0) = \{(q,bZ_0)\}\\
&\delta(q,a,a) = \{(q,aa)\} && \delta(q,b,b) = \{(q,bb)\}\\
&\delta(q,a,b) = \{(q,\varepsilon)\} && \delta(q,b,a) = \{(q,\varepsilon)\}\\
&\delta(q,\varepsilon,Z_0) = \{(f,Z_0)\}
\end{aligned}$$



Invariant: after reading u the stack is $a^{\#_a(u)-\#_b(u)}Z_0$ or $b^{\#_b(u)-\#_a(u)}Z_0$. Z_0 is on top exactly when the counts are equal, and only then can P move to f; f has no moves, so P accepts exactly when the counts are equal at the end of the input.

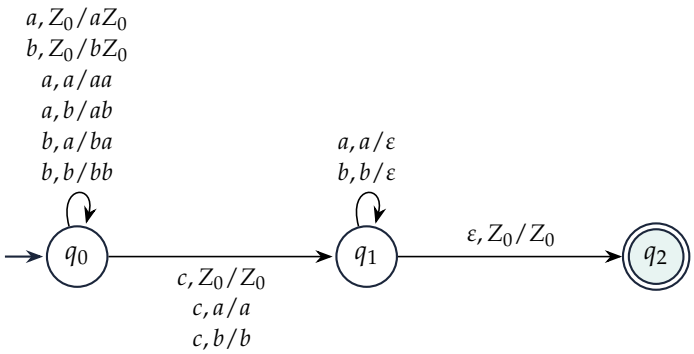
Remark. This PDA is not deterministic: in q with Z_0 on top it may read a or b and may also take the ε-move to f. A deterministic PDA for the same language makes f the start state and moves the two "read on Z_0" transitions there: $P' = (\{f,q\},\{a,b\},\{a,b,Z_0\},\delta',f,Z_0,\{f\})$ with $\delta'(f,a,Z_0) = \{(q,aZ_0)\}$, $\delta'(f,b,Z_0) = \{(q,bZ_0)\}$, the four moves $\delta'(q,a,a)$, $\delta'(q,b,b)$, $\delta'(q,a,b)$, $\delta'(q,b,a)$ as above, and $\delta'(q,\varepsilon,Z_0) = \{(f,Z_0)\}$.

2014–2015 • Q6(c) Source M23 **MARKS $9\frac{1}{3}$**

Construct a PDA state diagram for $L = \{wcw^R \mid w \in \{a,b\}^*\}$. Trace the accepting computation for *bbcbb* by configurations.

SOLUTION

The marker c tells the PDA where the middle is, so no guessing is needed: in every situation at most one move applies, and this PDA is deterministic.



$P = (\{q_0, q_1, q_2\}, \{a, b, c\}, \{a, b, Z_0\}, \delta, q_0, Z_0, \{q_2\})$. Configurations (IDs):

$$(q_0, bbcbb, Z_0) \vdash (q_0, bcbb, bZ_0) \vdash (q_0, cbb, bbZ_0) \vdash (q_1, bb, bbZ_0)$$
$$\vdash (q_1, b, bZ_0) \vdash (q_1, \varepsilon, Z_0) \vdash (q_2, \varepsilon, Z_0).$$

All input consumed in the accepting state q_2: accepted.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2014–2015 • Q7(a) Source M23 — **MARKS 7**

Prove that for every PDA P there is a CFG G with $L(P) = L(G)$.

2013–2014 • Q1(c) Source M19 — **MARKS $9\frac{1}{3}$**

Prove that for every PDA P there exists a CFG G with $L(P) = L(G)$.

2012–2013 • Q1(b) Source M17 — **MARKS $13\frac{1}{3}$**

Prove that L is context-free if and only if there exists a PDA accepting it.

2010–2011 • Q12(c) Source M13 — **MARKS 11**

Prove that if a PDA recognizes L, then L is context-free.

SOLUTION

SCOPE *2014–2015 Q7(a), 2013–2014 Q1(c) and 2010–2011 Q12(c) ask only for PDA ⇒ CFG (Part 2); 2012–2013 Q1(b) asks for both directions.*

Part 1: CFG ⇒ PDA

Given $G = (V, T, R, S)$, build the one-state PDA that simulates leftmost derivations on its stack and accepts by empty stack:

$$P = (\{q\}, T, V \cup T, \delta, q, S), \qquad \begin{aligned} &\delta(q, \varepsilon, A) = \{(q, \beta) \mid A \to \beta \text{ is a rule of } G\} && \text{for each variable } A,\\ &\delta(q, a, a) = \{(q, \varepsilon)\} && \text{for each terminal } a. \end{aligned}$$

A variable on top is replaced by the body of one of its rules (nondeterministic guess, first symbol of the body on top); a terminal on top must match the next input symbol and is popped.

Claim. $(q,w,S)\vdash^*(q,\varepsilon,\varepsilon)$ iff $S\Rightarrow_{lm}^* w$. If $S\Rightarrow_{lm}^* xA\alpha$ (x terminals, A the leftmost variable), then from (q,xy,S) the PDA reaches $(q,y,A\alpha)$: each leftmost step is an ε-move replacing the variable on top by the rule body, and the terminals in front of the next variable are matched by the moves $a,a/\varepsilon$. Conversely, by induction on the number of moves, in every reachable ID (q,y,γ) with $w=xy$ we have $S\Rightarrow_{lm}^* x\gamma$; at $(q,\varepsilon,\varepsilon)$ this gives $S\Rightarrow_{lm}^* w$. Hence $N(P)=L(G)$, and the $P_N\to P_F$ construction (2019–20 Q10(a)) gives a PDA accepting $L(G)$ by final state.

Part 2: PDA ⇒ CFG

By the $P_F\to P_N$ construction of 2019–20 Q10(a) we may assume $P=(Q,\Sigma,\Gamma,\delta,q_0,Z_0)$ accepts L by empty stack. Build $G=(V,\Sigma,R,S)$ whose variables are S and all triples $[pXq]$ ($p,q\in Q$, $X\in\Gamma$). $[pXq]$ is meant to generate exactly the strings w with $(p,w,X)\vdash^*(q,\varepsilon,\varepsilon)$: starting in p with X on top, P reads w, finally pops X (net) and is then in state q; what lies below X is never touched. Rules:

(i) $S\to[q_0Z_0p]$ for every $p\in Q$.

(ii) For every pair $(r,Y_1Y_2\cdots Y_k)\in\delta(q,a,X)$ ($a\in\Sigma\cup\{\varepsilon\}$, $k\geq 0$) and every list of states $r_1,\ldots,r_k$:

$$[qXr_k]\to a\,[rY_1r_1][r_1Y_2r_2]\cdots[r_{k-1}Y_kr_k];$$

for $k=0$, i.e. $(r,\varepsilon)\in\delta(q,a,X)$, this is $[qXr]\to a$.

Claim. $[qXp]\Rightarrow^* w$ iff $(q,w,X)\vdash^*(p,\varepsilon,\varepsilon)$.

If (induction on the number of moves). One move: $(p,\varepsilon)\in\delta(q,w,X)$ with $w\in\Sigma\cup\{\varepsilon\}$, and $[qXp]\to w$ is a rule. More moves: the first move is $(q,ax,X)\vdash(r,x,Y_1\cdots Y_k)$ with $w=ax$ and $(r,Y_1\cdots Y_k)\in\delta(q,a,X)$. Each Y_i is eventually popped (net); let r_i be the state just after Y_i is popped and w_i the input read meanwhile ($r_0=r$, $r_k=p$). Then $x=w_1\cdots w_k$ and $(r_{i-1},w_i,Y_i)\vdash^*(r_i,\varepsilon,\varepsilon)$ in fewer moves (input after w_i and stack below Y_i do not influence these moves). By the hypothesis $[r_{i-1}Y_ir_i]\Rightarrow^* w_i$, so $[qXp]\Rightarrow a[rY_1r_1]\cdots[r_{k-1}Y_kp]\Rightarrow^* aw_1\cdots w_k=w$.

Only if (induction on the length of the derivation). If the first step is $[qXp]\to a[rY_1r_1]\cdots[r_{k-1}Y_kp]$ and $[r_{i-1}Y_ir_i]\Rightarrow^* w_i$ with $w=aw_1\cdots w_k$, then by the hypothesis $(r_{i-1},w_i,Y_i)\vdash^*(r_i,\varepsilon,\varepsilon)$; adding the remaining input after w_i and the stack $Y_{i+1}\cdots Y_k$ below keeps each computation legal, so $(q,w,X)\vdash(r,w_1\cdots w_k,Y_1\cdots Y_k)\vdash^*(r_1,w_2\cdots w_k,Y_2\cdots Y_k)\vdash^*\cdots\vdash^*(p,\varepsilon,\varepsilon)$.

With $q=q_0$, $X=Z_0$: $S\Rightarrow[q_0Z_0p]\Rightarrow^* w$ iff $(q_0,w,Z_0)\vdash^*(p,\varepsilon,\varepsilon)$ for some p, i.e. $L(G)=N(P)=L$. So every PDA language is context-free.

Conclusion

Parts 1 and 2 give: *L is context-free* $\iff$ *some PDA accepts L.* ■

2013–2014 • Q1(a) Source M19 **MARKS 7**

Convert this CFG into an equivalent PDA: $E\to E+T\mid T$; $T\to T\times F\mid T/F\mid F$; $F\to(E)\mid a$.

SOLUTION

One-state construction (accepting by empty stack, start stack symbol E):

$$P=(\{q\},\{+,\times,/,(,),a\},\{E,T,F,+,\times,/,(,),a\},\delta,q,E)\quad\text{with}$$

$$\begin{aligned}
\delta(q,\varepsilon,E)&=\{(q,E{+}T),\ (q,T)\}\\
\delta(q,\varepsilon,T)&=\{(q,T{\times}F),\ (q,T/F),\ (q,F)\}\\
\delta(q,\varepsilon,F)&=\{(q,(E)),\ (q,a)\}\\
\delta(q,x,x)&=\{(q,\varepsilon)\}\quad\text{for each terminal } x\in\{+,\times,/,(,),a\}.
\end{aligned}$$

E.g. for $a+a$ (leftmost derivation $E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+F \Rightarrow a+a$):

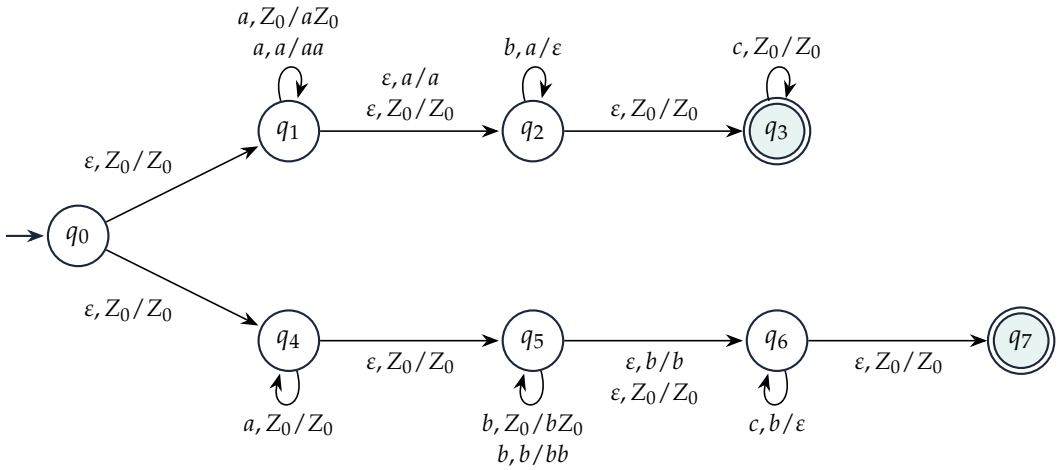
$$(q, a{+}a, E) \vdash (q, a{+}a, E{+}T) \vdash (q, a{+}a, T{+}T) \vdash (q, a{+}a, F{+}T) \vdash (q, a{+}a, a{+}T)$$
$$\vdash (q, {+}a, {+}T) \vdash (q, a, T) \vdash (q, a, F) \vdash (q, a, a) \vdash (q, \varepsilon, \varepsilon)$$ — accepted by empty stack.

2013–2014 • Q1(b) Source M19 **MARKS 7**

Construct a PDA state diagram for $\{a^i b^j c^k \mid i = j \text{ or } j = k, \text{ where } i, j, k \geq 0\}$.

SOLUTION

Guess the branch first. Upper branch ($i = j$): push a's, pop one per b, check Z_0, then read the c's without touching the stack. Lower branch ($j = k$): read the a's without touching the stack, push b's, pop one per c, check Z_0.



$P = (\{q_0, \ldots, q_7\}, \{a, b, c\}, \{a, b, Z_0\}, \delta, q_0, Z_0, \{q_3, q_7\})$.

2012–2013 • Q1(a) Source M17 **MARKS 10**

Formally define a PDA. Construct one for the same language as this CFG: $S \to BS \mid A$; $A \to 0A \mid \varepsilon$; $B \to BB1 \mid 2$.

SOLUTION

A PDA is a 7-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$: Q a finite set of states; Σ the input alphabet; Γ the stack alphabet; $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to$ finite subsets of $Q \times \Gamma^*$, where $(p, \gamma) \in \delta(q, a, X)$ means: in state q, reading a (or ε) with X on top of the stack, go to p and replace X by γ; $q_0 \in Q$ the start state; $Z_0 \in \Gamma$ the start symbol (initially the only symbol on the stack); $F \subseteq Q$ the accepting states. One-state construction (empty-stack acceptance, start stack symbol S):

$$P = (\{q\}, \{0, 1, 2\}, \{S, A, B, 0, 1, 2\}, \delta, q, S) \quad \text{with}$$

$\delta(q, \varepsilon, S) = \{(q, BS),\ (q, A)\}$, $\delta(q, \varepsilon, A) = \{(q, 0A),\ (q, \varepsilon)\}$, $\delta(q, \varepsilon, B) = \{(q, BB1),\ (q, 2)\}$, $\delta(q, 0, 0) = \delta(q, 1, 1) = \delta(q, 2, 2) = \{(q, \varepsilon)\}$.

Example: $S \Rightarrow BS \Rightarrow 2S \Rightarrow 2A \Rightarrow 20A \Rightarrow 20$; on input 20: $(q, 20, S) \vdash (q, 20, BS) \vdash (q, 20, 2S) \vdash (q, 0, S) \vdash (q, 0, A) \vdash (q, 0, 0A) \vdash (q, \varepsilon, A) \vdash (q, \varepsilon, \varepsilon)$ — stack empty, accepted.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q4(a) Source M17 **MARKS 8**

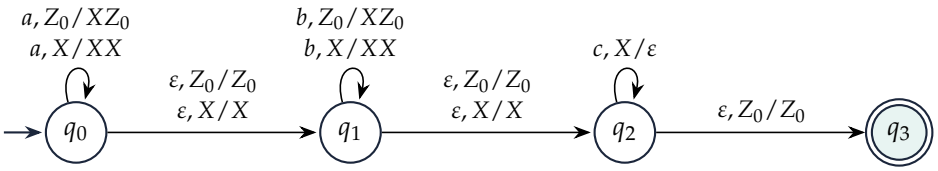
Construct a PDA state diagram recognizing $L = \{a^i b^j c^k \mid i + j = k\}$.

2011–2012 • Q4(a) Source M15 **MARKS 7**

Construct a PDA state diagram recognizing $L = \{a^i b^j c^k \mid i + j = k\}$.

SOLUTION

Push an X for every a and every b; pop one per c; accept when Z_0 is exposed.



$P = (\{q_0, q_1, q_2, q_3\}, \{a, b, c\}, \{X, Z_0\}, \delta, q_0, Z_0, \{q_3\})$.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q4(c) Source M18 **MARKS $8\frac{1}{3}$**

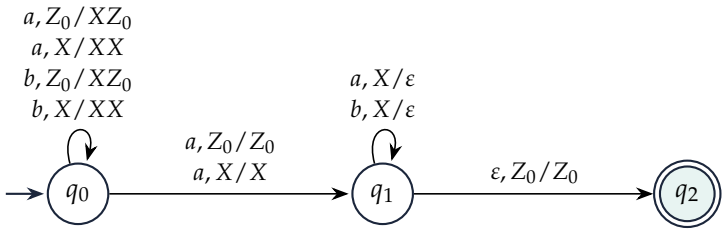
Construct a PDA state diagram accepting odd-length strings over $\{a, b\}$ with middle symbol a. Examples: *aaa*, *bbabb*, *baaba*.

2011–2012 • Q3(c) Source M15 **MARKS 7**

Construct a PDA state diagram for strings over $\{a, b\}$ of odd length with middle symbol a. Examples: *aaa*, *bbabb*, *baaba*.

SOLUTION

Push a marker X for every symbol of the first half (the symbols themselves do not matter), guess that the current a is the middle (read it without touching the stack), then pop one X for every symbol of the second half.



bbabb: $(q_0, bbabb, Z_0) \vdash (q_0, babb, XZ_0) \vdash (q_0, abb, XXZ_0) \vdash (q_1, bb, XXZ_0) \vdash (q_1, b, XZ_0) \vdash (q_1, \varepsilon, Z_0) \vdash (q_2, \varepsilon, Z_0)$ — accepted.

2011–2012 • Q4(b) Source M15 **MARKS 8**

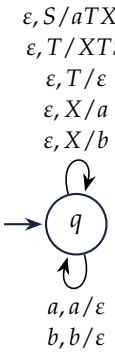
Convert this CFG to an equivalent PDA: $V = \{S, T, X\}$, $\Sigma = \{a, b\}$, start S, with rules $S \to aTXb$; $T \to XTS \mid \varepsilon$; $X \to a \mid b$.

SOLUTION

One-state construction (empty-stack acceptance, start stack symbol S):

$$P = (\{q\}, \{a, b\}, \{S, T, X, a, b\}, \delta, q, S) \quad \text{with}$$

$$\delta(q, \varepsilon, S) = \{(q, aTXb)\}, \quad \delta(q, \varepsilon, T) = \{(q, XTS),\ (q, \varepsilon)\}, \quad \delta(q, \varepsilon, X) = \{(q, a),\ (q, b)\},$$
$$\delta(q, a, a) = \{(q, \varepsilon)\}, \quad \delta(q, b, b) = \{(q, \varepsilon)\}.$$



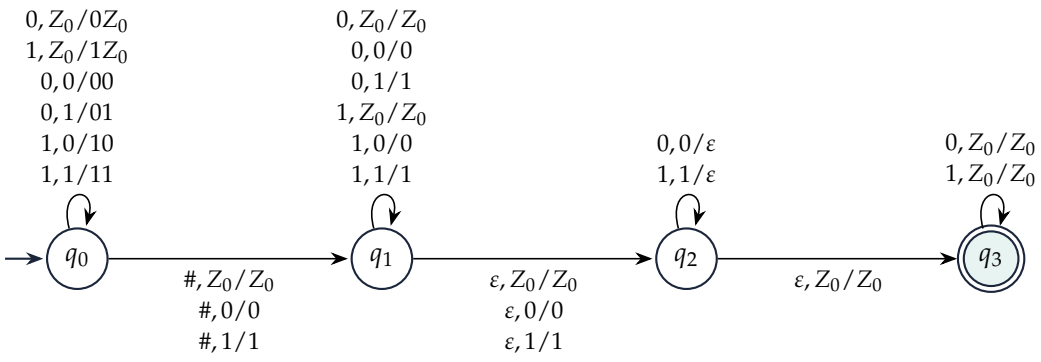
Example: input aab ($S \Rightarrow aTXb \Rightarrow aXb \Rightarrow aab$): $(q, aab, S) \vdash (q, aab, aTXb) \vdash (q, ab, TXb) \vdash (q, ab, Xb) \vdash (q, ab, ab) \vdash (q, b, b) \vdash (q, \varepsilon, \varepsilon)$ — accepted by empty stack.

2010–2011 • Q12(d) Source M13 **MARKS $5\frac{1}{3}$**

Give an informal description and a state diagram of a PDA recognizing this CFG, with start symbol S: $S \to TX$; $T \to 0T0 \mid 1T1 \mid \#X$; $X \to 0X \mid 1X \mid \varepsilon$.

SOLUTION

X generates any binary string and T generates $w\#x\,w^R$, so $L = \{w\#x\,w^R y \mid w, x, y \in \{0,1\}^*\}$. **Informally:** push the symbols before #; after # nondeterministically skip some symbols (x) without touching the stack; then pop while matching (w^R); when Z_0 reappears, go to the accepting state and read the rest (y) without touching the stack.



$P = (\{q_0, q_1, q_2, q_3\}, \{0, 1, \#\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_3\})$.

2010–2011 • Q13(a–b) Source M13 **MARKS (a) 8; (b) $7\frac{1}{3}$**

(a) Formally describe a PDA recognizing $L = \{w \in \{0,1\}^* \mid w = w^R\}$. (b) Convert the PDA from (a) to a CFG using standard conversion rules.

SOLUTION

(a) $P = (\{q_0, q_1, q_2\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \{q_2\})$ with

$$\begin{aligned}
&\delta(q_0,0,Z_0) = \{(q_0,0Z_0),\ (q_1,Z_0)\} && \delta(q_0,1,Z_0) = \{(q_0,1Z_0),\ (q_1,Z_0)\}\\
&\delta(q_0,0,0) = \{(q_0,00),\ (q_1,0)\} && \delta(q_0,0,1) = \{(q_0,01),\ (q_1,1)\}\\
&\delta(q_0,1,0) = \{(q_0,10),\ (q_1,0)\} && \delta(q_0,1,1) = \{(q_0,11),\ (q_1,1)\}\\
&\delta(q_0,\varepsilon,Z_0) = \{(q_1,Z_0)\},\ \ \delta(q_0,\varepsilon,0) = \{(q_1,0)\} && \delta(q_0,\varepsilon,1) = \{(q_1,1)\}\\
&\delta(q_1,0,0) = \{(q_1,\varepsilon)\},\ \ \delta(q_1,1,1) = \{(q_1,\varepsilon)\} && \delta(q_1,\varepsilon,Z_0) = \{(q_2,Z_0)\}
\end{aligned}$$

and all other values $\emptyset$. In q_0 the PDA pushes each symbol, or guesses that the symbol just read is the centre of an odd-length w (move to q_1 without pushing), or guesses the middle of an even-length w (ε-move to q_1); in q_1 it pops while matching and accepts when Z_0 is exposed. This is the lecture's PDA for $\{ww^R\}$ (2016–17 Q7(b)) plus the six odd-length moves to q_1.

(b) The $[pXq]$ construction (Part 2 of 2014–15 Q7(a) above) needs empty-stack acceptance. Let $P_N = (\{q_0,q_1\}, \{0,1\}, \{0,1,Z_0\}, \delta_N, q_0, Z_0)$ be P without q_2 and with the last move replaced by $\delta_N(q_1,\varepsilon,Z_0) = \{(q_1,\varepsilon)\}$. P_N empties its stack exactly when P would enter q_2, and afterwards no move is possible, so $N(P_N) = L(P) = L$. (The general $P_F \to P_N$ construction also works but gives a larger grammar.)

Productions, grouped by the kind of move they come from (a stands for 0 or 1, Y for the stack symbol 0, 1 or Z_0 under the move, and r, s for q_0 or q_1; one rule for each choice):

- start: $S \to [q_0 Z_0 q_0] \mid [q_0 Z_0 q_1]$;
- $(q_0, aY) \in \delta_N(q_0,a,Y)$: $[q_0 Y s] \to a\,[q_0 a r][r Y s]$;
- $(q_1, Y) \in \delta_N(q_0,a,Y)$: $[q_0 Y s] \to a\,[q_1 Y s]$; $\quad (q_1,Y) \in \delta_N(q_0,\varepsilon,Y)$: $[q_0 Y s] \to [q_1 Y s]$;
- $(q_1,\varepsilon) \in \delta_N(q_1,a,a)$: $[q_1 a q_1] \to a$; $\quad (q_1,\varepsilon) \in \delta_N(q_1,\varepsilon,Z_0)$: $[q_1 Z_0 q_1] \to \varepsilon$.

Simplify. P_N never returns from q_1 to q_0, so no $[q_1 Y q_0]$ has a rule; every rule for $[q_0 Y q_0]$ contains some $[q_1 Y' q_0]$ or $[q_0 a q_0]$, so no $[q_0 Y q_0]$ is generating either. In the rule $[q_0 Y q_1] \to a[q_0 a r][r Y q_1]$ only $r = q_1$ survives. What remains is

$$\begin{aligned}
&S \to [q_0 Z_0 q_1], \qquad [q_0 Y q_1] \to [q_1 Y q_1] \mid a[q_1 Y q_1] \mid a[q_0 a q_1][q_1 Y q_1],\\
&[q_1 0 q_1] \to 0, \quad [q_1 1 q_1] \to 1, \quad [q_1 Z_0 q_1] \to \varepsilon.
\end{aligned}$$

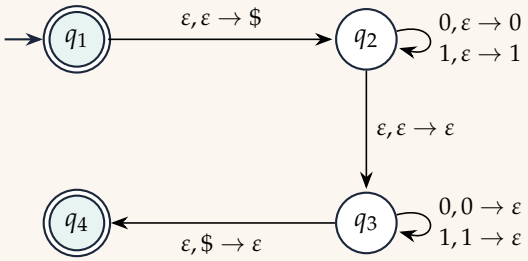
Renaming $A_Y = [q_0 Y q_1]$ and substituting the last three rules:

Answer. $S \to A_Z$, $\quad A_Z \to \varepsilon \mid 0 \mid 1 \mid 0A_0 \mid 1A_1$,
$A_0 \to 0 \mid 00 \mid 10 \mid 0A_00 \mid 1A_10$, $\quad A_1 \to 1 \mid 01 \mid 11 \mid 0A_01 \mid 1A_11$.

By induction A_c generates exactly $\{sc \mid s = s^R\}$ ($c \in \{0,1\}$), so A_Z (and S) generates $\varepsilon, 0, 1$ and all $0s0$, $1s1$ with s a palindrome — exactly the palindromes.

2010–2011 • Q13(c) Source M13 **MARKS 8**

Run the given PDA separately on 1001 and 0101. What does it do?



SOLUTION

The diagram is drawn in the paper's notation ($a, b \to c$: read a, pop b, push c). In Atif sir's notation Z_0 is on the stack from the start and plays the role of \$, so q_1 and its push-\$ move disappear (the accepting q_1 only served to accept ε, which is still accepted via $q_2 \to q_3 \to q_4$). The machine is then the lecture's PDA for $\{ww^R\}$ (2016–17 Q7(b)) with its states q_0, q_1, q_2 named q_2, q_3, q_4: $P = (\{q_2, q_3, q_4\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_2, Z_0, \{q_4\})$ with

$$\begin{array}{lll} \delta(q_2,0,Z_0) = \{(q_2,0Z_0)\} & \delta(q_2,0,0) = \{(q_2,00)\} & \delta(q_2,0,1) = \{(q_2,01)\} \\ \delta(q_2,1,Z_0) = \{(q_2,1Z_0)\} & \delta(q_2,1,0) = \{(q_2,10)\} & \delta(q_2,1,1) = \{(q_2,11)\} \\ \delta(q_2,\varepsilon,Z_0) = \{(q_3,Z_0)\} & \delta(q_2,\varepsilon,0) = \{(q_3,0)\} & \delta(q_2,\varepsilon,1) = \{(q_3,1)\} \\ \delta(q_3,0,0) = \{(q_3,\varepsilon)\} & \delta(q_3,1,1) = \{(q_3,\varepsilon)\} & \delta(q_3,\varepsilon,Z_0) = \{(q_4,Z_0)\}. \end{array}$$

1001. The branch that guesses the middle after two symbols:

$$\begin{aligned} &(q_2,1001,Z_0) \vdash (q_2,001,1Z_0) \vdash (q_2,01,01Z_0) \\ &\vdash (q_3,01,01Z_0) \vdash (q_3,1,1Z_0) \vdash (q_3,\varepsilon,Z_0) \vdash (q_4,\varepsilon,Z_0). \end{aligned}$$

Accepted. (Branches guessing the middle elsewhere die on a mismatch or leave input unread.)
0101. Pushed prefix u, then the remaining input must equal u^R. Possible guesses: $u = \varepsilon$ (needs $0101 = \varepsilon$), $u = 0$ (rest 101 must start with 0), $u = 01$ (rest 01 vs $u^R = 10$, mismatch), $u = 010$ (rest 1 vs 010), $u = 0101$ (rest ε, but Z_0 is not on top). Every branch dies: **rejected**.
What it does: it accepts exactly the even-length palindromes $\{ww^R \mid w \in \{0,1\}^*\}$ (including ε).

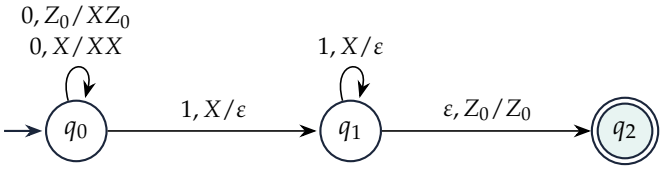
2009–2010 • Q10(a) Source M09 **MARKS 2 + 4 + 4 + 4 = 14**

Formally define a PDA. Design PDAs for (i) $\{0^n1^n \mid n \geq 1\}$; (ii) binary strings with equal numbers of 0's and 1's; (iii) all binary palindromes.

SOLUTION

Definition: see 2012–13 Q1(a) above.

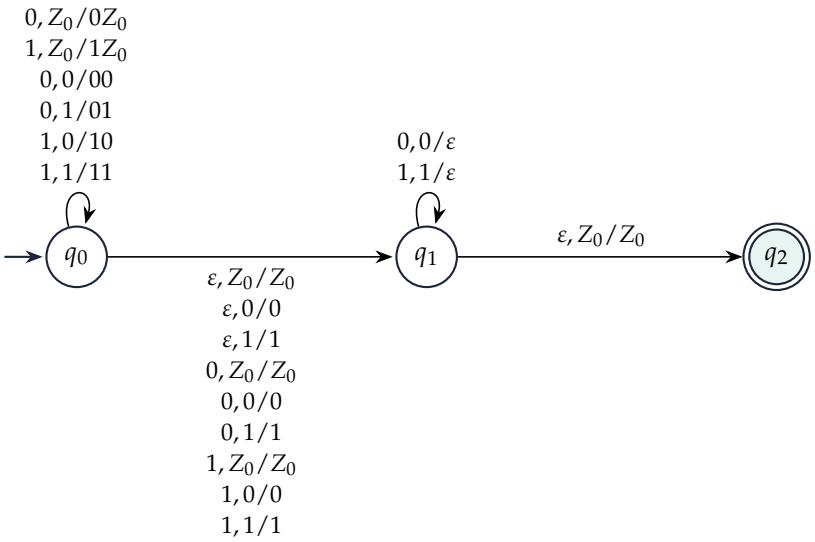
(i) $0^n1^n, n \geq 1$
$P = (\{q_0, q_1, q_2\}, \{0,1\}, \{X, Z_0\}, \delta, q_0, Z_0, \{q_2\})$ (the example of 2006–07 Q8):



(ii) Equal numbers of 0's and 1's

The surplus PDA of 2015–16 Q6(a) with $a \mapsto 0$, $b \mapsto 1$ (in the input and on the stack): $P = (\{q, f\}, \{0,1\}, \{0,1,Z_0\}, \delta, q, Z_0, \{f\})$.

(iii) All palindromes



This is the PDA of 2010–11 Q13(a). The middle move reads nothing (even length) or reads one symbol without pushing it (odd length).

2009–2010 • Q10(b) Source M09–M10 MARKS 3 + $6\frac{1}{3}$ = $9\frac{1}{3}$

Suppose the PDA $P = (\{p,q\}, \{0,1\}, \{Z,X\}, \delta, q, Z, \{p\})$ has the following transition functions:

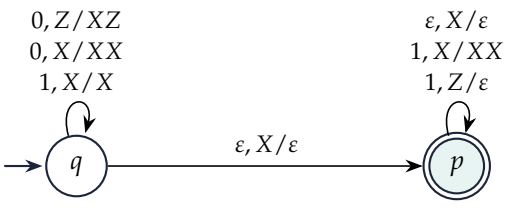
$$\begin{aligned}
\delta(q,0,Z) &= \{(q,XZ)\} \\
\delta(q,0,X) &= \{(q,XX)\} \\
\delta(q,1,X) &= \{(q,X)\} \\
\delta(q,\varepsilon,X) &= \{(p,\varepsilon)\} \\
\delta(p,\varepsilon,X) &= \{(p,\varepsilon)\} \\
\delta(p,1,X) &= \{(p,XX)\} \\
\delta(p,1,Z) &= \{(p,\varepsilon)\}
\end{aligned}$$

(i) Represent P by a transition diagram. (ii) Convert P to a PDA accepting by empty stack the same language that P accepts by final state. (Z plays the role of Z_0.)

SOLUTION

(i) Transition diagram

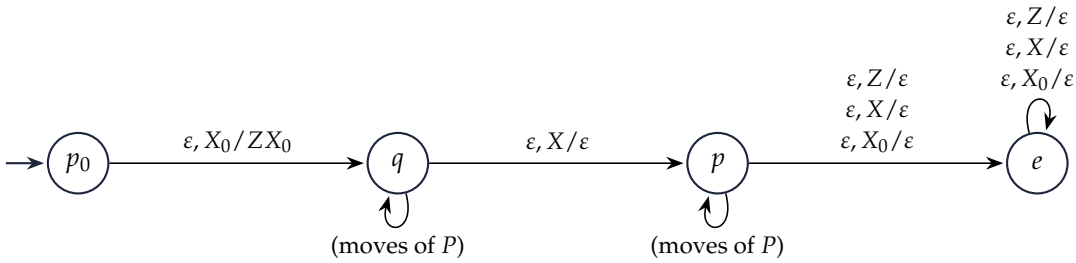
One arc label per rule of δ (start state q, final state p, Z initially on the stack):



(ii) Conversion to empty-stack acceptance

Apply the $P_F \to P_N$ construction from 2019–20 Q10(a) above. New states p_0 (start) and e (eraser), new bottom symbol X_0:

$$\delta_N(p_0, \varepsilon, X_0) = \{(q, ZX_0)\}, \quad \text{all seven moves of } P \text{ copied},$$
$$\delta_N(p, \varepsilon, Z) = \{(e, \varepsilon)\}, \quad \delta_N(p, \varepsilon, X) = \{(p, \varepsilon), (e, \varepsilon)\}, \quad \delta_N(p, \varepsilon, X_0) = \{(e, \varepsilon)\}$$
$$(p \text{ is the final state of } P;\ (p, \varepsilon) \in \delta_N(p, \varepsilon, X) \text{ is the copied move of } P),$$
$$\delta_N(e, \varepsilon, Z) = \{(e, \varepsilon)\}, \quad \delta_N(e, \varepsilon, X) = \{(e, \varepsilon)\}, \quad \delta_N(e, \varepsilon, X_0) = \{(e, \varepsilon)\}.$$



X_0 guarantees that the original move $\delta(p, 1, Z) = \{(p, \varepsilon)\}$, which empties P's own stack, does not by itself count as acceptance by P_N; acceptance happens only through e.

2009–2010 • Q11(a) Source M10 **MARKS 5**

Convert this grammar to a PDA accepting the same language by empty stack: $S \to AS \mid A$; $A \to 0A \mid 1B \mid 1$; $B \to 0B \mid 0$.

SOLUTION

$P = (\{q\}, \{0, 1\}, \{S, A, B, 0, 1\}, \delta, q, S)$, accepting by empty stack:

$$\delta(q, \varepsilon, S) = \{(q, AS), (q, A)\}, \quad \delta(q, \varepsilon, A) = \{(q, 0A), (q, 1B), (q, 1)\},$$
$$\delta(q, \varepsilon, B) = \{(q, 0B), (q, 0)\}, \quad \delta(q, 0, 0) = \{(q, \varepsilon)\}, \quad \delta(q, 1, 1) = \{(q, \varepsilon)\}.$$

2009–2010 • Q11(b) Source M10 **MARKS 13**

Convert $P = (\{p, q\}, \{0, 1\}, \{Z, X\}, \delta, q, Z)$ to a CFG, where $\delta(q, 1, Z) = \{(q, XZ)\}$; $\delta(q, 0, X) = \{(p, X)\}$; $\delta(p, 1, X) = \{(p, \varepsilon)\}$.

SOLUTION

Triple construction (empty-stack acceptance): variables $[rYs]$ meaning "from r, pop Y (net), ending in s"; $S \to [qZq] \mid [qZp]$.

- $\delta(q, 1, Z) \ni (q, XZ)$: $[qZs_2] \to 1\,[qXs_1][s_1Zs_2]$ for all $s_1, s_2 \in \{p, q\}$ (4 rules).
- $\delta(q, 0, X) \ni (p, X)$: $[qXs] \to 0\,[pXs]$ for $s \in \{p, q\}$ (2 rules).
- $\delta(p, 1, X) \ni (p, \varepsilon)$: $[pXp] \to 1$.

Simplify. Generating: $[pXp]$ ($\to 1$), hence $[qXp]$ ($\to 0[pXp]$). $[pXq]$ has no rules, so $[qXq]$ is not generating. No rule has head $[pZ\cdot]$, so $[pZp]$, $[pZq]$ are not generating; every rule for $[qZs_2]$ needs $[s_1Zs_2]$ with $[qXs_1]$ generating, i.e. $s_1 = p$, i.e. $[pZs_2]$ — not generating. Hence $[qZq]$, $[qZp]$, and S are **not generating**.

Answer. The grammar generates $\emptyset$: $N(P) = \emptyset$.
Indeed, P can push X on Z, but no transition ever pops Z, so the stack can never become empty.

2009–2010 • Q11(c) Source M10 **MARKS 5 $\frac{1}{3}$**

Briefly describe a deterministic PDA (DPDA) with an appropriate example.

SOLUTION

A PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is *deterministic* if in every situation at most one move is possible: for all q, a, X, (1) $|\delta(q, a, X)| \leq 1$ for $a \in \Sigma \cup \{\varepsilon\}$, and (2) if $\delta(q, a, X) \neq \emptyset$ for some $a \in \Sigma$ then $\delta(q, \varepsilon, X) = \emptyset$ (no choice between an ε-move and reading).
DPDAs (accepting by final state) recognise the *deterministic CFLs*, a proper subclass of the CFLs that strictly contains the regular languages; they are the basis of LR parsers, and every DCFL has an unambiguous grammar. (Languages accepted by DPDAs by empty stack have the prefix property.)
Example: $L_{wcwr} = \{wcw^R \mid w \in \{0,1\}^*\}$. $P = (\{q_0, q_1, q_2\}, \{0,1,c\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \{q_2\})$ with

$$\begin{array}{lll} \delta(q_0,0,Z_0) = \{(q_0,0Z_0)\} & \delta(q_0,0,0) = \{(q_0,00)\} & \delta(q_0,0,1) = \{(q_0,01)\} \\ \delta(q_0,1,Z_0) = \{(q_0,1Z_0)\} & \delta(q_0,1,0) = \{(q_0,10)\} & \delta(q_0,1,1) = \{(q_0,11)\} \\ \delta(q_0,c,Z_0) = \{(q_1,Z_0)\} & \delta(q_0,c,0) = \{(q_1,0)\} & \delta(q_0,c,1) = \{(q_1,1)\} \\ \delta(q_1,0,0) = \{(q_1,\varepsilon)\} & \delta(q_1,1,1) = \{(q_1,\varepsilon)\} & \delta(q_1,\varepsilon,Z_0) = \{(q_2,Z_0)\}. \end{array}$$

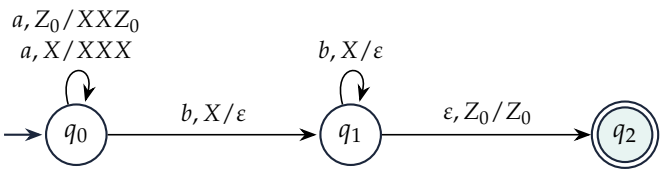
Every value of δ has at most one pair, and the only ε-move $\delta(q_1, \varepsilon, Z_0)$ has no competitor, since $\delta(q_1,0,Z_0) = \delta(q_1,1,Z_0) = \delta(q_1,c,Z_0) = \emptyset$. The centre marker c tells P when to switch from pushing to popping (this is the PDA of 2014–15 Q6(c) over $\{0,1\}$). By contrast $\{ww^R\}$ needs to *guess* the middle and has no DPDA.

2008–2009 • Q1(i) Source M06 **MARKS 10**

For $L = \{a^n b^{2n} \mid n > 0\}$, either design a PDA accepting it or prove that doing so is impossible.

SOLUTION

L is context-free ($S \to aSbb \mid abb$), so a PDA exists: push two X's per a (replace the top Z_0 by XXZ_0, or the top X by XXX), pop one per b.



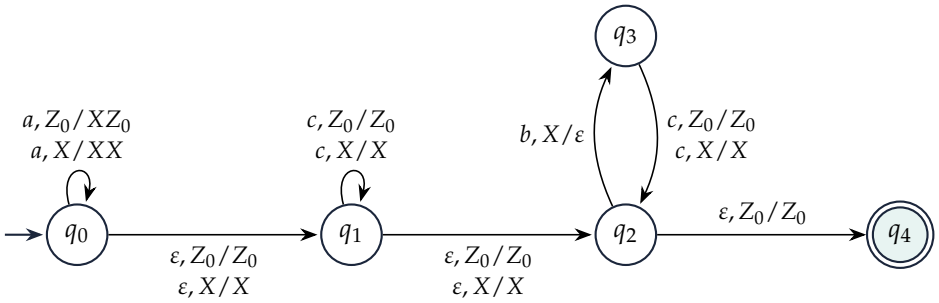
$P = (\{q_0, q_1, q_2\}, \{a, b\}, \{X, Z_0\}, \delta, q_0, Z_0, \{q_2\})$. The move $q_0 \to q_1$ requires an X on top, so $n > 0$ is enforced; $q_1 \to q_2$ requires Z_0 on top, i.e. exactly $2n$ b's.

2007–2008 • Q5 Source M05 **MARKS 10**

Draw a PDA transition diagram accepting $L = \{a^i c^k (bc)^i \mid i, k \geq 0\}$.

SOLUTION

Push one X per a; read the c^k without touching the stack; then each bc pair pops one X. A separate state for the c^k part prevents extra c's from being mixed into the $(bc)^i$ part.



$P = (\{q_0, \ldots, q_4\}, \{a, b, c\}, \{X, Z_0\}, \delta, q_0, Z_0, \{q_4\})$.

2006–2007 • Q8 Source M03 **MARKS 10**

Define a pushdown automaton (PDA) with an example. What is an instantaneous description (ID) of a PDA?

SOLUTION

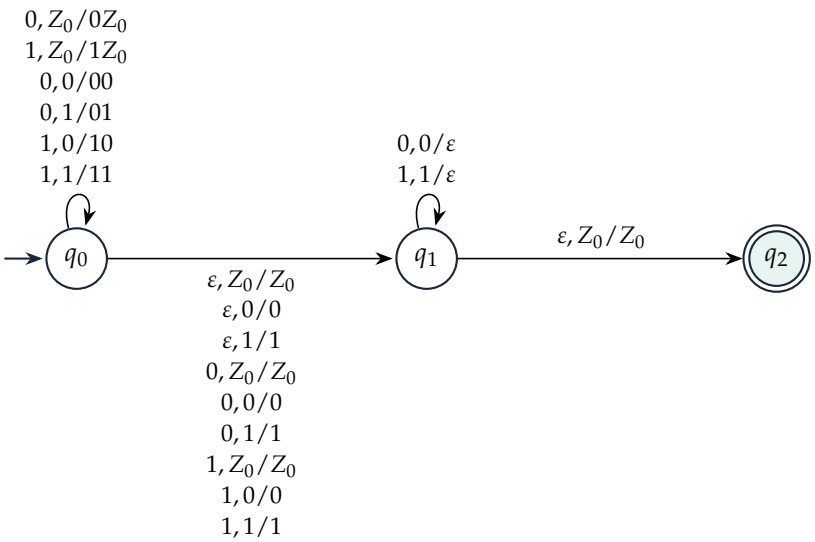
A PDA is a 7-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$: Q finite states; Σ input alphabet; Γ stack alphabet; $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to$ finite subsets of $Q \times \Gamma^*$; q_0 start state; $Z_0 \in \Gamma$ initial stack symbol; $F \subseteq Q$ accepting states. It is an ε-NFA with an unbounded stack.
Example: $\{0^n 1^n \mid n \geq 1\}$: $\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$, $\delta(q_0, 0, X) = \{(q_0, XX)\}$, $\delta(q_0, 1, X) = \{(q_1, \varepsilon)\}$, $\delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$, $\delta(q_1, \varepsilon, Z_0) = \{(q_2, Z_0)\}$, $F = \{q_2\}$.
Instantaneous description: a triple (q, w, γ) — current state q, the remaining input w, and the stack contents γ (top at the left). It captures the entire configuration. One move: $(q, aw, X\beta) \vdash (p, w, \alpha\beta)$ if $(p, \alpha) \in \delta(q, a, X)$. E.g. on 0011: $(q_0, 0011, Z_0) \vdash (q_0, 011, XZ_0) \vdash (q_0, 11, XXZ_0) \vdash (q_1, 1, XZ_0) \vdash (q_1, \varepsilon, Z_0) \vdash (q_2, \varepsilon, Z_0)$ — accepted.

2006–2007 • Q9 Source M03 **MARKS 10**

Draw a PDA transition diagram to accept palindromes of all lengths over $\{0, 1\}$.

SOLUTION

Same as 2009–10 Q10(a)(iii): push the first half; at the middle either read nothing (even length) or read one symbol without pushing it (odd length); then pop while matching, and accept when Z_0 is exposed.



$P = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_2\})$.

11

CFL pumping lemma and non-context-free languages

NOTE CFL Pumping Lemma. If L is context-free, there is an integer $p \geq 1$ (the pumping length) such that every $w \in L$ with $|w| \geq p$ can be written $w = uvxyz$ with (1) $|vxy| \leq p$, (2) $|vy| > 0$, (3) $uv^ixy^iz \in L$ for every $i \geq 0$. To show L is *not* context-free: assume L is context-free and let p be its pumping length; *we* choose $w \in L$ (in terms of p) with $|w| \geq p$; the *adversary* chooses any split $w = uvxyz$ obeying (1),(2); *we* choose i and show $uv^ixy^iz \notin L$ for *every* such split — a contradiction, so L is not context-free.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q7 Source N03 **MARKS 5 + 10 = 15**

Give a non-context-free language and prove it is not context-free using the CFL pumping lemma.

2021–2022 • Q8 Source M48 **MARKS 5 + 10 = 15**

Give an example of a non-context-free language. Prove it is not context-free using the CFL pumping lemma.

2016–2017 • Q7(c) Source M31 **MARKS 10**

Use the pumping lemma to show that $\{a^nb^nc^n \mid n \geq 0\}$ is not context-free.

2012–2013 • Q2(b) Source M17 **MARKS $13\frac{1}{3}$**

Formally state the pumping lemma for CFLs. Use it to show that $\{a^nb^nc^n \mid n \geq 0\}$ is not context-free.

SOLUTION

SCOPE *Only 2012–2013 Q2(b) asks for the formal statement; the other papers need just the example and its proof.*

Statement (formal): For every CFL L there exists an integer $p \geq 1$ such that for all $w \in L$ with $|w| \geq p$ there exist u, v, x, y, z with $w = uvxyz$, $|vxy| \leq p$, $|vy| > 0$, and $\forall i \geq 0 :\ uv^ixy^iz \in L$.

Example: $L = \{a^nb^nc^n \mid n \geq 0\}$ is not context-free.

Proof. Assume, for contradiction, that L is context-free, and let p be its pumping length. Choose $w = a^pb^pc^p \in L$, $|w| = 3p \geq p$. Let $w = uvxyz$ be any split with $|vxy| \leq p$ and $|vy| > 0$. Pump with $i = 2$.

- *Case 1: v or y contains two different symbols.* Then uv^2xy^2z contains them out of order (e.g. $\ldots ab \ldots ab \ldots$), so it is not of the form $a^*b^*c^*$ and not in L.
- *Case 2: v and y each consist of one repeated symbol.* Since $|vxy| \leq p$ and each block has length p, vxy cannot touch both the a-block and the c-block. Hence v and y together contain at most two of the three letters, and at least one letter appears (as $|vy| > 0$). Pumping up, uv^2xy^2z has more of the (one or two) touched letters but still exactly p of the untouched letter, so the three counts are not equal: $uv^2xy^2z \notin L$.

Every split fails condition (3) for $i = 2$; this contradicts the pumping lemma. L is not context-free.

■

(Other standard examples: $\{ww \mid w \in \{0,1\}^*\}$, $\{a^{n^2}\}$, $\{a^i b^j c^k \mid i < j < k\}$.)

2022–2023 • Q9 Source M52 **MARKS 15**

State and prove the CFL pumping lemma. You may use the theorem: if a CNF parse tree yields terminal string w and its longest path has length n, then $|w| \leq 2^{n-1}$.

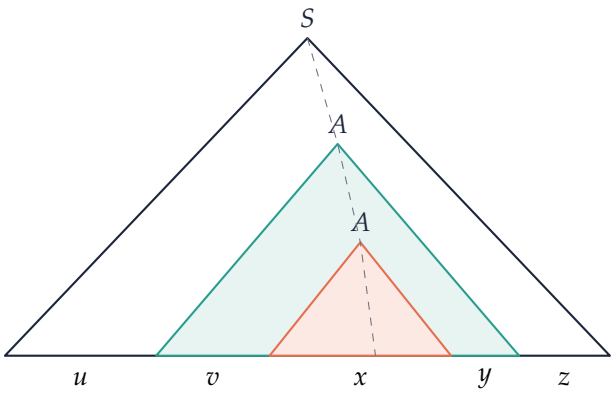
SOLUTION

Statement. If L is context-free, there is an integer $p \geq 1$ (the pumping length) such that every $w \in L$ with $|w| \geq p$ can be written $w = uvxyz$ with (1) $|vxy| \leq p$, (2) $|vy| > 0$, (3) $uv^i xy^i z \in L$ for every $i \geq 0$.

Proof. If $L = \emptyset$ or $L = \{\varepsilon\}$ any $p \geq 1$ works vacuously, so assume otherwise and let $G = (V, T, P, S)$ be a CNF grammar for $L - \{\varepsilon\}$ with $|V| = m$ variables. Let $p = 2^m$.

Take $w \in L$ with $|w| \geq p = 2^m$ (so $w \neq \varepsilon$ and $w \in L(G)$), and a parse tree for w. By the given theorem, if its longest path had length $\leq m$ then $|w| \leq 2^{m-1} < 2^m$. Hence the longest path has length $\geq m+1$ (edges), i.e. at least $m+2$ nodes; all but the last (a leaf) are variables, so the path contains at least $m+1$ variable occurrences.

Look at the *last* (lowest) $m+1$ variables on this longest path. By the pigeonhole principle two of them are equal: say $A_i = A_j = A$ with A_i above A_j. This gives the picture



where the subtree of the upper A yields vxy and the subtree of the lower A yields x. So $S \Rightarrow^* uAz$, $A \Rightarrow^* vAy$, $A \Rightarrow^* x$.

- **(3) Pumping.** Replacing the lower subtree by a copy of the upper one (repeatedly), or the upper subtree by the lower one, gives $S \Rightarrow^* uAz \Rightarrow^* uv^i Ay^i z \Rightarrow^* uv^i xy^i z$ for every $i \geq 0$; so $uv^i xy^i z \in L(G) \subseteq L$.
- **(2) $|vy| > 0$.** In CNF the upper A uses a rule $A \to BC$; one of B, C contains the lower A and the other yields a non-empty terminal string (a CNF grammar for $L - \{\varepsilon\}$ has no ε-rules), which is part of v or of y.
- **(1) $|vxy| \leq p$.** The path from the upper A down to the leaf, inside the longest path, has at most $m+1$ edges (we chose A_i among the last $m+1$ variables). It is a longest path of the upper A's subtree (a longer one there would give a longer path in the whole tree). By the theorem, the yield vxy of that subtree has length $\leq 2^{(m+1)-1} = 2^m = p$.

Hence $p = 2^m$ is a pumping length for L. ■

2014–2015 • Q6(b) Source M23 **MARKS 7**

Show that $L = \{a^n b^j c^k \mid k = jn\}$ over $\{a, b, c\}$ is not context-free.

SOLUTION

Assume, for contradiction, that L is context-free, and let p be its pumping length. Choose $w = a^p b^p c^{p^2} \in L$ ($k = p \cdot p$), $|w| = 2p + p^2 \geq p$. Let $w = uvxyz$ be any split with $|vxy| \leq p$, $|vy| > 0$, and pump with $i = 2$ (counts n', j', k' of a, b, c in uv^2xy^2z).

- v or y mixes two letters: $uv^2xy^2z \notin a^*b^*c^*$.
- vxy inside the a's: $n' > p$, $j' = p$, $k' = p^2$; $j'n' > p^2$. ×
- inside the b's: $n' = p$, $j' > p$; $j'n' > p^2 = k'$. ×
- inside the c's: $k' > p^2 = j'n'$. ×
- $v \subseteq a$'s, $y \subseteq b$'s (gains $\alpha, \beta \geq 0$, $\alpha + \beta \geq 1$): $(p + \alpha)(p + \beta) > p^2 = k'$. ×
- $v \subseteq b$'s, $y \subseteq c$'s (gains α in b, β in c, $\alpha + \beta \geq 1$, $\alpha + \beta \leq p$): need $p(p + \alpha) = p^2 + \beta$, i.e. $\beta = p\alpha$. If $\alpha = 0$ then $\beta = 0$, impossible; if $\alpha \geq 1$ then $\beta = p\alpha \geq p > p - \alpha \geq \beta$, impossible. ×
- vxy cannot touch both a's and c's, because the b-block has length p.

In every case $uv^2xy^2z \notin L$, contradicting the pumping lemma. Hence L is not context-free. ■

2013–2014 • Q2(c) Source M19 **MARKS 9 $\frac{1}{3}$**

Use the CFL pumping lemma to show these languages are not context-free: (i) $\{0^n1^n0^n1^n \mid n \geq 0\}$; (ii) $\{0^n\#0^{2n}\#0^{3n} \mid n \geq 0\}$.

SOLUTION

(i) Assume $L = \{0^n1^n0^n1^n \mid n \geq 0\}$ is context-free and let p be its pumping length. Choose $w = 0^p1^p0^p1^p \in L$, $|w| = 4p \geq p$, and let $w = uvxyz$ be any split with $|vxy| \leq p$, $|vy| > 0$. Pump with $i = 2$. If v or y straddles a block boundary, pumping breaks the four-block shape $0^*1^*0^*1^*$. Otherwise, since $|vxy| \leq p$, vxy lies within one block or two *adjacent* blocks, so pumping up lengthens at least one block (as $|vy| > 0$) while at least two blocks stay at length p. Then not all four blocks are equal: $uv^2xy^2z \notin L$. Contradiction; not context-free.

(ii) Assume $L = \{0^n\#0^{2n}\#0^{3n} \mid n \geq 0\}$ is context-free and let p be its pumping length. Choose $w = 0^p\#0^{2p}\#0^{3p} \in L$, $|w| = 6p + 2 \geq p$, and let $w = uvxyz$ be any split with $|vxy| \leq p$, $|vy| > 0$. Pump with $i = 2$. If v or y contains #, then uv^2xy^2z has more than two #'s: not in L. Otherwise v, y lie in the 0-blocks; since $|vxy| \leq p$ and the middle block has length $2p$, they touch at most two (adjacent) blocks. If the first block is untouched it stays 0^p, forcing $n = p$, but another block changed. If the first block is touched (it gains $\alpha \geq 0$, the second gains $\beta \geq 0$, $\alpha + \beta \geq 1$) the third block stays 0^{3p}, forcing $n = p$, so $\alpha = \beta = 0$ — contradiction. So $uv^2xy^2z \notin L$ in every case, contradicting the pumping lemma. Not context-free. ■

2013–2014 • Q7(c) Source M21 **MARKS 6**

Show that $\{a^n b^{2n} c^{3n} \mid n \geq 0\}$ is not context-free.

SOLUTION

Let $L = \{a^n b^{2n} c^{3n} \mid n \geq 0\}$. Assume, for contradiction, that L is context-free, and let p be its pumping length. Choose $w = a^p b^{2p} c^{3p} \in L$, $|w| = 6p \geq p$. Let $w = uvxyz$ be any split with $|vxy| \leq p$ and $|vy| > 0$. Pump with $i = 2$.

- *Case 1: v or y contains two different symbols.* Then uv^2xy^2z has the symbols out of order, so it is not in $a^*b^*c^*$ and not in L.
- *Case 2: v and y each lie inside one block.* vxy cannot touch both the a-block and the c-block, since it would have to contain the whole b-block and $|vxy| \geq 2p + 2 > p$.
  - If the c-block is untouched, uv^2xy^2z still has $3p$ c's, which forces $n = p$; but vy added at least one a or b, so there are more than p a's or more than $2p$ b's.
  - If the c-block is touched, the a-block is untouched: there are still p a's, which forces $n = p$; but vy added at least one b or c, so there are more than $2p$ b's or more than $3p$ c's.

  Either way $uv^2xy^2z \notin L$.

Every split fails for $i = 2$, contradicting the pumping lemma. Hence L is not context-free. ■

2008–2009 • Q1(ii) Source M06 **MARKS 13 $\frac{1}{3}$**

For $L = \{a^n b^{2n} c^n \mid n \geq 0\}$, either design a PDA accepting it or prove that doing so is impossible.

SOLUTION

PDAs accept exactly the context-free languages, so a PDA for L exists iff L is context-free. We show that L is **not** context-free.
Assume, for contradiction, that L is context-free, and let p be its pumping length. Choose $w = a^p b^{2p} c^p \in L$, $|w| = 4p \geq p$. Let $w = uvxyz$ be any split with $|vxy| \leq p$ and $|vy| > 0$. Pump with $i = 2$.

- *Case 1: v or y contains two different symbols.* Then uv^2xy^2z has the symbols out of order, so it is not in $a^*b^*c^*$ and not in L.
- *Case 2: v and y each lie inside one block.* vxy cannot touch both the a-block and the c-block, since it would have to contain the whole b-block and $|vxy| \geq 2p + 2 > p$.
  - If the c-block is untouched, uv^2xy^2z still has p c's, which forces $n = p$; but vy added at least one a or b, so there are more than p a's or more than $2p$ b's.
  - If the c-block is touched, the a-block is untouched: there are still p a's, which forces $n = p$; but vy added at least one b or c, so there are more than $2p$ b's or more than p c's.

  Either way $uv^2xy^2z \notin L$.

Every split fails for $i = 2$, contradicting the pumping lemma. Hence L is not context-free, and therefore **no PDA accepts** L: designing one is impossible. Intuitively, a single stack can compare the a's with the b's but has then forgotten n when the c's arrive.

12

Turing-machine design and computation

NOTE Conventions (Hopcroft–Motwani–Ullman, as in the slides). A TM is $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$; the tape is infinite in both directions and every cell outside the input holds the blank $B \in \Gamma - \Sigma$. $\delta(q, X) = (p, Y, D)$ means: in state q scanning X, go to state p, write Y and move the head one cell in direction $D \in \{L, R\}$ (a one-tape TM must move; only multitape machines also have S = stay). In tables the entry in row q, column X is $\delta(q, X)$; an empty entry means $\delta(q, X)$ is undefined: there is no move and M *halts*. M accepts w iff $q_0 w \vdash^* \alpha p \beta$ for some $p \in F$; our accepting states have no moves, so M halts as soon as it accepts, and halting in a non-accepting state means the input is rejected. A machine that computes a function halts in its final state h with the result on the tape. In a diagram an arc from q to p labelled $X/Y\,D$ means $\delta(q, X) = (p, Y, D)$; several labels on one arc are separated by commas or written one below the other. An ID $X_1 \cdots X_{i-1}\, q\, X_i \cdots X_n$ shows the state q to the left of the scanned symbol X_i and only the cells from the leftmost to the rightmost nonblank (plus the scanned blank when the head is outside that part); a B written over the first or last nonblank joins the surrounding blanks and is not shown. Storage in the state: a state such as r_b is the slides' pair $[r, b]$, a control state r with the bit b remembered. Multiple tracks: a checked symbol such as x stands for the two-track symbol $[*, 0]$ ($y = [*, 1]$), i.e. a check mark on the second track above the bit.

2023–2024 • Q9 Source N03 **MARKS 15**

Design a TM comparing binary w_1 and w_2. Initially the tape is $w_1 c w_2 c$. Write 0 after the second c if equal and 1 otherwise. You may assume equal lengths and use multiple tracks and state storage.

SOLUTION

Formal notation. State storage $[q, a, d]$ (a = the bit of w_1 being carried, d = the result so far, 0 = equal) and two tracks $[t, X]$ ($t \in \{B, *\}$ is the check mark, $X \in \{0, 1, c, B\}$ the symbol). $M = (Q, \Sigma, \Gamma, \delta, [q_0, B, 0], [B, B], \{[q_6, B, B]\})$ with input symbols $[B, 0], [B, 1], [B, c]$. The start state is $[q_0, B, 0]$; the final state $[q_6, B, B]$ has no moves, so M halts there after writing the answer (it computes by halting). Here $a, b \in \{0, 1\}$ and $\bar{a} = 1 - a$.

Transition rules:

1. $\delta([q_0, B, 0], [B, a]) = ([q_1, a, 0], [*, a], R)$
2. $\delta([q_1, a, 0], [B, b]) = ([q_1, a, 0], [B, b], R)$
3. $\delta([q_1, a, 0], [B, c]) = ([q_2, a, 0], [B, c], R)$
4. $\delta([q_2, a, 0], [*, b]) = ([q_2, a, 0], [*, b], R)$
5. $\delta([q_2, a, 0], [B, a]) = ([q_3, B, 0], [*, a], L)$
6. $\delta([q_3, B, 0], [*, b]) = ([q_3, B, 0], [*, b], L)$
7. $\delta([q_3, B, 0], [B, c]) = ([q_4, B, 0], [B, c], L)$
8. $\delta([q_4, B, 0], [B, 0/1]) = ([q_4, B, 0], [B, 0/1], L)$
9. $\delta([q_4, B, 0], [*, 0/1]) = ([q_0, B, 0], [*, 0/1], R)$
10. $\delta([q_0, B, 0], [B, c]) = ([q_5, B, 0], [B, c], R)$
11. $\delta([q_5, B, d], [*/B, 0/1/c]) = ([q_5, B, d], [*/B, 0/1/c], R) \quad [d = 0/1]$
12. $\delta([q_5, B, d], [B, B]) = ([q_6, B, B], [B, d], R)$
13. $\delta([q_2, a, 0], [B, \bar{a}]) = ([q_5, B, 1], [*, \bar{a}], R)$

2022–2023 • Q10 Source M52 **MARKS 15**

Design a TM computing bitwise NOT of binary w. Initially the tape is wc, with separator c. Insert the NOT of w in binary after c and terminate. Multiple tracks and state storage are allowed.

SOLUTION

Formal notation. State storage $[q, a]$ (a = the bit being carried) and two tracks $[t, X]$ ($t \in \{B, *\}$ is the check mark, $X \in \{0, 1, c, B\}$). $M = (Q, \Sigma, \Gamma, \delta, [q_0, B], [B, B], \{[q_5, B]\})$ with input symbols $[B, 0], [B, 1], [B, c]$. The start state is $[q_0, B]$; the final state $[q_5, B]$ has no moves, so M halts there with the NOT of w after c (it computes by halting). Here $a, b \in \{0, 1\}$ and $\bar{a} = 1 - a$.

Transition rules:

1. $\delta([q_0, B], [B, a]) = ([q_1, a], [*, a], R)$
2. $\delta([q_1, a], [B, b]) = ([q_1, a], [B, b], R)$
3. $\delta([q_1, a], [B, c]) = ([q_2, a], [B, c], R)$
4. $\delta([q_2, a], [*, 0/1]) = ([q_2, a], [*, 0/1], R)$
5. $\delta([q_2, a], [B, B]) = ([q_3, B], [*, \bar{a}], L)$
6. $\delta([q_3, B], [*, 0/1]) = ([q_3, B], [*, 0/1], L)$
7. $\delta([q_3, B], [B, c]) = ([q_4, B], [B, c], L)$
8. $\delta([q_4, B], [B, 0/1]) = ([q_4, B], [B, 0/1], L)$
9. $\delta([q_4, B], [*, 0/1]) = ([q_0, B], [*, 0/1], R)$
10. $\delta([q_0, B], [B, c]) = ([q_5, B], [B, c], R)$

2021–2022 • Q9 Source M48 MARKS 15

Construct a TM converting signed-magnitude binary w to two's complement. The leftmost bit is the MSB. If the MSB is 0, leave the number unchanged. If it is 1, scan from the LSB: copy zeros up to the first 1, copy that 1, and flip all remaining bits, leaving the MSB as 1. Finish with only the two's-complement number on the tape.

SOLUTION

Formal notation. $M = (\{q_0, q_1, q_2, q_3, q_4, q_5\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_5\})$. The start state is q_0; the final state q_5 has no moves, so M halts there with the two's-complement number on the tape (it computes by halting). An entry "—" means no move.

State	0	1	B
$\to q_0$	$(q_5, 0, R)$	$(q_1, 1, R)$	—
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	(q_2, B, L)
q_2	$(q_2, 0, L)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	$(q_3, 0, L)$	(q_4, B, R)
q_4	$(q_5, 1, R)$	—	—
$*q_5$	—	—	—

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2019–2020 • Q9 Source M42 MARKS 15

Design a TM computing AND of equal-length binary w_1 and w_2. Initially the tape is $w_1 c w_2 c$. Finish with their AND in binary after the second c. Multiple tracks and state storage are allowed.

January 2020 Term • Q9 Source M38 **MARKS 15**

Design a TM computing XOR of equal-length binary w_1 and w_2. Initially the tape contains $w_1 c w_2 c$. Finish with the XOR in binary after the second c.

2017–2018 • Q9 Source M34 **MARKS 15**

Design a TM computing bitwise OR of equal-length binary M_1 and M_2. Initially the tape is $M_1 \# M_2 \#$. Finish with the binary OR after the second #.

SOLUTION

Formal notation. State storage $[q, a, b]$ (a = the bit of w_1, b = the bit of w_2 being carried) and two tracks $[t, X]$ ($t \in \{B, *\}$ is the check mark, $X \in \{0, 1, c, B\}$). $M = (Q, \Sigma, \Gamma, \delta, [q_0, B, B], [B, B], \{[q_8, B, B]\})$ with input symbols $[B, 0], [B, 1], [B, c]$. The start state is $[q_0, B, B]$; the final state $[q_8, B, B]$ has no moves, so M halts there with the result after the second c (it computes by halting). Here $a, b \in \{0, 1\}$.

Rule of d (the bit written in rule 9): $d = a \wedge b$ for AND (2019–2020 Q9), $d = a \oplus b$ for XOR (January 2020 Term Q9), $d = a \vee b$ for OR (2017–2018 Q9; there the separator is #, so read $[B, \#]$ for $[B, c]$ in every rule).

a	b	AND	XOR	OR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	0	1

Transition rules:

1. $\delta([q_0, B, B], [B, a]) = ([q_1, a, B], [*, a], R)$
2. $\delta([q_1, a, B], [B, b]) = ([q_1, a, B], [B, b], R)$
3. $\delta([q_1, a, B], [B, c]) = ([q_2, a, B], [B, c], R)$
4. $\delta([q_2, a, B], [*, 0/1]) = ([q_2, a, B], [*, 0/1], R)$
5. $\delta([q_2, a, B], [B, b]) = ([q_3, a, b], [*, b], R)$
6. $\delta([q_3, a, b], [B, 0/1]) = ([q_3, a, b], [B, 0/1], R)$
7. $\delta([q_3, a, b], [B, c]) = ([q_4, a, b], [B, c], R)$
8. $\delta([q_4, a, b], [B, 0/1]) = ([q_4, a, b], [B, 0/1], R)$
9. $\delta([q_4, a, b], [B, B]) = ([q_5, B, B], [B, d], L)$ (where d is given by the rule above)
10. $\delta([q_5, B, B], [B, 0/1]) = ([q_5, B, B], [B, 0/1], L)$
11. $\delta([q_5, B, B], [B, c]) = ([q_6, B, B], [B, c], L)$
12. $\delta([q_6, B, B], [*/B, 0/1]) = ([q_6, B, B], [*/B, 0/1], L)$
13. $\delta([q_6, B, B], [B, c]) = ([q_7, B, B], [B, c], L)$
14. $\delta([q_7, B, B], [B, 0/1]) = ([q_7, B, B], [B, 0/1], L)$
15. $\delta([q_7, B, B], [*, 0/1]) = ([q_0, B, B], [*, 0/1], R)$
16. $\delta([q_0, B, B], [B, c]) = ([q_8, B, B], [B, c], R)$

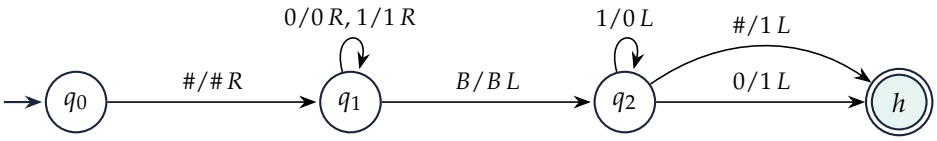
2016–2017 • Q5(b) Source M30 **MARKS 20**

Design a TM adding 1 to binary N. Initially the leftmost tape portion is #N (e.g. #101), with the head on #. Halt after replacing N by $N+1$; you may replace # by 0 or 1 if necessary. Briefly explain the working principle.

SOLUTION

Principle (pseudocode).

```
move right to the last bit of N
while bit == 1: write 0, move left      // propagate carry
if bit == 0: write 1, halt              // carry absorbed
if symbol == '#': write 1, halt         // N was all 1's: overflow
```



State	#	0	1	B
$\to q_0$	$(q_1,\#,R)$			
q_1		$(q_1,0,R)$	$(q_1,1,R)$	(q_2,B,L)
q_2	$(h,1,L)$	$(h,1,L)$	$(q_2,0,L)$	
$*h$				

Transition rules (numbered as in the slides):

1. $\delta(q_0,\#) = (q_1,\#,R)$ — step off #.
2. $\delta(q_1,a) = (q_1,a,R)$ for $a \in \{0,1\}$; $\delta(q_1,B) = (q_2,B,L)$ — go to the last bit.
3. $\delta(q_2,1) = (q_2,0,L)$ — $1+1$: write 0, carry on.
4. $\delta(q_2,0) = (h,1,L)$ — carry absorbed: halt.
5. $\delta(q_2,\#) = (h,1,L)$ — N was all 1's: # becomes the new leading 1; halt.

h has no moves, so M halts there. Examples: $q_0\#101 \vdash \#q_1101 \vdash^3 \#101q_1B \vdash \#10q_21 \vdash \#1q_200 \vdash \#h110$: #101 $\to$ #110 (5+1=6). #11 $\to$ the two 1's become 0, then # becomes 1: `100` (3+1=4) — the replaced # supplies the new leading digit.

2016–2017 • Q6(c) Source M31 **MARKS 10**

Define a TM for strings with equal numbers of 0's and 1's.

SOLUTION

(As in the slides the tape is infinite in both directions, with blanks on both sides of the input.) Repeatedly cross off the leftmost remaining symbol and one occurrence of the opposite symbol.

State	0	1	X	B	
$\to q_0$	(q_1, X, R)	(q_2, X, R)	(q_0, X, R)	(q_{acc}, B, R)	find next uncrossed symbol
q_1	$(q_1, 0, R)$	(q_3, X, L)	(q_1, X, R)		need a 1
q_2	(q_3, X, L)	$(q_2, 1, R)$	(q_2, X, R)		need a 0
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_3, X, L)	(q_0, B, R)	return to left end
$*q_{acc}$					accept

Transition rules (numbered as in the slides):

1. $\delta(q_0, 0) = (q_1, X, R)$, $\delta(q_0, 1) = (q_2, X, R)$ — cross off the leftmost remaining symbol, remember which.
2. $\delta(q_0, X) = (q_0, X, R)$; $\delta(q_0, B) = (q_{acc}, B, R)$ — skip crossed symbols; nothing left: accept.
3. $\delta(q_1, a) = (q_1, a, R)$ for $a \in \{0, X\}$; $\delta(q_1, 1) = (q_3, X, L)$ — find a 1 and cross it off.
4. $\delta(q_2, a) = (q_2, a, R)$ for $a \in \{1, X\}$; $\delta(q_2, 0) = (q_3, X, L)$ — find a 0 and cross it off.
5. $\delta(q_3, a) = (q_3, a, L)$ for $a \in \{0, 1, X\}$; $\delta(q_3, B) = (q_0, B, R)$ — return to the left end.

Formally $M = (\{q_0, q_1, q_2, q_3, q_{acc}\}, \{0,1\}, \{0, 1, X, B\}, \delta, q_0, B, \{q_{acc}\})$. Each round removes one 0 and one 1; M accepts (enters q_{acc}, which has no moves) when nothing is left. If a symbol has no partner, q_1 or q_2 reaches B, where δ is undefined: M halts without accepting. Example 0110: $X110 \to XX10$ (round 1), $XXX0 \to XXXX$ (round 2), then q_0 runs over the X's to B: accept.

2015–2016 • Q7(a) Source M27 **MARKS 13**

Design a TM for $\{a^n b^n c^n \mid n \geq 1\}$. Briefly describe its working principle.

SOLUTION

Formal notation. Two tracks $[t, X]$ ($t \in \{B, *\}$ is the check mark, $X \in \{a, b, c, B\}$). $M = (\{q_0, q_1, q_2, q_3, q_4, q_7\}, \Sigma, \Gamma, \delta, q_0, [B, B], \{q_7\})$ with input symbols $[B, a], [B, b], [B, c]$. The start state is q_0; M accepts by entering the final state q_7 ($F = \{q_7\}$). If no rule applies in any other state, M halts without accepting, i.e. it rejects.

Transition rules:

1. $\delta(q_0, [B, a]) = (q_1, [*, a], R)$
2. $\delta(q_1, [B, a]) = (q_1, [B, a], R)$
3. $\delta(q_1, [B, b]) = (q_2, [*, b], R)$
4. $\delta(q_1, [*, b]) = (q_1, [*, b], R)$
5. $\delta(q_2, [B, b]) = (q_2, [B, b], R)$
6. $\delta(q_2, [*, c]) = (q_2, [*, c], R)$
7. $\delta(q_2, [B, c]) = (q_3, [*, c], L)$
8. $\delta(q_3, [*/B, b/c]) = (q_3, [*/B, b/c], L)$
9. $\delta(q_3, [B, a]) = (q_3, [B, a], L)$
10. $\delta(q_3, [*, a]) = (q_0, [*, a], R)$
11. $\delta(q_0, [*, b]) = (q_4, [*, b], R)$
12. $\delta(q_4, [*, b/c]) = (q_4, [*, b/c], R)$
13. $\delta(q_4, [B, B]) = (q_7, [B, B], R) \quad (F = \{q_7\})$

2015–2016 • Q8(a) Source M27 **MARKS 13**

Design a TM computing the printed truncated-subtraction function $a \ominus b = \max(b - a, 0)$. Initially a and b are unary blocks of consecutive 1's separated by 0. Finish with only the unary

result surrounded by blanks. Briefly describe how it works.

SOLUTION

Formal notation. $M = (\{q_0, \ldots, q_7\}, \{0,1\}, \{0,1,B\}, \delta, q_0, B, \{q_6\})$, input 1^a01^b. The start state is q_0; the final state q_6 has no moves, so M halts there with the result on the tape (it computes by halting). An entry "—" means no move.

State	0	1	B
$\to q_0$	(q_5, B, R)	(q_1, B, R)	—
q_1	$(q_2, 0, R)$	$(q_1, 1, R)$	—
q_2	$(q_2, 0, R)$	$(q_3, 0, L)$	(q_7, B, L)
q_3	$(q_3, 0, L)$	$(q_4, 1, L)$	(q_0, B, R)
q_4	—	$(q_4, 1, L)$	(q_0, B, R)
q_5	(q_5, B, R)	$(q_5, 1, R)$	(q_6, B, R)
$*q_6$	—	—	—
q_7	(q_7, B, L)	(q_7, B, L)	(q_6, B, R)

How it works. Each round cancels one 1 of a against one 1 of b: q_0 erases the leftmost 1 of the a-block, q_1 runs to the separator 0, q_2 skips the 0's and turns the first remaining 1 of the b-block into 0, and q_3/q_4 walk back to the blank on the left. If the a-block is used up (q_0 reads 0), q_5 erases the 0's and keeps the 1's: 1^{b-a} remains. If the b-block is used up (q_2 reaches the blank), $b \le a$ and q_7 erases everything to its left: the result is 0 (blank tape).

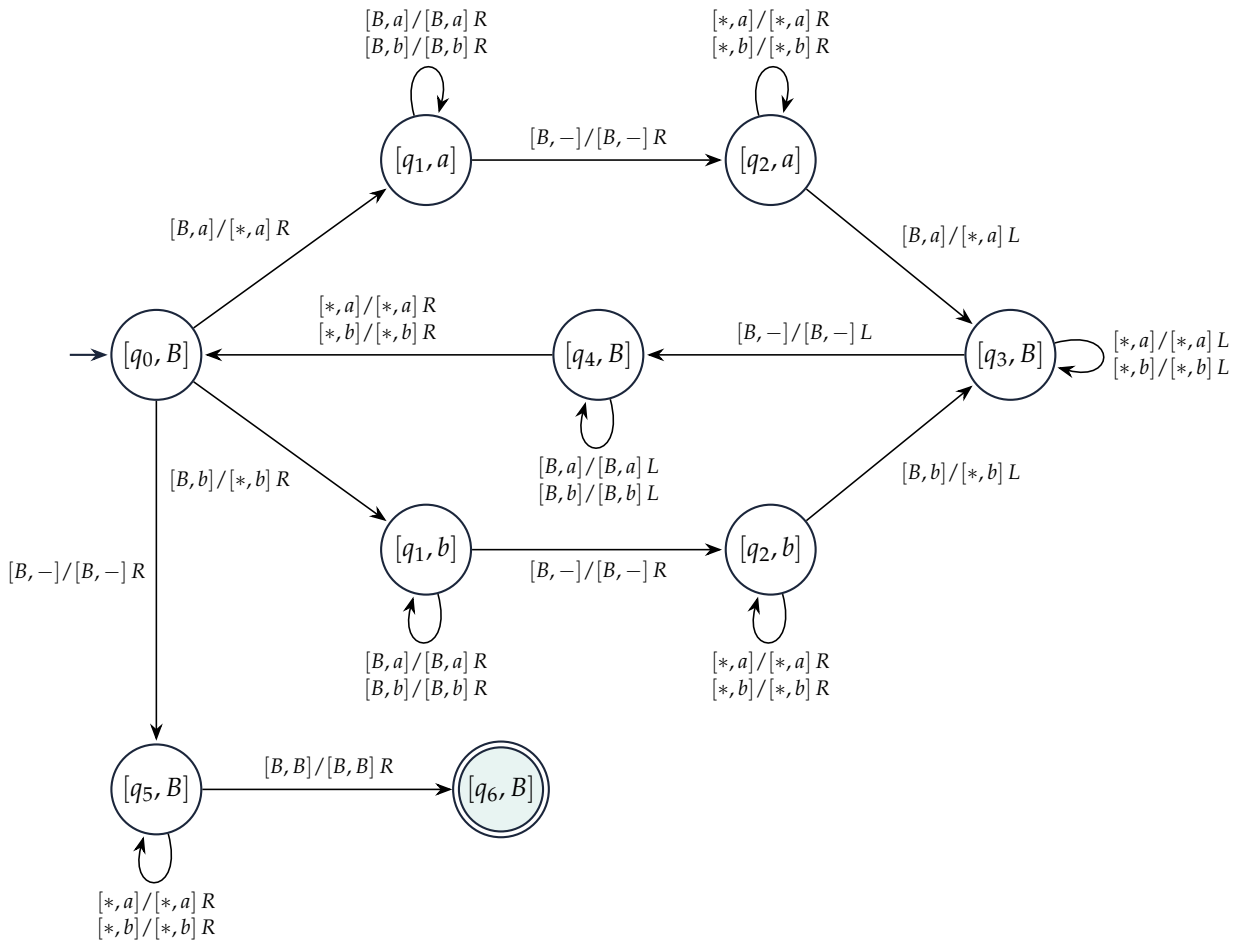
2014–2015 • Q8(b) Source M23 MARKS 7

Draw a state diagram of a TM deciding $L = \{w\text{-}w \mid w \in \{a,b\}^*\}$.

SOLUTION

Same procedure as the equality comparison of 2023–2024 Q9: check off the leftmost unchecked symbol of the left w and store it in the state, find the leftmost unchecked symbol right of "$-$", check it off if it is the same, and come back. When the left w is used up, every symbol on the right must already be checked.

Formal notation. State storage $[q, x]$ (x = the symbol being carried) and two tracks $[t, X]$ ($t \in \{B, *\}$ is the check mark, $X \in \{a, b, -, B\}$). $M = (Q, \Sigma, \Gamma, \delta, [q_0, B], [B, B], \{[q_6, B]\})$ with input symbols $[B, a], [B, b], [B, -]$. The start state is $[q_0, B]$; M accepts by entering the final state $[q_6, B]$. Where no rule applies (e.g. a mismatch, a missing or second "$-$", unequal lengths) M halts without accepting. Here $x, y \in \{a, b\}$.



Transition table (empty entry: no move), split into the unchecked and the checked columns:

State	$[B,a]$	$[B,b]$	$[B,-]$	$[B,B]$
$\rightarrow [q_0,B]$	$([q_1,a],[*,a],R)$	$([q_1,b],[*,b],R)$	$([q_5,B],[B,-],R)$	
$[q_1,a]$	$([q_1,a],[B,a],R)$	$([q_1,a],[B,b],R)$	$([q_2,a],[B,-],R)$	
$[q_1,b]$	$([q_1,b],[B,a],R)$	$([q_1,b],[B,b],R)$	$([q_2,b],[B,-],R)$	
$[q_2,a]$	$([q_3,B],[*,a],L)$			
$[q_2,b]$		$([q_3,B],[*,b],L)$		
$[q_3,B]$			$([q_4,B],[B,-],L)$	
$[q_4,B]$	$([q_4,B],[B,a],L)$	$([q_4,B],[B,b],L)$		
$[q_5,B]$				$([q_6,B],[B,B],R)$
$*[q_6,B]$				

State	$[*,a]$	$[*,b]$
$[q_2,a]$	$([q_2,a],[*,a],R)$	$([q_2,a],[*,b],R)$
$[q_2,b]$	$([q_2,b],[*,a],R)$	$([q_2,b],[*,b],R)$
$[q_3,B]$	$([q_3,B],[*,a],L)$	$([q_3,B],[*,b],L)$
$[q_4,B]$	$([q_0,B],[*,a],R)$	$([q_0,B],[*,b],R)$
$[q_5,B]$	$([q_5,B],[*,a],R)$	$([q_5,B],[*,b],R)$

(All other state/symbol pairs: no move.)

Transition rules:

1. $\delta([q_0,B],[B,x]) = ([q_1,x],[*,x],R)$
2. $\delta([q_1,x],[B,y]) = ([q_1,x],[B,y],R)$

3. $\delta([q_1, x], [B, -]) = ([q_2, x], [B, -], R)$
4. $\delta([q_2, x], [*, y]) = ([q_2, x], [*, y], R)$
5. $\delta([q_2, x], [B, x]) = ([q_3, B], [*, x], L)$
6. $\delta([q_3, B], [*, y]) = ([q_3, B], [*, y], L)$
7. $\delta([q_3, B], [B, -]) = ([q_4, B], [B, -], L)$
8. $\delta([q_4, B], [B, y]) = ([q_4, B], [B, y], L)$
9. $\delta([q_4, B], [*, y]) = ([q_0, B], [*, y], R)$
10. $\delta([q_0, B], [B, -]) = ([q_5, B], [B, -], R)$
11. $\delta([q_5, B], [*, y]) = ([q_5, B], [*, y], R)$
12. $\delta([q_5, B], [B, B]) = ([q_6, B], [B, B], R)$

2013–2014 • Q2(a) Source M19 **MARKS 7**

Build a TM recognizing $L = \{0^N 1^N 0^N \mid N \geq 0\}$.

SOLUTION

Same idea as the $a^n b^n c^n$ machine of 2015–2016 Q7(a): each round checks one 0 of block 1, one 1 and one 0 of block 3. Two changes are needed because blocks 1 and 3 use the same symbol 0: the return walk is split into q_3 (in block 3), q_5 (in block 2) and q_6 (in block 1), so a checked 0 of block 3 is never mistaken for the last checked 0 of block 1; and rule 18 accepts ε ($N = 0$).

Formal notation. Two tracks $[t, X]$ ($t \in \{B, *\}$ is the check mark, $X \in \{0, 1, B\}$). $M = (\{q_0, \ldots, q_7\}, \Sigma, \Gamma, \delta, q_0, [B, B], \{q_7\})$ with input symbols $[B, 0], [B, 1]$. The start state is q_0; M accepts by entering the final state q_7 ($F = \{q_7\}$). If no rule applies in any other state, M halts without accepting, i.e. it rejects.

Transition rules:

1. $\delta(q_0, [B, 0]) = (q_1, [*, 0], R)$
2. $\delta(q_1, [B, 0]) = (q_1, [B, 0], R)$
3. $\delta(q_1, [*, 1]) = (q_1, [*, 1], R)$
4. $\delta(q_1, [B, 1]) = (q_2, [*, 1], R)$
5. $\delta(q_2, [B, 1]) = (q_2, [B, 1], R)$
6. $\delta(q_2, [*, 0]) = (q_2, [*, 0], R)$
7. $\delta(q_2, [B, 0]) = (q_3, [*, 0], L)$
8. $\delta(q_3, [*, 0]) = (q_3, [*, 0], L)$
9. $\delta(q_3, [*/B, 1]) = (q_5, [*/B, 1], L)$
10. $\delta(q_5, [*/B, 1]) = (q_5, [*/B, 1], L)$
11. $\delta(q_5, [B, 0]) = (q_6, [B, 0], L)$
12. $\delta(q_5, [*, 0]) = (q_0, [*, 0], R)$
13. $\delta(q_6, [B, 0]) = (q_6, [B, 0], L)$
14. $\delta(q_6, [*, 0]) = (q_0, [*, 0], R)$
15. $\delta(q_0, [*, 1]) = (q_4, [*, 1], R)$
16. $\delta(q_4, [*, 1/0]) = (q_4, [*, 1/0], R)$
17. $\delta(q_4, [B, B]) = (q_7, [B, B], R)$ $\quad (F = \{q_7\})$
18. $\delta(q_0, [B, B]) = (q_7, [B, B], R)$ $\quad$ (empty input, $N = 0$)

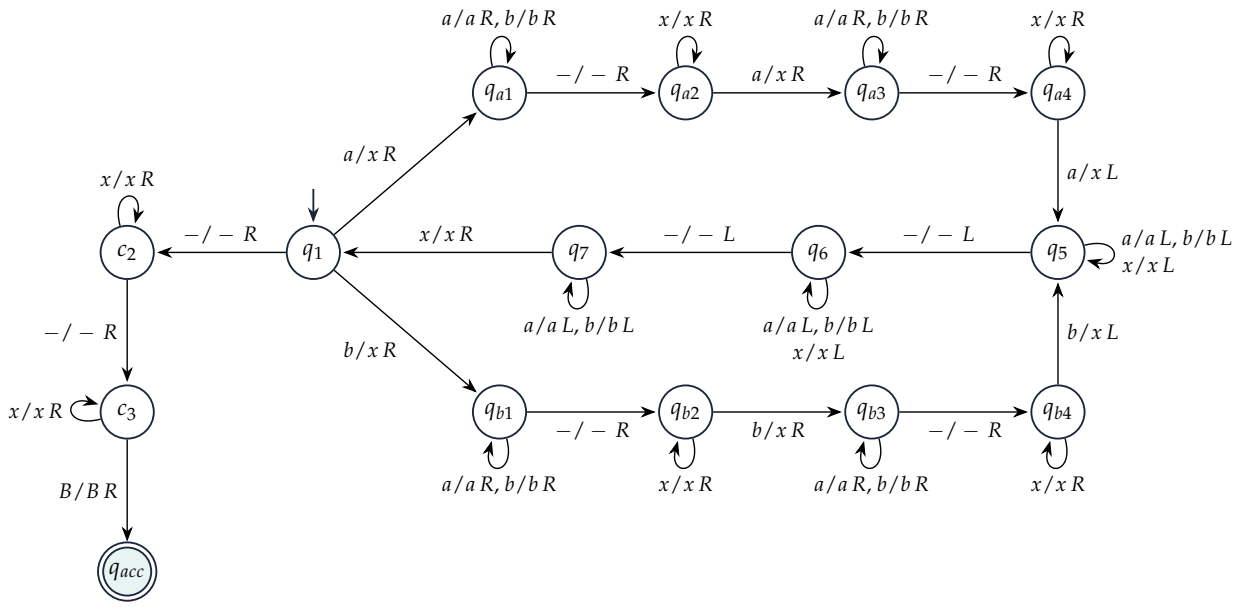
2012–2013 • Q3(b) Source M17 **MARKS 7**

For $\Sigma = \{-, a, b\}$, give a high-level description and state diagram of a TM deciding $L = \{w\text{-}w\text{-}w \mid w \in \{a,b\}^*\}$. You may assume a doubly infinite tape.

SOLUTION

High level. M = "Repeat: cross off (x) the leftmost unchecked symbol σ of block 1 and remember σ in the state; move right past the first "$-$" to the first unchecked symbol of block 2 — it must be σ, cross it off; move right past the second "$-$" to the first unchecked symbol of block 3 — it must be σ, cross it off; return left to the last x of block 1. When block 1 has no unchecked symbol left, verify that blocks 2 and 3 contain only x's and that there is no third "$-$"; accept." (This extends the w-w machine above by one more leg.)

Diagram. $q_{\sigma 1}, \ldots, q_{\sigma 4}$ carry $\sigma \in \{a, b\}$ through block 1, the checked part of block 2, the rest of block 2, and the checked part of block 3; q_5, q_6, q_7 return; c_2, c_3 do the final check.



Transition table:

State	a	b	-	x	B
$\rightarrow q_1$	(q_{a1}, x, R)	(q_{b1}, x, R)	$(c_2, \text{-}, R)$		
q_{a1}	(q_{a1}, a, R)	(q_{a1}, b, R)	$(q_{a2}, \text{-}, R)$		
q_{a2}	(q_{a3}, x, R)			(q_{a2}, x, R)	
q_{a3}	(q_{a3}, a, R)	(q_{a3}, b, R)	$(q_{a4}, \text{-}, R)$		
q_{a4}	(q_5, x, L)			(q_{a4}, x, R)	
q_{b1}	(q_{b1}, a, R)	(q_{b1}, b, R)	$(q_{b2}, \text{-}, R)$		
q_{b2}		(q_{b3}, x, R)		(q_{b2}, x, R)	
q_{b3}	(q_{b3}, a, R)	(q_{b3}, b, R)	$(q_{b4}, \text{-}, R)$		
q_{b4}		(q_5, x, L)		(q_{b4}, x, R)	
q_5	(q_5, a, L)	(q_5, b, L)	$(q_6, \text{-}, L)$	(q_5, x, L)	
q_6	(q_6, a, L)	(q_6, b, L)	$(q_7, \text{-}, L)$	(q_6, x, L)	
q_7	(q_7, a, L)	(q_7, b, L)		(q_1, x, R)	
c_2			$(c_3, \text{-}, R)$	(c_2, x, R)	
c_3				(c_3, x, R)	(q_{acc}, B, R)
$*q_{acc}$					

Transition rules ($\sigma \in \{a, b\}$):

1. $\delta(q_1, \sigma) = (q_{\sigma 1}, x, R)$ — cross off σ in block 1.
2. $\delta(q_{\sigma 1}, s) = (q_{\sigma 1}, s, R)$ for $s \in \{a, b\}$; $\delta(q_{\sigma 1}, \text{-}) = (q_{\sigma 2}, \text{-}, R)$ — go to block 2.
3. $\delta(q_{\sigma 2}, x) = (q_{\sigma 2}, x, R)$; $\delta(q_{\sigma 2}, \sigma) = (q_{\sigma 3}, x, R)$ — its first unchecked symbol must be σ.
4. $\delta(q_{\sigma 3}, s) = (q_{\sigma 3}, s, R)$ for $s \in \{a, b\}$; $\delta(q_{\sigma 3}, \text{-}) = (q_{\sigma 4}, \text{-}, R)$ — go to block 3.
5. $\delta(q_{\sigma 4}, x) = (q_{\sigma 4}, x, R)$; $\delta(q_{\sigma 4}, \sigma) = (q_5, x, L)$ — its first unchecked symbol must be σ.
6. $\delta(q_i, s) = (q_i, s, L)$ for $i \in \{5, 6\}$, $s \in \{a, b, x\}$; $\delta(q_5, \text{-}) = (q_6, \text{-}, L)$, $\delta(q_6, \text{-}) = (q_7, \text{-}, L)$ — back over blocks 3 and 2.
7. $\delta(q_7, s) = (q_7, s, L)$ for $s \in \{a, b\}$; $\delta(q_7, x) = (q_1, x, R)$ — to the last x of block 1; next round.
8. $\delta(q_1, \text{-}) = (c_2, \text{-}, R)$; $\delta(c_2, x) = (c_2, x, R)$; $\delta(c_2, \text{-}) = (c_3, \text{-}, R)$; $\delta(c_3, x) = (c_3, x, R)$; $\delta(c_3, B) = (q_{acc}, B, R)$ — blocks 2 and 3 fully checked: accept.

Missing transitions are undefined, so M halts without accepting (e.g. q_{a2} reading b, or c_3 reading a third "$-$"). q_5 walks left to the second "$-$", q_6 to the first "$-$", q_7 to the last x of block 1. Example *ab-ab-ab*: two rounds give *xx-xx-xx*; q_7 stops on the second x, q_1 reads "$-$", c_2, c_3 cross the two checked blocks and reach B: accept. Since the tape is doubly infinite no end marker is needed.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q3(c) Source M17 **MARKS 8 1/3**

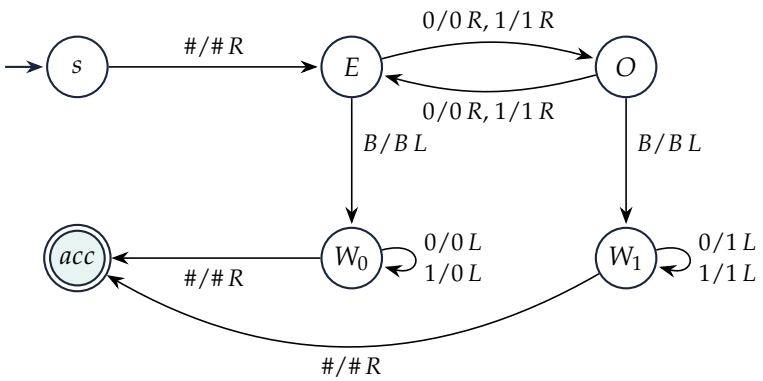
Give a TM state diagram that converts #w, $w \in \{0,1\}^*$, to #0^n if $n = |w|$ is even and to #1^n if n is odd. Enter the accepting state; the final head location does not matter. Do not show the reject state.

2011–2012 • Q3(a) Source M14 **MARKS 8 1/3**

Give a state diagram for a TM acting on #w, $w \in \{0,1\}^*$, with $n = |w|$. If n is even, replace #w by #0^n; if n is odd, replace it by #1^n. Enter the accept state after conversion.

SOLUTION

Scan right computing the parity of $|w|$ in the state; at the blank turn around and overwrite every symbol with 0 (even) or 1 (odd) until #.



State	#	0	1	B
$\to s$	$(E,\#,R)$			
E		$(O,0,R)$	$(O,1,R)$	(W_0,B,L)
O		$(E,0,R)$	$(E,1,R)$	(W_1,B,L)
W_0	$(acc,\#,R)$	$(W_0,0,L)$	$(W_0,0,L)$	
W_1	$(acc,\#,R)$	$(W_1,1,L)$	$(W_1,1,L)$	
$*acc$				

Transition rules (numbered as in the slides):

1. $\delta(s,\#) = (E,\#,R)$ — start; zero symbols read (even).
2. $\delta(E,a) = (O,a,R)$, $\delta(O,a) = (E,a,R)$ for $a \in \{0,1\}$ — keep the parity of $|w|$ in the state.
3. $\delta(E,B) = (W_0,B,L)$, $\delta(O,B) = (W_1,B,L)$ — end of w: n even / odd.
4. $\delta(W_0,a) = (W_0,0,L)$, $\delta(W_1,a) = (W_1,1,L)$ for $a \in \{0,1\}$ — overwrite every symbol.
5. $\delta(W_0,\#) = (acc,\#,R)$, $\delta(W_1,\#) = (acc,\#,R)$ — done: accept.

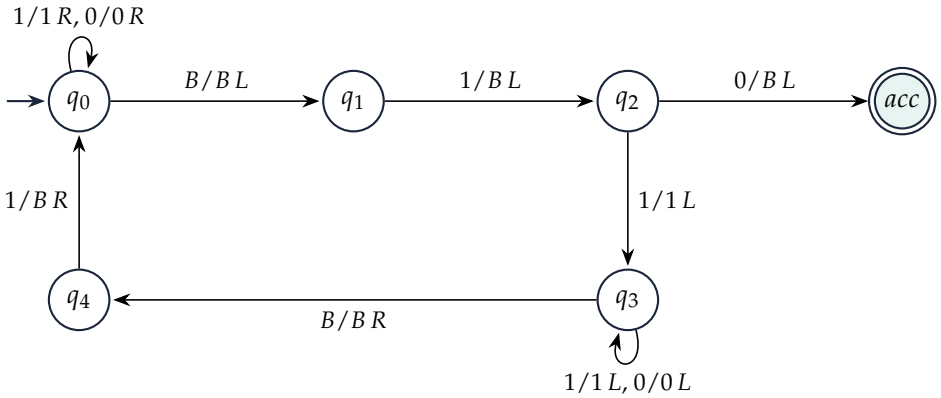
E/O: even/odd number of symbols read so far. Missing transitions (e.g. s on a bit) are undefined: M halts without accepting. Example: $s\#101 \vdash^* \#101\,O\,B \vdash \#10\,W_1\,1 \vdash^* W_1\#111 \vdash \#\,acc\,111$: ends with #111 ($n = 3$).

2011–2012 • Q1(a) Source M14 **MARKS 8 1/3**

Construct a TM state diagram computing $f(x,y) = x - y$, where $x \geq y$. Use modified unary representation: N is represented by $N+1$ copies of 1. Enter the accept state after computation; the final head location does not matter.

SOLUTION

Input $1^{x+1}\,0\,1^{y+1}$ (separator 0). Result should be 1^{x-y+1}. **Idea:** repeatedly erase the last 1 of the y-block; if that 1 was the last one of its block (a 0 is to its left) we have removed $y+1$ ones from y and y ones from x: erase the 0 and stop. Otherwise also erase the first 1 of the x-block and repeat. Since $x \geq y$, the x-block keeps $x+1-y \geq 1$ ones.



State	1	0	B
$\to q_0$	$(q_0, 1, R)$	$(q_0, 0, R)$	(q_1, B, L)
q_1	(q_2, B, L)		
q_2	$(q_3, 1, L)$	(acc, B, L)	
q_3	$(q_3, 1, L)$	$(q_3, 0, L)$	(q_4, B, R)
q_4	(q_0, B, R)		
$*acc$			

Transition rules (numbered as in the slides):

1. $\delta(q_0, s) = (q_0, s, R)$ for $s \in \{0, 1\}$; $\delta(q_0, B) = (q_1, B, L)$ — go to the right end.
2. $\delta(q_1, 1) = (q_2, B, L)$ — erase the last 1 of the y-block.
3. $\delta(q_2, 0) = (acc, B, L)$ — it was the last 1 of its block: erase the 0 and accept.
4. $\delta(q_2, 1) = (q_3, 1, L)$; $\delta(q_3, s) = (q_3, s, L)$ for $s \in \{0, 1\}$; $\delta(q_3, B) = (q_4, B, R)$ — otherwise return to the left end,
5. $\delta(q_4, 1) = (q_0, B, R)$ — erase the first 1 of the x-block and repeat.

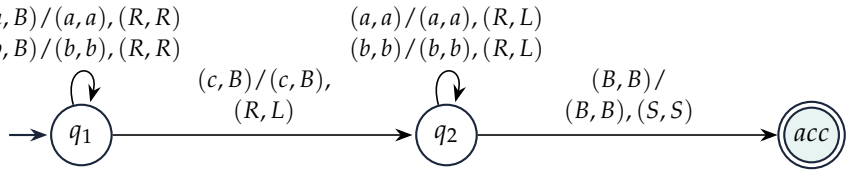
Example $x = 2$, $y = 1$: $111\,0\,11 \to 111\,0\,1 \to 11\,0\,1 \to 11\,0 \to 11 = 1$ in modified unary ✓. The machine enters acc (no moves) and halts.

2011–2012 • Q2(a) Source M14 MARKS $8\frac{1}{3}$

Construct the state diagram of a two-tape TM deciding $L = \{wcw^R \mid w \in \{a, b\}^*\}$. Keep the diagram as simple as possible and use both tapes.

SOLUTION

Copy w to tape 2; at c, move head 2 back onto the last copied symbol; then compare tape 1 moving right with tape 2 moving left. An arc label $(X_1, X_2)/(Y_1, Y_2), (D_1, D_2)$ means: head i reads X_i, writes Y_i and moves D_i, where a multitape TM may also use S = stay.



Transition table ($\sigma \in \{a, b\}$; columns = the pair of scanned symbols; entry $(p, (Y_1, Y_2), (D_1, D_2))$):

State	(σ, B)	(c, B)	(σ, σ)	(B, B)
$\to q_1$	$(q_1, (\sigma, \sigma), (R, R))$	$(q_2, (c, B), (R, L))$		
q_2			$(q_2, (\sigma, \sigma), (R, L))$	$(acc, (B, B), (S, S))$
$*acc$				

Transition rules ($\sigma \in \{a, b\}$):

1. $\delta(q_1, \sigma, B) = (q_1, (\sigma, \sigma), (R, R))$ — copy w onto tape 2.
2. $\delta(q_1, c, B) = (q_2, (c, B), (R, L))$ — at c: head 2 back onto the last copied symbol.
3. $\delta(q_2, \sigma, \sigma) = (q_2, (\sigma, \sigma), (R, L))$ — compare tape 1 (rightwards) with tape 2 (leftwards).
4. $\delta(q_2, B, B) = (acc, (B, B), (S, S))$ — both exhausted together: accept.

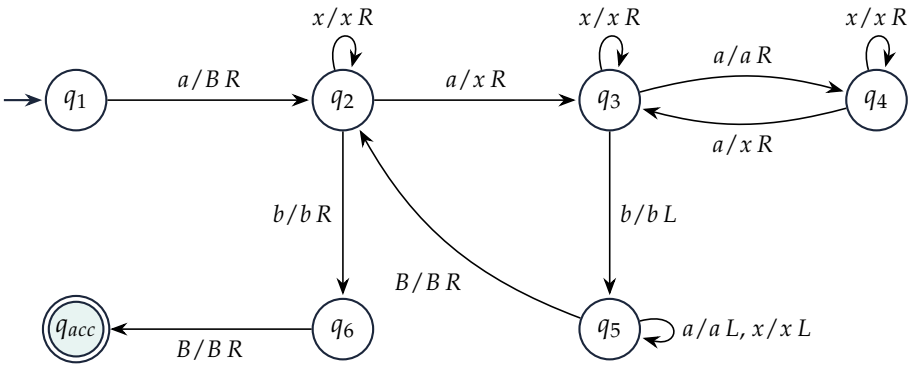
In q_2, tape 1 reads w^R left to right while tape 2 reads w right to left; both reach blanks together iff the parts match and have equal length. In every other situation there is no move and the machine halts without accepting.

2010–2011 • Q11(b) Source M12 **MARKS $8\frac{1}{3}$**

Describe a TM deciding $L = \{a^{2^n} b \mid n \geq 0\}$. Give its state diagram and configuration sequence for input *aaaab*.

SOLUTION

Idea: each sweep crosses off every other a, halving their number, with b as the right end-marker. The first a is replaced by B, so the blank to the left of the remaining a's marks their left end. If exactly one a remains (only x's between that blank and b), check that b is the last symbol and accept; if an odd number > 1 of a's remains, reject.



State	a	b	x	B
$\to q_1$	(q_2, B, R)			
q_2	(q_3, x, R)	(q_6, b, R)	(q_2, x, R)	
q_3	(q_4, a, R)	(q_5, b, L)	(q_3, x, R)	
q_4	(q_3, x, R)		(q_4, x, R)	
q_5	(q_5, a, L)		(q_5, x, L)	(q_2, B, R)
q_6				(q_{acc}, B, R)
$*q_{acc}$				

Transition rules (numbered as in the slides):

1. $\delta(q_1, a) = (q_2, B, R)$ — erase the first a; the blank marks the left end.
2. $\delta(q_2, x) = (q_2, x, R)$; $\delta(q_2, a) = (q_3, x, R)$ — cross off the first remaining a.
3. $\delta(q_3, a) = (q_4, a, R)$, $\delta(q_4, a) = (q_3, x, R)$; $\delta(q_3, x) = (q_3, x, R)$, $\delta(q_4, x) = (q_4, x, R)$ — leave and cross off the a's alternately.
4. $\delta(q_3, b) = (q_5, b, L)$ — even number of a's: the sweep is complete.
5. $\delta(q_5, s) = (q_5, s, L)$ for $s \in \{a, x\}$; $\delta(q_5, B) = (q_2, B, R)$ — back to the left end; next sweep.
6. $\delta(q_2, b) = (q_6, b, R)$ — exactly one a was left.
7. $\delta(q_6, B) = (q_{acc}, B, R)$ — b is the last symbol: accept.

$M = (\{q_1, \ldots, q_6, q_{acc}\}, \{a, b\}, \{a, b, x, B\}, \delta, q_1, B, \{q_{acc}\})$. Every missing transition is undefined, so M halts without accepting; e.g. q_4 on b (an odd number > 1 of a's), or q_6 on a non-blank (something follows the b, as in *abb* or *aba*).

IDs for *aaaab* (the B written over the first a joins the leading blanks, so it is not shown):

$$q_1aaaab \vdash q_2aaab \vdash xq_3aab \vdash xaq_4ab \vdash xaxq_3b \vdash xaq_5xb \vdash xq_5axb \vdash q_5xaxb \vdash q_5Bxaxb$$
$$\vdash q_2xaxb \vdash xq_2axb \vdash xxq_3xb \vdash xxxq_3b \vdash xxq_5xb \vdash xq_5xxb \vdash q_5xxxb \vdash q_5Bxxxb$$
$$\vdash q_2xxxb \vdash xq_2xxb \vdash xxq_2xb \vdash xxxq_2b \vdash xxxb\,q_6B \vdash xxxbB\,q_{acc}B.$$

M enters the accepting state: four a's $= 2^2$, accepted.

2010–2011 • Q12(b) Source M13 **MARKS 3**

Explain what the shown TM does. Do machines LR and RL always operate in the same way? Explain.

$$\longrightarrow R \xrightarrow{a \neq \sqcup} R \xrightarrow{b \neq \sqcup} R_{\sqcup}\, a\, R_{\sqcup}\, b$$

SOLUTION

(Lewis–Papadimitriou macro notation, with $\sqcup$ the blank and $\triangleright$ the left-end symbol: R = move right one cell; $R_{\sqcup}$ = move right until a blank; a symbol name = write it; an arrow labelled $a \neq \sqcup$ = continue if the scanned symbol is non-blank, remembering it as a.)
The machine moves right and reads a non-blank symbol a, moves right again and reads a non-blank b; then it goes right to the first blank (the end of the string), writes a, goes right to the next blank (the very next cell) and writes b. So it **copies the two symbols to the right of the head onto the end of the string**: e.g. $\underline{\triangleright}abba\sqcup \to \triangleright abbaab$. If either symbol is blank, it halts without writing.
The same machine in the slides' (Hopcroft) form, with storage in the state ($[q_2, a]$, $[q_3, a, b]$, $[q_4, b]$ for $a, b \neq B$):
Transition rules:

1. $\delta(q_0, X) = (q_1, X, R)$ for every $X \in \Gamma$ — the first R.
2. $\delta(q_1, a) = ([q_2, a], a, R)$ for $a \neq B$ — remember the non-blank symbol a; second R.
3. $\delta([q_2, a], b) = ([q_3, a, b], b, R)$ for $b \neq B$ — remember b too.
4. $\delta([q_3, a, b], X) = ([q_3, a, b], X, R)$ for $X \neq B$ — $R_{\sqcup}$: run to the first blank.
5. $\delta([q_3, a, b], B) = ([q_4, b], a, R)$ — write a there and move on.
6. $\delta([q_4, b], B) = (h, b, R)$ — write b in the next cell; h has no moves: halt.

$\delta(q_1, B)$, $\delta([q_2, a], B)$ and every move of h are undefined, so M halts (without writing if a blank was met). E.g. $q_0 \triangleright abba \vdash^* \triangleright abbaab\, h\, B$ (here $\triangleright$ is just an ordinary tape symbol).
LR **vs** RL**: not always the same.** In the slides' model the tape is infinite in both directions, so LR and RL both return the head to its starting cell and change nothing: there they always behave the same. They can differ only at a left end. In Lewis–Papadimitriou's model the leftmost cell holds $\triangleright$ and every move from $\triangleright$ is a right move: L started on $\triangleright$ is pushed right and then moves left again, ending on $\triangleright$. Hence started on $\triangleright$, LR ends one cell to the right of the start, while RL ends on the start cell. Away from the left end the two machines are equivalent.

2009–2010 • Q12(a) Source M10 **MARKS 2 + 3 = 5**

Formally define a TM. Write out in full a TM that scans right until finding two consecutive a's, then halts. Its alphabet should be $\{a, b, \sqcup, \triangleright\}$.

SOLUTION

Definition (Lewis–Papadimitriou, the book the question follows): $M = (K, \Sigma, \delta, s, H)$ with K finite states, Σ the alphabet containing $\sqcup$ and the left-end symbol $\triangleright$, $s \in K$ start, $H \subseteq K$ halting states, $\delta : (K - H) \times \Sigma \to K \times (\Sigma \cup \{\leftarrow, \rightarrow\})$ such that on $\triangleright$ the machine always moves right and $\triangleright$ is never written.
Machine: $K = \{q_0, q_1, h\}$, $s = q_0$, $H = \{h\}$:

	a	b	$\sqcup$	$\triangleright$
q_0	$(q_1, \rightarrow)$	$(q_0, \rightarrow)$	$(q_0, \rightarrow)$	$(q_0, \rightarrow)$
q_1	(h, a)	$(q_0, \rightarrow)$	$(q_0, \rightarrow)$	$(q_0, \rightarrow)$

q_1 means "the previous cell held a"; seeing a second a it halts (writing a = no change). If aa never occurs it runs forever — as specified.

Slides' (Hopcroft) form. $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ as in the topic note: Q states, Σ input symbols, $\Gamma \supseteq \Sigma$ tape symbols, $\delta(q, X) = (p, Y, D)$ with $D \in \{L, R\}$, q_0 start, $B \in \Gamma - \Sigma$ blank (= $\sqcup$), F final states. The machine: $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, \triangleright, B\}, \delta, q_0, B, \{q_2\})$ with

State	a	b	B	$\triangleright$
$\rightarrow q_0$	(q_1, a, R)	(q_0, b, R)	(q_0, B, R)	$(q_0, \triangleright, R)$
q_1	(q_2, a, R)	(q_0, b, R)	(q_0, B, R)	$(q_0, \triangleright, R)$
$*q_2$				

Transition rules (numbered as in the slides):

1. $\delta(q_0, a) = (q_1, a, R)$ — an a: remember it.
2. $\delta(q_0, s) = (q_0, s, R)$ for $s \in \{b, B, \triangleright\}$ — keep scanning.
3. $\delta(q_1, a) = (q_2, a, R)$ — second consecutive a: enter the final state q_2 (no moves), halt.
4. $\delta(q_1, s) = (q_0, s, R)$ for $s \in \{b, B, \triangleright\}$ — no aa here: start again.

q_2 has no moves, so M halts (and accepts) right after the second consecutive a: $q_0babaab \vdash bq_0abaab \vdash baq_1baab \vdash babq_0aab \vdash babaq_1ab \vdash babaaq_2b$. Without aa it moves right forever.

2009–2010 • Q12(b) Source M10 **MARKS 7**

Design a TM deciding $L = \{a^n c^n b^n d^n \mid n \geq 0\}$.

SOLUTION

Same strategy as $a^n b^n c^n$ with four blocks (block order a, c, b, d). Each round: $a \rightarrow X$ (leftmost), skip to the first c and mark it Y, skip to the first b and mark it Z, skip to the first d and mark it W, return to the rightmost X. (The markers must not be called B: that is the blank.) No separate format pass is needed — every symbol out of order finds no move.

State	a	c	b	d	X	Y	Z	W	B
$\rightarrow q_0$	(q_1, X, R)					(q_5, Y, R)			(q_{acc}, B, R)
q_1	(q_1, a, R)	(q_2, Y, R)				(q_1, Y, R)			
q_2		(q_2, c, R)	(q_3, Z, R)				(q_2, Z, R)		
q_3			(q_3, b, R)	(q_4, W, L)				(q_3, W, R)	
q_4	(q_4, a, L)	(q_4, c, L)	(q_4, b, L)		(q_0, X, R)	(q_4, Y, L)	(q_4, Z, L)	(q_4, W, L)	
q_5						(q_5, Y, R)	(q_5, Z, R)	(q_5, W, R)	(q_{acc}, B, R)
$*q_{acc}$									

Transition rules (numbered as in the slides):

1. $\delta(q_0, a) = (q_1, X, R)$ — mark the leftmost a.
2. $\delta(q_1, s) = (q_1, s, R)$ for $s \in \{a, Y\}$; $\delta(q_1, c) = (q_2, Y, R)$ — mark the first c.
3. $\delta(q_2, s) = (q_2, s, R)$ for $s \in \{c, Z\}$; $\delta(q_2, b) = (q_3, Z, R)$ — mark the first b.
4. $\delta(q_3, s) = (q_3, s, R)$ for $s \in \{b, W\}$; $\delta(q_3, d) = (q_4, W, L)$ — mark the first d.
5. $\delta(q_4, s) = (q_4, s, L)$ for $s \in \{a, c, b, Y, Z, W\}$; $\delta(q_4, X) = (q_0, X, R)$ — back to the rightmost X.
6. $\delta(q_0, Y) = (q_5, Y, R)$; $\delta(q_5, s) = (q_5, s, R)$ for $s \in \{Y, Z, W\}$ — no a left: only markers may follow.

7. $\delta(q_0, B) = (q_{acc}, B, R)$, $\delta(q_5, B) = (q_{acc}, B, R)$ — accept.

$M = (\{q_0, \ldots, q_5, q_{acc}\}, \{a, b, c, d\}, \{a, b, c, d, X, Y, Z, W, B\}, \delta, q_0, B, \{q_{acc}\})$; empty entries: no move, halt without accepting. Each round removes one symbol from each block, so all four counts are equal iff all blocks are exhausted together. Example: $q_0acbd \vdash Xq_1cbd \vdash XYq_2bd \vdash XYZq_3d \vdash XYq_4ZW \vdash Xq_4YZW \vdash q_4XYZW \vdash Xq_0YZW \vdash XYq_5ZW \vdash XYZq_5W \vdash XYZWq_5B \vdash XYZWBq_{acc}B$.

2009–2010 • Q12(c) Source M10 **MARKS 11 $\frac{1}{3}$**

Give a three-tape TM that subtracts two binary integers initially separated by a semicolon on its first tape. Assume the first integer is at least the second.

SOLUTION

Input on tape 1: $x;y$ with $x \geq y$ (y non-empty); tapes 2 and 3 are blank work tapes. Multitape moves may use S (stay).

(i) Copy (q_0, q_1). Move x from tape 1 to tape 2 (erasing tape 1); at ";" erase it and move y to tape 3 in the same way.

(ii) Align. At the blank after y all three heads step left: heads 2 and 3 are now on the LSBs of x and y, head 1 on the cell where the last bit of y was.

(iii) Subtract with the borrow in the state $[p, \beta]$ (storage in the state). Read u (tape 2) and v (tape 3; a blank counts as 0), compute $u - v - \beta$; write $d = (u - v - \beta) \bmod 2$ on tape 1, set $\beta' = 1$ iff $u - v - \beta < 0$, and move all three heads left.

u	v	β	write d	β'
0	0	0	0	0
0	1	0	1	1
1	0	0	1	0
1	1	0	0	0
0	0	1	1	1
0	1	1	0	1
1	0	1	0	0
1	1	1	1	1

(iv) Stop when head 2 reads the blank left of x (the borrow is then 0 because $x \geq y$): enter h. Tape 1 now holds $x - y$ with $|x|$ bits (leading zeros may be stripped by an extra pass).

Transition table ($a, u \in \{0, 1\}$, $v \in \{0, 1, B\}$; entry $(p, (Y_1, Y_2, Y_3), (D_1, D_2, D_3))$; d, β' from the table above):

State	(a, B, B)	$(;, B, B)$	(B, B, B)	(B, u, v)
$\to q_0$	$(q_0, (B, a, B), (R, R, S))$	$(q_1, (B, B, B), (R, S, S))$		
q_1	$(q_1, (B, B, a), (R, S, R))$		$([p, 0], (B, B, B), (L, L, L))$	
$[p, 0]$			$(h, (B, B, B), (S, S, S))$	$([p, \beta'], (d, u, v), (L, L, L))$
$[p, 1]$				$([p, \beta'], (d, u, v), (L, L, L))$
$*h$				

Transition rules ($a, u \in \{0, 1\}$, $v \in \{0, 1, B\}$):

1. $\delta(q_0, a, B, B) = (q_0, (B, a, B), (R, R, S))$ — move a bit of x to tape 2.
2. $\delta(q_0, ;, B, B) = (q_1, (B, B, B), (R, S, S))$ — erase the semicolon.

3. $\delta(q_1, a, B, B) = (q_1, (B, B, a), (R, S, R))$ — move a bit of y to tape 3.
4. $\delta(q_1, B, B, B) = ([p, 0], (B, B, B), (L, L, L))$ — end of y: all heads onto the LSBs, no borrow yet.
5. $\delta([p, \beta], B, u, v) = ([p, \beta'], (d, u, v), (L, L, L))$ — one column of the subtraction (blank $v = 0$).
6. $\delta([p, 0], B, B, B) = (h, (B, B, B), (S, S, S))$ — x used up: halt.

Example 1011; 0110 ($11 - 6$): LSB first: $1 - 0 = 1$; $1 - 1 = 0$; $0 - 1 \to 1$ borrow 1; $1 - 0 - 1 = 0$: tape 1 ends with $0101 = 5$ ✓. (Simulated for all $x \geq y$ with $|x| \leq 5$.)

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2008–2009 • Q3(b–c) Source M06 **MARKS (b) 13; (c) 8**

Design a TM computing bitwise XOR of two equal-length binary strings separated by #. For example, 0010#1011 should produce 1001. Make necessary assumptions. Then simulate your TM from (b) on input 1011#0010.

2006–2007 • Q11 Source M03 **MARKS 10**

Construct a Turing machine of any kind to compute XOR of two equal-length binary numbers. Initially the tape contains the two numbers separated by a comma. Replace the second number by their XOR.

SOLUTION

SCOPE *2006–2007 Q11 asks only for the machine: read the separator # as a comma and use the variant of the last two rows given below the table, so the first number is restored and the XOR replaces the second number. The simulation is asked only in 2008–2009 Q3(c).*

Assumptions. Input $w_1\#w_2$ with $|w_1| = |w_2| \geq 1$, head on the first symbol, blanks elsewhere. The XOR is built in place of w_2; for 2008–09 we finally erase $w_1\#$ so that only the result remains.
Idea. Check off the leftmost unchecked bit b of w_1 ($0 \to x$, $1 \to y$), carry b in the state (r_b, s_b), go to the leftmost *unprocessed* bit b' of w_2 and overwrite it with the *marked* result $b \oplus b'$ (0 as P, 1 as Q), return. When w_1 is used up, f turns P, Q back into $0, 1$, and g, e clean up.

State	0	1	x	y	P	Q	#	B
$\to q_0$	(r_0, x, R)	(r_1, y, R)					$(f, \#, R)$	
r_0	$(r_0, 0, R)$	$(r_0, 1, R)$					$(s_0, \#, R)$	
r_1	$(r_1, 0, R)$	$(r_1, 1, R)$					$(s_1, \#, R)$	
s_0	(k, P, L)	(k, Q, L)			(s_0, P, R)	(s_0, Q, R)		
s_1	(k, Q, L)	(k, P, L)			(s_1, P, R)	(s_1, Q, R)		
k	$(k, 0, L)$	$(k, 1, L)$	(q_0, x, R)	(q_0, y, R)	(k, P, L)	(k, Q, L)	$(k, \#, L)$	
f					$(f, 0, R)$	$(f, 1, R)$		(g, B, L)
g	$(g, 0, L)$	$(g, 1, L)$					(e, B, L)	
e			(e, B, L)	(e, B, L)				(h, B, R)
$*h$								

$M = (Q, \{0, 1, \#\}, \{0, 1, \#, x, y, P, Q, B\}, \delta, q_0, B, \{h\})$; h has no moves. s_b adds b to the bit it finds: with $b = 0$ it copies, with $b = 1$ it flips. In the slides' notation $r_b = [r, b]$, $s_b = [s, b]$:
Transition rules ($b \in \{0, 1\}$):

1. $\delta(q_0, 0) = ([r, 0], x, R)$, $\delta(q_0, 1) = ([r, 1], y, R)$ — check the next bit b of w_1, store it.
2. $\delta([r, b], a) = ([r, b], a, R)$ for $a \in \{0, 1\}$; $\delta([r, b], \#) = ([s, b], \#, R)$ — go to w_2.
3. $\delta([s, b], a) = ([s, b], a, R)$ for $a \in \{P, Q\}$ — skip the bits already processed.

4. $\delta([s,0],0) = (k,P,L)$, $\delta([s,0],1) = (k,Q,L)$, $\delta([s,1],0) = (k,Q,L)$, $\delta([s,1],1) = (k,P,L)$ — write $b \oplus a$, marked ($P = 0$, $Q = 1$).
5. $\delta(k,a) = (k,a,L)$ for $a \in \{0,1,P,Q,\#\}$; $\delta(k,a) = (q_0,a,R)$ for $a \in \{x,y\}$ — back to the last checked bit of w_1.
6. $\delta(q_0,\#) = (f,\#,R)$ — w_1 used up.
7. $\delta(f,P) = (f,0,R)$, $\delta(f,Q) = (f,1,R)$; $\delta(f,B) = (g,B,L)$ — unmark the result.
8. $\delta(g,a) = (g,a,L)$ for $a \in \{0,1\}$; $\delta(g,\#) = (e,B,L)$ — erase #.
9. $\delta(e,a) = (e,B,L)$ for $a \in \{x,y\}$; $\delta(e,B) = (h,B,R)$ — erase w_1 and halt.

Variant for 2006–07 (keep the comma, restore w_1): $\delta(g,',') = (e,',',L)$, $\delta(e,x) = (e,0,L)$, $\delta(e,y) = (e,1,L)$, $\delta(e,B) = (h,B,R)$; it ends with $w_1, (w_1 \oplus w_2)$ on the tape.
Simulation on 1011#0010 (tape after each round; the result is built in place of w_2):

round	carried b, found b'	tape
start		1011#0010
1	$1,0 \Rightarrow Q$ (1)	y011#Q010
2	$0,0 \Rightarrow P$ (0)	yx11#QP10
3	$1,1 \Rightarrow P$ (0)	yxy1#QPP0
4	$1,0 \Rightarrow Q$ (1)	yxyy#QPPQ
f	unmark	yxyy#1001
g,e	erase w_1#	1001

Result 1001 ✓ ($1011 \oplus 0010 = 1001$). IDs of round 1: $q_0$1011#0010 $\vdash$ y$r_1$011#0010 $\vdash^3$ y011r_1#0010 $\vdash$ y011#$s_1$0010 $\vdash$ y011k#Q010 $\vdash^4$ ky011#Q010 $\vdash$ y$q_0$011#Q010. End of the run: yxyyq_0#QPPQ $\vdash$ yxyy#fQPPQ $\vdash^4$ yxyy#1001fB $\vdash$ yxyy#100g1 $\vdash^4$ yxyy$g$#1001 $\vdash$ yxy$e$yB1001 $\vdash^4$ eBBBB1001 $\vdash$ hBBBB1001 — M halts with only 1001 on the tape (60 moves in all).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2007–2008 • Q7 Source M05 **MARKS 7 + 3 = 10**

Design a TM deciding $\{0^n1^n \mid n \geq 1\}$, using a transition table. Show its IDs for input 0011.

2006–2007 • Q10 Source M03 **MARKS 10**

Design a Turing machine to decide $\{0^n1^n \mid n \geq 1\}$. Draw its transition diagram.

SOLUTION

SCOPE *2007–2008 Q7 asks for the transition table and the IDs for 0011; 2006–2007 Q10 asks for the transition diagram.*

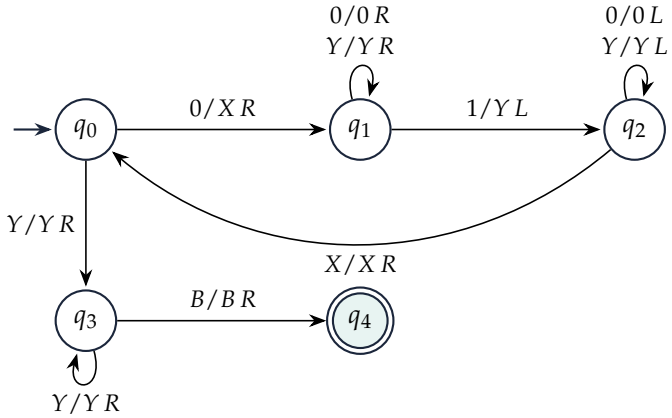
(Hopcroft–Motwani–Ullman Example 8.2, the slides' example.) Repeatedly change the leftmost 0 to X and the leftmost 1 to Y. $M = (\{q_0,q_1,q_2,q_3,q_4\},\{0,1\},\{0,1,X,Y,B\},\delta,q_0,B,\{q_4\})$:

State	0	1	X	Y	B
$\to q_0$	(q_1,X,R)			(q_3,Y,R)	
q_1	$(q_1,0,R)$	(q_2,Y,L)		(q_1,Y,R)	
q_2	$(q_2,0,L)$		(q_0,X,R)	(q_2,Y,L)	
q_3				(q_3,Y,R)	(q_4,B,R)
$*q_4$					

(Empty entry: no move; M halts there, and accepts only in q_4.)

Transition rules (numbered as in the slides):

1. $\delta(q_0, 0) = (q_1, X, R)$ — change the leftmost 0 to X.
2. $\delta(q_1, s) = (q_1, s, R)$ for $s \in \{0, Y\}$; $\delta(q_1, 1) = (q_2, Y, L)$ — find the first 1 and change it to Y.
3. $\delta(q_2, s) = (q_2, s, L)$ for $s \in \{0, Y\}$; $\delta(q_2, X) = (q_0, X, R)$ — back to the rightmost X; next round.
4. $\delta(q_0, Y) = (q_3, Y, R)$; $\delta(q_3, Y) = (q_3, Y, R)$ — all 0's used: skip the Y's.
5. $\delta(q_3, B) = (q_4, B, R)$ — no 1 left: accept.



IDs for 0011:

$q_0 0011 \vdash X q_1 011 \vdash X 0 q_1 11 \vdash X q_2 0 Y 1 \vdash q_2 X 0 Y 1 \vdash X q_0 0 Y 1 \vdash X X q_1 Y 1 \vdash X X Y q_1 1 \vdash X X q_2 Y Y$
$\vdash X q_2 X Y Y \vdash X X q_0 Y Y \vdash X X Y q_3 Y \vdash X X Y Y q_3 B \vdash X X Y Y B q_4 B$ (accept).

2007–2008 • Q8 Source M05 **MARKS 10**

Construct a TM of any kind that adds two equal-length binary numbers. Initially the numbers are separated by a comma; finish with only their sum on the tape.

SOLUTION

Use a **2-tape TM** (equivalent to a one-tape TM). Input on tape 1: x, y with $|x| = |y| = n$.

(i) Move head 1 right to the comma. Continue right, copying y onto tape 2 and erasing it from tape 1 (write B); erase the comma too. Now tape 1 holds x, tape 2 holds y.

(ii) Move both heads left onto the LSBs (head 1 to the last bit of x, head 2 to the last bit of y).

(iii) **Add with the carry in the state** ($[c, 0]/[c, 1]$): read u (tape 1), v (tape 2); $s = u + v + c$; write s mod 2 on tape 1, new carry $\lfloor s/2 \rfloor$; move both heads left.

u	v	c	write	c'
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

(iv) When head 1 reads the blank left of x: if the carry is 1, write 1 there (the sum has $n+1$ bits). Enter the final state h, which has no moves: halt.

Transition table ($a, b \in \{0,1\}$; entry $(p, (Y_1, Y_2), (D_1, D_2))$; $s = a + b + k$):

State	(a, B)	$(,, B)$	(B, B)	(B, b)	(a, b)
$\to q_0$	$(q_0, (a, B), (R, S))$	$(q_1, (B, B), (R, S))$			
q_1	$(q_1, (B, a), (R, R))$		$(q_2, (B, B), (L, L))$		
q_2				$(q_2, (B, b), (L, S))$	$([c, 0], (a, b), (S, S))$
$[c, 0]$			$(h, (B, B), (S, S))$		$([c, \lfloor s/2 \rfloor], (s \bmod 2, b), (L, L))$
$[c, 1]$			$(h, (1, B), (S, S))$		$([c, \lfloor s/2 \rfloor], (s \bmod 2, b), (L, L))$
$*h$					

Transition rules ($a, b \in \{0,1\}$, $k \in \{0,1\}$):

1. $\delta(q_0, a, B) = (q_0, (a, B), (R, S))$ — move head 1 over x.
2. $\delta(q_0, ,, B) = (q_1, (B, B), (R, S))$ — erase the comma.
3. $\delta(q_1, a, B) = (q_1, (B, a), (R, R))$ — move y to tape 2.
4. $\delta(q_1, B, B) = (q_2, (B, B), (L, L))$ — end of y; head 2 onto its LSB.
5. $\delta(q_2, B, b) = (q_2, (B, b), (L, S))$; $\delta(q_2, a, b) = ([c, 0], (a, b), (S, S))$ — head 1 back over the erased cells to the LSB of x.
6. $\delta([c, k], a, b) = ([c, \lfloor s/2 \rfloor], (s \bmod 2, b), (L, L))$ with $s = a + b + k$ — one column of the addition.
7. $\delta([c, 0], B, B) = (h, (B, B), (S, S))$, $\delta([c, 1], B, B) = (h, (1, B), (S, S))$ — write the final carry; halt.

Tape 1 now contains only $x + y$. Example 11, 01: sum computed as $1 + 1 = 0$ carry 1, $1 + 0 + 1 = 0$ carry 1, final carry written: 100 ($3 + 1 = 4$) ✓.

2007–2008 • Q11 Source M05 • $L(M)$ part **MARKS 4**

Describe $L(M)$ for $M = (\{q_0, q_1, q_f\}, \{0,1\}, \{0,1,B\}, \delta, q_0, B, \{q_f\})$, where $\delta(q_0, 0) = (q_0, B, R)$; $\delta(q_0, 1) = (q_1, B, R)$; $\delta(q_1, 1) = (q_1, B, R)$; $\delta(q_1, B) = (q_f, B, R)$.

SOLUTION

The machine. q_0 erases 0's moving right; the first 1 moves it to q_1; q_1 erases 1's; reaching the blank in q_1 accepts. Missing moves (q_0 on B, q_1 on 0) halt without accepting. So the input must be some 0's, then at least one 1, then nothing else. For example,

$$q_0 0011 \vdash B q_0 011 \vdash BB q_0 11 \vdash BBB q_1 1 \vdash BBBB q_1 B \vdash BBBBB q_f B \quad \text{(accept)},$$

while $q_0 010 \vdash B q_0 10 \vdash BB q_1 0$ halts without accepting ($\delta(q_1, 0)$ is undefined).

Answer. $L(M)$ = strings made of some 0's (possibly none) followed by at least one 1, with no 0 after any 1. In symbols: $0^*1^+ = \{0^m 1^n \mid m \geq 0,\ n \geq 1\}$ (and M halts on every input, so it is recursive).

2006–2007 • Q12 Source M03 • $L(M)$ part **MARKS part of 10**

Describe $L(M)$ for $M = (\{q_0, q_1, q_f\}, \{0,1\}, \{0,1,B\}, \delta, q_0, B, \{q_f\})$, where $\delta(q_0, 0) = (q_1, 1, R)$; $\delta(q_0, B) = (q_f, B, R)$; $\delta(q_1, 1) = (q_0, 0, R)$; $\delta(q_1, B) = (q_f, B, R)$.

SOLUTION

The machine. From q_0 it must read 0 (go to q_1); from q_1 it must read 1 (back to q_0); in either state a

blank accepts; any other symbol halts without accepting. So the input must alternate $0, 1, 0, 1, \ldots$ starting with 0, of any length. For example,

$$q_0010 \vdash 1q_110 \vdash 10q_00 \vdash 101q_1B \vdash 101Bq_fB \quad \text{(accept)},$$

while $q_0011 \vdash 1q_111 \vdash 10q_01$ halts without accepting ($\delta(q_0, 1)$ is undefined). (The tape ends up holding the complemented bits, which does not affect acceptance.)

Answer. $L(M)$ = strings in which 0's and 1's alternate, starting with 0 (no two equal symbols are adjacent), of any length including the empty string: $\varepsilon, 0, 01, 010, 0101, \ldots$ In symbols: $(01)^*(\varepsilon \cup 0)$.

13

Turing-machine models, simulation and encoding

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q10 Source N03 MARKS 5 + 10 = 15

What is a multitape TM? Explain how a one-tape TM with multiple tracks and state storage simulates it.

2019–2020 • Q10(b) Source M42 MARKS 7

Explain how a one-tape TM with multiple tracks and state storage can simulate a multitape TM.

2012–2013 • Q3(a) Source M17 MARKS 8

Prove that every multitape TM has an equivalent single-tape TM.

SOLUTION

SCOPE *2019–2020 Q10(b) and 2012–2013 Q3(a) ask only for the simulation (from "Simulation by one tape" on); the definition of a multitape TM is the extra part of 2023–2024 Q10.*

Multitape TM. A TM with a finite control and k tapes, each with its own independently moving head. Initially the input is on tape 1 with head 1 on its first symbol, all other cells of all tapes hold the blank B, and the control is in the start state. One move depends on the state and the k scanned symbols: $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k$ — it changes state, writes a symbol on every tape and moves each head left, right or not at all (S). Like the one-tape TM it accepts by entering an accepting state.

Simulation by one tape with $2k$ tracks (slides). Let M have k tapes. The single tape of N is divided into $2k$ tracks, so each cell holds a $2k$-tuple (a symbol of the larger alphabet $(\Gamma \times \{B, *\})^k$). For each tape i of M: one track holds tape i's contents, the other track holds a single marker $*$ under the position of head i.

head 1			*			
tape 1	a	b	b	a	*B*	*B*
head 2	*					
tape 2	1	0	1	*B*	*B*	*B*

One move of M is simulated by N as follows.

(i) *Gather.* N sweeps from the leftmost head marker to the rightmost. Each time it passes a head marker $*$ of tape i it records the symbol on the contents track of tape i in its finite control ("storage in the state"). It also keeps in its state the number of markers already visited, so it knows when all k have been seen and does not get lost. The state of N thus contains M's state plus the k scanned symbols — finitely many possibilities.

(ii) *Decide.* N now knows the whole argument of δ_M, hence the move: new state, k symbols to write, k directions.

(iii) *Update.* N revisits each head marker, overwrites the tape symbol on the contents track and moves the marker one cell left or right (or leaves it), then records M's new state in its own state.

(iv) The accepting states of N are those that record an accepting state of M: N accepts exactly when M does (and when M has no move, neither has N).

Correctness and cost. The tracks always mirror M's tapes and head positions, so $L(N) = L(M)$. After n moves of M the head markers are at most $2n$ cells apart, so one simulated move costs at most $4n + 2k = O(n)$ moves of N, and n moves cost $O(n^2)$: a single-tape TM is only polynomially slower. Hence every multitape TM has an equivalent single-tape TM, and multitape TMs accept exactly the recursively enumerable languages. ■

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2022–2023 • Q11(i–ii) Source M52 — **MARKS (i) 8; (ii) 7**

Justify true or false: (i) a deterministic TM can simulate n moves of a conventional computer in polynomial $P(n)$ moves; (ii) a conventional computer can simulate n moves of a nondeterministic TM in polynomial $P(n)$ moves.

2021–2022 • Q10(i–ii) Source M48 — **MARKS (i) 7; (ii) 8**

Justify true or false: (i) a one-tape TM with multiple tracks can simulate a multitape TM; (ii) a conventional computer can simulate n moves of a nondeterministic TM in polynomially many $P(n)$ moves.

January 2020 Term • Q10(i–ii) Source M38 — **MARKS (i) 7; (ii) 8**

Explain true or false: (i) a one-tape TM with multiple tracks and state storage can simulate a multitape TM; (ii) a deterministic TM can simulate n steps of a conventional computer in $P(n)$ steps for a polynomial P.

2017–2018 • Q10(i–iii) Source M34–M35 — **MARKS 15 (shared)**

Explain whether each is true or false: (i) A one-tape TM with multiple tracks can simulate a multitape TM. (ii) A deterministic TM can simulate n moves of a nondeterministic TM in $P(n)$ moves for a polynomial P. (iii) A deterministic TM can simulate n moves of a conventional computer in $P(n)$ moves for a polynomial P.

SOLUTION

SCOPE *2017–2018 asks all three statements A, B, C below; each of the other papers asks two of them (see the table at the end). "Conventional computer simulates an NTM" and "DTM simulates an NTM" are answered together in C, since a DTM and a computer simulate each other with polynomial overhead.*

A. "A one-tape TM with multiple tracks (and state storage) can simulate a multitape TM." — TRUE

Use $2k$ tracks: for each tape one track for contents and one for the head marker. Each move of the k-tape machine is simulated by a sweep that collects the k scanned symbols into the finite control and a sweep that writes and moves the markers (full construction in the previous solution). n moves cost $O(n^2)$ moves.

B. "A DTM can simulate n moves of a conventional computer in $P(n)$ moves." — TRUE

(Slides / Hopcroft–Motwani–Ullman §8.6.) Assume (i) each instruction produces a word at most one bit longer than its arguments (multiplication is done by repeated addition and shifting) and (ii) each instruction on words of length k can be done by a multitape TM in $O(k^2)$ steps. A multitape TM keeps: tape 1 = memory as \$ followed by address∗value# pairs in binary; tape 2 = instruction counter; tape 3 = memory address (or the value found there); further tapes for the computer's input and for scratch work. If c is the longest word or address in the program and d the number of words it occupies, then after n instructions every word and address has at most $c + n$ bits and at most $d + n$ addresses are in use, so tape 1 holds at most $2(c + n + 1)(d + n) = O(n^2)$ cells. Each of the fixed number of address look-ups per instruction takes $O(n^2)$ steps, the operation itself takes $O(n^2)$ by (ii), and shifting tape 1 over when a value changes length takes $O(n^2)$. So one instruction costs $O(n^2)$ and n instructions cost $O(n^3)$ steps of the multitape TM; converting it to one tape squares the time: $O(n^6)$ — still polynomial. Conversely, a computer can simulate a TM. Hence "polynomial on a TM" = "polynomial on a real computer", which justifies defining P with TMs.

C. "A DTM / conventional computer can simulate n moves of an NTM in $P(n)$ moves." — NOT KNOWN; believed FALSE

The known simulation (slides) keeps a queue of IDs of the NTM and explores them breadth-first. With at most m choices per move, n moves can reach $1 + m + m^2 + \cdots + m^n \leq (n + 1)m^n$ IDs, so the simulation takes exponential time. A polynomial simulation would mean every NTM running in polynomial time has a polynomial-time deterministic equivalent, i.e. NP ⊆ P, hence P = NP. Nobody knows such a simulation and it is widely believed not to exist. So the statement cannot be justified as true; as far as is known it is false (it is equivalent to the open problem P = NP).

Paper	item	answer
2022–23	(i) B; (ii) C	True; not known / believed false
2021–22	(i) A; (ii) C	True; not known / believed false
Jan 2020	(i) A; (ii) B	True; True
2017–18	(i) A; (ii) C; (iii) B	True; not known / believed false; True

2015–2016 • Q8(b) Source M27 MARKS $10\frac{1}{3}$

Describe how multiple tracks and multiple tapes extend a basic TM, and distinguish these extensions.

SOLUTION

Multiple tracks. The single tape is viewed as k parallel tracks; each cell holds a k-tuple such as $[X, Y, Z]$, read and written together by *one* head. Formally this is just a larger tape alphabet $\Gamma' = \Gamma_1 \times \cdots \times \Gamma_k$, so it is not a new model at all — a programming convenience (e.g. one track for data, one for check-marks).

Multiple tapes. k separate tapes, each with its *own head* moving independently; $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k$. This is a genuinely different machine: heads can be far apart (copy, compare, count without zig-zagging).

	Multi-track	Multi-tape
Heads	one head reads all tracks of one cell	one independent head per tape
Formally	ordinary TM with tuple alphabet	new transition function on k-tuples and k directions
Power	same as basic TM	same as basic TM (simulated with $2k$ tracks)
Speed	same	may be up to quadratically faster (e.g. $\{ww^R\}$ in $O(n)$ vs $\Theta(n^2)$)

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2014–2015 • Q7(c) Source M23 **MARKS 9$\frac{1}{3}$**

Prove nondeterministic and deterministic TMs are equivalent.

2013–2014 • Q3(c) Source M19 **MARKS 9$\frac{1}{3}$**

Show that every nondeterministic TM has an equivalent deterministic TM.

2012–2013 • Q4(b) Source M18 **MARKS 7**

Show that every nondeterministic TM has an equivalent deterministic TM.

SOLUTION

In an NTM M_N, $\delta(q, X)$ is a finite set of triples $\{(q_1, Y_1, D_1), \ldots, (q_k, Y_k, D_k)\}$ and at each step the machine may choose any one triple. M_N accepts w if *some* sequence of choices leads from q_0w to an ID with an accepting state. A DTM is an NTM with at most one choice everywhere, so one direction is trivial. Conversely:

Theorem. For every NTM M_N there is a DTM M_D with $L(M_D) = L(M_N)$.

Proof (slides). M_D is a two-tape DTM. Tape 1 holds a *queue* of IDs of M_N separated by $*$, initially just q_0w. One ID is marked (an x on its separator) as the *current* ID; the IDs to its left have been explored and are ignored from now on. Tape 2 is scratch. To process the current ID, M_D:

(i) examines the state and scanned symbol of the current ID (the moves of M_N for each state–symbol pair are built into M_D's finite control). If the state is accepting, M_D accepts and stops;

(ii) otherwise, if the pair has k moves, M_D copies the ID onto tape 2 and appends k copies of it at the end of the queue on tape 1;

(iii) it changes each of these k copies according to a different one of the k choices, giving the k successor IDs;

(iv) it returns to the current ID, erases the mark, marks the next ID to the right and repeats from step (i). (If there is no next ID, every branch has halted without accepting, and M_D halts without accepting.)

Correctness. New IDs go to the end of a queue, so M_D explores breadth-first: all IDs reachable in j moves are processed before any ID reachable only in $j+1$ moves. If m is the maximum number of choices of M_N, at most $1 + m + \cdots + m^n \le (n+1)m^n$ IDs are reachable in at most n moves. So if M_N reaches an accepting ID after n moves, M_D processes it after finitely many steps and

accepts. If M_N does not accept w, no accepting ID is ever generated and M_D does not accept. Hence $L(M_D) = L(M_N)$; finally convert the two-tape M_D to one tape (previous constructions). ■
A stack instead of a queue (depth-first search) would not work: it could follow an infinite branch forever and never reach an accepting ID on another branch. Cost: up to exponentially many IDs — the models are equal in power, not in efficiency. (Sipser's version, with an address tape enumerated in shortlex order, is the same breadth-first idea.)

2013–2014 • Q2(b) Source M19 **MARKS 7**

How can we recognize the left end of a TM tape? Explain.

SOLUTION

In the slides' basic model the tape is infinite in both directions, so the tape itself has no left end; what a machine needs to find is the left end of its input (or of the part of the tape it has used). Techniques:

(i) Blank boundary. All cells left of the input hold B, so moving left until the first B and then one step right finds the left end — e.g. the slides' monus machine finds its leftmost 0 "when it first meets a blank and then moves one cell to the right". This needs the machine never to write B inside the used part (otherwise use (ii)).

(ii) Mark on a second track. Replace the first symbol a by the marked symbol $[*, a]$ (or keep an end-marker $*$ on a lower track of the leftmost cell, as in the slides' simulation of a two-way tape by a semi-infinite one). The head is at the left end exactly when it scans a marked symbol; the mark is never erased.

(iii) Semi-infinite tape (slides: no cells left of the initial head position). Use the end-marker of (ii) on the leftmost cell, or shift the input one cell right at the start and write a special symbol such as \$ in the first cell. (In Sipser's model, where a left move at the leftmost cell leaves the head in place, a TM can also test: write a special mark, move left, and see whether the mark is still under the head.)

2013–2014 • Q3(a) Source M19 **MARKS 7**

Answer with reasoning: (i) Can a TM write the blank symbol on its tape? (ii) Can tape alphabet Γ equal input alphabet Σ? (iii) Can its head remain at the same location in two successive steps? (iv) Can a TM have just one state?

SOLUTION

(Model of the slides: $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$.)

(i) Yes. $B \in \Gamma$ and $\delta(q, X) = (p, Y, D)$ may write any $Y \in \Gamma$, including B — e.g. the slides' monus machine erases 0's by writing B.

(ii) No. By definition $B \in \Gamma$ but $B \notin \Sigma$ ("the blank is a tape symbol but not an input symbol"), so $\Sigma \subsetneq \Gamma$.

(iii) No (one-tape TM). Every move is (p, Y, D) with $D \in \{L, R\}$: "we require a move, and do not allow the head to remain stationary". So after each step the head is on a different cell; it can return to a cell only after two steps (e.g. R then L), which is how a stay is simulated. Only multitape TMs in the slides have the stationary option S.

(iv) Yes. Nothing in the 7-tuple requires two states. With $Q = \{q_0\}$ and $F = \emptyset$ the machine accepts nothing; with $F = \{q_0\}$ it accepts every input (its initial ID is already accepting). The slides themselves use "the TM with one state and no transitions" for invalid codes in the diagonalization argument. (Only in Sipser's model, which demands distinct $q_{accept} \neq q_{reject}$, are two states needed.)

2011–2012 • Q1(b) Source M14 **MARKS 7**

Give the proof idea that every TM with multiple tracks has an equivalent single-track TM.

SOLUTION

A k-track TM reads and writes, in one step, a whole column $[a_1, \ldots, a_k]$ of a single cell with one head. Define a single-track TM whose tape alphabet is $\Gamma' = \Gamma_1 \times \cdots \times \Gamma_k$ (each column becomes one "big" symbol), with blank $[B, \ldots, B]$ and input symbols $a \in \Sigma$ identified with $[a, B, \ldots, B]$. Its transition function is the same as the multi-track one, read on these tuple symbols. Every configuration corresponds one-to-one, each step to one step, so the two machines accept the same language in the same time. Multiple tracks are therefore just a way of structuring the tape alphabet.

2011–2012 • Q1(c) Source M14 **MARKS 6 + 2 = 8**

Explain an encoding technique for TMs. Use it to show that the set of TMs is countable.

SOLUTION

Encoding (slides / Hopcroft–Motwani–Ullman). Consider TMs $M = (Q, \{0,1\}, \Gamma, \delta, q_1, B, F)$. Number the states q_1 (start), q_2 (the only accepting state), $q_3, \ldots$; tape symbols $X_1 = 0$, $X_2 = 1$, $X_3 = B, \ldots$; directions $D_1 = L$, $D_2 = R$. A transition $\delta(q_i, X_j) = (q_k, X_l, D_m)$ is encoded as

$$0^i\,1\,0^j\,1\,0^k\,1\,0^l\,1\,0^m$$

(no two consecutive 1's inside, since all exponents are ≥ 1). The whole TM is the list of its transition codes separated by 11: $C_1\,11\,C_2\,11 \cdots 11\,C_n$.

Countability. Enumerate the binary strings as in the slides: w is the i-th string w_i when $1w$ is the binary numeral of i ($w_1 = \varepsilon$, $w_2 = 0$, $w_3 = 1$, $w_4 = 00, \ldots$ — shortlex order). Every TM has (at least one) code, a binary string; choosing one code per TM gives an injective map $M \mapsto \langle M \rangle$ into $\{0,1\}^*$, and different TMs (up to renaming) get different codes. An injection into a countable set shows that the set of TMs is countable: M_i, "the i-th TM", is the TM whose code is w_i (the one-state TM with no moves if w_i is not a valid code), and the list $M_1, M_2, \ldots$ contains every TM.

2011–2012 • Q2(b) Source M14 **MARKS 8**

Show that a language is recognized by a TM with a two-way infinite tape if and only if it is recognized by a TM with a one-way infinite tape.

SOLUTION

($\Leftarrow$) A TM with a one-way (semi-infinite) tape never moves left of its initial cell, so it is already a TM with a two-way tape that happens never to use the left half. (If the one-way model lets a left move at the first cell leave the head in place, the two-way TM first writes a marker in the cell left

of the input and, whenever it moves onto the marker, immediately moves right again.)
($\Rightarrow$) Let M have a two-way tape with cells $\ldots, X_{-2}, X_{-1}, X_0, X_1, X_2, \ldots$, where X_0 is the initial head position. Fold the tape at X_0 into a semi-infinite tape with **two tracks** (slides):

upper	X_0	X_1	X_2	X_3	X_4	$\cdots$
lower	$*$	X_{-1}	X_{-2}	X_{-3}	X_{-4}	$\cdots$

The upper track holds $X_0, X_1, X_2, \ldots$; the lower track holds $X_{-1}, X_{-2}, \ldots$ in reverse order; the lower track of the leftmost cell holds the end-marker $*$, so the simulating TM M' never falls off the left end. M' first turns each input symbol a into $[a, B]$ and the first cell into $[a, *]$. Its states are pairs $[q, U]$ / $[q, L]$: M's state and the track M' is on.

- On the upper track M' moves as M does; on the lower track it moves in the *opposite* direction (a left move of M is a right move of M').
- At the fold: if M' is on the upper track of the leftmost cell (X_0) and M moves left, M' moves right and switches to the lower track (X_{-1}). If M' is on the lower track at X_{-1} and M moves right, M' moves left; arriving at the cell whose lower track is $*$, it switches to the upper track (X_0).

Each move of M is one move of M', and the accepting states of M' are the pairs $[q, \cdot]$ with $q \in F$, so M' accepts iff M does. ■

2010–2011 • Q11(a) Source M12 MARKS 3 × 5 = 15

Describe equivalent single-tape TMs that completely simulate (i) a multitape TM; (ii) a nondeterministic single-tape TM; (iii) a TM with a two-dimensional tape extending infinitely in all directions.

SOLUTION

(i) Multitape. Use one tape with $2k$ tracks: for each tape of M a contents track and a track with a single head marker. Per step: one sweep over all markers stores the k scanned symbols in the finite control (it also counts the markers seen), then M's move is known; a second sweep rewrites the symbols and moves the markers; the new state of M is stored. Accept when the stored state of M is accepting. n moves cost $O(n^2)$. (Details in 2023–24 Q10 above.)
(ii) NTM. Explore M's IDs breadth-first. Keep on the tape a queue of IDs $* \mathrm{ID}_1 * \mathrm{ID}_2 * \cdots$ with the current ID marked. Repeatedly: if the current ID has an accepting state, accept; otherwise append all its successor IDs (one per choice of δ) at the end of the queue and move the mark to the next ID. (Details in 2014–15 Q7(c) above; the slides do this with a scratch tape, which is then removed by (i).)
(iii) 2-D tape. At any time only a finite rectangle of the plane is non-blank. Store it row by row on the 1-D tape: $\#\mathrm{row}_1\#\mathrm{row}_2\#\cdots\#\mathrm{row}_m\#$, all rows of equal length, the head position marked.

- Left/right moves: move the mark within the row.
- Up/down moves: move the mark to the same column of the previous/next row (count the column using marks, zig-zagging between the two rows).
- If the head leaves the rectangle: add a blank column to *every* row (shift contents) or a new blank row at the start/end.

Each 2-D step costs polynomially many 1-D steps, and the 1-D machine accepts exactly when the 2-D machine does.

The same construction with the slides' tools. Give the 1-D tape three tracks: track 1 holds the rows separated by #; track 2 holds a single head marker X under the cell M scans; track 3 holds temporary check marks. The finite control stores M's state and the scanned symbol (storage in the state), so the move of M is known after reading the marked cell. For an up/down move, N zig-zags: for every cell left of the marker in the current row it checks off (track 3) one cell of the neighbouring row, counting from that row's left #; the first unchecked cell then receives the marker, and the check marks are erased. When the rectangle must grow, N uses the shifting-over technique of the slides (copy the rest of the tape to the right by one cell, or via a scratch tape) to insert a blank cell into every row or to append a row of blanks. N accepts when the stored state of M is accepting.

2009–2010 • Q13(a) Source M10 **MARKS 4**

Do you agree that a TM may never stop? Justify with an example.

SOLUTION

Yes. In the slides' model a TM *halts* only when it is in a state q scanning a symbol X for which $\delta(q, X)$ is undefined (no move), and it accepts when it enters a final state (we may assume final states have no moves, so it halts on acceptance). Nothing forces either to happen. Example: $M = (\{q_0\}, \{a\}, \{a, B\}, \delta, q_0, B, \varnothing)$ with $\delta(q_0, a) = (q_0, a, R)$ and $\delta(q_0, B) = (q_0, B, R)$: on input a,

$$q_0a \vdash aq_0B \vdash aBq_0B \vdash aBBq_0B \vdash \cdots$$

M always has a move, so it moves right forever and never halts. More interestingly, the universal TM U on input (M, w) runs forever whenever M runs forever on w. This is why the recursively enumerable languages (a TM may run forever on strings not in the language) are distinguished from the recursive languages (a TM halts on every input), and why the halting problem is meaningful.

2008–2009 • Q3(a) Source M06 **MARKS $2\frac{1}{3}$**

Describe the formal notation of a Turing machine.

SOLUTION

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$: Q finite set of states; Σ input alphabet; $\Gamma \supseteq \Sigma$ tape alphabet; $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ partial transition function, $\delta(q, X) = (p, Y, D)$ means "in state q reading X: go to p, write Y, move D"; q_0 start state; $B \in \Gamma - \Sigma$ the blank; $F \subseteq Q$ accepting states. If $\delta(q, X)$ is undefined, M halts. An instantaneous description $X_1 \cdots X_{i-1}\, q\, X_i \cdots X_n$ shows the tape (leftmost to rightmost nonblank) with the head on X_i in state q; $\vdash$ denotes one move. $L(M) = \{w \in \Sigma^* \mid q_0w \vdash^* \alpha p\beta,\ p \in F\}$.

2007–2008 • Q9 Source M05 **MARKS 10**

Write short notes on (i) multitape Turing machines and (ii) multistack machines.

SOLUTION

(i) Multitape TMs. k tapes with independent heads; the input is on tape 1, all other cells blank. $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k$: one move changes the state, writes on every tape and moves each head left, right or not at all; acceptance is by entering an accepting state. They accept exactly

the recursively enumerable languages, like one-tape TMs (simulation with $2k$ tracks, see above), but are convenient and can be faster: the one-tape simulation of n steps takes $O(n^2)$ steps.
(ii) Multistack machines. A k-stack machine is a deterministic PDA with k stacks: it reads its input from an input source (not from a tape), and each move depends on the state, the input symbol (or ε) and the k stack tops, and replaces each top by a string.

- One stack: a DPDA — strictly weaker than TMs.
- **Two stacks are as powerful as a TM.** Stack 1 holds the tape left of the head (top = cell just left of the head); stack 2 holds the head cell and everything to its right (top = scanned cell). *Loading:* the machine first reads the whole input (followed by an end-marker), pushing it onto stack 1, then pops stack 1 and pushes each symbol onto stack 2; now stack 2 holds the input with its first symbol on top and stack 1 is empty, exactly the initial ID. *Simulation:* for $\delta(q, X) = (p, Y, R)$ pop X from stack 2 and push Y onto stack 1; for $\delta(q, X) = (p, Y, L)$ replace X by Y on stack 2, then pop the top of stack 1 and push it onto stack 2. An empty stack (only its bottom marker) is read as B. Accept when M enters an accepting state.

So 2-stack machines accept exactly the RE languages; by the slides even two-counter machines accept every RE language.

2007–2008 • Q10 Source M05 MARKS 10

Describe how a Turing machine can simulate a computer.

SOLUTION

Model the computer as memory words with addresses $0, 1, 2, \ldots$ (no limit on their number or length), the program stored in memory, and an instruction set where each instruction involves finitely many words, changes at most one word and produces a word at most one bit longer than its arguments (multiplication by repeated addition). A multitape TM simulates it (slides, HMU §8.6):

- **Tape 1 – memory:** \$ followed by $0{*}v_0\#1{*}v_1\#\ldots$, (address, value) pairs in binary; uninitialised words are absent.
- **Tape 2 – instruction counter** (address of the next instruction).
- **Tape 3 – memory address** or the contents of that address once found; further tapes hold the computer's input file and serve as scratch.

Instruction cycle: (1) search tape 1 for the address on tape 2 and fetch the instruction; (2) if it references memory, copy the operand address to tape 3 (marking the instruction on a second track of tape 1), find it on tape 1 and copy its value to tape 3; (3) perform the operation (copy, add by ripple-carry from the right ends, compare, …) using the scratch tape, writing the result back into tape 1 and shifting the rest of tape 1 over if the value changes length; (4) if it is not a jump, add 1 to the instruction counter, otherwise copy the target to tape 2. Halting instructions make the TM halt.
Time. With c the longest word or address in the program and d the number of program words, after n instructions all words and addresses have at most $c + n$ bits and at most $d + n$ addresses are used, so tape 1 has at most $2(c + n + 1)(d + n) = O(n^2)$ cells. Each look-up, each operation (assumed $O(k^2)$ on k-bit words) and each shifting-over costs $O(n^2)$, so one instruction costs $O(n^2)$ TM steps and n instructions $O(n^3)$ steps of the multitape TM; a one-tape TM needs at most $O(n^6)$. The TM is only polynomially slower. Together with the converse (a computer simulates a TM given unbounded storage), this is evidence for the Church–Turing thesis.

2006–2007 • Q4(c) Source M02 **MARKS $5\frac{1}{3}$**

Encode the Turing machine $M = (\{q_1, q_2, q_3\}, \{0,1\}, \{0,1,B\}, \delta, q_1, B, \{q_2\})$, with $\delta(q_1,1) = (q_3,0,R)$; $\delta(q_3,0) = (q_1,1,R)$; $\delta(q_3,1) = (q_2,0,R)$; $\delta(q_3,B) = (q_1,1,L)$.

SOLUTION

$X_1 = 0$, $X_2 = 1$, $X_3 = B$; $D_1 = L$, $D_2 = R$. Code of $\delta(q_i, X_j) = (q_k, X_l, D_m)$ is $0^i10^j10^k10^l10^m$.

Transition	(i,j,k,l,m)	Code
$\delta(q_1,1) = (q_3,0,R)$	$(1,2,3,1,2)$	$C_1 = 0100100010100$
$\delta(q_3,0) = (q_1,1,R)$	$(3,1,1,2,2)$	$C_2 = 0001010100100$
$\delta(q_3,1) = (q_2,0,R)$	$(3,2,2,1,2)$	$C_3 = 00010010010100$
$\delta(q_3,B) = (q_1,1,L)$	$(3,3,1,2,1)$	$C_4 = 00010001010010$

$$\langle M \rangle = C_1\,11\,C_2\,11\,C_3\,11\,C_4$$

$$= 0100100010100\,11\,0001010100100\,11\,00010010010100\,11\,00010001010010.$$

2006–2007 • Q13 Source M03 **MARKS 10**

What are multitape Turing machines and nondeterministic Turing machines (NTMs)? Do they accept any language not accepted by basic Turing machines?

SOLUTION

Multitape TM: k tapes, k independent heads, $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k$. **NTM:** $\delta(q, X)$ is a finite set of triples $\{(q_1, Y_1, D_1), \ldots, (q_k, Y_k, D_k)\}$ — several possible moves, of which one whole triple is chosen at each step; it accepts if *some* sequence of choices leads to an accepting state.
No new languages. Both are simulated by a basic one-tape deterministic TM: multitape by $2k$ tracks with head markers (quadratic slowdown); NTM by breadth-first exploration of its IDs kept in a queue (exponential slowdown). Both proofs appear earlier in this topic. Hence all three models accept exactly the recursively enumerable languages; the extensions only affect convenience and running time.

14

Decidability, RE languages and countability

NOTE **Vocabulary: *recursive* = *decidable* (some TM halts on every input and accepts exactly L); *recursively enumerable (RE)* = *Turing-recognisable* (some TM accepts exactly L, possibly looping on non-members).** $L_u = A_{TM} = \{\langle M, w\rangle \mid M$ **accepts** $w\}$; $L_d = \{w_i \mid w_i \notin L(M_i)\}$; $H = HALT_{TM} = \{\langle M, w\rangle \mid M$ **halts on** $w\}$.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q11 Source N04 **MARKS 15**

Give an example of a language that is not RE. Prove the halting problem is not recursive; you may assume that the universal language is not recursive.

2021–2022 • Q11 Source M48 **MARKS 6 + 9 = 15**

Define the diagonalization and universal languages. Show that the halting problem is not recursive.

2019–2020 • Q12 Source M43 **MARKS 4 + 11 = 15**

What is the halting problem? Prove it is RE but not recursive.

SOLUTION

Definitions / example. $L_d = \{w_i \mid w_i \notin L(M_i)\}$ (diagonalization language), where w_i is the i-th binary string ($1w_i$ is i in binary) and M_i is the TM whose code is w_i ($L(M_i) = \emptyset$ if w_i is not a valid code), is **not RE** (proof above); so is $\overline{L_u}$. The universal language is $L_u = \{\langle M, w\rangle \mid M$ accepts $w\}$. The **halting problem** is $H = \{\langle M, w\rangle \mid M$ is a TM that halts (accepts or rejects) on input $w\}$.
H is RE. TM U_H (built like the universal TM U) = "on $\langle M, w\rangle$: simulate M on w; if the simulation halts — whether M accepts or not — accept." U_H accepts exactly the pairs where M halts (and keeps running otherwise), so $L(U_H) = H$.
H is not recursive (reduction from L_u). Suppose a decider D_H for H existed. Build D_u:

> D_u = "On $\langle M, w\rangle$: (1) run D_H on $\langle M, w\rangle$. (2) If it rejects (M loops on w), *reject*. (3) If it accepts, simulate M on w — this is guaranteed to halt — and accept iff M accepted."

D_u always halts and accepts exactly L_u, so L_u would be recursive — contradicting the known (assumed) fact that L_u is not recursive. Hence H is not recursive. ■
Why L_u is not recursive (for completeness). If L_u were recursive, so would be $\overline{L_u}$. A TM that changes its input w to $w111w$ (the code of the pair (M_i, w_i) when $w = w_i$) and then runs the decider for $\overline{L_u}$ accepts w_i iff M_i does not accept w_i, i.e. it accepts L_d — contradicting the previous theorem.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2022–2023 • Q12 Source M52 **MARKS 5 + 10 = 15**

What is the diagonalization language? Prove that it is not RE.

2017–2018 • Q11 Source M35 **MARKS 9 + 6 = 15**

Define (i) the diagonalization language; (ii) the universal language; (iii) the halting problem. Show that one of them is not RE.

2007–2008 • Q12 Source M05 **MARKS 10**

Define the diagonalization language and prove it is not recursively enumerable.

2006–2007 • Q4(a) Source M01 **MARKS 10**

Prove that the diagonalization language is not recursively enumerable.

SOLUTION

Setup. Enumerate binary strings $w_1 = \varepsilon, w_2 = 0, w_3 = 1, w_4 = 00, \ldots$ (the i-th string is w_i where $1w_i$ is i in binary). Every TM over $\{0,1\}$ has a binary code; let M_i be the TM whose code is w_i (if w_i is not a valid code, M_i is a TM with no moves, so $L(M_i) = \emptyset$). Thus $M_1, M_2, \ldots$ lists *all* TMs.
Definitions.

- *Diagonalization language* $L_d = \{w_i \mid w_i \notin L(M_i)\}$: the strings not accepted by the TM whose code they are.
- *Universal language* $L_u = \{\langle M, w\rangle \mid M \text{ accepts } w\}$, the binary strings $\langle M\rangle 111 w$ coding a TM M and an input w accepted by M (Sipser's A_{TM}).
- *Halting problem* $H = \{\langle M, w\rangle \mid M \text{ halts on } w\}$.

Theorem. L_d is not RE.
Proof (diagonalization). Picture the infinite table with entry $(i, j) = 1$ iff $w_j \in L(M_i)$. Row i is the characteristic vector of $L(M_i)$.

	w_1	w_2	w_3	w_4	$\cdots$
M_1	0	1	1	0	
M_2	1	1	0	0	
M_3	0	0	1	1	
M_4	0	1	0	1	
$\vdots$					$\ddots$

L_d is obtained by *complementing the diagonal*: $w_i \in L_d$ iff entry $(i, i) = 0$. Suppose L_d were RE; then $L_d = L(M_k)$ for some k. Ask whether $w_k \in L_d$:

- If $w_k \in L_d$, then by definition of L_d, $w_k \notin L(M_k) = L_d$. Contradiction.
- If $w_k \notin L_d$, then by definition $w_k \in L(M_k) = L_d$. Contradiction.

Either way we have a contradiction, so no TM accepts L_d: L_d is not RE. (In table terms, L_d differs from every row i at column i.) ■
(L_u and H are RE but not recursive — see the next solution; $\overline{L_u}$ is also not RE.)

2019–2020 • Q11(a–e) Source M42 MARKS 15 (shared)

Give one example in each class: (a) regular; (b) context-free but not regular; (c) recursive but not context-free; (d) RE but not recursive; (e) not RE.

SOLUTION

(a) $\{w \in \{0,1\}^* \mid w \text{ ends in } 01\}$, or 0^*1^*.

(b) $\{0^n1^n \mid n \geq 0\}$ (CFG $S \to 0S1 \mid \varepsilon$; nonregular by pumping).

(c) $\{a^nb^nc^n \mid n \geq 0\}$ (decided by a TM; not context-free by the CFL pumping lemma).

(d) $L_u = A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$, or the halting problem H.

(e) L_d (diagonalization language), or $\overline{L_u}$.

January 2020 Term • Q11(a–b) Source M39 MARKS (a) 7; (b) 8

(a) Define RE and recursive languages. (b) Give examples, with definitions, of (i) a language that is not RE and (ii) an RE language that is not recursive.

SOLUTION

(a) L is *recursively enumerable* if $L = L(M)$ for some TM M: M accepts every $w \in L$, and on $w \notin L$ it either rejects or runs forever. L is *recursive* if $L = L(M)$ for a TM M that *halts on every input* (accepts members, rejects non-members). Every recursive language is RE; not conversely.
(b)(i) $L_d = \{w_i \mid w_i \notin L(M_i)\}$, where w_i is the i-th binary string and M_i the TM with code w_i. It differs from every $L(M_i)$ on the string w_i, so it is not RE.
(b)(ii) $L_u = \{\langle M, w\rangle \mid M \text{ accepts } w\}$. RE: the universal TM accepts it by simulating M on w. Not recursive: a decider for L_u would give a decider for $\overline{L_u}$ and hence a recogniser for L_d, impossible.

2016–2017 • Q7(a) Source M31 MARKS 15

Let $A = \{\langle G\rangle \mid G \text{ is a connected undirected graph}\}$. Can a TM decide A? If not, explain why. If so, discuss the design and demonstrate with an example.

SOLUTION

Yes — A is decidable. Encoding: $\langle G\rangle = (1,2,3,4)((1,2),(2,3),(3,1),(1,4))$ (node list, then edge list).
M = "On input $\langle G\rangle$:

(i) Check the encoding (nodes distinct, every edge joins listed nodes); reject if malformed.

(ii) Mark the first node of G (e.g. put a dot on it on the tape).

(iii) Repeat until no new node is marked: for each node u, if u is unmarked and there is an edge (u,v) or (v,u) with v marked, mark u.

(iv) Scan the node list: if all nodes are marked, *accept*; otherwise *reject*."

Each iteration of stage 3 marks at least one new node or stops, so it runs at most n times, each a finite scan of the tape: M always halts (in fact in polynomial time). It accepts exactly when every node is reachable from node 1.
Example. Above: mark 1; round 1 marks 2 (edge 1–2), 3 (3–1), 4 (1–4); all marked ⇒ accept. For

$(1,2,3)((1,2))$: mark 1, then 2; node 3 never gets marked ⇒ reject.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2016–2017 • Q8(a) Source M31 **MARKS 15**

Prove $A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM accepting } w\}$ is undecidable.

2014–2015 • Q8(c) Source M23 **MARKS 9 1/3**

Prove $A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM accepting } w\}$ is undecidable.

2013–2014 • Q4(c) Source M20 **MARKS 9 1/3**

Prove that $A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM accepting } w\}$ is undecidable.

SOLUTION

A_{TM} is the universal language L_u: the binary strings $\langle M\rangle 111 w$ coding a TM M (input alphabet $\{0,1\}$) and a string w with $w \in L(M)$. We use that the diagonalization language $L_d = \{w_i \mid w_i \notin L(M_i)\}$ is not RE (w_i = the i-th binary string, M_i = the TM whose code is w_i; proof: 2022–2023 Q12 above).

Step 1: if L_u were recursive, so would be $\overline{L_u}$. Let M halt on every input with $L(M) = L_u$. Make M's accepting states non-accepting with no moves, add a new accepting state r, and add a move to r for every (non-accepting state, symbol) pair on which M has no move. The new machine always halts and accepts exactly the strings M rejects, i.e. $\overline{L_u}$.

Step 2: a decider for $\overline{L_u}$ gives a TM for L_d. Let $\overline{M}$ decide $\overline{L_u}$. Build M':

> M' = "On input w: (1) change the input to $w111w$; (2) simulate $\overline{M}$ on $w111w$ and accept iff $\overline{M}$ accepts."

If $w = w_i$, then $w111w$ codes the pair (M_i, w_i), and $\overline{M}$ accepts it iff M_i does *not* accept w_i, i.e. iff $w_i \in L_d$. (If w_i is not a valid code, $L(M_i) = \emptyset$, $w_i \in L_d$ and $w111w \notin L_u$, so again M' accepts.) Hence $L(M') = L_d$.

Conclusion. L_d would be RE — contradicting the theorem that L_d is not RE. So $L_u = A_{TM}$ is not recursive, i.e. undecidable. ■ It is still RE: the universal TM U simulates M on w and accepts iff M accepts.

Remark (Sipser's direct version). Suppose a decider R for A_{TM} existed. Let D = "on input $\langle M\rangle$: run R on $\langle M, \langle M\rangle\rangle$ and output the opposite". D always halts, and D accepts $\langle M\rangle$ iff M does not accept $\langle M\rangle$. On its own description, D accepts $\langle D\rangle$ iff D does not accept $\langle D\rangle$ — a contradiction. This is the same diagonal argument, with D playing the role of a TM for L_d.

2015–2016 • Q7(b) Source M27 **MARKS 10 1/3**

Suppose P is undecidable. Explain how to use P and reduction to prove another problem Q undecidable.

SOLUTION

Reduce P *to* Q: show that a decider for Q could be used to build a decider for P.

(i) Assume, for contradiction, that Q is decided by a TM R.

(ii) Give a *computable* transformation f (a TM that always halts) mapping every instance x of P to an instance $f(x)$ of Q such that $x \in P \iff f(x) \in Q$ (a mapping reduction, $P \leq_m Q$).

(iii) Then S = "on x: compute $f(x)$, run R on it, answer as R" decides P.

(iv) This contradicts the undecidability of P; hence Q is undecidable.

Direction matters: reducing Q to P proves nothing. The same reduction also transfers non-RE-ness: if P is non-RE, then Q is non-RE (a recogniser for Q composed with f would recognise P).
Example: $P = A_{TM}$ $(= L_u)$, $Q = HALT_{TM}$. Let $f(\langle M, w\rangle) = \langle M', w\rangle$, where

> M' = "On input x: run M on x; if M accepts, accept; if M halts without accepting, loop forever."

f only edits the code of M, so it is computable, and M' halts on w iff M accepts w. Thus $\langle M, w\rangle \in A_{TM} \iff f(\langle M, w\rangle) \in HALT_{TM}$, and since A_{TM} is undecidable, so is $HALT_{TM}$. Similarly E_{TM}, $REGULAR_{TM}$, EQ_{TM} are shown undecidable.

2014–2015 • Q8(a) Source M23 **MARKS 7**

What are decidable, Turing-recognizable, and undecidable languages?

SOLUTION

- **Turing-recognisable (RE):** some TM accepts every string in L and does not accept others (it may reject or loop on them). Example: A_{TM}.
- **Decidable (recursive):** some TM (a decider) halts on every input, accepting exactly L. Example: A_{DFA}, $\{a^n b^n c^n\}$.
- **Undecidable:** not decidable — no TM halts on all inputs with the right answer. Undecidable languages may still be recognisable (A_{TM}, $HALT_{TM}$) or not even that ($\overline{A_{TM}}$, L_d).

Decidable $\subsetneq$ Turing-recognisable $\subsetneq$ all languages.

2013–2014 • Q3(b) Source M19 **MARKS 7**

Let A contain only the string s, where $s = 0$ if life will never be found on Mars and $s = 1$ if life is found there someday. Is A decidable? Why? Assume the question about life on Mars has an unambiguous YES or NO answer.

SOLUTION

Yes, A is decidable. Either $A = \{0\}$ or $A = \{1\}$. Both are finite languages, each decided by a trivial TM: M_0 accepts exactly the string 0, M_1 accepts exactly 1. One of M_0, M_1 decides A. Decidability asks whether a deciding TM *exists*, not whether *we know* which one it is. (A classic trick question from Sipser.)

2013–2014 • Q4(a) Source M20 **MARKS 7**

Prove that $A_{DFA} = \{\langle B, w\rangle \mid B \text{ is a DFA accepting } w\}$ is decidable.

SOLUTION

M = "On input $\langle B, w\rangle$:

(i) Check that the input encodes a DFA $B = (Q, \Sigma, \delta, q_0, F)$ and a string $w \in \Sigma^*$; if not, reject.

(ii) Simulate B on w: keep the current state on the tape (initially q_0); for each symbol of w look up $\delta(\text{current}, \text{symbol})$ in the description and update.

(iii) When w is exhausted, accept if the current state is in F, otherwise reject."

The simulation takes exactly $|w|$ steps of B, each a finite table lookup, so M always halts; it accepts iff B accepts w. Hence A_{DFA} is decidable. ■

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q2(a) Source M17 **MARKS 10**

Distinguish Turing-recognizable and decidable languages. Show that $L = \{\langle M, w\rangle \mid M \text{ is a TM and halts on } w\}$ is Turing-acceptable.

2011–2012 • Q2(c) Source M14 **MARKS 7**

Show that $L = \{\langle M, w\rangle \mid M \text{ is a TM and halts on input } w\}$, describing the halting problem, is Turing-acceptable.

SOLUTION

Distinction. A language is *Turing-recognisable* if some TM accepts exactly its members (it may loop on non-members); it is *decidable* if some TM always halts with the correct yes/no answer. Every decidable language is recognisable; A_{TM} and $HALT$ are recognisable but not decidable.

$L = HALT$ **is Turing-acceptable.** U_H = "On input $\langle M, w\rangle$: (1) check it is a valid encoding, else reject; (2) simulate M on w as the universal TM does; (3) if M ever halts — whether it accepts or not — *accept*."

If $\langle M, w\rangle \in L$, M halts after finitely many steps and U_H accepts. If $\langle M, w\rangle \notin L$, the simulation runs forever and U_H never accepts. So $L(U_H) = L$: the halting problem is Turing-acceptable (RE), although it is not decidable.

2011–2012 • Q3(b) Source M15 **MARKS 8**

Prove that the union of two recursively enumerable languages is recursively enumerable.

SOLUTION

Let M_1, M_2 recognise L_1, L_2. Build M = "On w: run M_1 and M_2 on w *alternately, one step at a time* (or on two tapes). If either accepts, accept."

If $w \in L_1 \cup L_2$, one of the machines accepts after finitely many of its steps, so M accepts. If $w \notin L_1 \cup L_2$, neither accepts and M does not accept. Thus $L(M) = L_1 \cup L_2$. Running them one after the other would fail: if M_1 loops on $w \in L_2$, M_2 would never get to run. ■

2011–2012 • Q4(c) Source M15 **MARKS 8 $\frac{1}{3}$**

Show that $L = \{\langle A, B, C\rangle \mid A, B, C \text{ are DFAs over the same alphabet } \Sigma \text{ and } L(C) = L(A) \cup L(B)\}$ is decidable.

SOLUTION

M = "On $\langle A, B, C\rangle$:

(i) Build a DFA U for $L(A) \cup L(B)$ by the product construction.

(ii) Build a DFA D for the symmetric difference $L(D) = (L(C) \cap \overline{L(U)}) \cup (\overline{L(C)} \cap L(U))$, using complement (swap accepting states) and product constructions — all are finite, terminating algorithms.

(iii) Test $L(D) = \emptyset$ (E_{DFA}): mark the start state; repeatedly mark states reachable by one transition from a marked state until nothing new is marked; if no accepting state is marked, $L(D) = \emptyset$.

(iv) Accept iff $L(D) = \emptyset$."

$L(D) = \emptyset \iff L(C) = L(A) \cup L(B)$, and every stage halts, so L is decidable. ■

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2010–2011 • Q10(c) Source M12 **MARKS 7**

Prove that a language is decidable if and only if both it and its complement are Turing-recognizable.

2008–2009 • Q4(d) Source M06 **MARKS 5**

Prove that if both L and its complement are RE, then both are recursive.

SOLUTION

SCOPE *2008–2009 Q4(d) asks only for the ($\Leftarrow$) direction (second paragraph).*

($\Rightarrow$) If M decides L, then M recognises L; and M' = "run M, swap accept/reject" decides (hence recognises) $\overline{L}$. In particular a decidable language and its complement are both RE.
($\Leftarrow$) Let M_1 recognise L and M_2 recognise $\overline{L}$. Build

> M = "On input w: run M_1 and M_2 on w *in parallel*, alternating one step of each (or using two tapes). If M_1 accepts, *accept*; if M_2 accepts, *reject*."

Every w is in L or in $\overline{L}$, so exactly one of M_1, M_2 eventually accepts; thus M always halts, and it accepts precisely the strings of L. Running in parallel (not one after the other) is essential, because M_1 may loop on $w \notin L$. So L is recursive, and $\overline{L}$ is recursive by the ($\Rightarrow$) argument. ■
Corollary: $\overline{A_{TM}}$ is not RE (else A_{TM} would be decidable).

2010–2011 • Q10(d) Source M12 **MARKS 4**

Formally describe the halting problem. Give an example of a language that is not Turing-recognizable.

SOLUTION

$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}$: given a program and an input, decide whether it eventually stops. It is RE but undecidable.
Not Turing-recognisable: $\overline{A_{TM}} = \{\langle M, w\rangle \mid M \text{ does not accept } w\}$ (if it were RE, A_{TM} would be decidable by the complement theorem), or L_d.

2010–2011 • Q12(a) Source M13 **MARKS 4**

Distinguish Turing-decidable and Turing-recognizable languages. What three conditions must a TM M satisfy to recognize a string w in L?

SOLUTION

Decidable: some TM halts on *all* inputs, accepting exactly L. Recognisable: some TM accepts exactly L but may loop on strings outside L.
For $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, M accepts w iff there is a sequence of IDs $C_1, C_2, \ldots, C_k$ such that

(i) C_1 is the **initial ID** of M on w: $C_1 = q_0 w$;

(ii) each C_i **yields** C_{i+1} by one legal move of δ: $C_i \vdash C_{i+1}$;

(iii) C_k is an **accepting ID**: $C_k = \alpha\, p\, \beta$ with $p \in F$.

That is, $q_0 w \vdash^* \alpha\, p\, \beta$ for some $p \in F$ and $\alpha, \beta \in \Gamma^*$.

2009–2010 • Q13(b) Source M10 **MARKS 4**

Prove that the complement of a recursive language is recursive.

SOLUTION

Let M be a TM that halts on every input and accepts L. Build $\overline{M}$ from M by making M's accepting halts reject and its rejecting halts accept (add a new accepting state reached whenever M would halt without accepting). $\overline{M}$ halts on every input because M does, and accepts w iff M rejects w iff $w \notin L$. So $\overline{L}$ is recursive. (This fails for RE languages: if M loops on w, swapping does not help.)

2009–2010 • Q13(c) Source M10 **MARKS 11**

Describe a universal TM with an example.

SOLUTION

A **universal TM** U takes as input the code of a TM M (with input alphabet $\{0,1\}$) together with an input w, written $\langle M\rangle 111 w$, and simulates M on w: U accepts iff M accepts w, so $L(U) = L_u$, the universal language. It is the theoretical model of a stored-program computer.
Construction (slides; multitape, then converted to one tape):

- Tape 1: the input $\langle M\rangle 111 w$ — the "program".
- Tape 2: M's tape, each symbol X_j written as 0^j, symbols separated by single 1's.
- Tape 3: M's current state q_i as 0^i.
- Tape 4: scratch.

Operation. (1) Check that tape 1 holds a legitimate TM code; if not, halt without accepting. (2)

Copy w to tape 2 in encoded form. (3) Write 0 (the start state q_1) on tape 3 and put head 2 on the first simulated cell. (4) To simulate a move: with state 0^i on tape 3 and the symbol 0^j under head 2, search tape 1 for a transition $0^i10^j10^k10^l10^m$; replace the state by 0^k, replace the symbol by 0^l (shifting tape 2 over via tape 4 if more or less space is needed), and move head 2 to the next 1 on the left ($m = 1$) or right ($m = 2$). (5) If no transition matches, M halts, and U halts too. (6) If M enters its accepting state q_2, U accepts.
Example. M: $\delta(q_1, 1) = (q_2, 0, R)$, code 0 1 00 1 00 1 0 1 00. Input $\langle M\rangle$111 1: U puts 00 (the code of symbol 1) on tape 2 and 0 on tape 3, finds the matching code, writes state 00 (q_2) and symbol 0, moves right; state q_2 is accepting, so U accepts, exactly as M accepts input 1.

2009–2010 • Q13(d) Source M10 MARKS $4\frac{1}{3}$

Explain the halting problem with an example.

SOLUTION

The halting problem asks: given any program (TM) M and input w, will M eventually stop on w? Formally $HALT = \{\langle M, w\rangle \mid M \text{ halts on } w\}$. No algorithm answers this for all pairs. **Example:** the program "on input a positive integer n: while $n \neq 1$: if n is even $n \leftarrow n/2$ else $n \leftarrow 3n + 1$" halts on every input tried, but whether it halts on *every* $n \geq 1$ is the open Collatz problem — a general halting decider would settle it, and countless other questions, automatically. Intuitively, a halting decider D_H leads to the paradoxical program D = "if D_H says I halt on my own code, loop forever; else halt", which halts iff it does not.

2008–2009 • Q4(b) Source M06 MARKS $3\frac{1}{3}$

Is the following problem decidable: whether the language accepted by a TM is palindromic? Argue your answer.

SOLUTION

Read "palindromic" as "$L(M)$ consists only of palindromes" (or "contains a palindrome"; the argument is identical). This is a property of the *language* $L(M)$, and it is non-trivial: $\emptyset$ and $\{0\}$ have it; $\{01\}$ does not. Problems about TMs that involve only the language the TM accepts (and are non-trivial) are undecidable (Rice's theorem), so it is **undecidable**.

2008–2009 • Q4(c) Source M06 MARKS 3

Give an example of an RE language that is not recursive.

SOLUTION

$L_u = \{\langle M, w\rangle \mid M \text{ accepts } w\}$: recognised by the universal TM, but not recursive (a decider would make L_d RE). The halting problem is another example.

2007–2008 • Q11 Source M05 MARKS 6

What are recursive, recursively enumerable (RE), and non-RE languages?

SOLUTION

Recursive: $L = L(M)$ for a TM M that halts on every input (accepting the members, halting without accepting on the others). **RE**: $L = L(M)$ for some TM M (which may run forever on non-members). **Non-RE**: accepted by no TM (e.g. L_d). Recursive $\subsetneq$ RE $\subsetneq$ all languages.

2006–2007 • Q3(c) Source M01 MARKS $5\frac{1}{3}$

What is meant by enumeration of binary strings? Find the 13th binary string.

SOLUTION

An enumeration is a listing $w_1, w_2, w_3, \ldots$ of all binary strings in which every string appears exactly once, so each string gets a position number. The standard order (shortlex): by length, then numerically; equivalently, w_i is the string such that $1w_i$ is the binary representation of i:

$$w_1 = \varepsilon,\ w_2 = 0,\ w_3 = 1,\ w_4 = 00,\ w_5 = 01,\ w_6 = 10,\ w_7 = 11,\ w_8 = 000, \ldots$$

$13 = 1101_2$, so w_{13} is 1101 without its leading 1:

Answer. $w_{13} = 101$.

(Check: $\varepsilon; 0, 1; 00, 01, 10, 11; 000, 001, 010, 011, 100, \mathbf{101}$ — the 13th.) This enumeration is what lets us number TMs as $M_1, M_2, \ldots$ in the diagonal argument.

2006–2007 • Q12 Source M03 MARKS part of 10

What is the language of a Turing machine? Define a recursive language.

SOLUTION

For $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, $L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \alpha\, p\, \beta$ for some $p \in F$ and $\alpha, \beta \in \Gamma^*\}$ — the inputs that drive M into an accepting state. L is *recursive* if $L = L(M)$ for some TM M that halts on every input: if $w \in L$, M accepts (and halts); if $w \notin L$, M eventually halts without entering an accepting state.

15

Complexity, reductions and cryptography

2023–2024 • Q1(a) Source N01 **MARKS 8**

What options are available when you confront a problem that appears computationally hard?

SOLUTION

First *establish* the hardness (prove NP-hardness by reducing a known NP-complete problem to it), so you stop searching for an exact fast algorithm. Then:

(i) **Special cases / restricted inputs:** the instances you actually face may be easy (2-SAT, planar or bounded-degree graphs, trees, small numbers ⇒ pseudo-polynomial DP for knapsack).

(ii) **Parameterised algorithms:** exponential only in a small parameter k (e.g. vertex cover in $O(2^k n)$).

(iii) **Approximation algorithms** with a guaranteed ratio (2-approx. vertex cover, Christofides for metric TSP).

(iv) **Heuristics and local search:** greedy, simulated annealing, genetic algorithms — good answers, no guarantee.

(v) **Randomised algorithms** (Monte Carlo, Las Vegas) with good expected behaviour.

(vi) **Smarter exact search:** branch-and-bound, backtracking, SAT/ILP solvers — exponential worst case but often fast in practice.

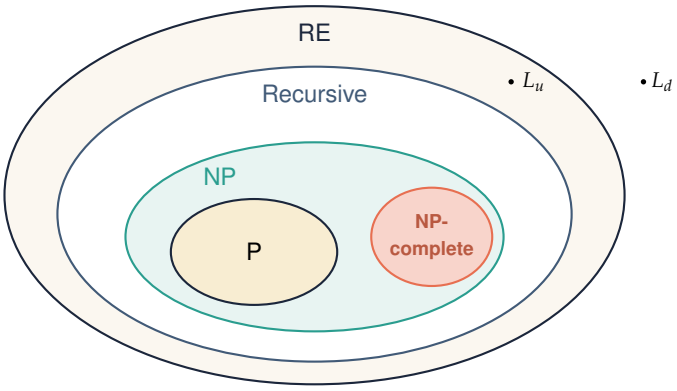
(vii) **Change/relax the problem** (e.g. LP relaxation) or accept average-case performance.

(viii) **Exploit it:** if a problem is truly hard, it may be useful for security (cryptography).

2023–2024 • Q12(i) Source N04 **MARKS 8**

Draw a Venn diagram of the widely believed relationships among P, NP, NP-complete, recursive, and RE languages.

SOLUTION



- P $\subseteq$ NP: a deterministic TM is a nondeterministic TM that never has a choice. **Believed** P $\neq$ NP (open problem).
- NP-complete problems (e.g. SAT) lie in NP; if P $\neq$ NP none of them is in P (a polynomial algorithm for one of them would give P = NP), so the NP-complete region is drawn inside NP and disjoint from P.
- NP $\subsetneq$ recursive: every branch of a polynomial-time NTM halts within $p(n)$ moves, so trying all branches one by one is an algorithm that always halts. The inclusion is proper: NP is contained in deterministic exponential time, and by the time-hierarchy theorem some decidable languages need more time than that.
- Recursive $\subsetneq$ RE: L_u is RE (universal TM) but not recursive. L_d is not RE, so it lies outside all regions.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q12(ii) Source N04 **MARKS 7**

State the relationship between PSPACE and NPSPACE and briefly justify.

2022–2023 • Q13(ii) Source M52 **MARKS 7**

State the relationship between PSPACE and NPSPACE, with justification.

SOLUTION

Answer. PSPACE = NPSPACE (Savitch's theorem: NSPACE$(f(n)) \subseteq$ SPACE$(f^2(n))$ for $f(n) \geq n$).

PSPACE / NPSPACE are the languages decided by a deterministic / nondeterministic TM that scans at most polynomially many tape cells (for an NTM: on any branch, all branches halting).

PSPACE $\subseteq$ NPSPACE**:** every DTM is an NTM.

NPSPACE $\subseteq$ PSPACE**.** Let NTM N decide L in space $f(n)$.

(i) *Bounded number of moves.* With s states and t tape symbols, N has at most $s\, f(n)\, t^{f(n)}$ IDs using $f(n)$ cells. An accepting computation need never repeat an ID, so if N accepts w it does so within $m = c^{1+f(n)}$ moves ($c = s + t$).

(ii) *Reachability.* $reach(I, J, m)$: "can N go from ID I to ID J in at most m moves?" If $m = 1$, check directly whether $I = J$ or $I \vdash J$. Otherwise try systematically every ID K of size at most $f(n)$ as a midpoint and return true if $reach(I, K, m/2)$ and $reach(K, J, m/2)$ are both true.

(iii) *Space.* The two recursive calls are made one after the other, so only one chain of stack frames $(I_1, J_1, m), (I_2, J_2, m/2), (I_3, J_3, m/4), \ldots$ is on the tape at a time: at most $\log_2 m = \log_2 c^{1+f(n)} = O(f(n))$ frames. A frame holds two IDs ($1 + f(n)$ cells each) and m in binary ($O(f(n))$ cells), i.e. $O(f(n))$ space. Total: $O(f^2(n))$.

(iv) *Deciding w.* With $I_0 = q_0 w$, run $reach(I_0, J, c^{1+f(n)})$ for every accepting ID J using at most $f(n)$ cells (enumerated systematically); accept iff one call returns true.

This deterministic simulation always halts and uses $O(f^2(n))$ space; if f is a polynomial, so is f^2. Hence NPSPACE $\subseteq$ PSPACE, and the two classes are equal. ■

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q13 Source N04 **MARKS 8 + 7 = 15**

Explain how Cook's theorem shows every problem in NP reduces to SAT. A nondeterministic TM has $\delta(q,t) = \{(q_1,t_1,R),(q_2,t_2,L)\}$. Construct its corresponding Boolean-formula rule under Cook's construction when the head is on cell i at step j. Assume the head is not on the leftmost or rightmost nonblank cell.

2022–2023 • Q13(i) Source M52 **MARKS 8**

Explain how Cook's theorem shows every NP problem reduces to SAT in polynomial time.

2021–2022 • Q12(i) Source M48 **MARKS 8**

Explain how Cook's theorem shows that every problem in NP reduces to SAT in polynomial time.

SOLUTION

SCOPE *2022–2023 Q13(i) and 2021–2022 Q12(i) ask only for Part 1.*

Part 1: Cook's theorem and the reduction to SAT

Cook's theorem: SAT∈NP is easy (guess a truth assignment and evaluate the formula in polynomial time). The main part is to show that *every* $L \in$ NP reduces to SAT in polynomial time.

Let $L = L(M)$ for a single-tape NTM $M = (Q,\Sigma,\Gamma,\delta,q_0,B,F)$ that makes at most $p(n)$ moves along any branch on an input of length n. If M accepts w ($|w| = n$), there is a sequence $\alpha_0 \vdash \alpha_1 \vdash \cdots \vdash \alpha_k$, $k \le p(n)$, where α_0 is the initial ID and α_k has an accepting state; in $p(n)$ moves only cells $0,\ldots,p(n)$ can be visited. We map (M,w) to a Boolean formula $F(M,w)$ that is satisfiable iff some computation branch accepts: a truth assignment describes one branch.

Variables ($0 \le i,j \le p(n)$, $q \in Q$, $t \in \Gamma$):

- $s(q,j)$: M is in state q during step j;
- l_{ij}: the tape head is on cell i during step j;
- $c(i,j,t)$: cell i contains symbol t during step j;
- $u(i,j)$ ($j < p(n)$): cell i is unchanged from step j to step $j+1$.

The seven rule families ($\overline{x}$ = NOT x):

Rule 1. Exactly one state per step: $\bigvee_{q\in Q} s(q,j)$, and $s(q,j) \to \bigwedge_{\hat{q}\in Q-\{q\}} \overline{s(\hat{q},j)}$.

Rule 2. Exactly one head position per step: $\bigvee_{i=0}^{p(n)} l_{ij}$, and $l_{kj} \to \bigwedge_{i\neq k} \overline{l_{ij}}$.

Rule 3. Exactly one symbol per cell: $\bigvee_{t\in\Gamma} c(i,j,t)$, and $c(i,j,t) \to \bigwedge_{\hat{t}\in\Gamma-\{t\}} \overline{c(i,j,\hat{t})}$.

Rule 4. An unchanged cell keeps its symbol: $c(i,j,t) \wedge u(i,j) \to c(i,j+1,t)$.

Rule 5. The next ID follows by δ: for $\delta(q,t) = \{(q_1,t_1,d_1),\ldots,(q_m,t_m,d_m)\}$,

$$\big[s(q,j) \wedge l_{ij} \wedge c(i,j,t)\big] \to \bigwedge_{k\neq i} u(k,j) \wedge \Big[\big(s(q_1,j{+}1) \wedge l_{\phi(1)(j+1)} \wedge c(i,j{+}1,t_1)\big) \vee \cdots \vee \big(s(q_m,j{+}1) \wedge l_{\phi(m)(j+1)} \wedge c(i,j{+}1,t_m)\big)\Big],$$

where $\phi(k) = i-1$ if $d_k = L$ and $i > 0$; $\phi(k) = i$ if $d_k = L$, $i = 0$ or $d_k = R$, $i = p(n)$; $\phi(k) = i+1$ if $d_k = R$ and $i < p(n)$.

Rule 6. Initial ID (one formula): $s(q_0,0) \wedge l_{00} \wedge c(0,0,w_1) \wedge \cdots \wedge c(n-1,0,w_n) \wedge c(n,0,B) \wedge \cdots \wedge c(p(n),0,B)$.

Rule 7. Acceptance (one formula): $s(q_a, p(n))$, q_a the accepting state (an accepting branch is taken to stay in q_a once it gets there).

Size. $F(M,w)$ is the AND of all these formulas. Each formula is written directly from its parameters, so $F(M,w)$ is computable from M and w. The largest family is Rule 5: one formula for each i,j,t,q, each containing the $O(p(n))$-long conjunction $\bigwedge_{k\neq i} u(k,j)$, in total $O(|Q|\,|\Gamma|\,p(n)^3)$ — a polynomial in n. So the construction takes polynomial time.

Correctness. An accepting branch $\alpha_0 \vdash \cdots \vdash \alpha_{p(n)}$ gives a satisfying assignment (set each variable as the IDs say). Conversely, in a satisfying assignment Rules 1–3 make every step a well-formed ID, Rule 6 makes step 0 the initial ID, Rules 4–5 make each step follow from the previous one by a move of δ, and Rule 7 makes the last state accepting — an accepting branch. Hence $w \in L \iff F(M,w) \in \text{SAT}$: every NP language reduces to SAT in polynomial time, and with SAT∈NP, SAT is NP-complete. Nondeterminism appears only as the $\vee$ in Rule 5: the satisfying assignment chooses the branch.

Part 2: the rule for $\delta(q,t) = \{(q_1,t_1,R),(q_2,t_2,L)\}$

Here $d_1 = R$, $d_2 = L$, and the head is not at either end, so $0 < i < p(n)$, $\phi(1) = i+1$ and $\phi(2) = i-1$. Rule 5 becomes

$$\big[s(q,j) \wedge l_{ij} \wedge c(i,j,t)\big] \;\to\; \bigwedge_{k\neq i} u(k,j) \wedge \Big[\big(s(q_1,j{+}1) \wedge l_{(i+1)(j+1)} \wedge c(i,j{+}1,t_1)\big) \vee \big(s(q_2,j{+}1) \wedge l_{(i-1)(j+1)} \wedge c(i,j{+}1,t_2)\big)\Big]$$

"If during step j the machine is in state q with the head on cell i holding t, then every other cell is unchanged, and during step $j+1$ either the state is q_1, cell i holds t_1 and the head is on cell $i+1$ (right move), or the state is q_2, cell i holds t_2 and the head is on cell $i-1$ (left move)." Together with Rule 4, $c(k,j,X) \wedge u(k,j) \to c(k,j+1,X)$ for $k \neq i$ and every $X \in \Gamma$, the unchanged cells keep their symbols. The symbol is written on cell i (where the head was), while the head moves to $i \pm 1$. There is one such formula for every i and j, each of size $O(p(n))$, so this part of $F(M,w)$ has size $O(p(n)^3)$.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q12(ii) Source M48 MARKS 7

Explain why a polynomial-time algorithm for any NP-complete problem implies P=NP.

January 2020 Term • Q12(a) Source M39 MARKS 7

State Cook's theorem. Explain why a polynomial-time algorithm for an NP-complete problem implies P=NP.

SOLUTION

SCOPE *2021–2022 Q12(ii) does not ask for the statement of Cook's theorem (first paragraph).*

Cook's theorem: SAT is NP-complete, i.e. SAT$\in$NP and every language in NP reduces to SAT in polynomial time (construction: 2023–2024 Q13 above).
Why a polynomial algorithm for an NP-complete problem gives P $=$ NP**.** L is NP-complete if (1) $L \in$ NP and (2) every $A \in$ NP has a polynomial-time reduction to L. Suppose L is NP-complete and has a deterministic decider running in time $q(m)$. Take any $A \in$ NP, with reduction f computable in time $p(n)$ such that $w \in A \iff f(w) \in L$; note $|f(w)| \leq p(n)$ (a TM writes at most one symbol per move). To decide A: compute $f(w)$, then run L's decider on it — total time $p(n) + q(p(n))$, a polynomial. So $A \in$ P, i.e. NP $\subseteq$ P; with P $\subseteq$ NP (every DTM is an NTM) we get P $=$ NP. ■

2021–2022 • Q13(i–iii) Source M48 **MARKS 15 (shared)**

Give the widely believed relationships, with brief justifications: (i) P and NP; (ii) P and PSPACE; (iii) PSPACE and NPSPACE.

SOLUTION

(i) P $\subseteq$ NP: a deterministic TM is a nondeterministic TM that never has a choice of moves. **Believed** P $\subsetneq$ NP (open): a polynomial-time NTM can in effect guess one of exponentially many candidate solutions and check it in polynomial time, and no polynomial algorithm is known for any NP-complete problem.

(ii) P $\subseteq$ PSPACE: a TM that makes at most $T(n)$ moves cannot visit more than $1 + T(n)$ cells, so polynomial time implies polynomial space. The same argument gives NP $\subseteq$ NPSPACE, and NPSPACE $=$ PSPACE by (iii); so P $\subseteq$ NP $\subseteq$ PSPACE. **Believed** P $\subsetneq$ PSPACE (open): space can be reused, time cannot.

(iii) PSPACE $=$ NPSPACE (**proved**, Savitch's theorem): an $f(n)$-space NTM is simulated deterministically in space $O(f^2(n))$ by the recursive reachability test; see 2023–2024 Q12(ii) above.

2019–2020 • Q13(a–c) Source M43 **MARKS 15 (shared)**

Explain true or false: (a) polynomial-time SAT implies P=NP; (b) languages decided by polynomial-time nondeterministic TMs are a superset of those decided by polynomial-space deterministic TMs; (c) polynomial-space deterministic and nondeterministic TMs decide the same class of languages.

SOLUTION

(a) True. SAT is NP-complete (Cook's theorem), and a polynomial-time algorithm for an NP-complete problem gives P $=$ NP (2021–2022 Q12(ii) above).

(b) False as stated (not known, and believed false). The statement says NP $\supseteq$ PSPACE. The known relation is the reverse: NP $\subseteq$ NPSPACE (polynomially many moves visit polynomially many cells) and NPSPACE $=$ PSPACE, so NP $\subseteq$ PSPACE. NP $\supseteq$ PSPACE would mean NP $=$ PSPACE, which is believed false.

(c) True: PSPACE $=$ NPSPACE by Savitch's theorem (2023–2024 Q12(ii) above).

January 2020 Term • Q1(a) Source M36 MARKS 6

Given an instance of Dominating Set, describe how a certificate can establish that the problem is in NP.

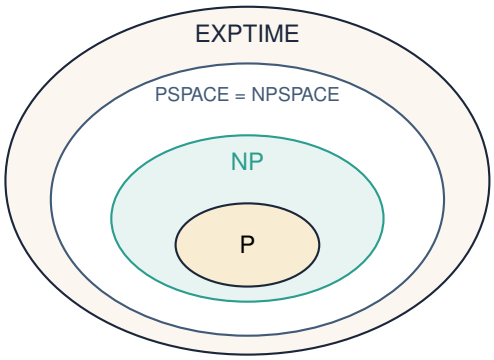
SOLUTION

Instance $\langle G, k\rangle$: does $G = (V, E)$ have a dominating set $D \subseteq V$ with $|D| \leq k$ (every vertex is in D or adjacent to a vertex of D)?
Certificate: the set D itself (a list of at most $k \leq |V|$ vertices — polynomial size).
Verifier $V(\langle G, k\rangle, D)$: (1) check $D \subseteq V$, no repeats, $|D| \leq k$ — $O(|V|)$; (2) mark every vertex of D and every neighbour of a vertex in D by scanning the edge list — $O(|V| + |E|)$; (3) accept iff all vertices are marked. This runs in polynomial time; a yes-instance has a certificate that makes V accept and a no-instance has none. Hence Dominating Set $\in$ NP (it is in fact NP-complete, by reduction from Vertex Cover).

January 2020 Term • Q12(b) Source M39 MARKS 8

Draw a Venn diagram of the widely believed relationships among P, NP, PSPACE, NPSPACE, and EXPTIME, and briefly justify.

SOLUTION



EXPTIME is the class of languages decided by a deterministic TM in time 2^{n^k} for some k.

- P $\subseteq$ NP: a DTM is an NTM. Believed proper.
- NP $\subseteq$ PSPACE: NP $\subseteq$ NPSPACE (in $T(n)$ moves at most $1 + T(n)$ cells are visited) and NPSPACE = PSPACE. Believed proper.
- PSPACE = NPSPACE: proved (Savitch; 2023–2024 Q12(ii) above), so they are one region.
- PSPACE $\subseteq$ EXPTIME: a $p(n)$-space TM with s states and t tape symbols has at most $s\,p(n)\,t^{p(n)}$ IDs, so if it accepts it does so within $c^{1+p(n)}$ moves ($c = s + t$); stopping after that many moves gives an exponential-time decider. Believed proper.

P $\subsetneq$ EXPTIME is *proved* (time hierarchy theorem), so at least one inclusion in P $\subseteq$ NP $\subseteq$ PSPACE $\subseteq$ EXPTIME is strict, but which one is not known.

2017–2018 • Q1(a) Source M32 MARKS 6

Define Boolean Satisfiability with examples.

SOLUTION

A Boolean formula is built from variables, $\wedge$, $\vee$, $\neg$ and parentheses. It is *satisfiable* if some assignment of TRUE/FALSE to its variables makes it true. SAT $= \{\langle\phi\rangle \mid \phi \text{ is satisfiable}\}$; CNF-SAT and 3-SAT restrict ϕ to a conjunction of clauses (with exactly 3 literals per clause for 3-SAT). Examples: $\phi_1 = (x \vee \neg y) \wedge (\neg x \vee y)$ is satisfiable ($x = y = 1$). $\phi_2 = x \wedge \neg x$ is unsatisfiable. $\phi_3 = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_3 \vee x_4)$ is satisfiable ($x_1 = 1$). SAT is in NP (certificate = assignment) and NP-complete (Cook).

2017–2018 • Q1(b) Source M32 **MARKS 20**

With the knowledge that the 3-SAT problem is NP, prove that graph 3-colorability is NP-complete.

SOLUTION

NOTE **The source says "3-SAT is NP"; for the proof we need that 3-SAT is NP-*complete* (Cook–Levin + reduction), which we use.**

3-COL $\in$ NP. Certificate: a colour for each vertex. Verify in $O(|E|)$ that adjacent vertices differ.
3-SAT $\leq_p$ 3-COL. Given ϕ with variables $x_1..x_n$ and clauses $C_1..C_m$, build G:

(i) **Palette:** a triangle on nodes T, F, B (true, false, base). They get three different colours; call the colours by these names.

(ii) **Variable gadget:** for each x_i, nodes x_i and $\neg x_i$, joined to each other and both joined to B. So one of them is coloured T and the other F — a truth assignment.

(iii) **Clause gadget** for $C = (a \vee b \vee c)$: an *OR-gadget* $\mathrm{OR}(u, v)$ consists of new nodes p, q, o with edges $u{-}p$, $v{-}q$, $p{-}q$, $p{-}o$, $q{-}o$. Property: if u, v are both F, then $p, q \in \{T, B\}$ are different, so o is forced to F; if u or v is T, o can be coloured T. Chain $o_1 = \mathrm{OR}(a, b)$, $o_2 = \mathrm{OR}(o_1, c)$ and join o_2 to both F and B, forcing $o_2 = T$.

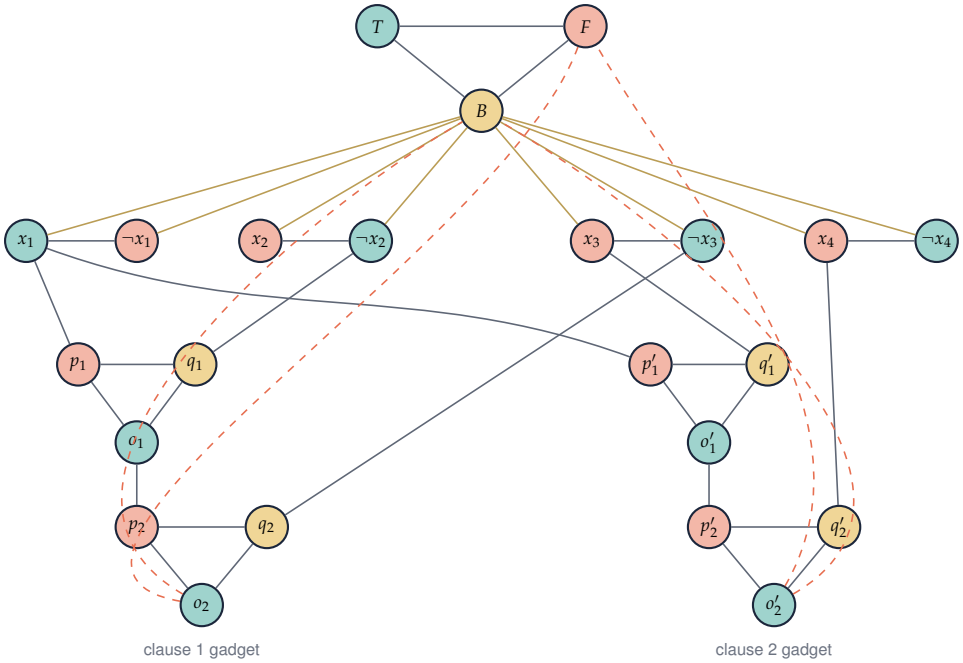
G has $3 + 2n + 6m$ nodes and $3 + 3n + 12m$ edges: polynomial.
Correctness. ($\Rightarrow$) If ϕ is satisfied, colour literals by their truth values; in each clause some literal is T, so by the OR property both gadgets can output T, and $o_2 = T$ is consistent with its edges to F, B. ($\Leftarrow$) In any 3-colouring, literal nodes are T/F consistently (variable gadget). If some clause had all three literals F, then $o_1 = F$ and then $o_2 = F$ — but o_2 is adjacent to F. So every clause has a true literal: ϕ is satisfiable.
Since 3-SAT is NP-complete and reduces to 3-COL in polynomial time, and 3-COL $\in$ NP, **3-COL is NP-complete**. ■

2017–2018 • Q1(c) Source M32 **MARKS 9**

Using the graph framework in 1(b), show the final 3-color graph for the 3-SAT formula $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_3 \vee x_4)$.

SOLUTION

Satisfying assignment used for the colouring: $x_1 = 1$, $x_2 = x_3 = x_4 = 0$. Colours: **T**, **F**, **B**.



Dashed edges join each clause output o_2 to F and B, forcing it to colour T. Every edge joins two different colours, so the graph is 3-coloured, confirming ϕ is satisfiable. (Clause 1: $o_1 = \mathrm{OR}(x_1, \neg x_2)$, $o_2 = \mathrm{OR}(o_1, \neg x_3)$; clause 2: $o'_1 = \mathrm{OR}(x_1, x_3)$, $o'_2 = \mathrm{OR}(o'_1, x_4)$.)

2017–2018 • Q12 Source M35 **MARKS 15**

The subgraph-isomorphism problem asks whether graph G_1 contains an exact copy of G_2 as a subgraph: choose a subset of G_1's nodes and a one-to-one correspondence with G_2 so the edges among those nodes form the copy. Prove this problem NP-complete.

SOLUTION

SUBISO $= \{\langle G_1, G_2\rangle \mid G_1$ has a subgraph isomorphic to $G_2\}$.

In NP. Certificate: an injective map $f : V(G_2) \to V(G_1)$. Verify in polynomial time that f is one-to-one and that for every edge $(u, v) \in E(G_2)$, $(f(u), f(v)) \in E(G_1)$.

NP-hard: CLIQUE $\leq_p$ SUBISO**.** CLIQUE $= \{\langle G, k\rangle \mid G$ has a k-clique$\}$ is NP-complete. Map $\langle G, k\rangle \mapsto \langle G_1, G_2\rangle = \langle G, K_k\rangle$, where K_k is the complete graph on k nodes (built in $O(k^2)$ time; if $k > |V(G)|$ output a fixed no-instance).

- If G has a k-clique S, any bijection $V(K_k) \to S$ maps every edge of K_k to an edge of G: K_k is a subgraph of G.
- If K_k embeds into G via f, the k image nodes are pairwise adjacent in G: a k-clique.

So $\langle G, k\rangle \in$ CLIQUE $\iff \langle G, K_k\rangle \in$ SUBISO. Being in NP and NP-hard, subgraph isomorphism is **NP-complete**. ■ (Under the "induced subgraph" reading the same reduction works, since an induced copy of K_k is also a clique. This is the same idea as the slides' reduction Clique $\leq_p$ Maximum Common Subgraph, where G_2 is a complete graph.)

2017–2018 • Q13(a–b) Source M35 **MARKS (a) 6; (b) 9**

(a) Define P, NP, PSPACE, and NPSPACE. (b) Give the widely believed relationship (equality or containment) for (i) P and NP; (ii) PSPACE and NPSPACE; (iii) NP and PSPACE. Briefly justify.

SOLUTION

(a)

- P: $L \in$ P if $L = L(M)$ for some deterministic TM M of time complexity $T(n)$ for a polynomial T (on every input of length n, M halts after at most $T(n)$ moves).
- NP: $L \in$ NP if $L = L(M)$ for a nondeterministic TM M and a polynomial $T(n)$ such that on an input of length n, M has no sequence of more than $T(n)$ moves. Equivalently, membership can be *checked* in polynomial time: $w \in L \iff$ some certificate c with $|c| \leq p(|w|)$ makes a polynomial-time verifier $V(w, c)$ accept.
- PSPACE: languages decided by a deterministic TM whose space complexity (the maximum number of tape cells scanned on an input of length n) is a polynomial.
- NPSPACE: languages decided by a nondeterministic TM, all of whose branches halt, scanning at most polynomially many cells on any branch.

(b)

(i) P $\subseteq$ NP, believed proper (2021–2022 Q13(i) above).

(ii) PSPACE $=$ NPSPACE, proved by Savitch's theorem (2023–2024 Q12(ii) above).

(iii) NP $\subseteq$ PSPACE: NP $\subseteq$ NPSPACE, since a TM making $T(n)$ moves visits at most $1 + T(n)$ cells, and NPSPACE $=$ PSPACE. Believed proper.

16
Out of syllabus

NOTE The questions in this topic use material that is not covered in Hakim sir's or Atif sir's slides (countability arguments, the formal proof of Rice's theorem, cryptography). They are kept for completeness and solved with the standard textbook method.

2022–2023 • Q1(a) Source M49 **MARKS 10**

Explain the roles of public and private keys in asymmetric cryptography, how they work together to secure communication between two parties, and the role of Theory of Computation in this setting.

SOLUTION

Keys. Each party (say Bob) generates a pair (K_{pub}, K_{priv}). The public key is published; the private key is kept secret. They are mathematically linked (in RSA: $n = pq$, e public, $d = e^{-1} \bmod \varphi(n)$ private), but computing K_{priv} from K_{pub} is believed infeasible.
Confidentiality. Alice encrypts m with Bob's public key: $c = E_{pub}(m)$ ($c = m^e \bmod n$). Only Bob, holding the private key, can decrypt: $m = D_{priv}(c)$ ($c^d \bmod n$). No shared secret had to be exchanged beforehand.
Authentication / integrity. Bob signs with his private key, $\sigma = D_{priv}(h(m))$; anyone verifies with his public key. In practice the slow public-key step is used to agree on a fast symmetric session key (hybrid encryption, TLS).
Role of Theory of Computation. Security rests on *computational complexity*: encryption and decryption with the key are in P (polynomial-time), whereas recovering m or K_{priv} without it requires solving a problem believed intractable (integer factoring, discrete logarithm). These are *one-way (trapdoor) functions* — easy to compute, hard to invert without the trapdoor. Their existence would imply P $\neq$ NP; if P = NP, keys could be found efficiently and public-key cryptography would collapse. Complexity theory gives the language (polynomial time, reductions, average-case hardness) to state and prove such security claims.

2019–2020 • Q1(b) Source M40 **MARKS 15**

Describe a scenario that specifically requires computationally hard problems, even though easy problems are generally preferable.

SOLUTION

Secure online banking / e-commerce. A customer sends a card number to a bank over the open Internet, where an eavesdropper can read every packet. The bank publishes an RSA public key (n, e) with $n = pq$ a product of two secret 1024-bit primes; the browser encrypts with it, and only the bank (knowing p, q, hence d) decrypts.

- The legitimate parties need *easy* problems: modular exponentiation is polynomial time.
- The attacker must be faced with a *hard* problem: factoring n (or computing e-th roots mod n). The best known algorithms take super-polynomial time, so even all the world's computers would need astronomically long.

If factoring were easy, anyone could read everyone's traffic. Here hardness is a feature, not an obstacle: the *gap* between easy (with the trapdoor) and hard (without it) is what gives security. Other examples: password hashing, digital signatures, proof-of-work in cryptocurrencies, CAPTCHAs.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2014–2015 • Q7(b) Source M23 **MARKS 7**

Prove that the set of infinite binary sequences is uncountable.

2010–2011 • Q10(a) Source M12 **MARKS 4**

Show that the set of all infinite binary sequences is uncountable.

SOLUTION

Let $\mathcal{B}$ be the set of infinite binary sequences. Suppose $\mathcal{B}$ were countable: then there is a list $s_1, s_2, s_3, \ldots$ containing every sequence, with $s_i = s_{i1}s_{i2}s_{i3}\cdots$. Define the diagonal-flipped sequence

$$t = t_1 t_2 t_3 \cdots, \qquad t_i = 1 - s_{ii}.$$

s_1 :	0	1	1	0	$\cdots$
s_2 :	1	0	0	1	$\cdots$
s_3 :	0	1	1	1	$\cdots$
s_4 :	1	1	0	0	$\cdots$
t :	1	1	0	1	$\cdots$

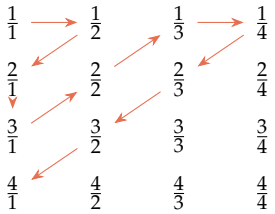
$t \in \mathcal{B}$, but $t \neq s_i$ for every i because they differ in position i. So the list misses t — contradiction. Hence $\mathcal{B}$ is uncountable. ■

2013–2014 • Q4(b) Source M20 **MARKS 7**

Prove that the rational numbers form a countable set.

SOLUTION

It suffices to list the positive rationals $\mathbb{Q}^+$ (then interleave $0, q_1, -q_1, q_2, -q_2, \ldots$). Arrange p/q in an infinite matrix, row p, column q:



Traverse the diagonals $p + q = 2, 3, 4, \ldots$ in a zig-zag; each diagonal is finite, so every p/q is reached after finitely many steps. Skip fractions not in lowest terms (e.g. $2/2$) so each rational appears once.

Following the arrows, this gives a correspondence $f : \mathbb{N} \to \mathbb{Q}^+$: $1, \frac{1}{2}, 2, 3, \frac{1}{3}, \frac{1}{4}, \frac{2}{3}, \frac{3}{2}, 4, \ldots$ Hence $\mathbb{Q}$ is countable. ■

2010–2011 • Q10(b) Source M12 MARKS $8\frac{1}{3}$

Using the findings of 10(a), prove that some languages are not Turing-recognizable.

SOLUTION

TMs are countable. Each TM has a finite description $\langle M \rangle \in \Sigma^*$, and Σ^* is countable (list by length, then lexicographically). So the set of TMs — and hence the set of Turing-recognisable languages $\{L(M)\}$ — is countable.

Languages are uncountable. List $\Sigma^* = \{s_1, s_2, s_3, \ldots\}$. Map each language $A \subseteq \Sigma^*$ to its *characteristic sequence* χ_A, whose i-th bit is 1 iff $s_i \in A$. E.g. for $\Sigma = \{0, 1\}$ and $A =$ "strings starting with 0":

Σ^*	ε	0	1	00	01	10	11	$\cdots$
χ_A	0	1	0	1	1	0	0	$\cdots$

This map is a bijection between the set of all languages and $\mathcal{B}$, which is uncountable by 10(a).

Conclusion. There are uncountably many languages but only countably many TMs, so some (indeed, most) languages are recognised by no TM. ■

2008–2009 • Q4(a) Source M06 MARKS 3 + 9 = 12

Describe Rice's theorem and sketch its proof.

SOLUTION

Rice's theorem. Every *non-trivial* property of the RE languages is undecidable. A property $\mathcal{P}$ is a set of RE languages; non-trivial means some RE language has it and some does not. Then $L_{\mathcal{P}} = \{\langle M \rangle \mid L(M) \in \mathcal{P}\}$ is undecidable. (It concerns the *language* of M, not M's structure.)

Proof sketch. Assume $\emptyset \notin \mathcal{P}$ (otherwise use $\overline{\mathcal{P}}$, which is also non-trivial; decidability is closed under complement). Since $\mathcal{P}$ is non-trivial, pick a TM M_L with $L = L(M_L) \in \mathcal{P}$. Reduce L_u to $L_{\mathcal{P}}$: given $\langle M, w \rangle$, construct M':

> M' = "On input x: simulate M on w; if it accepts, run M_L on x and accept iff M_L accepts."

If M accepts w, $L(M') = L \in \mathcal{P}$. If not, M' accepts nothing, $L(M') = \emptyset \notin \mathcal{P}$. The map $\langle M, w \rangle \mapsto \langle M' \rangle$ is computable, so a decider for $L_{\mathcal{P}}$ would decide L_u — impossible. ■